

# DSA Revision Notes

## Striver's A2Z DSA Sheet

### Batch 1 · Steps 1 & 2: Basics + Sorting

Purpose Quick revision before interviews & coding rounds

Covers 452 problems across 18 steps

Format Pattern · Intuition · Algorithm · Complexity · Traps

These notes are NOT for teaching — they are for fast recall. Scan any note in under 2 minutes.

#### TABLE OF CONTENTS

#### STEP 1 — LEARN THE BASICS

- 1.1 Time Complexity Calculations
- 1.5 Recursion — Core Concepts
- 1.6 Hashing — Core Concepts

#### STEP 2 — SORTING TECHNIQUES

- 2.1.1 Selection Sort
- 2.1.2 Bubble Sort
- 2.1.3 Insertion Sort
- 2.2.1 Merge Sort
- 2.2.4 Quick Sort
- Master Sorting Comparison Table

#### STEP 1 - Learn the Basics

Foundation for everything. These are not standalone contest problems — they are concepts and mental models you must internalize before proceeding.

### 1.1 · Time Complexity Calculations

#### WHAT IT'S ASKING

Given an algorithm, work out how its running time grows with input size  $N$  so you can label it with Big-O.

#### TOPIC

Fundamentals — Algorithm Analysis

#### CORE IDEA

Count the number of basic operations as a function of input size  $N$ . Drop constants and lower-order terms — only the dominant term matters. Always quote Worst Case in interviews unless told otherwise.

#### COMMON COMPLEXITIES (FASTEST → SLOWEST)

| Notation    | Name        | Typical Example                    |
|-------------|-------------|------------------------------------|
| $O(1)$      | Constant    | Array index access, HashMap lookup |
| $O(\log N)$ | Logarithmic | Binary Search                      |

|               |              |                                      |
|---------------|--------------|--------------------------------------|
| $O(N)$        | Linear       | Single loop over N elements          |
| $O(N \log N)$ | Linearithmic | Merge Sort, Heap Sort                |
| $O(N^2)$      | Quadratic    | Nested loops (bubble/selection sort) |
| $O(2^N)$      | Exponential  | All subsets, naive recursion         |
| $O(N!)$       | Factorial    | All permutations                     |

**KEY COUNTING RULES**

- Single loop over N  $\rightarrow O(N)$
- Two nested loops  $\rightarrow O(N^2)$
- Input halved each step  $\rightarrow O(\log N)$
- Recursion: branching factor B, depth D  $\rightarrow O(BD)$
- Sequential (non-nested) blocks  $\rightarrow$  add complexities, keep dominant

**COMMON MISTAKES**

- Forgetting that log base doesn't matter asymptotically
- Ignoring hidden inner loops (e.g. string copy  $O(N)$  inside a loop  $\rightarrow O(N^2)$ )
- Confusing space complexity of recursion — stack depth counts as space

**■ Interview Trigger**

Count operations  $\rightarrow$  drop constants  $\rightarrow$  keep the dominant term.

**1.5 · Recursion — Core Concepts****WHAT IT'S ASKING**

Understand what makes a function call itself correctly: base case, state reduction, and the call stack.

**TOPIC**

Recursion — Divide the problem into a smaller version of itself.

**THE THREE LAWS**

- Must have a base case (termination condition).
- Must move toward the base case with each call.
- Must call itself on a strictly smaller or simpler input.

**ALGORITHM — GENERIC RECURSIVE TEMPLATE**

- 1. Identify the base case — what is the smallest valid input?
- 2. Assume the recursive call works correctly for  $(n - 1)$ .
- 3. Use that result to build the answer for  $n$ .
- 4. Return the combined result.

**VISUALIZATION — CALL STACK FOR FACTORIAL(4)**

```

factorial(4)
  4 × factorial(3)
  3 × factorial(2)
  2 × factorial(1)
  1 ← base case
returns 2
returns 6
returns 24

```

**COMPLEXITY**

- Time:  $O(N)$  for linear recursion
- Space:  $O(N)$  call stack depth — watch for stack overflow on large  $N$

**ESSENTIAL PROBLEMS**

- Print 1 to  $N$  · Reverse Array · Fibonacci · Palindrome Check · Power( $x, n$ )

**COMMON MISTAKES**

- Missing base case → infinite recursion / stack overflow
- Not reducing the problem size → infinite recursion
- Forgetting to return the result of the recursive call

**■ Interview Trigger**

Trust the recursion — assume it works for  $N-1$ , build  $N$  from it.

## 1.6 · Hashing — Core Concepts

**WHAT IT'S ASKING**

Understand how hashing gives near  $O(1)$  lookups and where collisions come from.

**TOPIC / PATTERN**

Hashing — Frequency Counting /  $O(1)$  Lookup

**CORE IDEA**

Use a map or array to store the count or presence of elements, enabling  $O(1)$  access instead of  $O(N)$  search.

**WHEN TO IMMEDIATELY THINK HASHING**

- "Count frequency of elements"
- "Check if element exists / was seen before"
- "Find the duplicate / the missing element"
- "Most frequent or least frequent element"

**APPROACHES**

| Approach        | Time   | Space                | Notes                                               |
|-----------------|--------|----------------------|-----------------------------------------------------|
| Frequency Array | $O(N)$ | $O(\text{MAX\_VAL})$ | Only when value range is bounded (e.g. 0 to $10n$ ) |
| HashMap         | $O(N)$ | $O(N)$               | General case — works for any value range            |

**KEY OPERATIONS**

- `freq[x]++` → increment count of `x` in  $O(1)$
- `freq[x]` → retrieve count of `x` in  $O(1)$
- Iterate over map → find max/min frequency

**COMMON MISTAKES**

- Using an array when values can be negative — use `HashMap` instead
- Not initializing the frequency array to 0
- Forgetting that `HashMap` has  $O(1)$  average but  $O(N)$  worst (hash collisions)

**■ Interview Trigger**

Hashing trades space for time —  $O(N)$  space buys  $O(1)$  lookup.

**STEP 2 - Important Sorting Techniques**

Know every sort: how it works, its complexity, stability, and when to use it. Merge Sort and Quick Sort are the most interview-critical — understand both deeply.

**2.1.1 · Selection Sort****WHAT IT'S ASKING**

Repeatedly find the minimum of the unsorted part and swap it to the front.

**TOPIC / PATTERN**

Sorting — Repeated Minimum Selection

**CORE IDEA**

In each pass, find the minimum element from the unsorted portion and swap it into its correct position at the front.

**ALGORITHM**

1. For  $i = 0$  to  $N-2$ :
2. Find the index of the minimum element in `arr[i .. N-1]`.
3. Swap `arr[i]` with `arr[min_index]`.

**VISUALIZATION**

```
[ 5, 3, 1, 4, 2 ]
i=0 → min=1 at idx 2 → [ 1, 3, 5, 4, 2 ]
i=1 → min=2 at idx 4 → [ 1, 2, 5, 4, 3 ]
i=2 → min=3 at idx 4 → [ 1, 2, 3, 4, 5 ]
```

**COMPLEXITY**

| Approach       | Time     | Space  | Notes                                                    |
|----------------|----------|--------|----------------------------------------------------------|
| Selection Sort | $O(N^2)$ | $O(1)$ | Always $O(N^2)$ — no best-case optimization. Not stable. |

**COMMON MISTAKES**

- Looping to N instead of N-1 (last element is always already in place)
- Thinking it is stable — it is NOT; distant swaps can break relative order

**■ Interview Trigger**

Find the minimum in the unsorted part, place it at the front — repeat.

**2.1.2 · Bubble Sort****WHAT IT'S ASKING**

Repeatedly swap adjacent out-of-order elements until the array is sorted.

**TOPIC / PATTERN**

Sorting — Adjacent Swaps / Bubbling

**CORE IDEA**

Repeatedly compare and swap adjacent elements if they are out of order. The largest unsorted element 'bubbles up' to its correct position each pass.

**ALGORITHM**

1. For  $i = 0$  to  $N-2$ :
2. For  $j = 0$  to  $N-2-i$ :
3. If  $arr[j] > arr[j+1]$ : swap them.
4. Optimization: track if any swap occurred. If none → already sorted → break early.

**VISUALIZATION**

[ 5, 3, 1, 4, 2 ]

Pass 1 → 5 bubbles right → [ 3, 1, 4, 2, 5 ] Pass 2 → 4 bubbles right → [ 1, 3, 2, 4, 5 ] Pass 3 → 3 bubbles right → [ 1, 2, 3, 4, 5 ]

**COMPLEXITY**

| Approach       | Time        | Space  | Notes                                       |
|----------------|-------------|--------|---------------------------------------------|
| Without flag   | $O(N^2)$    | $O(1)$ | Always $N^2$ passes regardless              |
| With swap flag | $O(N)$ best | $O(1)$ | Best case: already sorted — 1 pass, 0 swaps |

**STABILITY**

- Stable — equal elements are never swapped, relative order preserved.

**COMMON MISTAKES**

- Forgetting the early-exit swap flag (interviewers specifically ask about this)
- Wrong inner loop bound ( $N-2-i$ ) → accessing out-of-bounds index

### ■ Interview Trigger

Largest bubbles to the end each pass — use a swap flag for  $O(N)$  best case.

## 2.1.3 · Insertion Sort

### WHAT IT'S ASKING

Build the sorted array one element at a time by inserting each new element into its correct place.

### TOPIC / PATTERN

Sorting — Build a Sorted Prefix (Like sorting cards in hand)

### CORE IDEA

Pick each element and insert it into its correct position within the already-sorted left portion by shifting larger elements one step right.

### ALGORITHM

- 1. For  $i = 1$  to  $N-1$ :
- 2.  $key = arr[i]$
- 3.  $j = i - 1$
- 4. While  $j \geq 0$  and  $arr[j] > key$ : shift  $arr[j]$  one step right,  $j--$ .
- 5. Place  $key$  at  $arr[j+1]$ .

### VISUALIZATION

```
[ 5, 3, 1, 4, 2 ]
i=1 key=3 → [ 3, 5, 1, 4, 2 ]
i=2 key=1 → [ 1, 3, 5, 4, 2 ]
i=3 key=4 → [ 1, 3, 4, 5, 2 ]
i=4 key=2 → [ 1, 2, 3, 4, 5 ]
```

### COMPLEXITY

| Approach   | Time     | Space  | Notes                                               |
|------------|----------|--------|-----------------------------------------------------|
| Worst case | $O(N^2)$ | $O(1)$ | Reverse-sorted input — maximum shifts               |
| Best case  | $O(N)$   | $O(1)$ | Already sorted — no shifts needed, just comparisons |

### STABILITY & PRACTICAL NOTES

- Stable — equal elements maintain relative order.
- Best sort for small arrays ( $N < 20$ ) or nearly-sorted data.
- Used internally by Python's Timsort and Java's Arrays.sort for small partitions.

### ■ Interview Trigger

Grow the sorted prefix by inserting each new element in its correct place.

## 2.2.1 · Merge Sort (Most Interview-Critical)

**WHAT IT'S ASKING**

Sort an array by recursively splitting it in half and merging the sorted halves back together.

**TOPIC / PATTERN**

Sorting — Divide and Conquer

**CORE IDEA**

Divide the array into two halves, recursively sort each half, then merge the two sorted halves. The key insight: merging two sorted arrays is  $O(N)$ .

**ALGORITHM**

- 1. Base case: if array has 0 or 1 element → already sorted, return.
- 2. Find  $mid = (low + high) / 2$ .
- 3. Recursively sort the left half  $[low .. mid]$ .
- 4. Recursively sort the right half  $[mid+1 .. high]$ .
- 5. Merge the two sorted halves using two pointers into a temp array.
- 6. Copy the temp array back into the original.

**VISUALIZATION — RECURSION TREE**

```
[ 5, 3, 1, 4, 2 ]
/  \
[ 5, 3, 1 ] [ 4, 2 ]
/ \ / \
[ 5, 3 ] [ 1 ] [ 4 ] [ 2 ]
/  \
[ 5 ] [ 3 ]
Merge up: [3,5] → [1,3,5] → [2,4] → [1,2,3,4,5]
```

**COMPLEXITY**

| Approach   | Time          | Space  | Notes                                                |
|------------|---------------|--------|------------------------------------------------------|
| Merge Sort | $O(N \log N)$ | $O(N)$ | Always $O(N \log N)$ — best, avg, and worst. Stable. |

**WHY  $O(N \log N)$ ?**

- $\log N$  levels in the recursion tree.
- $O(N)$  merge work per level.
- Total:  $\log N \times N = O(N \log N)$ .

**WHY IT MATTERS BEYOND SORTING**

- Directly used in: Count Inversions, Sort Linked List, External Sort.
- Only stable  $O(N \log N)$  sort → preferred when stability matters.

**COMMON MISTAKES**

- Forgetting to copy the temp array back to the original
- Wrong merge pointer logic — not handling remaining elements after one pointer exhausts

**■ Interview Trigger**

Divide until trivial, merge while maintaining order —  $O(N \log N)$  guaranteed.

## 2.2.4 · Quick Sort (Most Interview-Critical)

### WHAT IT'S ASKING

Sort an array by picking a pivot, partitioning around it, and recursively sorting each side.

### TOPIC / PATTERN

Sorting — Divide and Conquer (Pivot-Based Partitioning)

### CORE IDEA

Pick a pivot element. Partition the array so all elements smaller than the pivot are on its left and all larger are on its right. The pivot is then in its final sorted position. Recurse on both sides.

### ALGORITHM — LOMUTO PARTITION

- 1. Pick pivot = arr[high] (last element).
- 2. i = low – 1 (boundary of the 'smaller' region).
- 3. For j = low to high–1: if arr[j] ≤ pivot, then i++, swap arr[i] and arr[j].
- 4. Swap arr[i+1] with arr[high] → pivot is now at its correct index.
- 5. Recursively sort [low .. i] and [i+2 .. high].

### VISUALIZATION

```
pivot = 4 → [ 5, 3, 1, 4, 2 ]
After partition: [ 3, 1, 2, 4, 5 ]
                ^
                pivot in final place
Recurse: [ 3, 1, 2 ] and [ 5 ]
```

### PARTITIONING VARIANTS

| Approach   | Time              | Space       | Notes                                             |
|------------|-------------------|-------------|---------------------------------------------------|
| Lomuto     | $O(N \log N)$ avg | $O(\log N)$ | Simple. pivot = last element.                     |
| Hoare      | $O(N \log N)$ avg | $O(\log N)$ | Fewer swaps in practice. Two converging pointers. |
| 3-Way      | $O(N \log N)$ avg | $O(\log N)$ | Best when array has many duplicates (Dutch Flag). |
| Randomized | $O(N \log N)$ exp | $O(\log N)$ | Shuffle or random pivot — eliminates worst case.  |

### QUICK SORT VS MERGE SORT

Quick Sort Merge Sort

|                | Quick Sort        | Merge Sort       |
|----------------|-------------------|------------------|
| Time — Average | $O(N \log N)$     | $O(N \log N)$    |
| Time — Worst   | $O(N^2)$          | $O(N \log N)$    |
| Space          | $O(\log N)$ stack | $O(N)$ auxiliary |
| Stable         | No                | Yes              |

|          |                    |                    |
|----------|--------------------|--------------------|
| Cache    | Better (in-place)  | Worse (aux array)  |
| Use When | Memory constrained | Stability required |

### COMMON MISTAKES

- Not randomizing pivot →  $O(N^2)$  worst case on sorted / reverse-sorted input
- Confusing Lomuto vs Hoare partition index conventions
- Assuming Quick Sort is stable — it is NOT

### ■ Interview Trigger

Partition around pivot — smaller left, larger right — pivot lands in place — recurse.

Master Sorting Comparison Table

DSA Revision Notes | Striver A2Z · Batch 1: Steps 1 & 2 Page 10

| Algorithm      | Best          | Average       | Worst         | Space       | Stable | Key Note                       |
|----------------|---------------|---------------|---------------|-------------|--------|--------------------------------|
| Selection Sort | $O(N^2)$      | $O(N^2)$      | $O(N^2)$      | $O(1)$      | No     | Min-select each pass           |
| Bubble Sort    | $O(N)$        | $O(N^2)$      | $O(N^2)$      | $O(1)$      | Yes    | Swap flag gives $O(N)$ best    |
| Insertion Sort | $O(N)$        | $O(N^2)$      | $O(N^2)$      | $O(1)$      | Yes    | Best for nearly-sorted         |
| Merge Sort     | $O(N \log N)$ | $O(N \log N)$ | $O(N \log N)$ | $O(N)$      | Yes    | Guaranteed — divide & merge    |
| Quick Sort     | $O(N \log N)$ | $O(N \log N)$ | $O(N^2)$      | $O(\log N)$ | No     | Randomize pivot always         |
| Heap Sort      | $O(N \log N)$ | $O(N \log N)$ | $O(N \log N)$ | $O(1)$      | No     | In-place, not cache-friendly   |
| Counting Sort  | $O(N+K)$      | $O(N+K)$      | $O(N+K)$      | $O(K)$      | Yes    | Bounded integer range only     |
| Radix Sort     | $O(NK)$       | $O(NK)$       | $O(NK)$       | $O(N+K)$    | Yes    | Digit-by-digit, non-comparison |

### QUICK DECISION GUIDE

- Stability + guaranteed  $O(N \log N)$  → Merge Sort
- Memory constrained, average case matters → Quick Sort (randomized)
- Small array ( $N < 20$ ) or nearly-sorted → Insertion Sort
- Bounded integer range → Counting Sort / Radix Sort
- In-place +  $O(N \log N)$  worst case guaranteed → Heap Sort

DSA Revision Notes Striver's A2Z DSA Sheet

**ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING**

## 2.2.2 / 2.2.3 · Recursive Bubble Sort & Recursive Insertion Sort

### WHAT IT'S ASKING

Implement Bubble Sort and Insertion Sort using recursion instead of nested loops.

### TOPIC / PATTERN

Sorting — Recursion applied to classic sorts

### CORE IDEA

Same comparison/swap logic as the iterative versions, but the outer loop is replaced by a recursive call that shrinks the unsorted range by one each time.

### ALGORITHM

- 1. Recursive Bubble Sort: recurse on the first  $N-1$  elements, then do one bubble pass placing the largest remaining element at the end.
- 2. Base case:  $N==1 \rightarrow$  already sorted, return.
- 3. Recursive Insertion Sort: recursively sort the first  $N-1$  elements, then insert the  $N$ th element into its correct place by shifting.
- 4. Both still run in  $O(N^2)$  — recursion doesn't change complexity, only the control flow.

### COMPLEXITY

- Time:  $O(N^2)$  for both
- Space:  $O(N)$  recursion stack (extra vs. the iterative versions)

#### ■ Interview Trigger

Same swap/insert logic as iterative — just replace the outer for-loop with a recursive call on  $N-1$ .

# DSA Revision Notes

## Striver's A2Z DSA Sheet

### Batch 2 · Step 3: Arrays (Easy → Medium → Hard)

Problems 26 problems across Easy, Medium, and Hard

Patterns Two Pointers · Prefix Sum · Hashing · Kadane's · Dutch Flag · Greedy

Format Pattern · Intuition · Algorithm · Complexity · Traps · Trigger

Arrays are the most tested topic in coding interviews. Master every pattern here.

#### TABLE OF CONTENTS

##### EASY PROBLEMS (3.1)

- 3.1.1 Largest Element in an Array
- 3.1.2 Second Largest Element
- 3.1.3 Check if Array is Sorted
- 3.1.4 Remove Duplicates from Sorted Array
- 3.1.5 Left Rotate Array by One
- 3.1.6 Left Rotate Array by D Places
- 3.1.7 Move Zeros to End
- 3.1.8 Linear Search
- 3.1.9 Union and Intersection of Two Sorted Arrays
- 3.1.10 Find Missing Number
- 3.1.11 Maximum Consecutive Ones
- 3.1.12 Find the Single Number (XOR)
- 3.1.13 Longest Subarray with Sum K (Positives)
- 3.1.14 Longest Subarray with Sum K (Pos + Neg)

##### MEDIUM PROBLEMS (3.2)

- 3.2.1 Two Sum
- 3.2.2 Sort 0s 1s 2s — Dutch National Flag
- 3.2.3 Majority Element ( $> N/2$ ) — Boyer-Moore Voting
- 3.2.4 Kadane's Algorithm — Maximum Subarray Sum
- 3.2.5 Print Subarray with Max Sum
- 3.2.6 Stock Buy and Sell
- 3.2.7 Rearrange Alternating Positive Negative
- 3.2.8 Next Permutation
- 3.2.9 Leaders in an Array
- 3.2.10 Longest Consecutive Sequence
- 3.2.11 Set Matrix Zeros
- 3.2.12 Rotate Matrix 90 Degrees
- 3.2.13 Spiral Order Matrix
- 3.2.14 Count Subarrays with Given Sum

##### HARD PROBLEMS (3.3)

- 3.3.1 Pascal's Triangle
- 3.3.2 Majority Element ( $> N/3$ ) — Extended Voting
- 3.3.3 3-Sum Problem

- 3.3.4 4-Sum Problem
- 3.3.5 Largest Subarray with 0 Sum
- 3.3.6 Count Subarrays with XOR = K
- 3.3.7 Merge Overlapping Intervals
- 3.3.8 Merge Two Sorted Arrays Without Extra Space
- 3.3.9 Find Repeating and Missing Number
- 3.3.10 Count Inversions
- 3.3.11 Reverse Pairs
- 3.3.12 Maximum Product Subarray

## STEP 3 — Arrays

### 3.1 EASY PROBLEMS

#### 3.1.1 · Largest Element in an Array

##### WHAT IT'S ASKING

Find the largest element in an array.

##### TOPIC / PATTERN

Arrays — Linear Scan

##### CORE IDEA

Track the maximum value seen so far as you scan left to right in a single pass.

##### ALGORITHM

- 1. Initialize  $\text{max} = \text{arr}[0]$ .
- 2. For each element from index 1 to  $N-1$ : if  $\text{arr}[i] > \text{max}$ , update  $\text{max} = \text{arr}[i]$ .
- 3. Return  $\text{max}$ .

##### COMPLEXITY

| Approach    | Time          | Space  | Notes                           |
|-------------|---------------|--------|---------------------------------|
| Sorting     | $O(N \log N)$ | $O(1)$ | Sort and return last — overkill |
| Linear Scan | $O(N)$        | $O(1)$ | Optimal — single pass           |

##### COMMON MISTAKES

- Initializing  $\text{max} = 0$  — fails for all-negative arrays. Always init to  $\text{arr}[0]$ .

##### ■ Interview Trigger

Single pass, track running max —  $O(N)$ .

#### 3.1.2 · Second Largest Element (Without Sorting)

##### WHAT IT'S ASKING

Find the second-largest element in an array without sorting it.

**TOPIC / PATTERN**

Arrays — Two-Variable Linear Scan

**CORE IDEA**

Maintain two variables: largest and second\_largest. Update both in one pass.

**ALGORITHM**

- 1. Init largest =  $-\infty$ , second =  $-\infty$ .
- 2. For each element: if  $\text{arr}[i] > \text{largest} \rightarrow \text{second} = \text{largest}, \text{largest} = \text{arr}[i]$ .
- 3. Else if  $\text{arr}[i] > \text{second}$  AND  $\text{arr}[i] \neq \text{largest} \rightarrow \text{second} = \text{arr}[i]$ .
- 4. Return second ( $-\infty$  means no second largest exists).

**EDGE CASE**

- All elements equal  $\rightarrow$  second largest doesn't exist. Check for  $-\infty$  in result.

**COMPLEXITY**

| Approach    | Time   | Space  | Notes                 |
|-------------|--------|--------|-----------------------|
| Linear Scan | $O(N)$ | $O(1)$ | Single pass — optimal |

**■ Interview Trigger**

Track two maxima simultaneously — update both in one scan.

### 3.1.3 · Check if Array is Sorted

**WHAT IT'S ASKING**

Check whether a given array is sorted in non-decreasing order.

**TOPIC / PATTERN**

Arrays — Linear Scan

**ALGORITHM**

- 1. For  $i = 0$  to  $N-2$ : if  $\text{arr}[i] > \text{arr}[i+1] \rightarrow$  return false.
- 2. Return true.

**COMPLEXITY**

| Approach    | Time   | Space  | Notes                   |
|-------------|--------|--------|-------------------------|
| Linear Scan | $O(N)$ | $O(1)$ | Stop at first violation |

**COMMON MISTAKES**

- Checking  $\text{arr}[i] \geq \text{arr}[i+1]$  — equal elements are fine in non-strictly sorted.

**■ Interview Trigger**

Check adjacent pairs — first violation means unsorted.

### 3.1.4 · Remove Duplicates from Sorted Array

#### WHAT IT'S ASKING

Remove duplicates from a sorted array in place and return the new length.

#### TOPIC / PATTERN

Arrays — Two Pointers (slow/fast)

#### CORE IDEA

Use a slow pointer  $i$  to track the boundary of unique elements and a fast pointer  $j$  to scan forward.

#### ALGORITHM

- 1.  $i = 0$  (last unique element position).
- 2. For  $j = 1$  to  $N-1$ : if  $arr[j] \neq arr[i]$ , increment  $i$ , copy  $arr[j]$  to  $arr[i]$ .
- 3. Return  $i + 1$  (count of unique elements).

#### VISUALIZATION

```
[ 1, 1, 2, 3, 3, 4 ]
i → slow pointer (unique boundary)
j → fast scanner
arr[j] != arr[i] → i++, arr[i] = arr[j]
Result: [ 1, 2, 3, 4, _, _ ] return 4
```

#### COMPLEXITY

| Approach     | Time   | Space  | Notes                               |
|--------------|--------|--------|-------------------------------------|
| Two Pointers | $O(N)$ | $O(1)$ | In-place — optimal for sorted array |

#### ■ Interview Trigger

Slow pointer marks unique boundary — fast pointer discovers new values.

### 3.1.5 / 3.1.6 · Left Rotate Array by 1 and by D Places

#### WHAT IT'S ASKING

Shift every element of an array left by one position, and more generally by  $D$  positions.

#### TOPIC / PATTERN

Arrays — Reversal Algorithm

#### ROTATE BY 1

- Save  $arr[0]$ , shift all elements left by one, place saved at  $arr[N-1]$ .
- Time:  $O(N)$  · Space:  $O(1)$

#### ROTATE BY D — REVERSAL ALGORITHM (OPTIMAL)

- 1. Reverse the first  $D$  elements.
- 2. Reverse the remaining  $N-D$  elements.
- 3. Reverse the entire array.

**VISUALIZATION (D=2, N=5)**

```
[ 1, 2, 3, 4, 5 ] D=2
Step 1: Rev [0..1] → [ 2, 1, 3, 4, 5 ]
Step 2: Rev [2..4] → [ 2, 1, 5, 4, 3 ]
Step 3: Rev all → [ 3, 4, 5, 1, 2 ] ✓
```

**COMPLEXITY**

| Approach    | Time | Space | Notes                           |
|-------------|------|-------|---------------------------------|
| Extra Array | O(N) | O(N)  | Simple but uses extra space     |
| Reversal    | O(N) | O(1)  | Optimal — three reversal passes |

**COMMON MISTAKES**

- Not handling  $D > N$  — use  $D = D \% N$  first.

**■ Interview Trigger**

Reverse first D, reverse rest, reverse all — rotation done in-place.

**3.1.7 · Move Zeros to End****WHAT IT'S ASKING**

Move all zeros in an array to the end while keeping the relative order of non-zero elements.

**TOPIC / PATTERN**

Arrays — Two Pointers

**CORE IDEA**

Maintain a pointer  $j$  for the next non-zero position. Scan and swap non-zeros to  $j$ .

**ALGORITHM**

- 1.  $j = 0$  (next position for non-zero element).
- 2. For  $i = 0$  to  $N-1$ : if  $arr[i] \neq 0$ , swap  $arr[i]$  and  $arr[j]$ ,  $j++$ .

**VISUALIZATION**

```
[ 0, 1, 0, 3, 2 ]
j=0
i=1: arr[1]=1 ≠ 0 → swap(arr[1], arr[0]) → [ 1, 0, 0, 3, 2 ], j=1
i=3: arr[3]=3 ≠ 0 → swap(arr[3], arr[1]) → [ 1, 3, 0, 0, 2 ], j=2
i=4: arr[4]=2 ≠ 0 → swap(arr[4], arr[2]) → [ 1, 3, 2, 0, 0 ] ✓
```

**COMPLEXITY**

| Approach     | Time | Space | Notes                                         |
|--------------|------|-------|-----------------------------------------------|
| Two Pointers | O(N) | O(1)  | Stable relative order of non-zeros maintained |

### ■ Interview Trigger

Non-zero pointer tracks the front — push all non-zeros forward, zeros fall back.

## 3.1.10 · Find Missing Number in Array [0..N]

### WHAT IT'S ASKING

Find the one number missing from an array containing N distinct numbers from 0 to N.

### TOPIC / PATTERN

Arrays — Math (Expected Sum) or XOR

### APPROACH 1 — SUM FORMULA

- Expected sum of 0..N =  $N*(N+1)/2$ .
- Missing = Expected Sum – Actual Sum.

### APPROACH 2 — XOR (NO OVERFLOW RISK)

- XOR all array elements with XOR of 0..N.
- Every present number cancels (a XOR a = 0). Missing number remains.

### COMPLEXITY

| Approach    | Time   | Space  | Notes                              |
|-------------|--------|--------|------------------------------------|
| Sum Formula | $O(N)$ | $O(1)$ | Risk: integer overflow for large N |
| XOR         | $O(N)$ | $O(1)$ | Preferred — no overflow possible   |

### COMMON MISTAKES

- Sum approach: use long/int64 to avoid overflow.

### ■ Interview Trigger

XOR everything 0..N with all array elements — missing number survives.

## 3.1.11 · Maximum Consecutive Ones

### WHAT IT'S ASKING

Count the maximum number of consecutive 1s in a binary array.

### TOPIC / PATTERN

Arrays — Linear Scan with Counter

### ALGORITHM

1. count = 0, maxCount = 0.
2. For each element: if 1 → count++, else count = 0.
3. maxCount = max(maxCount, count) every step.

### COMPLEXITY

| Approach    | Time   | Space  | Notes       |
|-------------|--------|--------|-------------|
| Linear Scan | $O(N)$ | $O(1)$ | Single pass |

#### ■ Interview Trigger

Reset counter on 0, track running max — one pass.

### 3.1.12 · Find the Single Number (Others Appear Twice)

#### WHAT IT'S ASKING

Find the single number that appears once while every other number appears exactly twice.

#### TOPIC / PATTERN

Arrays / Bit Manipulation — XOR

#### CORE IDEA

XOR of a number with itself = 0. XOR of a number with 0 = itself. XOR all elements — pairs cancel, the single number remains.

#### ALGORITHM

– result = 0. For each element: result ^= arr[i]. Return result.

#### VISUALIZATION

```
[ 4, 1, 2, 1, 2 ]
0 ^ 4 ^ 1 ^ 2 ^ 1 ^ 2
= 0 ^ 4 ^ (1^1) ^ (2^2)
= 0 ^ 4 ^ 0 ^ 0 = 4 ✓
```

#### COMPLEXITY

| Approach | Time   | Space  | Notes                               |
|----------|--------|--------|-------------------------------------|
| HashMap  | $O(N)$ | $O(N)$ | Count frequencies — straightforward |
| XOR      | $O(N)$ | $O(1)$ | Optimal — no extra space            |

#### ■ Interview Trigger

XOR all elements — pairs vanish, the lone number survives.

### 3.1.13 / 3.1.14 · Longest Subarray with Sum K

#### WHAT IT'S ASKING

Find the length of the longest subarray whose elements sum to exactly K.

#### TOPIC / PATTERN

Arrays — Sliding Window (positives only) or Prefix Sum + HashMap (pos+neg)

#### CASE 1 — ALL POSITIVE ELEMENTS: SLIDING WINDOW

- Expand window by moving right. If  $\text{sum} > K$ , shrink from left.
- Track max window size whenever  $\text{sum} == K$ .

**CASE 2 — POSITIVES + NEGATIVES: PREFIX SUM + HASHMAP**

- Maintain running prefix sum. Store first occurrence index of each prefix sum.
- If  $(\text{prefixSum} - K)$  exists in map → subarray from that index+1 to current has sum  $K$ .
- Update max length accordingly.

**VISUALIZATION — PREFIX SUM APPROACH**

```
arr = [ 1, -1, 5, -2, 3 ], K = 3
prefix: 0, 1, 0, 5, 3, 6
At i=4: prefix=3, 3-3=0 exists at idx 0 → length = 4-0 = 4 ✓
```

**COMPLEXITY**

| Approach       | Time     | Space  | Notes                                   |
|----------------|----------|--------|-----------------------------------------|
| Brute          | $O(N^2)$ | $O(1)$ | Check all subarrays — TLE for large $N$ |
| Sliding Window | $O(N)$   | $O(1)$ | Only valid for non-negative arrays      |
| Prefix+HashMap | $O(N)$   | $O(N)$ | Works for all cases including negatives |

**COMMON MISTAKES**

- Using sliding window on arrays with negative numbers — window won't shrink correctly.
- Not initializing map with  $\{0: -1\}$  — misses subarrays starting from index 0.

**■ Interview Trigger**

Positives only → sliding window. Negatives present → prefix sum + hashmap.

**3.2 MEDIUM PROBLEMS****3.2.1 · Two Sum****WHAT IT'S ASKING**

Given an array and a target, find the indices of the two numbers that add up to the target.

**TOPIC / PATTERN**

Arrays — HashMap or Two Pointers

**CORE IDEA**

For each element  $x$ , check if  $(\text{target} - x)$  has been seen before. If yes, we have our pair.

**ALGORITHM — HASHMAP**

1. For each index  $i$ :  $\text{complement} = \text{target} - \text{arr}[i]$ .
2. If complement exists in map → return  $[\text{map}[\text{complement}], i]$ .
3. Otherwise store  $\text{arr}[i] \rightarrow i$  in the map.

**ALGORITHM — TWO POINTERS (SORTED ARRAY ONLY)**

- 1. Sort the array (or use original indices carefully).
- 2.  $L = 0$ ,  $R = N-1$ . While  $L < R$ :
- 3.  $sum = arr[L] + arr[R]$ . If  $sum == target \rightarrow$  found. If  $sum < target \rightarrow L++$ . Else  $R--$ .

**COMPLEXITY**

| Approach     | Time          | Space  | Notes                                         |
|--------------|---------------|--------|-----------------------------------------------|
| Brute        | $O(N^2)$      | $O(1)$ | Check all pairs                               |
| HashMap      | $O(N)$        | $O(N)$ | One pass — handles unsorted + returns indices |
| Two Pointers | $O(N \log N)$ | $O(1)$ | Needs sorted array — lose original indices    |

**INTERVIEW NOTE**

- If the problem asks for indices  $\rightarrow$  use HashMap (two pointers loses original indices after sort).
- If it asks for existence/values and array is sorted  $\rightarrow$  use two pointers.

*Related: [LeetCode 167 Two Sum II](#), [LeetCode 15 3Sum](#), [LeetCode 18 4Sum](#)*

**■ Interview Trigger**

Store complement while scanning —  $O(N)$  lookup via hashmap.

**3.2.2 · Sort Array of 0s, 1s, 2s — Dutch National Flag****WHAT IT'S ASKING**

Sort an array containing only 0s, 1s, and 2s in a single pass.

**TOPIC / PATTERN**

Arrays — Dutch National Flag (3-way partition with 3 pointers)

**CORE IDEA**

Maintain three regions:  $[0..low-1] = 0s$ ,  $[low..mid-1] = 1s$ ,  $[high+1..N-1] = 2s$ . mid is the explorer.

**ALGORITHM**

- 1.  $low = 0$ ,  $mid = 0$ ,  $high = N-1$ .
- 2. While  $mid \leq high$ :
- 3.  $arr[mid] == 0 \rightarrow swap(mid, low)$ ,  $low++$ ,  $mid++$ .
- 4.  $arr[mid] == 1 \rightarrow mid++$  (already in correct region).
- 5.  $arr[mid] == 2 \rightarrow swap(mid, high)$ ,  $high--$  (DO NOT  $mid++$  — check new  $arr[mid]$ ).

**VISUALIZATION**

```
[ 2, 0, 1, 2, 0, 1 ]
L H
M
```

$arr[M]=2 \rightarrow swap(M,H)$ ,  $H-- \rightarrow [1, 0, 1, 2, 0, 2]$   $arr[M]=1 \rightarrow M++ \rightarrow [1, 0, 1, 2, 0, 2]$  ... continues until  $[0, 0, 1, 1, 2, 2]$  ✓

**COMPLEXITY**

| Approach            | Time   | Space  | Notes                                       |
|---------------------|--------|--------|---------------------------------------------|
| Count & Fill        | $O(N)$ | $O(1)$ | Two passes — count 0s/1s/2s, then overwrite |
| Dutch National Flag | $O(N)$ | $O(1)$ | One pass — optimal                          |

**COMMON MISTAKES**

- Incrementing mid after swapping with high — the swapped-in element is unchecked.

*Related: [LeetCode 75 Sort Colors](#)*

**■ Interview Trigger**

Three pointers partition in one pass — never move mid after swapping with high.

**3.2.3 · Majority Element (> N/2 times) — Boyer-Moore Voting****WHAT IT'S ASKING**

Find the element that occurs more than  $N/2$  times in the array (guaranteed to exist).

**TOPIC / PATTERN**

Arrays — Boyer-Moore Voting Algorithm

**CORE IDEA**

The majority element (appearing  $> N/2$  times) can never be fully cancelled out. Use a candidate and a vote count — votes cancel between different elements.

**ALGORITHM**

1. candidate = arr[0], votes = 1.
2. For  $i = 1$  to  $N-1$ : if votes == 0 → candidate = arr[i], votes = 1.
3. Else if arr[i] == candidate → votes++. Else votes--.
4. Verify candidate appears  $> N/2$  times (if problem guarantees majority exists, skip verification).

**VISUALIZATION**

```
[ 2, 2, 1, 1, 2 ]
cand=2 votes=1
arr[1]=2 → votes=2
arr[2]=1 → votes=1
arr[3]=1 → votes=0
arr[4]=2 → cand=2, votes=1 → candidate = 2 ✓
```

**COMPLEXITY**

| Approach | Time     | Space  | Notes                         |
|----------|----------|--------|-------------------------------|
| Brute    | $O(N^2)$ | $O(1)$ | Count each element explicitly |
| HashMap  | $O(N)$   | $O(N)$ | Store frequencies             |

|             |               |        |                                       |
|-------------|---------------|--------|---------------------------------------|
| Sorting     | $O(N \log N)$ | $O(1)$ | $N/2$ index always holds majority     |
| Boyer-Moore | $O(N)$        | $O(1)$ | Optimal — single pass, no extra space |

**COMMON MISTAKES**

- Not verifying the candidate — if majority not guaranteed to exist, must verify.

*Related: [LeetCode 169 Majority Element](#), [LeetCode 229 Majority Element II](#)*

**■ Interview Trigger**

Majority element can never be voted out — last standing candidate wins.

**3.2.4 / 3.2.5 · Kadane's Algorithm — Maximum Subarray Sum****WHAT IT'S ASKING**

Find the contiguous subarray with the largest sum, and print that subarray.

**TOPIC / PATTERN**

Arrays — Kadane's Algorithm (Dynamic Programming / Greedy)

**CORE IDEA**

At each index, the maximum subarray ending here is either: (a) just the current element (start fresh), or (b) extend the previous subarray. Carry the sum forward only if it's positive.

**ALGORITHM**

- 1.  $\text{maxSum} = \text{arr}[0]$ ,  $\text{currentSum} = \text{arr}[0]$ .
- 2. For  $i = 1$  to  $N-1$ :
- 3.  $\text{currentSum} = \max(\text{arr}[i], \text{currentSum} + \text{arr}[i])$ .
- 4.  $\text{maxSum} = \max(\text{maxSum}, \text{currentSum})$ .
- 5. Return  $\text{maxSum}$ .

**EXTENDED — PRINT THE SUBARRAY**

- Track start and end indices. When  $\text{currentSum} < \text{arr}[i]$ , reset  $\text{start} = i$ .
- When  $\text{maxSum}$  updates, record  $\text{end} = i$  and save start.

**VISUALIZATION**

```
[ -2, 1, -3, 4, -1, 2, 1, -5, 4 ]
cur: -2 1 -2 4 3 5 6 1 5
max: -2 1 1 4 4 5 6 6 6 → maxSum = 6
Subarray: [ 4, -1, 2, 1 ] ✓
```

**KEY INSIGHT**

- If  $\text{currentSum}$  goes negative, adding it to a future element only hurts — reset to 0.

**COMPLEXITY**

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|          |          |        |                                              |
|----------|----------|--------|----------------------------------------------|
| Brute    | $O(N^3)$ | $O(1)$ | All subarrays + sum each                     |
| Better   | $O(N^2)$ | $O(1)$ | Prefix sum to compute subarray sum in $O(1)$ |
| Kadane's | $O(N)$   | $O(1)$ | Optimal — single pass                        |

**COMMON MISTAKES**

- Initializing `maxSum = 0` — fails for all-negative arrays. Use `arr[0]` or  $-\infty$ .

*Related: [LeetCode 53 Maximum Subarray](#), [LeetCode 152 Maximum Product Subarray](#)*

**■ Interview Trigger**

A negative running sum only hurts — reset to 0 and start fresh.

**3.2.6 · Stock Buy and Sell (Single Transaction)****WHAT IT'S ASKING**

Given prices for  $N$  days, find the maximum profit from buying and selling the stock once.

**TOPIC / PATTERN**

Arrays — Greedy / Track Running Minimum

**CORE IDEA**

For each day, compute profit if we sell today (`price[i] - min_price_so_far`). Track the running minimum buy price and the maximum profit seen.

**ALGORITHM**

- 1. `minPrice = prices[0]`, `maxProfit = 0`.
- 2. For  $i = 1$  to  $N-1$ :
- 3. `profit = prices[i] - minPrice`.
- 4. `maxProfit = max(maxProfit, profit)`.
- 5. `minPrice = min(minPrice, prices[i])`.

**COMPLEXITY**

| Approach | Time     | Space  | Notes                           |
|----------|----------|--------|---------------------------------|
| Brute    | $O(N^2)$ | $O(1)$ | Try all buy-sell pairs          |
| Optimal  | $O(N)$   | $O(1)$ | Track running min — single pass |

**COMMON MISTAKES**

- Selling before buying — always update `minPrice` AFTER computing profit at  $i$ .

*Related: [LeetCode 121](#), [122 \(multiple\)](#), [123 \(at most 2\)](#), [188 \(at most K\)](#) — all DP variants*

**■ Interview Trigger**

Track the cheapest buy price so far — profit at each day = `price - min_so_far`.

### 3.2.8 · Next Permutation

#### WHAT IT'S ASKING

Rearrange an array of numbers into the lexicographically next greater permutation.

#### TOPIC / PATTERN

Arrays — 3-Step Algorithm (Find breakpoint, find swap, reverse)

#### CORE IDEA

Find the longest decreasing suffix — just before it is the 'breakpoint'. Swap breakpoint with the next greater element in the suffix, then reverse the suffix.

#### ALGORITHM

- 1. Find the rightmost index  $i$  where  $arr[i] < arr[i+1]$  — this is the breakpoint.
- 2. If no such index exists → array is the last permutation → reverse entire array.
- 3. Find the rightmost  $j > i$  where  $arr[j] > arr[i]$ . Swap  $arr[i]$  and  $arr[j]$ .
- 4. Reverse the suffix  $arr[i+1 .. N-1]$ .

#### VISUALIZATION

```
[ 1, 3, 5, 4, 2 ]
Step 1: breakpoint i=1 (arr[1]=3 < arr[2]=5)
Step 3: rightmost j where arr[j] > 3 → j=3 (arr[3]=4)
swap → [ 1, 4, 5, 3, 2 ]
Step 4: reverse suffix [2..4] → [ 1, 4, 2, 3, 5 ] ✓
```

#### COMPLEXITY

| Approach | Time   | Space  | Notes              |
|----------|--------|--------|--------------------|
| 3-Step   | $O(N)$ | $O(1)$ | In-place — optimal |

#### COMMON MISTAKES

– Reversing instead of sorting the suffix — since it's already in descending order, reversing gives ascending.

*Related: [LeetCode 31 Next Permutation](#), [LeetCode 46/47 Permutations](#)*

#### ■ Interview Trigger

Find dip → swap with next-greater → reverse suffix → next permutation found.

### 3.2.10 · Longest Consecutive Sequence

#### WHAT IT'S ASKING

Find the length of the longest run of consecutive integers present in an unsorted array.

#### TOPIC / PATTERN

Arrays — HashSet

#### CORE IDEA

For each number that is the START of a sequence (num-1 not in set), count how long the streak extends. Only start counting from sequence starts.

#### ALGORITHM

- 1. Insert all elements into a HashSet.
- 2. For each num: if (num-1) not in set → num is a sequence start.
- 3. Count streak: while (num + streak) in set → streak++.
- 4. Update maxStreak.

#### COMPLEXITY

| Approach | Time          | Space  | Notes                                          |
|----------|---------------|--------|------------------------------------------------|
| Brute    | $O(N^3)$      | $O(1)$ | Check all pairs                                |
| Sorting  | $O(N \log N)$ | $O(1)$ | Sort and count consecutive runs                |
| HashSet  | $O(N)$        | $O(N)$ | Optimal — each element processed at most twice |

#### COMMON MISTAKES

- Not checking if num-1 exists before counting — leads to  $O(N^2)$  from redundant counts.

*Related: [LeetCode 128 Longest Consecutive Sequence](#)*

#### ■ Interview Trigger

Only count from sequence starts (num-1 absent) — each number touched at most twice.

### 3.2.11 · Set Matrix Zeros

#### WHAT IT'S ASKING

Given a matrix, if a cell is 0, set its entire row and column to 0 — in place.

#### TOPIC / PATTERN

Arrays / Matrix — In-Place Marking

#### CORE IDEA

Use the first row and first column as markers to avoid extra space. A separate flag tracks whether the first row/column themselves should be zeroed.

#### ALGORITHM

- 1. Check if first row or first column contains 0 (save flags firstRowZero, firstColZero).
- 2. For all (i, j) with  $i > 0, j > 0$ : if  $\text{matrix}[i][j] == 0$ , mark  $\text{matrix}[i][0] = 0$  and  $\text{matrix}[0][j] = 0$ .
- 3. For  $i > 0, j > 0$ : if  $\text{matrix}[i][0] == 0$  or  $\text{matrix}[0][j] == 0$  → set  $\text{matrix}[i][j] = 0$ .
- 4. Use flags to zero out first row/column if needed.

#### COMPLEXITY

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|              |                 |          |                                        |
|--------------|-----------------|----------|----------------------------------------|
| Extra Space  | $O(M \times N)$ | $O(M+N)$ | Store zero positions separately        |
| Row/Col Sets | $O(M \times N)$ | $O(M+N)$ | Track rows and cols with zeros         |
| In-Place     | $O(M \times N)$ | $O(1)$   | Use first row/col as markers — optimal |

**COMMON MISTAKES**

- Zeroing cells immediately while scanning — corrupts markers. Always do it in two passes.

*Related: [LeetCode 73 Set Matrix Zeroes](#)*

**■ Interview Trigger**

Use first row and column as markers — two-pass in-place  $O(1)$  space.

**3.2.12 · Rotate Matrix 90 Degrees Clockwise****WHAT IT'S ASKING**

Rotate an  $N \times N$  matrix 90 degrees clockwise in place.

**TOPIC / PATTERN**

Matrix — Transpose + Reverse Rows

**CORE IDEA**

90° clockwise rotation = Transpose the matrix, then reverse each row.

**ALGORITHM**

- 1. Transpose: swap  $\text{matrix}[i][j]$  with  $\text{matrix}[j][i]$  for all  $i < j$ .
- 2. Reverse each row: reverse the elements within every row.

**VISUALIZATION**

```
Original: Transposed: Reversed rows:
1 2 3 1 4 7 7 4 1
4 5 6 → 2 5 8 → 8 5 2
7 8 9 3 6 9 9 6 3 ✓
```

**ANTI-CLOCKWISE**

- Transpose first, then reverse each column (or reverse rows first, then transpose).

**COMPLEXITY**

| Approach     | Time     | Space    | Notes                         |
|--------------|----------|----------|-------------------------------|
| Extra Matrix | $O(N^2)$ | $O(N^2)$ | Direct copy to new matrix     |
| In-Place     | $O(N^2)$ | $O(1)$   | Transpose + reverse — optimal |

*Related: [LeetCode 48 Rotate Image](#)*

**■ Interview Trigger**

Clockwise 90° = Transpose then reverse each row. Anti-clockwise = reverse each row then transpose.

### 3.2.13 · Print Matrix in Spiral Order

**WHAT IT'S ASKING**

Print the elements of a matrix by traversing it in a spiral order.

**TOPIC / PATTERN**

Matrix — Boundary Shrinking with Four Pointers

**ALGORITHM**

- 1. Define boundaries: top=0, bottom=M-1, left=0, right=N-1.
- 2. While top ≤ bottom and left ≤ right:
- 3. Print left→right along top row. top++.
- 4. Print top→bottom along right column. right--.
- 5. If top ≤ bottom: print right→left along bottom row. bottom--.
- 6. If left ≤ right: print bottom→top along left column. left++.

**COMPLEXITY**

| Approach      | Time   | Space | Notes                        |
|---------------|--------|-------|------------------------------|
| Four Pointers | O(M×N) | O(1)  | Visit each cell exactly once |

**COMMON MISTAKES**

- Not checking top ≤ bottom / left ≤ right before printing bottom row / left column — causes duplicates in

single-row/column matrices.

*Related: LeetCode 54 Spiral Matrix, LeetCode 59 Spiral Matrix II*

**■ Interview Trigger**

Shrink four boundaries layer by layer — always check bounds before each direction.

### 3.2.14 · Count Subarrays with Given Sum K

**WHAT IT'S ASKING**

Count how many subarrays sum to exactly K (including negative numbers).

**TOPIC / PATTERN**

Arrays — Prefix Sum + HashMap

**CORE IDEA**

A subarray [j+1..i] has sum K when  $\text{prefixSum}[i] - \text{prefixSum}[j] == K$ , i.e.  $\text{prefixSum}[j] == \text{prefixSum}[i] - K$ . Count how many times that value appeared.

**ALGORITHM**

- 1. prefixSum = 0, count = 0. Map = {0: 1} (empty prefix).
- 2. For each element: prefixSum += arr[i].
- 3. count += map.get(prefixSum – K, 0).
- 4. map[prefixSum]++.

**COMPLEXITY**

| Approach       | Time     | Space  | Notes     |
|----------------|----------|--------|-----------|
| Brute          | $O(N^2)$ | $O(1)$ | All pairs |
| Prefix+HashMap | $O(N)$   | $O(N)$ | Optimal   |

**COMMON MISTAKES**

- Forgetting to initialize map with {0: 1} — misses subarrays that start from index 0.
- Querying map BEFORE updating it — lookup first, then insert.

*Related: [LeetCode 560 Subarray Sum Equals K](#)*

**■ Interview Trigger**

Prefix sum magic: count how many times (prefixSum – K) appeared before.

**3.3 HARD PROBLEMS****3.3.1 · Pascal's Triangle****WHAT IT'S ASKING**

Print the first N rows of Pascal's Triangle, and answer element/row queries about it.

**TOPIC / PATTERN**

Arrays / Math — Three Variants

**VARIANT 1 — VALUE AT (ROW, COL)**

- Answer =  $C(\text{row}-1, \text{col}-1) = (\text{row}-1)! / ((\text{col}-1)! \times (\text{row}-\text{col})!)$ .
- Compute iteratively: multiply (row-1)(row-2)...(row-col+1) / (col-1)!

**VARIANT 2 — PRINT ROW R**

- Start with 1. Multiply by  $(R-1-i) / (i+1)$  for  $i = 0$  to  $R-2$ .

**VARIANT 3 — PRINT ALL N ROWS**

- Each row  $i$ : start with [1]. Each element = sum of two elements directly above.

**VISUALIZATION**

```

Row 1: 1
Row 2: 1 1
Row 3: 1 2 1
Row 4: 1 3 3 1
Row 5: 1 4 6 4 1

```

**COMPLEXITY**

| Approach | Time     | Space    | Notes            |
|----------|----------|----------|------------------|
| Optimal  | $O(N^2)$ | $O(N^2)$ | Build row by row |

**■ Interview Trigger**

Pascal's cell =  $C(\text{row}-1, \text{col}-1)$ . Each row element = sum of two above.

**3.3.2 · Majority Element (> N/3 times) — Extended Voting****WHAT IT'S ASKING**

Find all elements that occur more than  $N/3$  times in the array (at most two can exist).

**TOPIC / PATTERN**

Arrays — Extended Boyer-Moore Voting (2 candidates)

**CORE IDEA**

At most 2 elements can appear  $> N/3$  times. Run voting with 2 candidate slots — eliminate triplets of distinct elements.

**ALGORITHM**

- 1. Find candidates: two slots (cand1, votes1), (cand2, votes2).
- 2. For each element: if it matches cand1  $\rightarrow$  votes1++. Else if matches cand2  $\rightarrow$  votes2++.
- 3. Else if votes1==0  $\rightarrow$  cand1=element, votes1=1. Else if votes2==0  $\rightarrow$  cand2=element, votes2=1.
- 4. Else votes1--, votes2--.
- 5. Verify both candidates by counting actual occurrences — keep those with count  $> N/3$ .

**COMPLEXITY**

| Approach      | Time   | Space  | Notes              |
|---------------|--------|--------|--------------------|
| HashMap       | $O(N)$ | $O(N)$ | Count frequencies  |
| Extended Vote | $O(N)$ | $O(1)$ | Optimal — two-pass |

**COMMON MISTAKES**

- Not verifying candidates at the end — voting only finds candidates, not guarantees.

*Related: [LeetCode 229 Majority Element II](#)*

**■ Interview Trigger**

At most 2 majority ( $>N/3$ ) elements exist — use 2 candidate slots and verify.

**3.3.3 / 3.3.4 · 3-Sum and 4-Sum Problems****WHAT IT'S ASKING**

Find all unique triplets/quadruplets in the array that sum to a given target.

**TOPIC / PATTERN**

Arrays — Sort + Two Pointers (avoid duplicates carefully)

**3-SUM ALGORITHM**

- 1. Sort the array.
- 2. Fix element  $\text{arr}[i]$ . Use two pointers  $L=i+1$ ,  $R=N-1$  to find pair summing to  $-\text{arr}[i]$ .
- 3. If  $\text{sum} < \text{target} \rightarrow L++$ . If  $\text{sum} > \text{target} \rightarrow R--$ . If  $\text{sum} == \text{target} \rightarrow \text{record, skip duplicates}$ .
- 4. Skip duplicate values of  $\text{arr}[i]$  to avoid duplicate triplets.

**4-SUM ALGORITHM**

- Add an outer loop fixing  $\text{arr}[j]$ , then run 3-Sum logic inside.  $O(N^3)$  total.

**COMPLEXITY**

| Approach          | Time     | Space  | Notes                   |
|-------------------|----------|--------|-------------------------|
| Brute 3Sum        | $O(N^3)$ | $O(1)$ | All triplets — TLE      |
| Sort+Two Pointers | $O(N^2)$ | $O(1)$ | Optimal for 3Sum        |
| 4Sum              | $O(N^3)$ | $O(1)$ | Two fixed + two pointer |

**COMMON MISTAKES**

- Not skipping duplicates  $\rightarrow$  duplicate triplets/quadruplets in output.
- Integer overflow in 4Sum — use long for the target comparisons.

*Related: [LeetCode 15 3Sum](#), [LeetCode 18 4Sum](#), [LeetCode 16 3Sum Closest](#)*

**■ Interview Trigger**

Sort first, fix outer elements, use two pointers inside — skip duplicates at every level.

**3.3.5 · Largest Subarray with 0 Sum****WHAT IT'S ASKING**

Find the length of the largest subarray whose elements sum to zero.

**TOPIC / PATTERN**

Arrays — Prefix Sum + HashMap (store first occurrence index)

**CORE IDEA**

If the prefix sum at index  $i$  equals the prefix sum at index  $j$  ( $i < j$ ), then the subarray  $(i+1, j)$  has sum 0. Find the largest such span.

**ALGORITHM**

- 1.  $\text{Map} = \{0: -1\}$  (prefix sum 0 at index -1 before the array).
- 2.  $\text{prefixSum} = 0$ . For each  $i$ :
- 3.  $\text{prefixSum} += \text{arr}[i]$ .
- 4. If  $\text{prefixSum}$  in  $\text{map} \rightarrow \text{length} = i - \text{map}[\text{prefixSum}]$ . Update  $\text{maxLen}$ .

- 5. Else  $\text{map}[\text{prefixSum}] = i$  (store FIRST occurrence only).

#### COMPLEXITY

| Approach       | Time     | Space  | Notes         |
|----------------|----------|--------|---------------|
| Brute          | $O(N^2)$ | $O(1)$ | All subarrays |
| Prefix+HashMap | $O(N)$   | $O(N)$ | Optimal       |

#### COMMON MISTAKES

- Overwriting map with later indices — we want the earliest occurrence for maximum length.

#### ■ Interview Trigger

Same prefix sum twice means zero-sum subarray in between — maximize the gap.

### 3.3.6 · Count Subarrays with XOR = K

#### WHAT IT'S ASKING

Count the number of subarrays whose XOR equals K.

#### TOPIC / PATTERN

Arrays — Prefix XOR + HashMap

#### CORE IDEA

Analogous to prefix sum. If  $\text{prefixXOR}[i] \text{ XOR } \text{prefixXOR}[j] == K$ , then subarray  $(j+1, i)$  has XOR = K. Equivalently:  $\text{prefixXOR}[j] == \text{prefixXOR}[i] \text{ XOR } K$ .

#### ALGORITHM

- 1. Map = {0: 1}.  $\text{prefixXOR} = 0$ , count = 0.
- 2. For each element:  $\text{prefixXOR} \oplus= \text{arr}[i]$ .
- 3.  $\text{count} += \text{map.get}(\text{prefixXOR} \oplus K, 0)$ .
- 4.  $\text{map}[\text{prefixXOR}]++$ .

#### COMPLEXITY

| Approach       | Time     | Space  | Notes         |
|----------------|----------|--------|---------------|
| Brute          | $O(N^2)$ | $O(1)$ | All subarrays |
| Prefix XOR Map | $O(N)$   | $O(N)$ | Optimal       |

#### KEY OBSERVATION

- XOR obeys the same prefix identity as sum: if  $A \text{ XOR } B = K$ , then  $A = B \text{ XOR } K$ .

#### ■ Interview Trigger

Replace 'sum' with 'XOR' — prefix XOR map works identically to prefix sum map.

### 3.3.7 · Merge Overlapping Intervals

**WHAT IT'S ASKING**

Given a list of intervals, merge all overlapping intervals into one.

**TOPIC / PATTERN**

Arrays — Sort by Start + Greedy Merge

**CORE IDEA**

Sort intervals by start time. Scan linearly — if the current interval overlaps the last merged interval, extend it. Otherwise start a new merged interval.

**ALGORITHM**

- 1. Sort intervals by start time.
- 2. Add first interval to result.
- 3. For each subsequent interval:
- 4. If  $\text{start} \leq \text{result.last.end}$  → overlap → update  $\text{result.last.end} = \max(\text{result.last.end}, \text{end})$ .
- 5. Else → no overlap → append interval to result.

**VISUALIZATION**

```

Intervals: [(1,3), (2,6), (8,10), (15,18)]
Sorted: [(1,3), (2,6), (8,10), (15,18)]
Merge (1,3)+(2,6) → (1,6)
(1,6) and (8,10) → no overlap → add (8,10)
Result: [(1,6), (8,10), (15,18)] ✓

```

**COMPLEXITY**

| Approach  | Time          | Space  | Notes             |
|-----------|---------------|--------|-------------------|
| Sort+Scan | $O(N \log N)$ | $O(N)$ | Dominated by sort |

**COMMON MISTAKES**

- Forgetting to sort first — algorithm only works on sorted intervals.

*Related: [LeetCode 56 Merge Intervals](#), [LeetCode 57 Insert Interval](#)*

**■ Interview Trigger**

Sort by start → scan and greedily merge overlapping intervals.

**3.3.8 · Merge Two Sorted Arrays Without Extra Space****WHAT IT'S ASKING**

Merge two sorted arrays in place without using extra space.

**TOPIC / PATTERN**

Arrays — Gap Method (Shell Sort variant) or Two Pointer with Swap

**APPROACH 1 — TWO POINTER + SWAP (SIMPLE)**

- $i$  = end of  $\text{arr1}$ ,  $j$  = start of  $\text{arr2}$ . While  $i \geq 0$  and  $j < \text{len}(\text{arr2})$ :
- If  $\text{arr1}[i] > \text{arr2}[j]$  → swap them. Then sort each array.  $O((M+N) \log(M+N))$  effectively.

**APPROACH 2 — GAP METHOD (OPTIMAL O(1) SPACE)**

- gap =  $\text{ceil}((M+N)/2)$ . Compare elements that are 'gap' apart across both arrays and swap if needed.
- Halve gap each iteration. Repeat until gap = 0.

**COMPLEXITY**

| Approach   | Time                 | Space  | Notes                              |
|------------|----------------------|--------|------------------------------------|
| Swap+Sort  | $O((M+N) \log(M+N))$ | $O(1)$ | Simpler to implement               |
| Gap Method | $O((M+N) \log(M+N))$ | $O(1)$ | True $O(1)$ — gap halves each pass |

**COMMON MISTAKES**

- Using extra space — question specifically forbids it.

*Related: LeetCode 88 Merge Sorted Array (allows extra space)*

**■ Interview Trigger**

Gap method: compare elements gap-positions apart, swap if needed, halve gap —  $O(1)$  space.

**3.3.9 · Find Repeating and Missing Number****WHAT IT'S ASKING**

Given an array where one number is duplicated and another is missing, find both.

**TOPIC / PATTERN**

Arrays — Math (Sum + Sum of Squares) or XOR

**CORE IDEA — MATH APPROACH**

- Let repeating = A, missing = B.
- $S1 = \text{sum}(\text{arr}) - \text{expected\_sum} \rightarrow A - B = S1$ .
- $S2 = \text{sum}(\text{arr}^2) - \text{expected\_sum\_squares} \rightarrow A^2 - B^2 = S2 \rightarrow (A+B) = S2/S1$ .
- Solve:  $A = (S1 + S2/S1) / 2$ ,  $B = A - S1$ .

**COMPLEXITY**

| Approach | Time     | Space  | Notes                   |
|----------|----------|--------|-------------------------|
| Brute    | $O(N^2)$ | $O(1)$ | Count each number       |
| HashMap  | $O(N)$   | $O(N)$ | Store frequencies       |
| Math     | $O(N)$   | $O(1)$ | Two equations — optimal |
| XOR      | $O(N)$   | $O(1)$ | Bit-wise — no overflow  |

**COMMON MISTAKES**

- Integer overflow in sum of squares — use long/int64.

### ■ Interview Trigger

Two equations, two unknowns:  $(A-B)$  from sum,  $(A+B)$  from sum of squares.

## 3.3.10 / 3.3.11 · Count Inversions and Reverse Pairs

### WHAT IT'S ASKING

Count how many pairs  $(i, j)$  are inversions, i.e.  $i < j$  but  $arr[i] > arr[j]$  (and the 'reverse pairs' variant with  $arr[i] > 2 \times arr[j]$ ).

### TOPIC / PATTERN

Arrays — Modified Merge Sort

### CORE IDEA

During the merge step of merge sort, when a right-half element is placed before left-half elements, all remaining left-half elements form inversions (or reverse pairs) with it. Count them.

### COUNT INVERSIONS — $(i < j \text{ AND } arr[i] > arr[j])$

– During merge: if  $arr[left] > arr[right]$ , all remaining left-half elements ( $mid-left+1$ ) form inversions with  $arr[right]$ .

### REVERSE PAIRS — $(i < j \text{ AND } arr[i] > 2 \times arr[j])$

- Before merging, use two pointers to count pairs satisfying  $arr[i] > 2 \times arr[j]$  across halves.
- Then perform the regular merge to sort.

### COMPLEXITY

| Approach   | Time          | Space  | Notes                        |
|------------|---------------|--------|------------------------------|
| Brute      | $O(N^2)$      | $O(1)$ | All pairs — TLE              |
| Merge Sort | $O(N \log N)$ | $O(N)$ | Count during merge — optimal |

### COMMON MISTAKES

- For reverse pairs: counting and merging in the same pass gives wrong results — count first, then merge.

*Related: [LeetCode 493 Reverse Pairs](#), [LeetCode 315 Count Smaller After Self](#)*

### ■ Interview Trigger

Embed the count inside merge sort — inversions accumulate during the merge step.

## 3.3.12 · Maximum Product Subarray

### WHAT IT'S ASKING

Find the contiguous subarray with the largest product.

### TOPIC / PATTERN

Arrays — Track Both Max and Min (negatives flip signs)

**CORE IDEA**

A negative number can turn the minimum product into the maximum. Track both the current max and current min product at every step.

**ALGORITHM**

- 1.  $\text{maxProd} = \text{arr}[0]$ ,  $\text{minProd} = \text{arr}[0]$ ,  $\text{result} = \text{arr}[0]$ .
- 2. For  $i = 1$  to  $N-1$ :
- 3.  $\text{candidates} = \{\text{arr}[i], \text{arr}[i] \times \text{maxProd}, \text{arr}[i] \times \text{minProd}\}$ .
- 4.  $\text{maxProd} = \max(\text{candidates})$ ,  $\text{minProd} = \min(\text{candidates})$ .
- 5.  $\text{result} = \max(\text{result}, \text{maxProd})$ .

**WHY TRACK MINIMUM?**

[ 2, 3, -2, 4 ] At -2:  $\text{maxProd}=6$ ,  $\text{minProd}=6$  new  $\text{max} = \max(-2, 6 \times -2, 6 \times -2) = -2$  new  $\text{min} = \min(-2, -12, -12) = -12$  At 4:  $\text{max} = \max(4, -8, -48) = 4 \rightarrow \text{result} = 6$

**COMPLEXITY**

| Approach | Time     | Space  | Notes                  |
|----------|----------|--------|------------------------|
| Brute    | $O(N^2)$ | $O(1)$ | All subarrays          |
| Optimal  | $O(N)$   | $O(1)$ | Track both max and min |

**COMMON MISTAKES**

- Only tracking max — misses the negative×negative opportunity.
- Forgetting 0 resets both max and min to 0 (handled automatically by taking  $\text{arr}[i]$  alone as candidate).

*Related: [LeetCode 152 Maximum Product Subarray](#), [LeetCode 53 Maximum Subarray \(sum\)](#)*

**■ Interview Trigger**

Track both max and min — a negative flips them, unlocking hidden maximums.

DSA Revision Notes Striver's A2Z DSA Sheet

**ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING****3.1.8 · Linear Search****WHAT IT'S ASKING**

Find the index of a target value in an array by scanning it from left to right.

**TOPIC / PATTERN**

Arrays — Brute-force scan

**CORE IDEA**

Scan every element left to right and compare to the target; return the index the moment it matches.

**ALGORITHM**

- 1. For  $i = 0$  to  $N-1$ : if  $\text{arr}[i] == \text{target} \rightarrow \text{return } i$ .
- 2. If loop finishes without a match  $\rightarrow \text{return } -1$ .

**COMPLEXITY**

- Time:  $O(N)$
- Space:  $O(1)$

**■ Interview Trigger**

No ordering assumption needed — only choice when the array isn't sorted.

### 3.1.9 · Union and Intersection of Two Sorted Arrays

**WHAT IT'S ASKING**

Given two sorted arrays, find their union (all distinct elements) and intersection (common elements).

**TOPIC / PATTERN**

Arrays — Two Pointers on sorted input

**CORE IDEA**

Walk both arrays with two pointers; for union take the smaller (skipping duplicates), for intersection only advance both when they're equal.

**ALGORITHM**

1.  $i=0, j=0$ . While  $i < N$  and  $j < M$ : compare  $arr1[i]$  and  $arr2[j]$ .
2. Union: add the smaller to the result (skip if same as last added), advance that pointer; if equal, add once and advance both.
3. Intersection: if equal, add to result and advance both; else advance the pointer at the smaller value.
4. Append any remaining tail for union (not needed for intersection).

**COMPLEXITY**

- Time:  $O(N+M)$
- Space:  $O(N+M)$  for the result

**■ Interview Trigger**

Sorted + two pointers → linear union/intersection without a hash set.

### 3.2.7 · Rearrange Array in Alternating Positive-Negative Order

**WHAT IT'S ASKING**

Rearrange an array with equal positives and negatives so the signs alternate, starting with a positive.

**TOPIC / PATTERN**

Arrays — Two Pointers / Extra Array

**CORE IDEA**

Given equal counts of positives and negatives, place them alternately starting with a positive, preserving each group's relative order.

**ALGORITHM**

- 1. Create a result array of size N. posIdx=0 (even slots), negIdx=1 (odd slots).
- 2. Scan input; positives go to result[posIdx] (posIdx+=2), negatives go to result[negIdx] (negIdx+=2).
- 3. Copy result back (or return it).
- 4. Variant: if counts are unequal, place the leftover elements of the larger group at the end in order.

**COMPLEXITY**

- Time:  $O(N)$
- Space:  $O(N)$

**■ Interview Trigger**

Even indices for positives, odd indices for negatives — one pass, no sorting.

### 3.2.9 · Leaders in an Array

**WHAT IT'S ASKING**

Find every element that is strictly greater than all elements to its right.

**TOPIC / PATTERN**

Arrays — Suffix Scan

**CORE IDEA**

An element is a leader if it's strictly greater than every element to its right. Scan right to left, tracking the max seen so far.

**ALGORITHM**

- 1. maxSoFar = arr[N-1]; it's always a leader.
- 2. For  $i = N-2$  down to 0: if  $arr[i] > \text{maxSoFar}$  → arr[i] is a leader, update maxSoFar.
- 3. Reverse the collected leaders to restore left-to-right order (optional).

**COMPLEXITY**

- Time:  $O(N)$
- Space:  $O(1)$  extra (excluding output)

**■ Interview Trigger**

Scan from the right, keep a running max — anything bigger than max-so-far is a leader.

# DSA Revision Notes

## Striver's A2Z DSA Sheet

### Batch 3 · Step 4: Binary Search (1D · Answers · 2D)

Problems 37 problems — 1D Arrays · Search Space · 2D Matrices

Patterns Classic BS · Lower/Upper Bound · BS on Answer · 2D BS

Format Pattern · Intuition · Algorithm · Complexity · Traps · Trigger

Binary Search is not just about sorted arrays — it's a mindset for eliminating half the search space.

#### TABLE OF CONTENTS

##### 4.1 BS ON 1D ARRAYS

- 4.1.1 Classic Binary Search
- 4.1.2 Implement Lower Bound
- 4.1.3 Implement Upper Bound
- 4.1.4 Search Insert Position
- 4.1.5 Floor and Ceil in Sorted Array
- 4.1.6 First and Last Occurrence of a Number
- 4.1.7 Count Occurrences in Sorted Array
- 4.1.8 Search in Rotated Sorted Array I
- 4.1.9 Search in Rotated Sorted Array II (Duplicates)
- 4.1.10 Find Minimum in Rotated Sorted Array
- 4.1.11 How Many Times Has the Array Been Rotated?
- 4.1.12 Single Element in a Sorted Array
- 4.1.13 Find Peak Element

##### 4.2 BS ON ANSWERS (SEARCH SPACE)

- 4.2.1 Square Root in log N
- 4.2.2 Nth Root of a Number
- 4.2.3 Koko Eating Bananas
- 4.2.4 Minimum Days to Make M Bouquets
- 4.2.5 Find the Smallest Divisor
- 4.2.6 Capacity to Ship Packages within D Days
- 4.2.7 Kth Missing Positive Number
- 4.2.8 Aggressive Cows
- 4.2.9 Book Allocation Problem
- 4.2.10 Split Array — Largest Sum
- 4.2.11 Painter's Partition
- 4.2.12 Minimize Max Distance to Gas Station
- 4.2.13 Median of Two Sorted Arrays
- 4.2.14 Kth Element of Two Sorted Arrays

##### 4.3 BS ON 2D ARRAYS

- 4.3.1 Row with Maximum 1s
- 4.3.2 Search in a 2D Matrix (Staircase)
- 4.3.3 Search in Row and Column Sorted Matrix
- 4.3.4 Find Peak Element in 2D Matrix

## 4.3.5 Matrix Median

**STEP 4 — Binary Search**

The Core Idea: If you can check whether a mid-point value satisfies a condition in  $O(1)$  or  $O(\log N)$ , and the answer space is monotonic (all valid on one side, all invalid on the other), Binary Search applies.

Universal Binary Search Template

low = 0, high = N-1 (or search space boundary)

while low <= high:

mid = low + (high - low) // 2 # avoids overflow

if check(mid): # condition met

ans = mid; high = mid - 1 # or low = mid + 1

else:

low = mid + 1 # or high = mid - 1

return ans

– Always use mid = low + (high - low) // 2 to avoid integer overflow.

– Loop condition low <= high for searching a value; low < high for converging to a boundary.

– The decision: what does it mean when check(mid) is True? Move left (high=mid-1) or right (low=mid+1)?

## 4.1 Binary Search on 1D Arrays

**4.1.1 · Classic Binary Search****WHAT IT'S ASKING**

Find a target value's index in a sorted array using binary search.

**TOPIC / PATTERN**

Binary Search — Find exact value in sorted array

**ALGORITHM**

- 1. low=0, high=N-1.
- 2. While low ≤ high: mid = low + (high-low)//2.
- 3. If arr[mid]==target → return mid.
- 4. If arr[mid] < target → low = mid+1 (search right).
- 5. Else → high = mid-1 (search left).
- 6. Return -1 if not found.

**VISUALIZATION**

```
arr = [ 1, 3, 5, 7, 9, 11 ], target = 7
L=0 H=5 mid=2 arr[2]=5 < 7 → L=3
L=3 H=5 mid=4 arr[4]=9 > 7 → H=3
L=3 H=3 mid=3 arr[3]=7 == 7 → return 3 ✓
```

**COMPLEXITY**

| Approach      | Time        | Space  | Notes                        |
|---------------|-------------|--------|------------------------------|
| Binary Search | $O(\log N)$ | $O(1)$ | Halve search space each step |

**COMMON MISTAKES**

- mid = (low+high)//2 can overflow in languages with fixed int size — use low+(high-low)//2.

### ■ Interview Trigger

Sorted +  $O(\log N)$  search  $\rightarrow$  Binary Search. Halve the space every step.

## 4.1.2 / 4.1.3 · Lower Bound and Upper Bound

### WHAT IT'S ASKING

Find the first index where a value could be inserted to keep the array sorted (lower bound), and the first index strictly greater than the value (upper bound).

### TOPIC / PATTERN

Binary Search — Boundary Finding

### DEFINITIONS

- Lower Bound: first index where  $\text{arr}[i] \geq \text{target}$  (first position target could be inserted to keep sorted order).
- Upper Bound: first index where  $\text{arr}[i] > \text{target}$ .

### LOWER BOUND ALGORITHM

1.  $\text{low}=0$ ,  $\text{high}=N$ ,  $\text{ans}=N$ .
2. While  $\text{low} \leq \text{high}$ :  $\text{mid} = \text{low} + (\text{high} - \text{low}) // 2$ .
3. If  $\text{arr}[\text{mid}] \geq \text{target} \rightarrow \text{ans} = \text{mid}$ ,  $\text{high} = \text{mid} - 1$  (go left to find earlier occurrence).
4. Else  $\rightarrow \text{low} = \text{mid} + 1$ .

### UPPER BOUND ALGORITHM

- Same logic but condition is  $\text{arr}[\text{mid}] > \text{target}$ .

### VISUALIZATION

```
arr = [ 1, 3, 3, 5, 7 ], target = 3
Lower Bound  $\rightarrow$  index 1 (first index where  $\text{arr}[i] \geq 3$ )
Upper Bound  $\rightarrow$  index 3 (first index where  $\text{arr}[i] > 3$ )
Count of 3s = Upper - Lower = 3 - 1 = 2
```

### COMPLEXITY

| Approach    | Time        | Space  | Notes                     |
|-------------|-------------|--------|---------------------------|
| BS Boundary | $O(\log N)$ | $O(1)$ | Lean toward left boundary |

### COMMON MISTAKES

- Returning low vs ans — always maintain a separate ans variable.
- High should be N (not N-1) since answer can be past the last element.

### ■ Interview Trigger

Lower bound leans left ( $\geq$ ). Upper bound leans left ( $>$ ). Difference = count.

## 4.1.4 · Search Insert Position

**WHAT IT'S ASKING**

Find the index at which a target should be inserted into a sorted array to keep it sorted.

**TOPIC / PATTERN**

Binary Search — Lower Bound (first position  $\geq$  target)

**CORE IDEA**

This is exactly the Lower Bound problem. Find the first index where  $\text{arr}[i] \geq \text{target}$ . If target exists, returns its index. If not, returns the insertion point.

**COMPLEXITY**

| Approach    | Time        | Space  | Notes                              |
|-------------|-------------|--------|------------------------------------|
| Lower Bound | $O(\log N)$ | $O(1)$ | Same as lower bound implementation |

*Related: LeetCode 35 Search Insert Position*

**■ Interview Trigger**

Search Insert Position = Lower Bound. Return first index where  $\text{arr}[i] \geq \text{target}$ .

**4.1.5 · Floor and Ceil in Sorted Array****WHAT IT'S ASKING**

Find the floor (largest element  $\leq$  target) and ceil (smallest element  $\geq$  target) of a number in a sorted array.

**DEFINITIONS**

- Floor(x): largest element in array  $\leq x$ .
- Ceil(x): smallest element in array  $\geq x$ .

**ALGORITHM**

- Ceil(x) = Lower Bound of x.
- Floor(x): binary search, track last element seen that is  $\leq x$ .

**VISUALIZATION**

```
arr = [ 1, 2, 4, 6, 10 ], x = 5
Floor(5) = 4 (largest  $\leq$  5)
Ceil(5) = 6 (smallest  $\geq$  5)
```

**COMPLEXITY**

| Approach      | Time        | Space  | Notes                 |
|---------------|-------------|--------|-----------------------|
| Binary Search | $O(\log N)$ | $O(1)$ | Two separate BS calls |

**■ Interview Trigger**

Floor = last element  $\leq x$ . Ceil = first element  $\geq x$  = Lower Bound.

## 4.1.6 / 4.1.7 · First/Last Occurrence and Count of a Number

### WHAT IT'S ASKING

Find the first and last position of a given target in a sorted array, and count its occurrences.

### TOPIC / PATTERN

Binary Search — Two boundary searches

### FIRST OCCURRENCE

- Lower Bound of target. Verify `arr[result] == target`.

### LAST OCCURRENCE

- Upper Bound of target minus 1. Verify `arr[result] == target`.

### COUNT OF OCCURRENCES

- `count = upper_bound(target) - lower_bound(target)`.

### COMPLEXITY

| Approach         | Time        | Space  | Notes              |
|------------------|-------------|--------|--------------------|
| 2× Binary Search | $O(\log N)$ | $O(1)$ | Two boundary calls |

### COMMON MISTAKES

- Forgetting to verify the found index actually equals target — target may not exist.

*Related: [LeetCode 34 Find First and Last Position of Element in Sorted Array](#)*

#### ■ Interview Trigger

First = lower bound. Last = upper bound – 1. Count = upper – lower.

## 4.1.8 · Search in Rotated Sorted Array I

### WHAT IT'S ASKING

Search for a target in a sorted array that has been rotated at an unknown pivot, with all elements distinct.

### TOPIC / PATTERN

Binary Search — Rotated Array (no duplicates)

### CORE OBSERVATION

One half is ALWAYS sorted. Use that fact to decide which half to search.

### ALGORITHM

1. `low=0, high=N-1`.
2. `mid = low+(high-low)//2`. If `arr[mid]==target` → return mid.
3. Check which half is sorted:
4. Left sorted (`arr[low] ≤ arr[mid]`): if target in `[arr[low], arr[mid]]` → `high=mid-1`. Else `low=mid+1`.
5. Right sorted (`arr[mid] ≤ arr[high]`): if target in `(arr[mid], arr[high]]` → `low=mid+1`. Else `high=mid-1`.

### VISUALIZATION

```
arr = [ 4, 5, 6, 7, 0, 1, 2 ], target = 0
L=0 H=6 mid=3 arr[3]=7
Left sorted [4..7]: target 0 NOT in [4,7] → search right: L=4
L=4 H=6 mid=5 arr[5]=1
Right sorted [1..2]: target 0 NOT in [1,2] → search left: H=4
L=4 H=4 mid=4 arr[4]=0 == target → return 4 ✓
```

**COMPLEXITY**

| Approach      | Time        | Space  | Notes                            |
|---------------|-------------|--------|----------------------------------|
| Brute         | $O(N)$      | $O(1)$ | Linear scan                      |
| Binary Search | $O(\log N)$ | $O(1)$ | One half always sorted — optimal |

**COMMON MISTAKES**

- Using strict  $<$  instead of  $<=$  when checking sorted half boundaries.

*Related: [LeetCode 33 Search in Rotated Sorted Array](#)*

**■ Interview Trigger**

One half is always sorted — check which, then decide which half contains the target.

**4.1.9 · Search in Rotated Sorted Array II (With Duplicates)****WHAT IT'S ASKING**

Same rotated-array search, but the array may contain duplicate values.

**TOPIC / PATTERN**

Binary Search — Rotated Array with Duplicates

**CORE PROBLEM**

When  $arr[low] == arr[mid] == arr[high]$ , we cannot determine which half is sorted. In this case, shrink both pointers:  $low++$ ,  $high--$ .

**ALGORITHM**

- Same as Rotated I, but add this special case:
- If  $arr[low] == arr[mid] == arr[high] \rightarrow low++, high--, \text{continue}$ .

**COMPLEXITY**

| Approach | Time        | Space  | Notes                                  |
|----------|-------------|--------|----------------------------------------|
| Best/Avg | $O(\log N)$ | $O(1)$ | No duplicates encountered              |
| Worst    | $O(N)$      | $O(1)$ | All elements same — forced linear scan |

**COMMON MISTAKES**

- Forgetting the ambiguous case — causes incorrect half selection with duplicates.

*Related: [LeetCode 81 Search in Rotated Sorted Array II](#)*

### ■ Interview Trigger

Duplicates at both ends → ambiguous → shrink both sides by 1 and continue.

## 4.1.10 / 4.1.11 · Find Minimum in Rotated Array / Rotation Count

### WHAT IT'S ASKING

Find the minimum element in a rotated sorted array, and figure out how many times it was rotated.

### TOPIC / PATTERN

Binary Search — Find the Pivot (point of rotation)

### CORE IDEA

The minimum element is at the rotation point. The sorted half always has its leftmost element  $\geq$  its rightmost. Eliminate the sorted half — the minimum is in the unsorted half.

### ALGORITHM

- 1. low=0, high=N-1, ans=INF.
- 2. If  $\text{arr}[\text{low}] \leq \text{arr}[\text{high}] \rightarrow$  this half is sorted,  $\text{min} = \text{arr}[\text{low}]$ .  $\text{high} = \text{mid} - 1$ .
- 3. Else if left half sorted ( $\text{arr}[\text{low}] \leq \text{arr}[\text{mid}]$ ):  $\text{ans} = \min(\text{ans}, \text{arr}[\text{low}])$ ,  $\text{low} = \text{mid} + 1$  (min is in right).
- 4. Else right half sorted:  $\text{ans} = \min(\text{ans}, \text{arr}[\text{mid}])$ ,  $\text{high} = \text{mid} - 1$  (min is in left).

### ROTATION COUNT

- Rotation count = index of the minimum element.

### COMPLEXITY

| Approach      | Time        | Space  | Notes                         |
|---------------|-------------|--------|-------------------------------|
| Binary Search | $O(\log N)$ | $O(1)$ | Discard sorted half each time |

Related: [LeetCode 153 Find Minimum in Rotated Sorted Array](#), [LeetCode 154 \(with duplicates\)](#)

### ■ Interview Trigger

Minimum is at the rotation pivot — discard the sorted half, search the unsorted side.

## 4.1.12 · Single Element in a Sorted Array

### WHAT IT'S ASKING

Find the single element in a sorted array where every other element appears exactly twice.

### TOPIC / PATTERN

Binary Search — Parity / Index Property

### CORE OBSERVATION

Before the single element, pairs occupy (even, odd) index pairs. After the single element, pairs shift to (odd, even). Use this parity break.

### ALGORITHM

- 1. low=0, high=N-1.
- 2. mid = low+(high-low)//2. Ensure mid is even (if odd, mid--).
- 3. If arr[mid]==arr[mid+1] → pair is intact, single is to the right → low=mid+2.
- 4. Else → pair is broken, single is at mid or to the left → high=mid.

### VISUALIZATION

```
[ 1, 1, 2, 2, 3, 3, 4, 5, 5 ]
0 1 2 3 4 5 6 7 8
mid=4 (even): arr[4]=3, arr[5]=3 → pair intact → low=6
mid=7 (odd → use 6): arr[6]=4, arr[7]=5 → pair broken → high=6
low=high=6 → arr[6]=4 ✓
```

### COMPLEXITY

| Approach      | Time     | Space | Notes                  |
|---------------|----------|-------|------------------------|
| XOR           | O(N)     | O(1)  | XOR all — pairs cancel |
| Binary Search | O(log N) | O(1)  | Optimal — parity trick |

*Related: LeetCode 540 Single Element in a Sorted Array*

#### ■ Interview Trigger

Pairs break parity at the single element — binary search on even indices.

## 4.1.13 · Find Peak Element

### WHAT IT'S ASKING

Find any peak element (one that is greater than both its neighbors) in an array.

### TOPIC / PATTERN

Binary Search — Move toward the higher neighbour

### CORE IDEA

A peak always exists. If arr[mid] < arr[mid+1], a peak exists on the right (the value keeps increasing). If arr[mid] < arr[mid-1], peak is on the left. Otherwise arr[mid] is a peak.

### ALGORITHM

- 1. Handle edge cases: arr[0]>arr[1] → peak at 0. arr[N-1]>arr[N-2] → peak at N-1.
- 2. low=1, high=N-2.
- 3. mid=low+(high-low)//2.
- 4. If arr[mid]>arr[mid-1] and arr[mid]>arr[mid+1] → return mid.
- 5. If arr[mid] < arr[mid+1] → low=mid+1 (peak is right).
- 6. Else → high=mid-1 (peak is left).

### COMPLEXITY

| Approach | Time | Space | Notes               |
|----------|------|-------|---------------------|
| Linear   | O(N) | O(1)  | Scan for first peak |

|               |             |        |                              |
|---------------|-------------|--------|------------------------------|
| Binary Search | $O(\log N)$ | $O(1)$ | Move toward higher neighbour |
|---------------|-------------|--------|------------------------------|

*Related: LeetCode 162 Find Peak Element, LeetCode 852 Peak Index in Mountain Array*

#### ■ Interview Trigger

Move toward the higher neighbour — a peak is guaranteed to exist there.

### 4.2 Binary Search on Answers (Search Space)

Pattern Recognition: When a problem asks to minimize/maximize a value subject to a constraint, and the feasibility function is monotonic (if  $X$  works,  $X+1$  also works), binary search on the answer.

Template for BS on Answer low = min\_possible\_answer high = max\_possible\_answer while low <= high: mid = low + (high - low) // 2 if canAchieve(mid): # is mid a feasible answer? ans = mid high = mid - 1 # try smaller (minimize) # OR: low = mid+1 (maximize) else: low = mid + 1 # mid too small return ans

## 4.2.1 / 4.2.2 · Square Root and Nth Root in log N

### WHAT IT'S ASKING

Compute the integer square root, and more generally the Nth root, of a number using binary search.

### TOPIC / PATTERN

Binary Search on Answer — Find integer floor of  $\sqrt[n]{n}$  or N-th root

### SQUARE ROOT ALGORITHM

- low=1, high=n. Binary search for largest mid where  $\text{mid} * \text{mid} \leq n$ .
- If  $\text{mid} * \text{mid} \leq n \rightarrow \text{ans} = \text{mid}$ , low=mid+1. Else high=mid-1.

### NTH ROOT ALGORITHM

- low=1, high=n. Binary search for mid where  $\text{mid}^m == n$ .
- Write a helper power(mid,m,n) that returns: 1 if result==n, 0 if n (avoid overflow).

### COMPLEXITY

| Approach        | Time        | Space  | Notes                              |
|-----------------|-------------|--------|------------------------------------|
| Newton's Method | $O(\log n)$ | $O(1)$ | Faster convergence but not pure BS |
| Binary Search   | $O(\log n)$ | $O(1)$ | Clean and interview-safe           |

### COMMON MISTAKES

- mid\*mid overflow — use long or a safe power helper.

#### ■ Interview Trigger

Binary search on answer space  $[1..n]$  — find largest mid satisfying the condition.

## 4.2.3 · Koko Eating Bananas

### WHAT IT'S ASKING

Given piles of bananas and H hours, find the minimum eating speed so Koko finishes all piles in time.

### TOPIC / PATTERN

Binary Search on Answer — Minimize eating speed K

### PROBLEM SUMMARY

Given N piles of bananas and H hours, find the minimum integer eating speed K such that all piles can be eaten in H hours.

### KEY INSIGHT

Hours to finish pile[i] at speed K =  $\text{ceil}(\text{pile}[i] / K)$ . As K increases, total hours decreases (monotonic) → binary search on K.

### ALGORITHM

- low=1, high=max(piles). Binary search for minimum K.
- canFinish(K): total\_hours = sum(ceil(pile/K) for each pile). Return total\_hours ≤ H.
- If canFinish(mid) → ans=mid, try smaller (high=mid-1). Else low=mid+1.

### COMPLEXITY

| Approach      | Time                                         | Space  | Notes           |
|---------------|----------------------------------------------|--------|-----------------|
| Brute         | $O(\text{max}(\text{piles}) \times N)$       | $O(1)$ | Try every speed |
| Binary Search | $O(N \times \log(\text{max}(\text{piles})))$ | $O(1)$ | Optimal         |

Binary Search  $O(N \times \log(\text{max}(\text{piles})))$   $O(1)$  Optimal

### COMMON MISTAKES

- Using integer division instead of ceiling division.  $\text{ceil}(a/b) = (a+b-1)/b$ .

*Related: LeetCode 875 Koko Eating Bananas*

#### ■ Interview Trigger

Monotonic feasibility: more speed = fewer hours → binary search the speed.

## 4.2.4 · Minimum Days to Make M Bouquets

### WHAT IT'S ASKING

Given days each flower takes to bloom, find the minimum number of days needed to make M bouquets of K adjacent flowers.

### TOPIC / PATTERN

Binary Search on Answer — Minimize days

### PROBLEM SUMMARY

Given bloom times for N flowers and requirements of M bouquets of K consecutive flowers, find the minimum number of days to wait.

### ALGORITHM

- low=min(bloomDay), high=max(bloomDay).

– canMake(day): count consecutive bloomed flowers. If we form K in a row → bouquet++. Return bouquets

≥ M.

– Binary search for minimum day where canMake returns True.

#### EDGE CASE

– If  $M \times K > N \rightarrow$  impossible, return -1.

#### COMPLEXITY

| Approach          | Time                            | Space  | Notes                       |
|-------------------|---------------------------------|--------|-----------------------------|
| BS + Linear Check | $O(N \times \log(\max - \min))$ | $O(1)$ | Feasibility check is $O(N)$ |

BS + Linear Check  $O(N \times \log(\max - \min))$   $O(1)$  Feasibility check is  $O(N)$

*Related: LeetCode 1482 Minimum Number of Days to Make M Bouquets*

#### ■ Interview Trigger

More days = more flowers bloom = easier to form bouquets → binary search on days.

## 4.2.5 · Find the Smallest Divisor

### WHAT IT'S ASKING

Find the smallest divisor such that the sum of  $(\text{array}[i] / \text{divisor})$ , rounded up, stays  $\leq$  a given threshold.

### TOPIC / PATTERN

Binary Search on Answer — Minimize divisor

### PROBLEM SUMMARY

Find the smallest divisor D such that  $\text{sum}(\text{ceil}(\text{arr}[i]/D)) \leq \text{threshold}$ .

### ALGORITHM

- low=1, high=max(arr). Binary search for minimum D.
- check(D): return  $\text{sum}(\text{ceil}(a/D) \text{ for } a \text{ in arr}) \leq \text{threshold}$ .

### COMPLEXITY

| Approach    | Time              | Space  | Notes                 |
|-------------|-------------------|--------|-----------------------|
| BS + Linear | $O(N \log(\max))$ | $O(1)$ | Standard BS on answer |

*Related: LeetCode 1283 Find the Smallest Divisor Given a Threshold*

#### ■ Interview Trigger

Larger divisor → smaller sum → binary search for minimum valid divisor.

## 4.2.6 · Capacity to Ship Packages within D Days

### WHAT IT'S ASKING

Find the minimum ship capacity needed to ship all packages within D days without reordering them.

#### TOPIC / PATTERN

Binary Search on Answer — Minimize ship capacity

#### PROBLEM SUMMARY

Find the minimum weight capacity of a ship to ship all packages (in order) within D days.

#### ALGORITHM

- low=max(weights) (must carry heaviest package), high=sum(weights) (carry all in 1 day).
- canShip(cap): greedily assign packages to days. If adding next package exceeds cap → new day. Return

days ≤ D.

- Binary search for minimum cap where canShip returns True.

#### COMPLEXITY

| Approach          | Time                    | Space  | Notes   |
|-------------------|-------------------------|--------|---------|
| BS + Greedy Check | $O(N \log(\text{sum}))$ | $O(1)$ | Optimal |

#### COMMON MISTAKES

- low = 0 instead of max(weights) — capacity must be at least the heaviest package.

*Related: LeetCode 1011 Capacity to Ship Packages, similar to Book Allocation / Painter's Partition*

#### ■ Interview Trigger

Higher capacity = fewer days. Binary search for minimum capacity satisfying D days.

## 4.2.7 · Kth Missing Positive Number

#### WHAT IT'S ASKING

Find the Kth positive integer that is missing from a given sorted array of positive integers.

#### TOPIC / PATTERN

Binary Search — Count missing numbers before index

#### CORE IDEA

Missing numbers before arr[i] = arr[i] - (i+1). Binary search for the last index where missing count < K.

#### ALGORITHM

- low=0, high=N-1.
- missing(mid) = arr[mid] - (mid+1).
- If missing(mid) < K → low=mid+1. Else high=mid-1.
- Answer = high+1 + K (after loop, low points just past the insertion point).

#### COMPLEXITY

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|               |             |        |                    |
|---------------|-------------|--------|--------------------|
| Linear        | $O(N)$      | $O(1)$ | Scan until Kth gap |
| Binary Search | $O(\log N)$ | $O(1)$ | Optimal            |

*Related: LeetCode 1539 Kth Missing Positive Number*

#### ■ Interview Trigger

Missing count at index  $i = \text{arr}[i] - (i+1)$ . Binary search for insertion boundary.

## 4.2.8 · Aggressive Cows

### WHAT IT'S ASKING

Place cows in stalls at positions on a line so that the minimum distance between any two cows is as large as possible.

### TOPIC / PATTERN

Binary Search on Answer — Maximize minimum distance

### PROBLEM SUMMARY

Place  $C$  cows in  $N$  stalls (given positions) such that the minimum distance between any two cows is maximized.

### ALGORITHM

- Sort stall positions.
- $\text{low}=1$ ,  $\text{high}=\text{stalls}[N-1]-\text{stalls}[0]$  (max possible gap).
- $\text{canPlace}(\text{minDist})$ : greedily place cows — place first cow at  $\text{stalls}[0]$ , next cow at first stall with gap  $\geq$

$\text{minDist}$ . Return  $\text{cows\_placed} \geq C$ .

- Binary search for maximum  $\text{minDist}$  where  $\text{canPlace}$  returns True.

### VISUALIZATION

```
Stalls: [ 1, 2, 4, 8, 9 ], C=3 cows
Try minDist=3: place at 1, 4, 8 → 3 cows placed ✓ → try larger
Try minDist=4: place at 1, 8 → only 2 cows ✗ → try smaller
Answer: minDist = 3
```

### COMPLEXITY

| Approach           | Time                     | Space  | Notes                      |
|--------------------|--------------------------|--------|----------------------------|
| Sort + BS + Greedy | $O(N \log N + N \log D)$ | $O(1)$ | $D = \text{max stall gap}$ |

Sort + BS + Greedy  $O(N \log N + N \log D)$   $O(1)$   $D = \text{max stall gap}$

### WHY THIS PATTERN MATTERS

- This is the archetypal 'maximize minimum' problem. Same idea: Book Allocation, Painter's Partition, Split

Array.

*Related: GFG Aggressive Cows, LeetCode 2517 Maximum Tastiness of Candy Basket*

### ■ Interview Trigger

Maximize minimum distance → binary search on the answer, greedy feasibility check.

## 4.2.9 / 4.2.10 / 4.2.11 · Book Allocation · Split Array · Painter's Partition

### WHAT IT'S ASKING

Split an array into groups (books/pages, contiguous subarrays, or painters) to minimize the maximum sum/load any one group gets.

### TOPIC / PATTERN

Binary Search on Answer — Minimize maximum (all three are the SAME problem)

### PROBLEM SUMMARY

Divide an array into K contiguous groups to minimize the maximum group sum. Book allocation = Split Array Largest Sum = Painter's Partition.

### ALGORITHM

- low=max(arr) (min possible max — at least the largest element), high=sum(arr).
- canDivide(maxSum): greedily assign elements to groups without exceeding maxSum. Count groups needed.
- If groups ≤ K → feasible. Binary search for minimum maxSum.

### VISUALIZATION

```
arr=[10,20,30,40], K=2 partitions
low=40, high=100
mid=70: groups=[10+20+30]=60, [40] → 2 groups ≤ 2 ✓ → ans=70, try smaller
mid=55: groups=[10+20], [30], too many → try larger ...
Answer: 60
```

### COMPLEXITY

| Approach    | Time                    | Space  | Notes                                      |
|-------------|-------------------------|--------|--------------------------------------------|
| BS + Greedy | $O(N \log(\text{sum}))$ | $O(1)$ | Feasibility is a greedy left-to-right scan |

### COMMON MISTAKES

- Setting low=0 — the maximum partition must be at least as large as the maximum element.

*Related: [LeetCode 410 Split Array Largest Sum](#), [LeetCode 1011 Ship Packages](#), [GFG Book Allocation](#)*

### ■ Interview Trigger

Minimize the maximum → binary search the answer, greedily count partitions.

## 4.2.12 · Minimize Max Distance to Gas Station

### WHAT IT'S ASKING

Place gas stations along a line to minimize the maximum distance between any two adjacent stations, given you can add K more stations.

#### TOPIC / PATTERN

Binary Search on Answer (on doubles) — Minimize maximum gap

#### PROBLEM SUMMARY

Add K gas stations to minimize the maximum distance between any two adjacent stations.

#### ALGORITHM

- Binary search on the answer D (a decimal value, use precision of  $1e-6$ ).
- $\text{canPlace}(D)$ : for each gap, how many stations needed =  $\text{floor}(\text{gap}/D)$ . Sum must  $\leq K$ .
- If  $\text{canPlace}(\text{mid}) \rightarrow \text{high}=\text{mid}$ . Else  $\text{low}=\text{mid}$ . Run  $\sim 100$  iterations for precision.

#### COMPLEXITY

| Approach      | Time              | Space  | Notes                                      |
|---------------|-------------------|--------|--------------------------------------------|
| BS on doubles | $O(N \times 100)$ | $O(1)$ | $\sim 100$ iterations for $1e-6$ precision |

#### COMMON MISTAKES

- Using integer BS — answer is a decimal. Must use floating point binary search.

*Related: LeetCode 774 Minimize Max Distance to Gas Station*

#### ■ Interview Trigger

Floating point BS: run  $\sim 100$  iterations (or until  $\text{high}-\text{low} < 1e-6$ ).

## 4.2.13 · Median of Two Sorted Arrays (Hard)

#### WHAT IT'S ASKING

Given two sorted arrays, find the median of all elements combined, faster than merging them.

#### TOPIC / PATTERN

Binary Search — Partition both arrays to find median directly

#### CORE IDEA

Binary search on the smaller array. For each partition of array1, determine the corresponding partition of array2. Adjust until left halves of both arrays together form the correct lower half of the merged array.

#### ALGORITHM

- Binary search on arr1 (smaller). Partition:  $\text{left1}=\text{mid1}$  elements from arr1,  $\text{left2}=(n1+n2+1)//2-\text{mid1}$  from arr2.
- Valid partition:  $\text{maxLeft1} \leq \text{minRight2}$  AND  $\text{maxLeft2} \leq \text{minRight1}$ .
- If  $\text{maxLeft1} > \text{minRight2} \rightarrow \text{move left} (\text{high}=\text{mid1}-1)$ . Else  $\rightarrow \text{move right} (\text{low}=\text{mid1}+1)$ .
- Median = average of  $\text{max}(\text{maxLeft1}, \text{maxLeft2})$  and  $\text{min}(\text{minRight1}, \text{minRight2})$  for even total.

#### COMPLEXITY

| Approach      | Time                 | Space    | Notes                                |
|---------------|----------------------|----------|--------------------------------------|
| Merge+Find    | $O(N+M)$             | $O(N+M)$ | Merge both arrays then find median   |
| Binary Search | $O(\log(\min(N,M)))$ | $O(1)$   | Optimal — always BS on smaller array |

**COMMON MISTAKES**

- Not handling out-of-bounds partitions — use  $-\text{INF}$  and  $+\text{INF}$  as sentinels.
- Binary searching on the larger array — always pick the smaller to minimize iterations.

*Related: LeetCode 4 Median of Two Sorted Arrays*

**■ Interview Trigger**

Partition both arrays so left halves together have the right count — adjust partition by BS.

**4.2.14 · Kth Element of Two Sorted Arrays****WHAT IT'S ASKING**

Given two sorted arrays, find the Kth smallest element among all elements combined.

**TOPIC / PATTERN**

Binary Search — Similar to Median of Two Sorted Arrays

**ALGORITHM**

- Binary search on arr1. Partition so that left parts together have exactly K elements.
- Same partition validity check as median problem.
- Answer =  $\max(\text{maxLeft1}, \text{maxLeft2})$  — largest element in the left partition.

**COMPLEXITY**

| Approach      | Time                 | Space  | Notes   |
|---------------|----------------------|--------|---------|
| Binary Search | $O(\log(\min(N,M)))$ | $O(1)$ | Optimal |

*Related: GFG Kth Element of Two Sorted Arrays, LeetCode 4 (median is a special case)*

**■ Interview Trigger**

K-th element = max of the left partition when both arrays are correctly partitioned.

**4.3 Binary Search on 2D Arrays****4.3.1 · Row with Maximum Number of 1s****WHAT IT'S ASKING**

Find the row in a binary matrix that contains the maximum number of 1s.

**TOPIC / PATTERN**

2D Array — Binary Search per row (each row is sorted: 0s then 1s)

**ALGORITHM**

- For each row, use `lower_bound` to find the first 1 (index of first 1 = `lower_bound(target=1)`).
- Ones in row = `N - lower_bound_index`. Track max.

**COMPLEXITY**

| Approach   | Time            | Space  | Notes                     |
|------------|-----------------|--------|---------------------------|
| Brute      | $O(M \times N)$ | $O(1)$ | Count 1s each row         |
| BS per row | $O(M \log N)$   | $O(1)$ | Binary search in each row |

**■ Interview Trigger**

Each row is sorted — lower bound gives first 1, rest of the row is all 1s.

**4.3.2 · Search in a 2D Matrix (Row-Column Sorted)****WHAT IT'S ASKING**

Search for a target value in an  $m \times n$  matrix where each row and column is sorted.

**TOPIC / PATTERN**

2D Matrix — Treat as flattened 1D sorted array OR Staircase Search

**VARIANT 1 — MATRIX SORTED ROW BY ROW, LAST OF ROW < FIRST OF NEXT ROW**

- Treat as a single sorted array of size  $M \times N$ . Map 1D index  $\rightarrow$  2D: `row=mid//N`, `col=mid%N`.

VARIANT 2 — EACH ROW SORTED, EACH COLUMN SORTED (BUT NO GUARANTEE BETWEEN ROWS)

- Start at top-right corner. If `val==target`  $\rightarrow$  found. If `val > target`  $\rightarrow$  `col--`. If `val < target`  $\rightarrow$  `row++`.

**VISUALIZATION — STAIRCASE**

```
Start at top-right: arr[0][N-1]
Too big  $\rightarrow$  move left (eliminate column)
Too small  $\rightarrow$  move down (eliminate row)
Each step eliminates a row or column  $\rightarrow O(M+N)$ 
```

**COMPLEXITY**

| Approach          | Time                  | Space  | Notes                                    |
|-------------------|-----------------------|--------|------------------------------------------|
| Flattened BS (V1) | $O(\log(M \times N))$ | $O(1)$ | Only works when rows are globally sorted |
| Staircase (V2)    | $O(M+N)$              | $O(1)$ | Works for row+col sorted matrices        |

Related: [LeetCode 74 Search a 2D Matrix](#), [LeetCode 240 Search a 2D Matrix II](#)

**■ Interview Trigger**

Globally sorted  $\rightarrow$  flatten and BS. Row+col sorted  $\rightarrow$  staircase from top-right.

### 4.3.4 · Find Peak Element in 2D Matrix

#### WHAT IT'S ASKING

Find a peak element in a 2D matrix (greater than all four neighbors) using binary search.

#### TOPIC / PATTERN

2D Binary Search — Binary search on columns, find max in column

#### ALGORITHM

- Binary search on columns: low=0, high=N-1.
- For mid column, find the row with maximum element in that column (maxRow).
- If matrix[maxRow][mid] > left and right neighbours → it's a 2D peak, return.
- Else move toward the larger neighbour.

#### COMPLEXITY

| Approach      | Time            | Space  | Notes                                |
|---------------|-----------------|--------|--------------------------------------|
| Brute         | $O(M \times N)$ | $O(1)$ | Check all cells                      |
| Binary Search | $O(M \log N)$   | $O(1)$ | Find column max + compare neighbours |

*Related: LeetCode 1901 Find a Peak Element II*

#### ■ Interview Trigger

Binary search columns — find column max — if it dominates neighbours it's a 2D peak.

### 4.3.5 · Matrix Median

#### WHAT IT'S ASKING

Find the median of all elements in a row-wise sorted matrix.

#### TOPIC / PATTERN

Binary Search on Answer — Count elements  $\leq$  mid across all rows

#### CORE IDEA

Each row is sorted. Binary search on the value space [min..max]. For a candidate median M, count how many elements  $\leq$  M using upper\_bound on each row. Median is the smallest M where count  $> (M \times N) // 2$ .

#### ALGORITHM

- low=min of all row-minimums, high=max of all row-maximums.
- countLE(mid): sum upper\_bound(row, mid) for each row.
- Binary search for minimum mid where countLE  $> (M \times N) // 2$ .

#### COMPLEXITY

| Approach | Time            | Space   | Notes            |
|----------|-----------------|---------|------------------|
| Sort all | $O(MN \log MN)$ | $O(MN)$ | Flatten and sort |

|               |                                 |        |                             |
|---------------|---------------------------------|--------|-----------------------------|
| Binary Search | $O(M \log N \log(\max - \min))$ | $O(1)$ | Optimal — BS on value space |
|---------------|---------------------------------|--------|-----------------------------|

Binary Search  $O(M \log N \log(\max - \min))$   $O(1)$  Optimal — BS on value space

*Related: GFG Matrix Median, LeetCode 378 Kth Smallest Element in Sorted Matrix*

#### ■ Interview Trigger

Binary search the answer value — count elements  $\leq$  mid using upper\_bound per row.

### BINARY SEARCH — MASTER CHEATSHEET

| Pattern        | Trigger Keywords                            | low / high       | Direction                    |
|----------------|---------------------------------------------|------------------|------------------------------|
| Classic BS     | 'sorted array', 'find target', 'O(log N)'   | 0 / N-1          | Target side                  |
| Lower Bound    | 'first $\geq$ target', 'insert position'    | 0 / N            | arr[mid] $\geq$ x fi go left |
| Upper Bound    | 'first > target', 'count occurrences'       | 0 / N            | arr[mid] > x fi go left      |
| Rotated Array  | 'rotated sorted', 'find in rotated'         | 0 / N-1          | Check sorted half            |
| Peak Element   | 'no neighbors larger', 'peak'               | 1 / N-2          | Go toward higher neighbor    |
| Single Element | 'all pairs except one'                      | 0 / N-1          | Parity of even index         |
| BS on Answer   | 'minimize max', 'maximize min', 'at most K' | 1 / max_val      | Feasibility function         |
| Float BS       | 'minimize distance', 'precision needed'     | lo / hi (float)  | ~100 iterations              |
| 2D Flattened   | 'M×N matrix globally sorted'                | 0 / M×N-1        | row=mid//N, col=mid%N        |
| Staircase      | 'row+col sorted matrix'                     | top-right corner | Eliminate row or column      |

### THE 3 QUESTIONS TO ASK

- 1. What is the search space? Array indices, or the answer value range?
- 2. What is the condition? Is it monotonic (once true, always true after a point)?
- 3. Which direction to move? When condition is met, do we want smaller (go left) or larger (go right)?

DSA Revision Notes Striver's A2Z DSA Sheet

### ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING

## 4.3.6 · Search in a 2D Matrix (Flattened / Row-Major)

### WHAT IT'S ASKING

Search for a target in an  $m \times n$  matrix where each row's last element is smaller than the next row's first, treating it as one flattened sorted array.

### TOPIC / PATTERN

Binary Search — Treat 2D matrix as a flattened 1D sorted array

### CORE IDEA

When each row's last element is smaller than the next row's first element, the whole matrix is really one sorted array in disguise — binary search over indices  $[0, N*M-1]$  and convert back to (row, col).

### ALGORITHM

- 1.  $low=0$ ,  $high=N*M-1$ .
- 2.  $mid = (low+high)/2 \rightarrow row = mid/M$ ,  $col = mid\%M$ .
- 3. Compare  $matrix[row][col]$  to target; move low/high as in classic binary search.

### COMPLEXITY

- Time:  $O(\log(N*M))$
- Space:  $O(1)$

#### ■ Interview Trigger

Row-major index math ( $row = mid/M$ ,  $col = mid\%M$ ) turns a 2D matrix into a 1D binary search.

# DSA Revision Notes

## Striver's A2Z DSA Sheet

### Batch 4 · Steps 5 & 18: Strings (Basic · Medium · Advanced)

Problems 15 problems — Basic, Medium, and Advanced string techniques

Patterns Two Pointers · Sliding Window · Hashing · KMP · Z-Function · Rabin-Karp

Format Pattern · Intuition · Algorithm · Complexity · Traps · Trigger

Strings = character arrays with extra tricks. Master hashing, window patterns, and KMP.

#### TABLE OF CONTENTS

##### 5.1 BASIC & EASY STRING PROBLEMS

- 5.1.1 Remove Outermost Parentheses
- 5.1.2 Reverse Words in a String
- 5.1.3 Largest Odd Number in a String
- 5.1.4 Longest Common Prefix
- 5.1.5 Isomorphic Strings
- 5.1.6 Check if String is Rotation of Another
- 5.1.7 Check if Two Strings are Anagrams

##### 5.2 MEDIUM STRING PROBLEMS

- 5.2.1 Sort Characters by Frequency
- 5.2.2 Maximum Nesting Depth of Parentheses
- 5.2.3 Roman Number to Integer and Vice Versa
- 5.2.4 Implement Atoi
- 5.2.5 Count Number of Substrings
- 5.2.6 Longest Palindromic Substring
- 5.2.7 Sum of Beauty of All Substrings
- 5.2.8 Reverse Every Word in a String

##### 18.1 ADVANCED STRING PROBLEMS

- 18.1.1 Hashing in Strings — Theory
- 18.1.2 Rabin-Karp Algorithm
- 18.1.3 Z-Function
- 18.1.4 KMP Algorithm / LPS Array
- 18.1.5 Shortest Palindrome
- 18.1.6 Longest Happy Prefix
- 18.1.7 Count Palindromic Subsequences
- 18.1.8 Count and Say
- 18.1.9 Minimum Bracket Reversals
- STEPS 5 & 18 — Strings
- 5.1 Basic & Easy String Problems

---

### 5.1.1 · Remove Outermost Parentheses

#### WHAT IT'S ASKING

Given a string of balanced parentheses, remove the outermost parentheses of every primary group.

**TOPIC / PATTERN**

Strings — Stack depth counter

**CORE IDEA**

Track nesting depth with a counter. Add character to result only when depth > 0 (opening) or depth > 1 (closing). The outermost brackets are those at depth 0→1 and 1→0.

**ALGORITHM**

- 1. depth = 0, result = "".
- 2. For each char c: if c=='(' → if depth > 0 append c; depth++.
- 3. If c==')' → depth--; if depth > 0 append c.

**VISUALIZATION**

```
s = '(()())()'
depth: 0→1 1→2 2→1 1→0 | 0→1 1→2 2→1 1→0
add?:  N Y Y N | N Y Y N
result: '()' + '()' = '(()())' ✓
```

**COMPLEXITY**

| Approach    | Time | Space | Notes                      |
|-------------|------|-------|----------------------------|
| Linear Scan | O(N) | O(N)  | Single pass, result string |

*Related: [LeetCode 1021 Remove Outermost Parentheses](#)*

**■ Interview Trigger**

Depth counter: skip chars at depth 0 (open) and depth 1 (close) — those are outermost.

## 5.1.2 · Reverse Words in a String

**WHAT IT'S ASKING**

Reverse the order of the words in a string, collapsing extra spaces.

**TOPIC / PATTERN**

Strings — Split / Two Pointers

**CORE IDEA**

Split the string by spaces, filter empty tokens, reverse the list, rejoin with single space.

**ALGORITHM (SIMPLE)**

- 1. Split string on whitespace (handles multiple spaces automatically).
- 2. Filter out empty strings.
- 3. Reverse the word list.
- 4. Join with single space.

**IN-PLACE VARIANT (O(1) SPACE)**

- 1. Reverse the entire string.
- 2. Reverse each individual word in place.

- 3. Clean up extra spaces.

#### COMPLEXITY

| Approach        | Time   | Space  | Notes                     |
|-----------------|--------|--------|---------------------------|
| Split & Reverse | $O(N)$ | $O(N)$ | Easy — language built-ins |
| In-Place        | $O(N)$ | $O(1)$ | Harder — reverse twice    |

*Related: LeetCode 151 Reverse Words in a String*

#### ■ Interview Trigger

Reverse all, then reverse each word — words flip back to correct order.

### 5.1.3 · Largest Odd Number in a String

#### WHAT IT'S ASKING

Among the numbers formed by substrings of a digit string, find the largest one that's odd.

#### TOPIC / PATTERN

Strings — Greedy scan from right

#### CORE IDEA

A number is odd if its last digit is odd. Scan from the rightmost digit left — the first odd digit found gives the answer (substring from start to that index).

#### ALGORITHM

- For  $i = N-1$  down to 0: if  $\text{int}(s[i]) \% 2 \neq 0 \rightarrow$  return  $s[0..i]$ .
- If no odd digit found  $\rightarrow$  return empty string.

#### COMPLEXITY

| Approach   | Time   | Space  | Notes           |
|------------|--------|--------|-----------------|
| Right scan | $O(N)$ | $O(1)$ | At most N steps |

*Related: LeetCode 1903 Largest Odd Number in a String*

#### ■ Interview Trigger

Scan right to left — first odd digit = rightmost cutoff for largest odd number.

### 5.1.4 · Longest Common Prefix

#### WHAT IT'S ASKING

Find the longest common prefix string among an array of strings.

#### TOPIC / PATTERN

Strings — Vertical scan or Sort trick

**APPROACH 1 — VERTICAL SCAN**

- Take `strs[0]` as the candidate prefix. For each character position, check if ALL strings have that character

at that position. Stop at first mismatch.

**APPROACH 2 — SORT TRICK**

- Sort the array of strings. The common prefix of the entire array = common prefix of first and last string

only (they differ the most).

**COMPLEXITY**

| Approach      | Time          | Space  | Notes                            |
|---------------|---------------|--------|----------------------------------|
| Vertical Scan | $O(S)$        | $O(1)$ | $S$ = total chars across strings |
| Sort Trick    | $O(S \log N)$ | $O(1)$ | Sort dominates                   |

**COMMON MISTAKES**

- Accessing `s[i]` when  $i \geq \text{len}(s)$  — always check length bounds.

*Related: [LeetCode 14 Longest Common Prefix](#)*

**■ Interview Trigger**

Check each character position across all strings — stop at first mismatch.

## 5.1.5 · Isomorphic Strings

**WHAT IT'S ASKING**

Check whether two strings follow the same character-to-character mapping pattern (isomorphic).

**TOPIC / PATTERN**

Strings — Bidirectional HashMap

**CORE IDEA**

Two strings are isomorphic if characters can be consistently replaced. Need two maps:  $s \rightarrow t$  and  $t \rightarrow s$ . Both directions must be consistent.

**ALGORITHM**

1. For each position  $i$ :  $cs = s[i]$ ,  $ct = t[i]$ .
2. If  $cs$  in `mapST` and `mapST[cs] != ct` → return False.
3. If  $ct$  in `mapTS` and `mapTS[ct] != cs` → return False.
4. Else: `mapST[cs]=ct`, `mapTS[ct]=cs`.

**WHY TWO MAPS?**

$s='ab'$ ,  $t='aa'$  →  $a \rightarrow a$ ,  $b \rightarrow a$  (ok one way) But  $a \rightarrow a$  AND  $a \leftarrow b$  and  $a \leftarrow a$  conflicts → need reverse map

**COMPLEXITY**

| Approach     | Time | Space | Notes                    |
|--------------|------|-------|--------------------------|
| Two HashMaps | O(N) | O(1)  | Only 256 ASCII chars max |

Related: [LeetCode 205 Isomorphic Strings](#)

#### ■ Interview Trigger

Use two maps ( $s \rightarrow t$  and  $t \rightarrow s$ ) — consistent mapping must hold in BOTH directions.

## 5.1.6 · Check if String is Rotation of Another

### WHAT IT'S ASKING

Check whether one string is a rotation of another using only string concatenation.

### TOPIC / PATTERN

Strings — Concatenation trick

### CORE IDEA

If  $s_2$  is a rotation of  $s_1$ , then  $s_2$  will always be a substring of  $s_1+s_1$ .

### ALGORITHM

- 1. If  $\text{len}(s_1) \neq \text{len}(s_2) \rightarrow$  return False.
- 2. Check if  $s_2$  is a substring of  $(s_1+s_1)$ .
- 3. Use KMP or built-in find for O(N) substring check.

### VISUALIZATION

```
s1 = 'abcde', s2 = 'cdeab'
s1+s1 = 'abcdeabcde'
'cdeab' found at index 2 → True ✓
```

### COMPLEXITY

| Approach           | Time | Space | Notes                     |
|--------------------|------|-------|---------------------------|
| Concatenate+Search | O(N) | O(N)  | KMP or built-in substring |

Related: [LeetCode 796 Rotate String](#)

#### ■ Interview Trigger

$s_2$  is rotation of  $s_1$  iff  $s_2$  is a substring of  $s_1+s_1$ .

## 5.1.7 · Check if Two Strings are Anagrams

### WHAT IT'S ASKING

Check whether two strings are anagrams of each other.

### TOPIC / PATTERN

Strings — Frequency Count

**ALGORITHM**

- 1. If  $\text{len}(s) \neq \text{len}(t) \rightarrow \text{return False}$ .
- 2. Count frequency of each character in  $s$  (increment) and  $t$  (decrement).
- 3. If all counts are 0  $\rightarrow$  anagram. Else  $\rightarrow$  not anagram.

**COMPLEXITY**

| Approach   | Time          | Space  | Notes                     |
|------------|---------------|--------|---------------------------|
| Sort       | $O(N \log N)$ | $O(1)$ | Sort both and compare     |
| Freq Array | $O(N)$        | $O(1)$ | 26 letters — $O(1)$ space |

**COMMON MISTAKES**

- Forgetting the length check — different lengths can never be anagrams.

*Related: [LeetCode 242 Valid Anagram](#), [LeetCode 438 Find All Anagrams in a String](#)*

**■ Interview Trigger**

Anagram = same character frequencies. One pass: increment for  $s$ , decrement for  $t$ .

## 5.2 Medium String Problems

**5.2.1 · Sort Characters by Frequency****WHAT IT'S ASKING**

Sort the characters of a string by descending frequency.

**TOPIC / PATTERN**

Strings — Frequency Count + Sort / Max Heap

**ALGORITHM**

- 1. Count character frequencies using a HashMap.
- 2. Sort characters by frequency (descending).
- 3. Build result string: repeat each char by its frequency.

**HEAP VARIANT**

- Push (freq, char) pairs into a max-heap. Pop and build result greedily.

**COMPLEXITY**

| Approach | Time              | Space  | Notes                                  |
|----------|-------------------|--------|----------------------------------------|
| Sort     | $O(N + K \log K)$ | $O(K)$ | $K$ = unique chars ( $\leq 26$ or 128) |
| Max Heap | $O(N + K \log K)$ | $O(K)$ | Same — heap just for elegance          |

*Related: [LeetCode 451 Sort Characters By Frequency](#)*

**■ Interview Trigger**

Count → sort by frequency → rebuild string. Heap optional.

## 5.2.2 · Maximum Nesting Depth of Parentheses

### WHAT IT'S ASKING

Find the maximum depth of nested parentheses in a valid parentheses string.

### TOPIC / PATTERN

Strings — Counter

### ALGORITHM

- Scan string. Track current depth: depth++ on '(', depth-- on ')'.  
– maxDepth = max(maxDepth, depth) after each increment.

### COMPLEXITY

| Approach | Time | Space | Notes               |
|----------|------|-------|---------------------|
| Linear   | O(N) | O(1)  | Single pass counter |

*Related: LeetCode 1614 Maximum Nesting Depth of Parentheses*

**■ Interview Trigger**

Track running depth, record peak — one pass.

## 5.2.3 · Roman Number to Integer and Vice Versa

### WHAT IT'S ASKING

Convert a Roman numeral to an integer and an integer to a Roman numeral.

### TOPIC / PATTERN

Strings — Greedy with lookup table

### ROMAN TO INTEGER

- Scan left to right. If current value < next value → subtract current (e.g. IV = 5–4).  
– Else add current value.

### INTEGER TO ROMAN

- Greedy: maintain list of (value, symbol) pairs in descending order.  
– For each pair, while num >= value: append symbol, subtract value.

### KEY SYMBOL TABLE

**M=1000, CM=900, D=500, CD=400, C=100, XC=90,**

**L=50, XL=40, X=10, IX=9, V=5, IV=4, I=1**

### COMPLEXITY

| Approach    | Time   | Space  | Notes                         |
|-------------|--------|--------|-------------------------------|
| Greedy Scan | $O(N)$ | $O(1)$ | Lookup table is constant size |

Related: [LeetCode 13 Roman to Integer](#), [LeetCode 12 Integer to Roman](#)

#### ■ Interview Trigger

Roman to Int: if current < next, subtract. Integer to Roman: greedy largest symbol fit.

## 5.2.4 · Implement Atoi (String to Integer)

### WHAT IT'S ASKING

Implement `atoi()`: convert a string to a 32-bit signed integer the way C's `atoi` does, handling whitespace, sign, and overflow.

### TOPIC / PATTERN

Strings — State Machine / Step-by-step parsing

### ALGORITHM

- 1. Skip leading whitespace.
- 2. Detect optional sign ('+' or '-'). Default = positive.
- 3. Read digits until non-digit or end. Build integer digit by digit.
- 4. Clamp to  $[INT\_MIN, INT\_MAX]$  range.

### CLAMP CHECK (TO AVOID OVERFLOW)

Before:  $result = result * 10 + digit$  If  $result > INT\_MAX // 10 \rightarrow$  will overflow  $\rightarrow$  clamp If  $result == INT\_MAX // 10$  and  $digit > 7 \rightarrow$  clamp

### COMPLEXITY

| Approach | Time   | Space  | Notes             |
|----------|--------|--------|-------------------|
| Linear   | $O(N)$ | $O(1)$ | Parse in one pass |

### COMMON MISTAKES

- Not handling overflow before it happens — check BEFORE multiplying.
- Forgetting multiple leading spaces or signs like '--42'.

Related: [LeetCode 8 String to Integer \(atoi\)](#)

#### ■ Interview Trigger

Skip spaces  $\rightarrow$  read sign  $\rightarrow$  read digits  $\rightarrow$  clamp. Check overflow BEFORE multiply.

## 5.2.5 · Count Number of Substrings / Sum of Beauty of All Substrings

### WHAT IT'S ASKING

Count the number of substrings with all distinct characters, or sum the 'beauty' (max freq – min freq) of every substring.

#### TOPIC / PATTERN

Strings — Sliding Window + Frequency Map

#### COUNT SUBSTRINGS WITH ALL 3 CHARACTERS (A,B,C)

- Use sliding window: expand right, shrink left when all 3 chars present.
- For each valid window ending at right, count = left (all shorter windows are also valid).

#### SUM OF BEAUTY (MAX\_FREQ - MIN\_FREQ ACROSS ALL SUBSTRINGS)

- Fix left boundary. Expand right, maintain frequency map.
- At each right, track max and min frequency directly from the map.

#### COMPLEXITY

| Approach        | Time                 | Space  | Notes                                 |
|-----------------|----------------------|--------|---------------------------------------|
| Brute           | $O(N^2)$ or $O(N^3)$ | $O(1)$ | All substrings                        |
| Sliding Window  | $O(N)$               | $O(1)$ | For count-of-3-chars variant          |
| Fixed Left Scan | $O(N^2)$             | $O(1)$ | For beauty sum — unavoidable $O(N^2)$ |

*Related: LeetCode 1358 Number of Substrings Containing All Three Characters, LeetCode 1781*

#### ■ Interview Trigger

Count valid windows: when all chars present, every earlier left is also valid.

## 5.2.6 · Longest Palindromic Substring (Without DP)

#### WHAT IT'S ASKING

Find the longest palindromic substring in a string without using DP.

#### TOPIC / PATTERN

Strings — Expand Around Center

#### CORE IDEA

Every palindrome has a center. Expand outward from each possible center (N centers for odd length, N–1 centers for even length). Track longest.

#### ALGORITHM

- For each index  $i$  (0 to  $N-1$ ):
- Expand around  $(i, i)$  for odd-length palindromes.
- Expand around  $(i, i+1)$  for even-length palindromes.
- While  $left \geq 0$  and  $right < N$  and  $s[left] == s[right]$ : expand.
- Track max palindrome found.

#### VISUALIZATION

```
s = 'babad'
Center i=1 ('a'): expand → 'bab' (length 3)
Center i=2 ('b'): expand → 'aba' (length 3)
Answer: 'bab' or 'aba' ✓
```

**COMPLEXITY**

| Approach        | Time     | Space  | Notes                                |
|-----------------|----------|--------|--------------------------------------|
| Brute           | $O(N^3)$ | $O(1)$ | All substrings + palindrome check    |
| Expand Center   | $O(N^2)$ | $O(1)$ | Optimal without DP or Manacher       |
| Manacher's Algo | $O(N)$   | $O(N)$ | Advanced — all palindromes in $O(N)$ |

*Related: LeetCode 5 Longest Palindromic Substring, LeetCode 647 Palindromic Substrings*

**■ Interview Trigger**

Expand from every center —  $2N-1$  centers total.  $O(N^2)$  without DP.

**18.1 Advanced String Problems**

These algorithms are pattern matchers and prefix analyzers. Understanding KMP and Z-function is essential for any string-heavy role. They eliminate redundant comparisons by preprocessing the pattern.

**18.1.1 · Hashing in Strings — Theory****WHAT IT'S ASKING**

Understand how polynomial rolling hash values are computed and used for fast string comparison.

**TOPIC / PATTERN**

Strings — Polynomial Rolling Hash

**CORE IDEA**

Assign each string a numeric hash so comparison is  $O(1)$  instead of  $O(N)$ . Use polynomial rolling hash:  $H(s) = s[0]*p^0 + s[1]*p^1 + \dots + s[n]*p^n \pmod{M}$ .

**KEY FORMULAS**

$\text{hash}(s) = \sum(s[i] * p^i) \pmod{M}$   $p = 31$  (lowercase) or  $53$  (mixed case)  $M = 10^9 + 7$  (large prime to reduce collisions) Substring  $\text{hash}[l..r] = (\text{hash}[r+1] - \text{hash}[l] * p^{(r-l+1)}) \pmod{M}$

**WHY USE IT**

- Compare two substrings in  $O(1)$  after  $O(N)$  preprocessing.
- Find all occurrences of pattern in text in  $O(N+M)$  on average.

**COMMON MISTAKES**

- Hash collision — use double hashing (two different mod values) for safety.
- Negative mod in some languages — always add  $M$  before taking mod.

**■ Interview Trigger**

Precompute prefix hashes in  $O(N)$  — then any substring hash is  $O(1)$ .

## 18.1.2 · Rabin-Karp Algorithm

### WHAT IT'S ASKING

Find all occurrences of a pattern in a text using rolling-hash based Rabin-Karp matching.

### TOPIC / PATTERN

Strings — Rolling Hash for Pattern Matching

### CORE IDEA

Compute hash of pattern. Slide a window of same length across text, computing rolling hash in  $O(1)$  per step. On hash match, verify with direct comparison.

### ALGORITHM

- 1. Compute hash of pattern  $P$  (length  $M$ ).
- 2. Compute hash of first window of text  $T[0..M-1]$ .
- 3. Slide window: remove leftmost char, add new rightmost char ( $O(1)$  rolling update).
- 4. If hashes match  $\rightarrow$  verify character by character (avoid false positives).

### ROLLING HASH UPDATE

$$\text{new\_hash} = (\text{old\_hash} - s[\text{left}] * p^{(M-1)}) * p + s[\text{right}]$$

### COMPLEXITY

| Approach   | Time            | Space  | Notes                                    |
|------------|-----------------|--------|------------------------------------------|
| Naive      | $O(N \times M)$ | $O(1)$ | Compare at every position                |
| Rabin-Karp | $O(N+M)$ avg    | $O(1)$ | $O(N \times M)$ worst if many collisions |

| Approach | Time     | Space  | Notes                             |
|----------|----------|--------|-----------------------------------|
| KMP      | $O(N+M)$ | $O(M)$ | Guaranteed linear — no collisions |

*Related: LeetCode 28 Find the Index of the First Occurrence in a String*

**■ Interview Trigger**

Roll the hash window —  $O(1)$  update per step, verify only on hash match.

## 18.1.3 · Z-Function

### WHAT IT'S ASKING

Compute the Z-array of a string, where  $Z[i]$  is the length of the longest substring starting at  $i$  that matches the string's prefix.

**TOPIC / PATTERN**

Strings — Z-Array for prefix matching

**DEFINITION**

$Z[i]$  = length of the longest substring starting at index  $i$  that is also a prefix of the string.  $Z[0]$  is undefined (or set to  $N$ ).

**ALGORITHM**

- 1. Maintain a window  $[L, R]$  — the rightmost Z-box (substring = prefix).
- 2. For each  $i$ : if  $i < R$ , use previously computed  $Z$  values to initialize  $Z[i]$ .
- 3. Extend  $Z[i]$  by naive matching from the known boundary.
- 4. Update  $[L, R]$  if the new Z-box extends further right.

**VISUALIZATION**

```
s = 'aabxaa'
Z = [_, 1, 0, 0, 2, 1]
Z[4]=2: s[4..5]='aa' matches prefix s[0..1]='aa'
```

**PATTERN MATCHING WITH Z-FUNCTION**

- Concatenate: combined = pattern + '\$' + text.
- Compute  $Z$  on combined. Positions where  $Z[i] == \text{len}(\text{pattern})$  are matches.

**COMPLEXITY**

| Approach   | Time   | Space  | Notes                               |
|------------|--------|--------|-------------------------------------|
| Z-Function | $O(N)$ | $O(N)$ | Linear — reuses computed $Z$ values |

*Related: [LeetCode 796 Rotate String](#), various competitive programming problems*

**■ Interview Trigger**

$Z[i]$  = longest prefix of  $s$  that starts at  $i$ . Use '\$' separator for pattern search.

**18.1.4 · KMP Algorithm / LPS Array****WHAT IT'S ASKING**

Find all occurrences of a pattern in a text in linear time using the KMP failure/LPS array.

**TOPIC / PATTERN**

Strings — Knuth-Morris-Pratt Pattern Matching

**CORE IDEA**

Preprocess the pattern to build an LPS (Longest Proper Prefix which is also Suffix) array. On mismatch, use LPS to skip redundant comparisons — never move text pointer backward.

**STEP 1 - BUILD LPS ARRAY**

- $LPS[0] = 0$  always.

- $len=0, i=1$ . While  $i < M$ :
- If  $pattern[i]==pattern[len]$ :  $LPS[i]=len+1, i++, len++$ .
- Else if  $len > 0$ :  $len = LPS[len-1]$  (fall back, don't increment  $i$ ).
- Else:  $LPS[i]=0, i++$ .

## STEP 2 - KMP SEARCH

- $i$  = text pointer,  $j$  = pattern pointer. Both start at 0.
- If  $text[i]==pattern[j]$ :  $i++, j++$ .
- If  $j==M$ : match found at index  $(i-j)$ . Reset  $j = LPS[j-1]$ .
- Else if mismatch: if  $j > 0 \rightarrow j = LPS[j-1]$ . Else  $i++$ .

### LPS VISUALIZATION

pattern = 'AABAAB' index: 0 1 2 3 4 5

**LPS: 0 1 0 1 2 3**

$LPS[5]=3$ : 'AAB' is both prefix and suffix of 'AABAAB'

### WHY IT WORKS

- On mismatch at  $j$ ,  $LPS[j-1]$  tells us the longest prefix of pattern that still matches — skip re-comparing it.

### COMPLEXITY

| Approach | Time            | Space  | Notes                                    |
|----------|-----------------|--------|------------------------------------------|
| Naive    | $O(N \times M)$ | $O(1)$ | Compare at every text position           |
| KMP      | $O(N+M)$        | $O(M)$ | Linear — LPS skips redundant comparisons |

### APPLICATIONS

- Pattern matching · String rotation check · Repeated substring detection · Shortest palindrome

*Related: [LeetCode 28 Implement strStr\(\)](#), [LeetCode 459 Repeated Substring Pattern](#), [LeetCode 214 Shortest Palindrome](#)*

#### ■ Interview Trigger

KMP never moves the text pointer back — LPS tells us how far to skip on mismatch.

## 18.1.5 / 18.1.6 · Shortest Palindrome and Longest Happy Prefix

### WHAT IT'S ASKING

Find the shortest palindrome formable by adding characters to the front of a string, and the longest prefix that is also a suffix (happy prefix).

### TOPIC / PATTERN

Strings — KMP LPS Array

### SHORTEST PALINDROME

Find the shortest palindrome by prepending characters to  $s$ . Equivalently: find the longest prefix of  $s$  that is a palindrome, then prepend the reverse of the remaining suffix.

#### ALGORITHM

- 1.  $combined = s + \# + reverse(s)$ .
- 2. Compute LPS of  $combined$ .
- 3.  $LPS[last] = \text{length of longest palindromic prefix of } s$ .
- 4.  $Answer = reverse(s[LPS[last]:]) + s$ .

#### LONGEST HAPPY PREFIX

A happy prefix is both a prefix and suffix (but not the whole string). This is exactly  $LPS[N-1]$  of the string. Just build the LPS array and return  $s[:LPS[N-1]]$ .

#### COMPLEXITY

| Approach | Time   | Space  | Notes                          |
|----------|--------|--------|--------------------------------|
| KMP LPS  | $O(N)$ | $O(N)$ | Both reduce to LPS computation |

Related: [LeetCode 214 Shortest Palindrome](#), [LeetCode 1392 Longest Happy Prefix](#)

#### ■ Interview Trigger

Shortest palindrome and longest happy prefix both reduce to one KMP LPS computation.

## 18.1.9 · Minimum Bracket Reversals

#### WHAT IT'S ASKING

Given a string of brackets, find the minimum number of reversals needed to make it balanced.

#### TOPIC / PATTERN

Strings — Stack / Counter to cancel matched pairs

#### CORE IDEA

Cancel all matched  $()$  pairs first. The remaining unmatched string is of the form  $\}}...\{{\{$  (some closing then some opening). Count how many reversals needed.

#### ALGORITHM

- 1. If  $N$  is odd  $\rightarrow$  impossible, return  $-1$ .
- 2. Use a counter to cancel matched pairs. Track unmatched open and close brackets.
- 3. After cancellation:  $open = \text{unmatched '(', close} = \text{unmatched ')'}$ .
- 4.  $Answer = \text{ceil}(open/2) + \text{ceil}(close/2)$ .

#### WHY?

Unmatched:  $)$  needs 1 reversal to become  $() \rightarrow \text{ceil}(2/2)=1$  Unmatched:  $(((($  needs 2 reversals:  $(((( \rightarrow ()()$ , then fix  $\rightarrow \text{ceil}(3/2)=2$

#### COMPLEXITY

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|               |      |      |                      |
|---------------|------|------|----------------------|
| Stack/Counter | O(N) | O(1) | Two-variable counter |
|---------------|------|------|----------------------|

*Related: GFG Minimum Number of Bracket Reversals*

#### ■ Interview Trigger

Cancel matched pairs first — remaining unmatched brackets need  $\text{ceil}(\text{open}/2) + \text{ceil}(\text{close}/2)$  reversals.

## 18.1.8 · Count and Say

### WHAT IT'S ASKING

Count and Say: generate the  $n$ th term of the sequence where each term describes the digits of the previous one.

### TOPIC / PATTERN

Strings — Simulation / Run-Length Encoding

### CORE IDEA

Each term describes the previous term by run-length encoding: count consecutive identical digits and say 'count then digit'.

### ALGORITHM

- 1. Start with result = '1'.
- 2. For each iteration up to  $n$ : scan result, count runs of same char.
- 3. Build next = concatenation of (count)(char) for each run.
- 4. result = next. Repeat.

### VISUALIZATION

```
n=1: '1'
n=2: one 1 → '11'
n=3: two 1s → '21'
n=4: one 2, one 1 → '1211'
n=5: one 1, one 2, two 1s → '111221'
```

### COMPLEXITY

| Approach   | Time            | Space  | Notes                                             |
|------------|-----------------|--------|---------------------------------------------------|
| Simulation | $O(N \times L)$ | $O(L)$ | $L$ = length of $n$ th term (grows exponentially) |

*Related: LeetCode 38 Count and Say*

#### ■ Interview Trigger

Read and encode runs: for each group of same chars, output count + char.

### STRINGS — MASTER CHEATSHEET

| Pattern | Trigger Keywords | Key Tool | Complexity |
|---------|------------------|----------|------------|
|---------|------------------|----------|------------|

|                     |                                               |                     |                    |
|---------------------|-----------------------------------------------|---------------------|--------------------|
| Frequency Count     | 'anagram', 'unique chars', 'most frequent'    | Array[26] / HashMap | O(N)               |
| Two Pointers        | 'palindrome', 'reverse', 'compare ends'       | L and R pointers    | O(N)               |
| Sliding Window      | 'substring with K chars', 'all chars present' | Window + freq map   | O(N)               |
| Expand from Center  | 'longest palindromic substring' (no DP)       | 2N-1 centers        | O(N <sup>2</sup> ) |
| Stack / Depth       | 'valid parentheses', 'nesting depth'          | Stack or counter    | O(N)               |
| Rolling Hash        | 'find all occurrences', 'pattern in text'     | Rabin-Karp          | O(N) avg           |
| Z-Function          | 'prefix match at every position'              | Z-array             | O(N)               |
| KMP / LPS           | 'pattern search', 'shortest palindrome'       | LPS array           | O(N+M)             |
| Sort Trick          | 'check rotation', 'common prefix'             | Concatenate / Sort  | O(N log N)         |
| Run-Length Encoding | 'compress string', 'count and say'            | Group consecutive   | O(N)               |

### KMP VS Z-FUNCTION — WHEN TO USE WHICH

KMP Z-Function

|             | KMP                                             | Z-Function                                      |
|-------------|-------------------------------------------------|-------------------------------------------------|
| Core output | LPS[i]: longest prefix=suffix in pattern[0..i]  | Z[i]: longest prefix of s starting at i         |
| Best for    | Pattern matching, shortest palindrome, repeated | Pattern matching, rotation check, string search |
| Separator   | pattern + '#' + text                            | pattern + '\$' + text                           |
| Match check | j == len(pattern)                               | Z[i] == len(pattern)                            |
| Complexity  | O(N+M) time, O(M) space                         | O(N) time, O(N) space                           |

Best for Pattern matching, shortest palindrome, repeated Pattern substring matching, rotation check, string search

### THE GOLDEN RULES OF STRING PROBLEMS

- 'Anagram / permutation' → frequency array of 26 chars.
- 'Palindrome check' → two pointers from ends OR expand from center.
- 'Substring with constraint' → sliding window + shrink on violation.
- 'Pattern in text, O(N)' → KMP or Z-function (never naive O(N×M)).
- 'Is rotation / does it contain as substring' → concatenate + KMP or Z.
- 'Compare substrings fast' → polynomial rolling hash.

DSA Revision Notes Striver's A2Z DSA Sheet

**ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING**

## 5.2.8 · Reverse Every Word in a String

### WHAT IT'S ASKING

Reverse the characters within each word of a string while keeping word order and spacing the same.

### TOPIC / PATTERN

Strings — In-place / Two Pointer per word

### CORE IDEA

Unlike reversing word ORDER, this reverses the CHARACTERS within each word while keeping word order and spacing intact.

### ALGORITHM

- 1. Split on spaces (or scan word boundaries in place).
- 2. For each word, reverse its characters using two pointers (start/end swap inward).
- 3. Rejoin words with single spaces (or write back in place if using two pointers directly on the string).

### COMPLEXITY

- Time:  $O(N)$
- Space:  $O(1)$  extra if done in place,  $O(N)$  if rebuilding

#### ■ Interview Trigger

Don't confuse with 5.1.2 (reverse word ORDER) — here each word's letters flip, order stays.

# DSA Revision Notes

## Striver's A2Z DSA Sheet

### Batch 5 · Steps 6 & 7: Linked List + Recursion

Problems 25 problems — Singly LL, Doubly LL, and Recursion patterns

Patterns Fast/Slow Pointers · Reversal · Recursion Backtracking · Subsequence Generation

Format Pattern · Intuition · Algorithm · Complexity · Traps · Trigger

Linked Lists test pointer manipulation. Recursion tests your ability to trust smaller subproblems.

#### TABLE OF CONTENTS

##### 6.1 / 6.2 LL FUNDAMENTALS

6.1 Singly Linked List — Node, Insert, Delete, Traverse

6.2 Doubly Linked List — Insert, Delete, Reverse

##### 6.3 MEDIUM PROBLEMS OF LL

6.3.1 Middle of Linked List (Tortoise-Hare)

6.3.2 Reverse a Linked List (Iterative & Recursive)

6.3.3 Detect a Loop in LL (Floyd's Cycle Detection)

6.3.4 Find Starting Point of the Loop

6.3.5 Length of Loop in LL

6.3.6 Check if LL is a Palindrome

6.3.7 Segregate Odd and Even Nodes

6.3.8 Remove Nth Node from the Back

6.3.9 Delete the Middle Node

6.3.10 Sort a Linked List (Merge Sort on LL)

6.3.11 Sort LL of 0s, 1s, 2s

6.3.12 Find Intersection Point of Two LLs

6.3.13 Add 1 to a Number Represented as LL

6.3.14 Add Two Numbers Represented as LL

##### 6.5 HARD PROBLEMS OF LL

6.5.1 Reverse LL in Groups of Size K

6.5.2 Rotate a Linked List

6.5.3 Flatten a Multilevel Linked List

6.5.4 Clone a Linked List with Random Pointer

##### 7.1 / 7.2 RECURSION FUNDAMENTALS

7.1.1 Recursive Implementation of atoi()

7.1.2 Pow(x, n) — Fast Exponentiation

7.2.1 Generate All Binary Strings

7.2.2 Generate Parentheses

7.2.3 Print All Subsequences / Power Set

7.2.4 Combination Sum

7.2.5 Subset Sum I & II

##### 7.3 HARD RECURSION / BACKTRACKING

- 7.3.1 Letter Combinations of a Phone Number
- 7.3.2 Palindrome Partitioning
- 7.3.3 Word Search
- 7.3.4 N-Queens
- 7.3.5 Rat in a Maze
- 7.3.6 Word Break
- 7.3.7 M-Coloring Problem
- 7.3.8 Sudoku Solver

## STEP 6 — Linked List

6.1 / 6.2 Fundamentals — Singly & Doubly LL

### 6.1 · Singly Linked List — Core Operations

#### WHAT IT'S ASKING

Implement insertion, deletion, and traversal on a singly linked list from scratch.

#### TOPIC / PATTERN

Linked List — Node-based structure, pointer manipulation

#### CORE IDEA

A Node has data + a pointer (next) to the following node. Unlike arrays, insertion/deletion doesn't require shifting — just pointer rewiring.

#### KEY OPERATIONS

- Insert at head: `newNode.next = head`, `head = newNode`.  $O(1)$ .
- Insert at tail: Traverse to last node, `lastNode.next = newNode`.  $O(N)$ .
- Delete head: `head = head.next`.  $O(1)$ .
- Delete by value: Traverse, track previous node, `prev.next = curr.next`.  $O(N)$ .
- Traversal: while `curr != null`: `process(curr)`, `curr = curr.next`.  $O(N)$ .

#### VISUALIZATION

```
head → [1] → [2] → [3] → [4] → null
Insert 0 at head: [0] → [1] → [2] → [3] → [4] → null
Delete node [2]: [0] → [1] → [3] → [4] → null
```

#### COMMON MISTAKES

- Losing reference to head/next before rewiring — always save next pointer BEFORE modifying.
- Not handling empty list or single-node list edge cases.

#### ■ Interview Trigger

LL = nodes + pointers. Always save the next pointer before you break the link.

### 6.2 · Doubly Linked List — Core Operations

#### WHAT IT'S ASKING

Implement insertion, deletion, and traversal on a doubly linked list from scratch.

**TOPIC / PATTERN**

Linked List — Bidirectional traversal with prev + next pointers

**CORE IDEA**

Each node has both next and prev pointers, enabling  $O(1)$  backward traversal and  $O(1)$  deletion when you already have the node reference.

**KEY OPERATIONS**

- Insert after node X: `newNode.next=X.next`, `newNode.prev=X`, `X.next.prev=newNode` (if exists), `X.next=newNode`.
- Delete node X: `X.prev.next = X.next`, `X.next.prev = X.prev` (handle null checks).
- Reverse DLL: Swap prev/next for every node, then swap head and tail.

**DLL VS SINGLY LL**

| Approach  | Time                 | Space                | Notes              |
|-----------|----------------------|----------------------|--------------------|
| Singly LL | $O(1)$ insert head   | $O(N)$ delete b      | y Lveasluse memory |
| Doubly LL | $O(1)$ insert/delete | gOiv(eNn) nsoedaerch | Extra prev pointer |

valuememory

Doubly LL  $O(1)$  insert/delete given  $O(N)$  node search Extra prev pointer

**COMMON MISTAKES**

- Forgetting to update BOTH prev and next pointers — DLL needs both directions consistent.

**■ Interview Trigger**

DLL = next AND prev. Every insert/delete must update both directions.

**6.3 Medium Problems of Linked List****6.3.1 · Middle of Linked List — Tortoise-Hare****WHAT IT'S ASKING**

Find the middle node of a linked list in one pass using slow and fast pointers.

**TOPIC / PATTERN**

Linked List — Fast & Slow Pointers

**CORE IDEA**

Move slow pointer by 1 step and fast pointer by 2 steps. When fast reaches the end, slow is at the middle.

**ALGORITHM**

- 1. `slow = head`, `fast = head`.
- 2. While `fast != null` and `fast.next != null`: `slow = slow.next`, `fast = fast.next.next`.
- 3. Return `slow` (middle node).

**VISUALIZATION**

```
[1]→[2]→[3]→[4]→[5]→null
slow, fast=1
step1: slow=2, fast=3
step2: slow=3, fast=5
fast.next=null → stop → middle = 3 ✓
```

**COMPLEXITY**

| Approach  | Time | Space | Notes                           |
|-----------|------|-------|---------------------------------|
| Two Pass  | O(N) | O(1)  | Count length, then traverse N/2 |
| Fast/Slow | O(N) | O(1)  | Single pass — optimal           |

*Related: LeetCode 876 Middle of the Linked List*

**■ Interview Trigger**

Slow moves 1, fast moves 2 — when fast hits the end, slow is at the middle.

**6.3.2 · Reverse a Linked List — Iterative & Recursive****WHAT IT'S ASKING**

Reverse a singly linked list, both iteratively and recursively.

**TOPIC / PATTERN**

Linked List — Pointer Reversal

**ITERATIVE ALGORITHM**

- 1. prev = null, curr = head.
- 2. While curr != null: next = curr.next, curr.next = prev, prev = curr, curr = next.
- 3. Return prev (new head).

**RECURSIVE ALGORITHM**

- 1. Base case: if head==null or head.next==null → return head.
- 2. newHead = reverse(head.next).
- 3. head.next.next = head. head.next = null.
- 4. Return newHead.

**VISUALIZATION — ITERATIVE**

```
null ← [1] [2]→[3]→[4]→null
prev curr
Step: next=2, curr.next=prev(null), prev=1, curr=2
null ← [1] ← [2] [3]→[4]→null ...continue until curr=null
```

**COMPLEXITY**

| Approach  | Time | Space | Notes                      |
|-----------|------|-------|----------------------------|
| Iterative | O(N) | O(1)  | Preferred — constant space |

|           |        |        |                      |
|-----------|--------|--------|----------------------|
| Recursive | $O(N)$ | $O(N)$ | Call stack depth = N |
|-----------|--------|--------|----------------------|

Related: [LeetCode 206 Reverse Linked List](#), [LeetCode 92 Reverse Linked List II](#)

#### ■ Interview Trigger

Three pointers: prev, curr, next — rewire curr.next to point backward.

## 6.3.3 / 6.3.4 / 6.3.5 · Detect Loop, Find Start, Length of Loop — Floyd's

### WHAT IT'S ASKING

Detect if a linked list has a cycle, find where the cycle begins, and find its length — using Floyd's algorithm.

Algorithm

### TOPIC / PATTERN

Linked List — Floyd's Cycle Detection (Tortoise and Hare)

### DETECT LOOP

- slow=head, fast=head. Move slow by 1, fast by 2. If they meet → loop exists.
- If fast reaches null → no loop.

### FIND STARTING POINT OF LOOP

After slow and fast meet inside the loop, reset one pointer to head. Move both at speed 1 — they will meet exactly at the loop's start.

- 1. Detect meeting point using Floyd's algorithm.
- 2. ptr1 = head, ptr2 = meeting point.
- 3. Move both by 1 step until ptr1 == ptr2 → that's the loop start.

### WHY THIS WORKS (MATH)

Let  $L$  = distance head to loop start Let  $C$  = loop length,  $x$  = distance from loop start to meeting point  
 slow traveled:  $L + x$  fast traveled:  $L + x + nC$  ( $n$  loops extra) Since  $\text{fast} = 2 \times \text{slow}$ :  $L + x + nC = 2(L + x) \rightarrow L = nC - x$   
 This means: distance from head to loop start equals distance from meeting point to loop start (mod  $C$ )

### LENGTH OF LOOP

- From the meeting point, keep moving one pointer until it returns to the same node. Count steps.

### COMPLEXITY

| Approach | Time   | Space  | Notes                    |
|----------|--------|--------|--------------------------|
| HashSet  | $O(N)$ | $O(N)$ | Track visited nodes      |
| Floyd's  | $O(N)$ | $O(1)$ | Optimal — no extra space |

Related: [LeetCode 141 Linked List Cycle](#), [LeetCode 142 Linked List Cycle II](#)

#### ■ Interview Trigger

Fast=2x slow. They meet inside the loop. Reset one to head, move both by 1 to find loop start.

### 6.3.6 · Check if Linked List is a Palindrome

#### WHAT IT'S ASKING

Check whether a linked list reads the same forwards and backwards.

#### TOPIC / PATTERN

Linked List — Fast/Slow + Reverse Second Half

#### ALGORITHM

- 1. Find the middle using fast/slow pointers.
- 2. Reverse the second half of the list.
- 3. Compare first half and reversed second half node by node.
- 4. (Optional) Restore the list by reversing second half back.

#### VISUALIZATION

```
[1]→[2]→[3]→[2]→[1]→null
Middle found at node [3]
Reverse second half: [1]→[2]→[3] & [1]→[2] (reversed)
Compare: 1==1, 2==2 → palindrome ✓
```

#### COMPLEXITY

| Approach          | Time | Space | Notes                               |
|-------------------|------|-------|-------------------------------------|
| Extra Array       | O(N) | O(N)  | Copy values, check array palindrome |
| Fast/Slow+Reverse | O(N) | O(1)  | Optimal — in-place                  |

*Related: [LeetCode 234 Palindrome Linked List](#)*

#### ■ Interview Trigger

Find middle, reverse second half, compare both halves — O(1) space.

### 6.3.7 · Segregate Odd and Even Nodes (by position)

#### WHAT IT'S ASKING

Rearrange a linked list so all nodes at odd positions come before all nodes at even positions.

#### TOPIC / PATTERN

Linked List — Two Separate Chains

#### ALGORITHM

- 1. Maintain two pointers: odd (1st,3rd,5th...) and even (2nd,4th,6th...).
- 2. Traverse, linking odd nodes together and even nodes together.
- 3. At the end, connect tail of odd chain to head of even chain.

#### COMPLEXITY

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|              |        |        |                              |
|--------------|--------|--------|------------------------------|
| Two pointers | $O(N)$ | $O(1)$ | Single pass, rewire in-place |
|--------------|--------|--------|------------------------------|

*Related: LeetCode 328 Odd Even Linked List*

#### ■ Interview Trigger

Build two chains by position parity, then merge odd-tail to even-head.

## 6.3.8 · Remove Nth Node from the Back

### WHAT IT'S ASKING

Remove the Nth node from the end of a linked list in one pass.

### TOPIC / PATTERN

Linked List — Fast/Slow with Gap

### ALGORITHM

- 1. fast = head. Move fast N steps ahead.
- 2. slow = head. Move both slow and fast together until fast.next == null.
- 3. slow is now just before the node to remove. slow.next = slow.next.next.

### VISUALIZATION (N=2)

```
[1]→[2]→[3]→[4]→[5]→null, N=2
fast moves 2 ahead: fast=3
Move both together: slow=1→2→3, fast=3→4→5
```

fast.next=null, slow=3 → remove slow.next(4) → [1]→[2]→[3]→[5] ✓

### COMPLEXITY

| Approach       | Time   | Space  | Notes                      |
|----------------|--------|--------|----------------------------|
| Two Pass       | $O(N)$ | $O(1)$ | Count length, then remove  |
| One Pass (Gap) | $O(N)$ | $O(1)$ | Optimal — single traversal |

### COMMON MISTAKES

– Off-by-one in the gap — use a dummy node before head to simplify edge case of removing the head itself.

*Related: LeetCode 19 Remove Nth Node From End of List*

#### ■ Interview Trigger

Fast starts N steps ahead — when fast hits the end, slow is right before the target.

## 6.3.9 · Delete the Middle Node

### WHAT IT'S ASKING

Delete the middle node of a linked list in one pass.

**TOPIC / PATTERN**

Linked List — Fast/Slow Pointers (track previous of slow)

**ALGORITHM**

- 1. slow=head, fast=head, prev=null.
- 2. While fast and fast.next: prev=slow, slow=slow.next, fast=fast.next.next.
- 3. prev.next = slow.next (skip the middle node).

**COMPLEXITY**

| Approach  | Time   | Space  | Notes       |
|-----------|--------|--------|-------------|
| Fast/Slow | $O(N)$ | $O(1)$ | Single pass |

*Related: LeetCode 2095 Delete the Middle Node of a Linked List*

**■ Interview Trigger**

Same middle-finding trick — just track the node before slow to unlink it.

**6.3.10 / 6.3.11 · Sort a Linked List (Merge Sort) and Sort 0s 1s 2s in LL****WHAT IT'S ASKING**

Sort a linked list (via merge sort), and specifically sort one that only contains 0s, 1s, and 2s.

**TOPIC / PATTERN**

Linked List — Merge Sort adapted for LL

**MERGE SORT ON LL**

- 1. Base case: if head==null or head.next==null → return head.
- 2. Find middle using fast/slow, split into two halves.
- 3. Recursively sort each half.
- 4. Merge two sorted LLs (like merging two sorted arrays, but with pointers).

**WHY MERGE SORT (NOT QUICK SORT) FOR LL?**

- LL has  $O(1)$  insertion but no random access — merge sort's sequential merge fits naturally. Quick sort

needs random access for efficient partitioning.

**SORT 0S, 1S, 2S (BY CHANGING LINKS, NOT VALUES)**

- Maintain 3 dummy heads: zeroHead, oneHead, twoHead. Traverse once, append each node to the appropriate chain. Link the three chains together.

**COMPLEXITY**

| Approach      | Time          | Space       | Notes                            |
|---------------|---------------|-------------|----------------------------------|
| Merge Sort LL | $O(N \log N)$ | $O(\log N)$ | Stack space for recursion        |
| Sort 0-1-2 LL | $O(N)$        | $O(1)$      | Three-way partition, single pass |

Related: [LeetCode 148 Sort List](#)

#### ■ Interview Trigger

LL favors merge sort (sequential access). Sort 0-1-2: three dummy chains, one pass.

## 6.3.12 · Find Intersection Point of Two Linked Lists

### WHAT IT'S ASKING

Given two linked lists that merge at some node, find that intersection node.

### TOPIC / PATTERN

Linked List — Two Pointers with Length Difference Equalization

### CORE IDEA

If two lists intersect, the part after intersection is identical. Make both pointers travel the SAME total distance by switching lists at the end — this naturally compensates for the length difference.

### ALGORITHM

- 1.  $p1 = \text{headA}$ ,  $p2 = \text{headB}$ .
- 2. While  $p1 \neq p2$ :  $p1 = p1.\text{next}$  or headB if  $p1$  reaches null. Same for  $p2$  with headA.
- 3. Return  $p1$  (intersection node, or null if no intersection).

### WHY IT WORKS

Length A =  $a$ , Length B =  $b$ , common suffix =  $c$   
 $p1$  travels:  $a + (b - c) = a + b - c$   
 $p2$  travels:  $b + (a - c) = a + b - c$   
 Equal total distance → they meet exactly at intersection!

### COMPLEXITY

| Approach         | Time     | Space  | Notes                                      |
|------------------|----------|--------|--------------------------------------------|
| HashSet          | $O(N+M)$ | $O(N)$ | Store all nodes of list A, check list B    |
| Length Diff      | $O(N+M)$ | $O(1)$ | Compute lengths, advance longer list first |
| Two Pointer Swap | $O(N+M)$ | $O(1)$ | Optimal — elegant, no length calc needed   |

Related: [LeetCode 160 Intersection of Two Linked Lists](#)

#### ■ Interview Trigger

Switch lists when you hit null — both pointers cover equal total distance, meeting at intersection.

## 6.3.13 / 6.3.14 · Add 1 to LL Number and Add Two LL Numbers

### WHAT IT'S ASKING

Add 1 to a number represented as a linked list, and add two numbers each represented as a linked list.

### TOPIC / PATTERN

Linked List — Reverse + Carry Propagation, or Recursive Carry

#### ADD 1 TO A NUMBER (DIGITS STORED AS LL)

- 1. Reverse the list (so least significant digit is first).
- 2. Add 1, propagate carry through the list.
- 3. If carry remains after the last node, append a new node.
- 4. Reverse back.

#### ADD TWO NUMBERS (TWO LLS, DIGITS IN LISTS)

- 1. Traverse both lists simultaneously, adding corresponding digits + carry.
- 2. Create new nodes for the result list. Continue until both lists exhausted and carry is 0.

#### COMPLEXITY

| Approach    | Time           | Space            | Notes                        |
|-------------|----------------|------------------|------------------------------|
| Add 1 to LL | $O(N)$         | $O(1)$ or $O(N)$ | $O(1)$ if reversing in-place |
| Add Two LLS | $O(\max(N,M))$ | $O(\max(N,M))$   | New list for result          |

*Related: LeetCode 2 Add Two Numbers, LeetCode 369 Plus One Linked List*

#### ■ Interview Trigger

Reverse to align least-significant digits, add with carry, reverse back.

### 6.5 Hard Problems of Linked List

## 6.5.1 · Reverse LL in Groups of Size K

#### WHAT IT'S ASKING

Reverse a linked list in groups of K nodes at a time.

#### TOPIC / PATTERN

Linked List — Iterative reversal applied per group, recursively chained

#### ALGORITHM

- 1. Check if there are at least K nodes remaining (else return head unchanged).
- 2. Reverse the first K nodes (standard iterative reversal, stop after K).
- 3. Recursively call the function on the remaining list — get newHead of next group.
- 4. Connect: original head (now the tail of reversed group) .next = newHead from recursion.
- 5. Return the new head of this group (the Kth node, now at front).

#### VISUALIZATION (K=3)

```
[1]→[2]→[3]→[4]→[5]→[6]→null
Reverse first 3: [3]→[2]→[1] then [4]→[5]→[6]→null
Recursively reverse next 3: [6]→[5]→[4]→null
Connect: [3]→[2]→[1]→[6]→[5]→[4]→null ✓
```

#### COMPLEXITY

| Approach  | Time   | Space    | Notes                                   |
|-----------|--------|----------|-----------------------------------------|
| Recursive | $O(N)$ | $O(N/K)$ | Stack depth = number of groups          |
| Iterative | $O(N)$ | $O(1)$   | Use a dummy node + loop — optimal space |

Related: [LeetCode 25 Reverse Nodes in k-Group](#)

#### ■ Interview Trigger

Check K nodes exist → reverse them → recurse on rest → stitch tail to recursive head.

## 6.5.2 · Rotate a Linked List

### WHAT IT'S ASKING

Rotate a linked list to the right by K places.

### TOPIC / PATTERN

Linked List — Make Circular, then Break at New Point

### ALGORITHM

- 1. Find length N and the tail node.
- 2. Connect tail.next = head (make it circular).
- 3.  $k = k \% N$ . Find new tail at position  $(N - k - 1)$  from old head.
- 4. newHead = newTail.next. newTail.next = null (break the circle).

### VISUALIZATION (K=2)

```
[1]→[2]→[3]→[4]→[5]→null, k=2
Make circular: [1]→[2]→[3]→[4]→[5]→[1]...
New tail at position (5-2-1)=2 → node [3]
Break after [3]: [4]→[5]→[1]→[2]→[3]→null ✓
```

### COMPLEXITY

| Approach       | Time   | Space  | Notes                                      |
|----------------|--------|--------|--------------------------------------------|
| Circular trick | $O(N)$ | $O(1)$ | Two passes — find length, then break point |

Related: [LeetCode 61 Rotate List](#)

#### ■ Interview Trigger

Make the list circular, then break it at the correct new tail position.

## 6.5.3 · Flatten a Multilevel Linked List

### WHAT IT'S ASKING

Flatten a multilevel linked list where nodes can have both a next and a child pointer, into a single-level sorted list.

### TOPIC / PATTERN

Linked List — Recursive Merge (similar to merging two sorted LLs by child pointer)

### CORE IDEA

Each node may have a 'child' pointer to another sorted sub-list. Recursively flatten from the bottom-right, merging each level's list with the next.

### ALGORITHM

- 1. Base case: if head==null or head.next==null → return head.
- 2. Recursively flatten head.next first.
- 3. Merge head's own child list (if any) with the flattened head.next using a merge-two-sorted-list routine.
- 4. Return the merged head.

### COMPLEXITY

| Approach        | Time            | Space  | Notes                                           |
|-----------------|-----------------|--------|-------------------------------------------------|
| Recursive Merge | $O(N \times M)$ | $O(N)$ | N=main list, M=avg child list size; stack space |

*Related: InterviewBit Flatten a Linked List, similar to LeetCode 430 Flatten Multilevel DLL*

#### ■ Interview Trigger

Flatten right-to-left recursively — merge each node's child list into the result.

## 6.5.4 · Clone a Linked List with Random Pointer

### WHAT IT'S ASKING

Deep-clone a linked list where each node has an extra pointer that can point to any node in the list (or null).

### TOPIC / PATTERN

Linked List — HashMap mapping, or  $O(1)$  space Interleaving Trick

#### APPROACH 1 — HASHMAP

- Pass 1: create a copy of each node, map original→copy.
- Pass 2: set copy.next and copy.random using the map.

#### APPROACH 2 — $O(1)$ SPACE (INTERLEAVING)

- 1. Insert each cloned node right after its original (interleave):  $A \rightarrow A' \rightarrow B \rightarrow B' \dots$
- 2. Set random pointers for cloned nodes:  $A'.random = A.random.next$  (the clone of A.random).
- 3. Separate the two lists: restore original list's next pointers, extract clone list.

### VISUALIZATION

```
Original: [A]→[B]→[C]
Interleaved: [A]→[A']→[B]→[B']→[C]→[C']
A.random=C → A'.random = A.random.next = C' ✓
Separate: [A]→[B]→[C] and [A']→[B']→[C']
```

**COMPLEXITY**

| Approach     | Time | Space | Notes                    |
|--------------|------|-------|--------------------------|
| HashMap      | O(N) | O(N)  | Simple, two passes       |
| Interleaving | O(N) | O(1)  | Optimal — no extra space |

*Related: [LeetCode 138 Copy List with Random Pointer](#)*

**■ Interview Trigger**

Interleave clone nodes with originals — random pointers transfer via next of random.

**STEP 7 - Recursion****7.1 Get a Strong Hold****7.1.2 · Pow(x, n) — Fast Exponentiation****WHAT IT'S ASKING**

Implement `pow(x, n)` — compute  $x$  raised to  $n$  efficiently using fast exponentiation.

**TOPIC / PATTERN**

Recursion — Divide and Conquer (Binary Exponentiation)

**CORE IDEA**

$x^n = (x^{(n/2)})^2$  if  $n$  is even, or  $x \times (x^{(n/2)})^2$  if  $n$  is odd. This halves the problem each time instead of multiplying  $n$  times.

**ALGORITHM**

- 1. Base case: if  $n == 0 \rightarrow$  return 1.
- 2.  $half = pow(x, n/2)$ .
- 3. If  $n$  is even  $\rightarrow$  return  $half \times half$ .
- 4. Else  $\rightarrow$  return  $x \times half \times half$ .
- 5. Handle negative  $n$ : return  $1 / pow(x, -n)$ .

**COMPLEXITY**

| Approach            | Time     | Space    | Notes                    |
|---------------------|----------|----------|--------------------------|
| Naive Loop          | O(N)     | O(1)     | Multiply $n$ times       |
| Fast Exponentiation | O(log N) | O(log N) | Halve exponent each call |

**COMMON MISTAKES**

- Not handling negative exponents or  $x=0$  with negative  $n$  (division by zero).

Related: LeetCode 50 Pow(x, n)

#### ■ Interview Trigger

Halve the exponent each call — square the result, multiply extra x if odd.

## 7.2 · Subsequence Pattern — Universal Template

### WHAT IT'S ASKING

Learn the take/not-take template used to generate all subsequences of an array recursively.

### TOPIC / PATTERN

Recursion — Take/Not-Take (Pick/Skip) Decision Tree

### CORE IDEA

At every index, you have exactly 2 choices: include this element in the subsequence, or skip it. This branches into  $2^N$  total subsequences.

Universal Take/Not-Take Template

```
def solve(index, current):
    if index == N: process(current) # base case
    return # Choice 1: take arr[index]
    current.append(arr[index])
    solve(index+1, current)
    current.pop() # backtrack
    # Choice 2: skip arr[index]
    solve(index+1, current)
```

### RECURSION TREE (N=2, ELEMENTS [A,B])

solve(0, []) / \ take A: [A] skip A: [] / \ / \ take B: [A,B] skip B: [A] take B: [B] skip B: []

### COMPLEXITY

| Approach      | Time     | Space  | Notes                                |
|---------------|----------|--------|--------------------------------------|
| Take/Not-Take | $O(2^N)$ | $O(N)$ | $2^N$ leaves, $O(N)$ recursion depth |

– ALWAYS backtrack (undo the take) before exploring the skip branch — this is what makes it work.

#### ■ Interview Trigger

Every index: take it or skip it. This single template solves all subsequence problems.

### 7.2.1 · Generate All Binary Strings (No Consecutive 1s)

#### WHAT IT'S ASKING

Generate all binary strings of length N that contain no two consecutive 1s.

#### TOPIC / PATTERN

Recursion — Take/Not-Take with constraint

#### ALGORITHM

- At each position, try placing '0' (always allowed) and '1' (only if previous char wasn't '1').
- Base case: length == N → add to result.

#### COMPLEXITY

| Approach     | Time               | Space  | Notes                            |
|--------------|--------------------|--------|----------------------------------|
| Backtracking | $O(\text{Fib}(N))$ | $O(N)$ | Bounded by Fibonacci-like growth |

#### ■ Interview Trigger

Place 0 freely, place 1 only if last char wasn't 1 — prune invalid branches early.

## 7.2.2 · Generate Parentheses

### WHAT IT'S ASKING

Generate all combinations of  $N$  pairs of well-formed (balanced) parentheses.

### TOPIC / PATTERN

Recursion / Backtracking — Constraint Tracking

### CORE IDEA

Track count of open and close brackets used. Can add '(' if  $\text{open} < N$ . Can add ')' only if  $\text{close} < \text{open}$  (never exceed open brackets).

### ALGORITHM

- 1. solve(current, open, close):
- 2. If  $\text{len}(\text{current}) == 2N$ : add to result, return.
- 3. If  $\text{open} < N$ : solve(current+'(', open+1, close).
- 4. If  $\text{close} < \text{open}$ : solve(current+')', open, close+1).

### COMPLEXITY

| Approach     | Time                      | Space  | Notes                          |
|--------------|---------------------------|--------|--------------------------------|
| Backtracking | $O(4^N / \text{sqrt}(N))$ | $O(N)$ | Catalan number of valid combos |

Related: [LeetCode 22 Generate Parentheses](#)

#### ■ Interview Trigger

Add '(' if  $\text{open} < N$ . Add ')' only if  $\text{close} < \text{open}$ . Never let close exceed open.

## 7.2.3 · Print All Subsequences / Power Set

### WHAT IT'S ASKING

Print all subsequences of an array, i.e. its power set.

### TOPIC / PATTERN

Recursion — Take/Not-Take (direct application of the template)

### ALGORITHM

- Use the universal take/not-take template. At  $\text{index} == N$ , add current subsequence to result.

**HANDLING DUPLICATES (SUBSET SUM II STYLE)**

- Sort the array first. Skip duplicate elements at the SAME recursion level (only the first occurrence at each

level branches into 'take').

**COMPLEXITY**

| Approach      | Time              | Space  | Notes                              |
|---------------|-------------------|--------|------------------------------------|
| Take/Not-Take | $O(2^N \times N)$ | $O(N)$ | $2^N$ subsets, $O(N)$ to copy each |

*Related: LeetCode 78 Subsets, LeetCode 90 Subsets II*

**■ Interview Trigger**

Power set = take/not-take template applied directly, no extra constraint.

**7.2.6 / 7.2.8 / 7.2.9 · Combination Sum, Subset Sum I & II****WHAT IT'S ASKING**

Find all unique combinations/subsets of numbers that sum to a given target, with and without reusing elements.

**TOPIC / PATTERN**

Recursion — Take/Not-Take with Sum Target Tracking

**COMBINATION SUM (ELEMENTS REUSABLE)**

- At each index: take  $\text{arr}[i]$  and stay at  $i$  (can reuse), or move to  $i+1$  (skip).
- Base case:  $\text{target} == 0 \rightarrow$  valid combo found.  $\text{target} < 0 \rightarrow$  prune.

**COMBINATION SUM II (ELEMENTS NOT REUSABLE, HAS DUPLICATES)**

- Sort array. At each level, skip duplicate values to avoid duplicate combinations.
- Move to  $i+1$  after taking (no reuse).

**SUBSET SUM (COUNT OR CHECK EXISTENCE)**

- Same take/not-take, but track running sum instead of building the list explicitly (if only count/existence needed).

**COMPLEXITY**

| Approach           | Time     | Space  | Notes                                         |
|--------------------|----------|--------|-----------------------------------------------|
| Combination Sum    | $O(2^T)$ | $O(T)$ | $T$ = target value, exponential in worst case |
| Combination Sum II | $O(2^N)$ | $O(N)$ | $N$ = array length, with duplicate pruning    |

**COMMON MISTAKES**

- Combination Sum I: forgetting elements can repeat — must NOT move to  $i+1$  after taking.

- Combination Sum II: not skipping duplicates at the same level → duplicate combos in output.

*Related: LeetCode 39 Combination Sum, LeetCode 40 Combination Sum II, LeetCode 216 Combination Sum III*

#### ■ Interview Trigger

Reusable elements → stay at same index. No reuse → move forward + skip duplicates.

### 7.3 Trying Out All Combos — Hard Backtracking

The Backtracking Template: Choose → Explore → Un-choose. Every hard recursion problem follows this exact 3-step rhythm.

## 7.3.1 · Letter Combinations of a Phone Number

### WHAT IT'S ASKING

Given a phone keypad, return all possible letter combinations a given digit string could represent.

### TOPIC / PATTERN

Backtracking — Cartesian product of choices per digit

### ALGORITHM

- Map each digit to its letters (2→'abc', 3→'def', etc.).
- solve(index, current): if index==len(digits) → add current to result.
- For each letter mapped to digits[index]: solve(index+1, current+letter).

### COMPLEXITY

| Approach     | Time     | Space  | Notes                             |
|--------------|----------|--------|-----------------------------------|
| Backtracking | $O(4^N)$ | $O(N)$ | Max 4 letters per digit (7 and 9) |

*Related: LeetCode 17 Letter Combinations of a Phone Number*

#### ■ Interview Trigger

Each digit maps to letters — backtrack through Cartesian product of all digits.

## 7.3.2 · Palindrome Partitioning

### WHAT IT'S ASKING

Partition a string into all possible groups of palindromic substrings.

### TOPIC / PATTERN

Backtracking — Partition points + palindrome check

### ALGORITHM

1. solve(start, path): if start==N → valid partition, add path to result.
2. For end = start to N-1: if s[start..end] is palindrome:
3. path.append(s[start..end]), solve(end+1, path), path.pop() (backtrack).

### COMPLEXITY

| Approach     | Time              | Space  | Notes                                           |
|--------------|-------------------|--------|-------------------------------------------------|
| Backtracking | $O(2^N \times N)$ | $O(N)$ | Exponential partitions, $O(N)$ palindrome check |

**OPTIMIZATION**

- Precompute `palindrome[i][j]` using DP to make each check  $O(1)$  instead of  $O(N)$ .

*Related: [LeetCode 131 Palindrome Partitioning](#), [LeetCode 132 Palindrome Partitioning II](#)*

**■ Interview Trigger**

Try every prefix as a palindrome cut — recurse on the remainder, backtrack.

### 7.3.3 · Word Search

**WHAT IT'S ASKING**

Search a 2D grid of letters for a given word, moving to adjacent cells without reusing a cell.

**TOPIC / PATTERN**

Backtracking — DFS on grid with visited marking

**ALGORITHM**

1. For each cell, if it matches `word[0]`, start DFS.
2. `DFS(row, col, index)`: if `index==len(word)` → found.
3. Check bounds, visited, and character match. Mark visited (or use a sentinel).
4. Explore all 4 directions recursively. Unmark (backtrack) after exploring.

**COMPLEXITY**

| Approach         | Time                       | Space  | Notes                                   |
|------------------|----------------------------|--------|-----------------------------------------|
| DFS Backtracking | $O(M \times N \times 4^L)$ | $O(L)$ | $L$ =word length, 4 directions per cell |

**COMMON MISTAKES**

- Forgetting to unmark visited cells after backtracking — corrupts other search paths.

*Related: [LeetCode 79 Word Search](#), [LeetCode 212 Word Search II \(uses Trie\)](#)*

**■ Interview Trigger**

DFS in 4 directions, mark visited, backtrack by unmarking — classic grid search.

### 7.3.4 · N-Queens

**WHAT IT'S ASKING**

Place  $N$  queens on an  $N \times N$  chessboard so that no two queens attack each other.

**TOPIC / PATTERN**

Backtracking — Place one queen per row, validate column/diagonal

#### ALGORITHM

- 1. solve(row): if row==N → valid solution found, save board.
- 2. For col = 0 to N-1: if isValid(row, col):
- 3. Place queen at (row, col). solve(row+1). Remove queen (backtrack).

#### INVALID CHECK (O(1) WITH HASHING)

- Track 3 sets: usedCols, usedDiagonals (row-col), usedAntiDiagonals (row+col).
- Position (row,col) is safe if col, (row-col), and (row+col) are all unused.

#### VISUALIZATION

```
4-Queens example placement:
. Q . .
. . . Q
Q . . .
. . Q .
```

#### COMPLEXITY

| Approach          | Time             | Space    | Notes                             |
|-------------------|------------------|----------|-----------------------------------|
| Naive Validation  | $O(N! \times N)$ | $O(N^2)$ | Check entire board each placement |
| Hashed Validation | $O(N!)$          | $O(N)$   | $O(1)$ validity check using sets  |

Related: [LeetCode 51 N-Queens](#), [LeetCode 52 N-Queens II](#)

#### ■ Interview Trigger

One queen per row — track columns and both diagonals with sets for  $O(1)$  validation.

## 7.3.5 · Rat in a Maze

#### WHAT IT'S ASKING

Find a path for a rat from the top-left to the bottom-right of a maze, moving only through open cells.

#### TOPIC / PATTERN

Backtracking — DFS with path tracking on grid

#### ALGORITHM

- 1. solve(row, col, path): if (row,col) is destination → save path.
- 2. If out of bounds, blocked, or visited → return.
- 3. Mark visited. Try all 4 directions (often in D,L,R,U order for lexicographic results).
- 4. Append direction to path, recurse, pop from path (backtrack). Unmark visited.

#### COMPLEXITY

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|                  |                       |          |                                            |
|------------------|-----------------------|----------|--------------------------------------------|
| DFS Backtracking | $O(4^{(N \times N)})$ | $O(N^2)$ | Exponential worst case, visited grid space |
|------------------|-----------------------|----------|--------------------------------------------|

Related: [GFG Rat in a Maze](#), [LeetCode 980 Unique Paths III](#)

#### ■ Interview Trigger

DFS with a visited grid — explore all 4 directions, backtrack by unmarking.

## 7.3.6 · Word Break

### WHAT IT'S ASKING

Determine if a given string can be segmented into a sequence of words from a dictionary.

### TOPIC / PATTERN

Recursion + Memoization (pure recursion is exponential — needs DP)

### ALGORITHM

- 1. solve(start): if start==N → return True (memoize with a cache on 'start').
- 2. For end = start+1 to N: if s[start..end] in wordSet and solve(end) → return True.
- 3. Return False if no valid split found.

### COMPLEXITY

| Approach       | Time     | Space  | Notes                                     |
|----------------|----------|--------|-------------------------------------------|
| Pure Recursion | $O(2^N)$ | $O(N)$ | Exponential — recomputes same subproblems |
| Memoized       | $O(N^2)$ | $O(N)$ | Cache results per starting index          |

Related: [LeetCode 139 Word Break](#), [LeetCode 140 Word Break II](#)

#### ■ Interview Trigger

Try every split point — memoize on starting index to avoid recomputation.

## 7.3.7 · M-Coloring Problem

### WHAT IT'S ASKING

Color a graph's vertices with at most M colors so that no two adjacent vertices share a color.

### TOPIC / PATTERN

Backtracking — Graph coloring, validate neighbor colors

### ALGORITHM

- 1. solve(node): if node==N → all colored validly, return True.
- 2. For color = 1 to M: if isValid(node, color) (no adjacent node has same color):
- 3. Assign color[node]=color. If solve(node+1) → return True. Else unassign (backtrack).

**COMPLEXITY**

| Approach     | Time     | Space  | Notes                                      |
|--------------|----------|--------|--------------------------------------------|
| Backtracking | $O(M^N)$ | $O(N)$ | M colors, N nodes — exponential worst case |

*Related: GFG M-Coloring Problem, related to Graph Bipartite Check (M=2 case)*

**■ Interview Trigger**

Try each color per node, validate against neighbors, backtrack on failure.

**7.3.8 · Sudoku Solver****WHAT IT'S ASKING**

Fill a partially-filled 9×9 Sudoku board so it satisfies all Sudoku rules.

**TOPIC / PATTERN**

Backtracking — Fill empty cells, validate row/col/box

**ALGORITHM**

- 1. Find next empty cell (row, col).
- 2. For digit = 1 to 9: if isValid(row, col, digit):
- 3. Place digit. Recursively solve rest. If success → return True.
- 4. Else remove digit (backtrack), try next digit.
- 5. If no digit works → return False (triggers backtrack at caller).

**ISVALID CHECK**

- Check digit not already in same row, same column, and same 3×3 box.
- Box index:  $(row/3)*3 + col/3$  region boundaries: rows  $[3*(r/3), 3*(r/3)+2]$ , similarly for cols.

**COMPLEXITY**

| Approach     | Time                  | Space           | Notes                                                      |
|--------------|-----------------------|-----------------|------------------------------------------------------------|
| Backtracking | $O(9^{(N \times N)})$ | $O(N \times N)$ | Exponential worst case, very fast in practice with pruning |

**COMMON MISTAKES**

- Continuing to search other cells after a full board fill — return True immediately to stop all recursion.

*Related: LeetCode 37 Sudoku Solver, LeetCode 36 Valid Sudoku*

**■ Interview Trigger**

Fill cell by cell, validate row/col/box, backtrack on dead end — classic CSP backtracking.

Addendum — 12 Missing Problems (Restored) Step 6 (Linked List) was missing all 3 problems from 6.4 Medium Problems of DLL. Step 7 (Recursion) was missing 3 problems from 7.1, 4 from 7.2, and 2 from 7.3. Added back below in their correct subsections.

## 6.4 Medium Problems of DLL (Addendum)

**6.4.1 · Delete All Occurrences of a Key in DLL****WHAT IT'S ASKING**

Delete every node in a doubly linked list that has a given value.

**TOPIC / PATTERN**

Doubly Linked List — unlink matching nodes in a single pass

**ALGORITHM**

- Traverse from head. For every node whose value equals the key, unlink it: set `node.prev.next = node.next`

(if `node.prev` exists, else advance head), and `node.next.prev = node.prev` (if `node.next` exists).

- Advance to the next node (saved before unlinking) and repeat until the end of the list.

**COMPLEXITY**

| Approach           | Time   | Space  | Notes                                                         |
|--------------------|--------|--------|---------------------------------------------------------------|
| Single pass unlink | $O(N)$ | $O(1)$ | No extra list needed — DLL supports $O(1)$ unlinking per node |

unlinking per node

*Related: Delete a node in DLL, Remove duplicates from sorted DLL*

**■ Interview Trigger**

Deleting nodes matching a value in a DLL: always unlink using BOTH prev and next pointers, and special-case when the node being removed is the head.

**6.4.2 · Find Pairs with Given Sum in DLL****WHAT IT'S ASKING**

Given an unsorted doubly linked list and a target sum, find all pairs of nodes that add up to it.

**TOPIC / PATTERN**

Doubly Linked List — two-pointer on a sorted DLL

**ALGORITHM**

- Works only when the DLL is sorted. Find `left = head` and `right = the last node` (traverse once, or maintain a

tail pointer).

- While `left != right` and `right.next != left`: let `sum = left.val + right.val`.
- If `sum == target`: record the pair, move `left = left.next` and `right = right.prev`.
- If `sum < target`: move `left = left.next` (need a larger sum).
- If `sum > target`: move `right = right.prev` (need a smaller sum).

**WHY A DLL HELPS**

- A singly linked list would need to reverse itself (or use extra space) to walk backward from the tail; a DLL's

prev pointers give that backward walk for free, mirroring the classic sorted-array two-pointer trick.

#### COMPLEXITY

| Approach    | Time   | Space  | Notes                                 |
|-------------|--------|--------|---------------------------------------|
| Two-pointer | $O(N)$ | $O(1)$ | Requires the DLL to already be sorted |

*Related: 2Sum Problem (Step 3), Two Sum In BST*

#### ■ Interview Trigger

“Pair sum” on a SORTED structure with backward traversal available → two-pointer from both ends, not a hashset.

## 6.4.3 · Remove Duplicates from Sorted DLL

### WHAT IT'S ASKING

Remove duplicate values from a sorted doubly linked list.

### TOPIC / PATTERN

Doubly Linked List — adjacent-duplicate removal

### ALGORITHM

- Because the list is sorted, all duplicates of a value are adjacent — a single forward pass suffices.
- `current = head`. While `current` and `current.next`: if `current.val == current.next.val`, unlink `current.next` (`current.next = current.next.next`; fix the new `current.next.prev` if it exists) and do NOT advance `current`.
- Otherwise advance `current = current.next`.

### COMPLEXITY

| Approach    | Time   | Space  | Notes                                                              |
|-------------|--------|--------|--------------------------------------------------------------------|
| Single pass | $O(N)$ | $O(1)$ | Same shape as removing duplicates from a sorted singly linked list |

sorted singly linked list

*Related: Remove duplicates from Sorted array (Step 3)*

#### ■ Interview Trigger

“Sorted” + “remove duplicates” is always an adjacent-comparison single pass — never needs a hashset.

## 7.1 Get a Strong Hold (Addendum)

### 7.1.3 · Count Good Numbers

**WHAT IT'S ASKING**

Count how many 'good' numbers of length  $N$  exist, where even-index digits must be even and odd-index digits must be prime, computed efficiently with fast exponentiation.

**TOPIC / PATTERN**

Recursion — combinatorics + fast exponentiation

**CORE IDEA**

– A digit string of length  $n$  is “good” if every even index (0-indexed) holds an even digit  $\{0,2,4,6,8\}$  and every odd index holds a prime digit  $\{2,3,5,7\}$ .

– Count of even indices =  $\text{ceil}(n/2)$ , count of odd indices =  $\text{floor}(n/2)$ .

– Total good numbers =  $5^{\text{ceil}(n/2)} \times 4^{\text{floor}(n/2)} \pmod{1e9+7}$ .

**ALGORITHM**

– Compute both powers using recursive fast exponentiation (binary exponentiation), not a loop of  $n$  multiplications —  $n$  can be up to  $10^{15}$ .

**COMPLEXITY**

| Approach            | Time        | Space                       | Notes                                             |
|---------------------|-------------|-----------------------------|---------------------------------------------------|
| Fast exponentiation | $O(\log n)$ | $O(\log n)$ recursion depth | Same recursive halving idea as $\text{Pow}(x, n)$ |

recursion depth

*Related:  $\text{Pow}(x, n)$*

**■ Interview Trigger**

Huge exponent ( $n$  up to  $10^{15}$ ) → always binary/fast exponentiation, never a direct loop.

**7.1.4 · Sort a Stack Using Recursion****WHAT IT'S ASKING**

Sort a stack so it's in ascending order, using only recursion (no extra data structure).

**TOPIC / PATTERN**

Recursion as implicit storage — no extra array/stack allowed

**ALGORITHM**

–  $\text{sortStack}(\text{stack})$ : if stack is empty, return. Pop the top element, recursively call  $\text{sortStack}$  on the remaining stack.

– Then call a helper  $\text{insertSorted}(\text{stack}, \text{element})$ : if stack is empty OR  $\text{stack.top()} \leq \text{element}$ , push element and return.

– Otherwise pop the top, recursively  $\text{insertSorted}$  with the same element, then push the popped value back

— this bubbles the element to its correct sorted position.

#### COMPLEXITY

| Approach            | Time     | Space  | Notes                                                      |
|---------------------|----------|--------|------------------------------------------------------------|
| Recursive insertion | $O(N^2)$ | $O(N)$ | $O(N)$ call-stack depth acts as the only auxiliary storage |

auxiliary storage

*Related: Reverse a stack using recursion*

#### ■ Interview Trigger

“No extra data structure” + stack/recursion in the same sentence → use the call stack itself as storage: pop, recurse, then insert-back-in-order.

## 7.1.5 · Reverse a Stack Using Recursion

### WHAT IT'S ASKING

Reverse the elements of a stack using only recursion (no extra stack).

### TOPIC / PATTERN

Recursion as implicit storage — same trick as sorting a stack

### ALGORITHM

- reverseStack(stack): if stack is empty, return. Pop the top element, recursively reverse the remaining stack.
- Then call a helper insertAtBottom(stack, element): if stack is empty, push element and return.
- Otherwise pop the top, recursively insertAtBottom with the same element, then push the popped value

back — this places the element at the very bottom.

#### COMPLEXITY

| Approach                   | Time     | Space  | Notes                                                         |
|----------------------------|----------|--------|---------------------------------------------------------------|
| Recursive insert-at-bottom | $O(N^2)$ | $O(N)$ | Each of $N$ elements triggers an $O(N)$ insert-at-bottom pass |

insert-at-bottom pass

*Related: Sort a stack using recursion*

#### ■ Interview Trigger

Reversing a stack with no extra structure is the SAME two-function pattern as sorting a stack, just with insert-at-bottom instead of insert-in-order.

## 7.2 Subsequences Pattern (Addendum)

## 7.2.4 / 7.2.5 · Count / Check Existence of a Subsequence with Sum K

### WHAT IT'S ASKING

Count, or check the existence of, a subsequence of an array whose elements sum to K.

### TOPIC / PATTERN

Recursion — take / not-take template

### COUNT ALL SUBSEQUENCES (RECURRENCE)

- $f(\text{idx}, \text{target})$ : if  $\text{idx} == n$ , return 1 if  $\text{target} == 0$  else 0.
- return  $f(\text{idx}+1, \text{target})$  [not-take] +  $f(\text{idx}+1, \text{target} - \text{arr}[\text{idx}])$  [take, only if  $\text{arr}[\text{idx}] \leq \text{target}$ ].

### CHECK IF ONE EXISTS (RECURRENCE)

- Same shape, but combine branches with logical OR instead of addition, so the recursion can short-circuit

and return true the moment either branch succeeds — no need to explore the full tree.

### COMPLEXITY

| Approach                | Time     | Space  | Notes                                                                                                          |
|-------------------------|----------|--------|----------------------------------------------------------------------------------------------------------------|
| Take/not-take recursion | $O(2^N)$ | $O(N)$ | The “exists” version early-exits; “count” must explore fully. Both are the un-memoized ancestor of DP-14/DP-17 |

explore fully. Both are the un-memoized ancestor of DP-14/DP-17

*Related: Subset Sum Equal to Target (DP-14), Count Subsets with Sum K (DP-17)*

### ■ Interview Trigger

Any “count subsequences with property X” or “does a subsequence with property X exist” starts as this exact take/not-take recursion before it gets memoized into a DP table.

## 7.2.7 · Combination Sum II

### WHAT IT'S ASKING

Find all unique combinations of numbers from an array (each number used at most once) that sum to a target.

### TOPIC / PATTERN

Backtracking — each element used at most once, duplicates in input

### ALGORITHM

- Sort the array first — this is what makes duplicate-skipping and early pruning possible.
- At each recursion level, iterate  $i$  from the current start index: skip  $i$  if  $i > \text{start}$  and  $\text{arr}[i] == \text{arr}[i-1]$  (this avoids picking the “same value” twice at the SAME tree level, which would create duplicate combinations).
- If  $\text{arr}[i] > \text{remaining target}$ , break immediately (sorted array, nothing further can work).

- Otherwise include `arr[i]`, recurse with `(i+1, target - arr[i])` — note `i+1`, not `i`, since each element is used once
- then backtrack.

**COMPLEXITY**

| Approach                       | Time    | Space  | Notes                                                                |
|--------------------------------|---------|--------|----------------------------------------------------------------------|
| Sort + backtrack with dup-skip | $O(2N)$ | $O(N)$ | The dup-skip check is what distinguishes this from Combination Sum I |

$N$

dup-skip this from Combination Sum I

*Related: Combination Sum (7.2.4), Subset Sum-II*

**■ Interview Trigger**

“Each number used at most once” + duplicate values in the input → sort first, then skip `i > start` && `arr[i] == arr[i-1]` to dedupe at the same recursion level.

**7.2.10 · Combination Sum III****WHAT IT'S ASKING**

Find all unique combinations of  $k$  numbers from 1-9 that sum to a given target, with no repeated numbers in a combination.

**TOPIC / PATTERN**

Backtracking — fixed count  $k$ , fixed digit pool 1–9

**ALGORITHM**

- Pick exactly  $k$  distinct digits from 1–9 (each usable once) that sum to the target.
- Standard backtracking: try digits in increasing order starting from the last chosen + 1; maintain a running count and sum.
- Prune as soon as `count > k` or `sum > target`; when `count == k`, check if `sum == target` and record if so.

**COMPLEXITY**

| Approach             | Time        | Space  | Notes                                                                 |
|----------------------|-------------|--------|-----------------------------------------------------------------------|
| Bounded backtracking | $O(C(9,k))$ | $O(k)$ | Search space is tiny — at most $C(9,k)$ combinations of single digits |

combinations of single digits

*Related: Combination Sum (7.2.4), Combination Sum II (7.2.6)*

**■ Interview Trigger**

Fixed digit pool (1–9) + fixed count  $k$  is a much smaller search space than general Combination Sum — prune on count, not just on sum.

## 7.3 Trying Out All Combos / Hard (Addendum)

**7.3.9 · Expression Add Operators****WHAT IT'S ASKING**

Given a target and an array of digit-characters, insert +, -, or nothing between them to reach the target.

**TOPIC / PATTERN**

Backtracking over string positions — careful multiplication handling

**ALGORITHM**

- At each recursion step, try every possible next operand as a substring starting at the current position (skip multi-digit operands that start with a leading zero, except the single digit “0” itself).
- For each operand, branch three ways: append it with +, with –, or with  $\times$ .
- For + and –, the running total updates directly. For  $\times$ , precedence must be respected: remove the PREVIOUS operand's contribution from the running total, then add back (previous  $\times$  current) instead.
- Track the “last operand value” alongside the running total specifically to undo/redo it correctly across  $\times$ .

**COMPLEXITY**

| Approach                         | Time    | Space  | Notes                                                                       |
|----------------------------------|---------|--------|-----------------------------------------------------------------------------|
| Backtracking + precedence fix-up | $O(4N)$ | $O(N)$ | Roughly 3 operator choices $\times$ operand-length choices at each position |

fix-up operand-length choices at each position

*Related: K-th Permutation Sequence (7.3.10)*

**■ Interview Trigger**

Backtracking that must respect operator precedence ( $\times$  before + / –)  $\rightarrow$  always carry the previous operand separately so it can be subtracted out and reapplied.

**7.3.10 · K-th Permutation Sequence****WHAT IT'S ASKING**

Find the K-th permutation of the sequence [1, 2, ..., N] without generating all permutations.

**TOPIC / PATTERN**

Factorial number system — avoid generating all permutations

**ALGORITHM**

- Precompute factorials  $0!$  through  $(n-1)!$ . Convert  $k$  to 0-indexed:  $k = k - 1$ .

- Maintain a list of remaining candidate digits [1, 2, ..., n]. For each position (n candidates remaining):
- $\text{index} = k / (\text{remaining}-1)!$  — pick `candidates[index]`, remove it from the list, and append it to the result.
- Update  $k = k \% (\text{remaining}-1)!$ , then move to the next position with one fewer candidate.

**COMPLEXITY**

| Approach                | Time     | Space  | Notes                                                                                                         |
|-------------------------|----------|--------|---------------------------------------------------------------------------------------------------------------|
| Factorial number system | $O(N^2)$ | $O(N)$ | $O(N^2)$ from list removal;<br>$O(N \log N)$ possible with a Fenwick tree — both crush<br>$O(N!)$ brute force |

with a Fenwick tree — both crush  $O(N!)$  brute force

*Related: [Expression Add Operators \(7.3.9\)](#)*

**■ Interview Trigger**

“K-th permutation” without generating all of them → factorial number system: each digit position is decided by  $k / (\text{remaining})!$ , never brute-force enumeration.

**LINKED LIST + RECURSION — MASTER CHEATSHEET****LINKED LIST PATTERN TRIGGERS**

| Pattern            | Trigger Keywords                          | Key Tool                           |
|--------------------|-------------------------------------------|------------------------------------|
| Fast/Slow Pointers | 'middle', 'cycle', 'palindrome LL'        | 2 pointers, different speeds       |
| Reversal           | 'reverse', 'rotate', 'reverse in groups'  | prev/curr/next rewiring            |
| Dummy Node         | 'remove Nth', 'merge lists', edge at head | Simplifies head-removal edge cases |
| Two List Merge     | 'merge sorted lists', 'flatten'           | Like merging two sorted arrays     |
| HashMap/Set        | 'intersection', 'clone with random ptr'   | $O(N)$ space trade-off             |
| Floyd's Cycle      | 'detect loop', 'find loop start'          | Meet point math: $L = nC - x$      |

**RECURSION / BACKTRACKING PATTERN TRIGGERS**

| Pattern            | Trigger Keywords                                  | Key Tool                          |
|--------------------|---------------------------------------------------|-----------------------------------|
| Take/Not-Take      | 'all subsequences', 'subsets', 'combination sum'  | 2 branches per index              |
| Backtrack + Undo   | 'all permutations', 'N-Queens', 'Sudoku'          | Choose fi Explore fi Un-choose    |
| DFS on Grid        | 'word search', 'rat in maze', 'number of islands' | 4-directional DFS + visited       |
| Constraint Pruning | 'valid parentheses', 'palindrome partition'       | Check validity before recursing   |
| Memoization Needed | Pure recursion is exponential & slow              | Cache on (index) or (index,state) |

|                     |                         |                               |
|---------------------|-------------------------|-------------------------------|
| Fast Exponentiation | 'pow(x,n)', 'x^n mod p' | Halve exponent, square result |
|---------------------|-------------------------|-------------------------------|

### THE GOLDEN RULES

- LL problems almost always reduce to: fast/slow pointers OR pointer reversal OR dummy node trick.
- Recursion = trust the smaller subproblem. Don't trace the whole tree — trust solve(n-1) works.
- Backtracking = always undo your choice before trying the next one. Forgetting this corrupts state.
- If recursion explores overlapping subproblems repeatedly → add memoization (signals upcoming DP

topic).

DSA Revision Notes Striver's A2Z DSA Sheet

### ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING

## 7.1.1 · Recursive Implementation of atoi()

### WHAT IT'S ASKING

Implement atoi() (string to integer conversion) using a recursive helper instead of a loop.

### TOPIC / PATTERN

Recursion — String to Integer parsing

### CORE IDEA

Same rules as the iterative atoi (skip leading spaces, optional sign, digits until non-digit, clamp to 32-bit range) but the digit accumulation is done via a recursive helper instead of a loop.

### ALGORITHM

1. Skip leading whitespace, capture an optional +/- sign.
2. Recursive helper(i, acc): if s[i] isn't a digit or i is out of bounds → return acc.
3. Otherwise acc = acc\*10 + digit, recurse on i+1.
4. Apply sign, then clamp the result to  $[-2^{31}, 2^{31} - 1]$ .

### COMPLEXITY

- Time:  $O(N)$
- Space:  $O(N)$  recursion stack (vs  $O(1)$  for the iterative version)

#### ■ Interview Trigger

Same parsing rules as iterative atoi — the only change is accumulating digits through recursive calls.

# DSA Revision Notes

## Striver's A2Z DSA Sheet

### Batch 6 · Steps 8 & 9: Bit Manipulation + Stacks & Queues

Problems 32 problems — Bit tricks, Monotonic Stack, Implementation

Patterns Bit Masking · Monotonic Stack · Two Stacks · Sliding Window Max

Format Pattern · Intuition · Algorithm · Complexity · Traps · Trigger

Bit tricks unlock  $O(1)$  operations. Monotonic stacks unlock  $O(N)$  for 'next greater' style problems.

#### TABLE OF CONTENTS

#### 8.1 BIT MANIPULATION — CONCEPTS

- 8.1.1 Check if i-th Bit is Set
- 8.1.2 Check if Number is Odd
- 8.1.3 Check if Number is Power of 2
- 8.1.4 Count Number of Set Bits
- 8.1.5 Set/Unset the Rightmost Unset Bit
- 8.1.6 Swap Two Numbers (XOR trick)
- 8.1.7 Divide Two Integers Without /, ×, %

#### 8.2 BIT MANIPULATION — INTERVIEW PROBLEMS

- 8.2.1 Count Bits to Flip (A to B)
- 8.2.2 Number Appearing Odd Number of Times
- 8.2.3 Power Set (Bit Manipulation)
- 8.2.4 XOR of Numbers from L to R
- 8.2.5 Two Numbers Appearing Odd Number of Times

#### 8.3 ADVANCED MATHS

- 8.3.1 Print Prime Factors
- 8.3.2 All Divisors of a Number
- 8.3.3 Sieve of Eratosthenes
- 8.3.4 Prime Factorization using Sieve

#### 9.1 / 9.2 STACK & QUEUE FUNDAMENTALS

- 9.1 Implement Stack/Queue using Array, LL, each other
- 9.1.7 Check for Balanced Parentheses
- 9.1.8 Implement Min Stack
- 9.2 Infix / Prefix / Postfix Conversions

#### 9.3 MONOTONIC STACK / QUEUE

- 9.3.1 Next Greater Element
- 9.3.2 Next Greater Element II (Circular)
- 9.3.3 Next Smaller Element
- 9.3.5 Trapping Rainwater
- 9.3.6 Sum of Subarray Minimums
- 9.3.7 Asteroid Collision
- 9.3.9 Remove K Digits

9.3.10 Largest Rectangle in Histogram

9.3.11 Maximal Rectangle

## 9.4 IMPLEMENTATION PROBLEMS

9.4.1 Sliding Window Maximum

9.4.2 Stock Span Problem

9.4.3 The Celebrity Problem

9.4.5 LRU Cache

## STEP 8 — Bit Manipulation

8.1 Concepts

## CORE BIT TRICKS REFERENCE

Approach Time Space Notes

Check i-th bit  $O(1)$   $O(1)$   $(n \gg i) \& 1$

Set i-th bit  $O(1)$   $O(1)$   $n \mid (1 \ll i)$

Clear i-th bit  $O(1)$   $O(1)$   $n \& \sim(1 \ll i)$

Toggle i-th bit  $O(1)$   $O(1)$   $n \wedge (1 \ll i)$

Check power of 2  $O(1)$   $O(1)$   $n \& (n-1) == 0$  ( $n > 0$ )

Remove last set bit  $O(1)$   $O(1)$   $n \& (n-1)$

Isolate last set bit  $O(1)$   $O(1)$   $n \& (-n)$

Count set bits  $O(\log N)$   $O(1)$  Brian Kernighan: loop  $n = n \& (n-1)$

8.1.1 – 8.1.4 · Bit Checks, Odd Check, Power of 2, Count Set Bits

## TOPIC / PATTERN

Bit Manipulation — Fundamental bitwise operations

## CHECK I-TH BIT

–  $(n \gg i) \& 1$  — shift right by i, then mask the last bit.

## CHECK ODD

–  $n \& 1 == 1$  — the last bit determines parity. Faster than  $n \% 2$ .

## CHECK POWER OF 2

A power of 2 has exactly one set bit (e.g.  $8 = 1000$ ).  $n \& (n-1)$  clears the lowest set bit. If result is 0, only one bit

was set.

$n = 8 = 1000$

$n-1 = 7 = 0111$

$n \& (n-1) = 1000 \& 0111 = 0000 \rightarrow$  power of 2 ✓

## COUNT SET BITS — BRIAN KERNIGHAN'S ALGORITHM

– While  $n \neq 0$ :  $n = n \& (n-1)$ , count++. Each iteration removes one set bit.

## COMPLEXITY

Approach Time Space Notes

Naive loop (32 bits)  $O(32)$   $O(1)$  Check every bit position

Brian Kernighan's  $O(\text{set bits})$   $O(1)$  Only iterates as many times as set bits

Interview Trigger

$n \& (n-1)$  clears the lowest set bit — the building block for power-of-2 check and bit counting.

## 8.1.5 / 8.1.6 / 8.1.7 · Set/Unset Bits, Swap via XOR, Divide Without Operators

### WHAT IT'S ASKING

Check/set the  $i$ -th bit, swap two numbers using XOR, and divide two integers without using  $/$ ,  $\times$ , or  $\%$ .

### TOPIC / PATTERN

Bit Manipulation — Manipulation tricks

### SET THE RIGHTMOST UNSET BIT

- $n | (n+1)$  — adding 1 carries through trailing 1s and sets the first 0.

### SWAP TWO NUMBERS WITHOUT TEMP VARIABLE

- $a = a \oplus b$ ;  $b = a \oplus b$ ;  $a = a \oplus b$ ; — uses XOR's self-inverse property.

### DIVIDE TWO INTEGERS (NO $/$ , $\times$ , $\%$ )

Use bit shifts to find how many times divisor fits, doubling each time (like binary search).

1. result=0, sign handling for negatives done separately (work with positives).
2. While dividend  $\geq$  divisor: find largest  $k$  such that  $\text{divisor} \times 2^k \leq \text{dividend}$ .
3. Subtract  $\text{divisor} \times 2^k$  from dividend, add  $2^k$  to result. Repeat.

### COMPLEXITY

| Approach       | Time        | Space  | Notes                                           |
|----------------|-------------|--------|-------------------------------------------------|
| XOR Swap       | $O(1)$      | $O(1)$ | No temp variable needed                         |
| Bitwise Divide | $O(\log N)$ | $O(1)$ | Doubling divisor each step — like binary search |

### COMMON MISTAKES

- XOR swap fails if  $a$  and  $b$  are the SAME memory location ( $a \oplus a = 0$ , data loss).

#### ■ Interview Trigger

XOR is its own inverse:  $a \oplus b \oplus b = a$ . This powers swap-without-temp and single-number tricks.

## 8.2 Bit Manipulation — Interview Problems

### 8.2.1 · Count Number of Bits to be Flipped to Convert A to B

#### WHAT IT'S ASKING

Find the minimum number of bit flips required to convert integer A into integer B.

#### TOPIC / PATTERN

Bit Manipulation — XOR + Count Set Bits

#### CORE IDEA

$A \oplus B$  has a set bit exactly where A and B differ. Count the set bits in  $(A \oplus B)$  to get the number of flips needed.

#### ALGORITHM

- 1.  $\text{xorVal} = A \oplus B$ .
- 2. Return count of set bits in  $\text{xorVal}$  (Brian Kernighan's).

**COMPLEXITY**

| Approach    | Time        | Space  | Notes                      |
|-------------|-------------|--------|----------------------------|
| XOR + Count | $O(\log N)$ | $O(1)$ | One XOR, then bit counting |

**■ Interview Trigger**

XOR marks differing bits — count them to know flips required.

**8.2.2 / 8.2.5 · Number(s) Appearing Odd Number of Times****WHAT IT'S ASKING**

Find the number(s) that appear an odd number of times while all others appear an even number of times.

**TOPIC / PATTERN**

Bit Manipulation — XOR cancellation

**SINGLE NUMBER APPEARING ODD TIMES**

- XOR all elements. Even-count pairs cancel to 0, the odd one survives.

**TWO NUMBERS APPEARING ODD TIMES (OTHERS APPEAR EVEN TIMES)**

XOR all elements  $\rightarrow$  result =  $A \oplus B$  (the two odd-occurring numbers, since all even-occurring numbers cancel). Find any set bit in this XOR — it means A and B differ at that bit. Partition all numbers by that bit; XOR within each partition isolates A and B separately.

- 1.  $\text{xorAll} = \text{XOR of all elements} = A \oplus B$ .
- 2.  $\text{rightmostSetBit} = \text{xorAll} \& (-\text{xorAll})$  (isolates lowest set bit).
- 3. Partition numbers into two groups based on whether this bit is set.
- 4. XOR each group separately  $\rightarrow$  group1 gives A, group2 gives B.

**COMPLEXITY**

| Approach          | Time   | Space  | Notes                       |
|-------------------|--------|--------|-----------------------------|
| HashMap           | $O(N)$ | $O(N)$ | Count frequencies           |
| XOR (single)      | $O(N)$ | $O(1)$ | One pass                    |
| XOR (two numbers) | $O(N)$ | $O(1)$ | Two passes, partition trick |

*Related: [LeetCode 136 Single Number](#), [LeetCode 137 Single Number II](#), [LeetCode 260 Single Number III](#)*

**■ Interview Trigger**

XOR all  $\rightarrow$  isolates  $A \oplus B$  for two-odd-numbers. Use lowest set bit to split A from B.

**8.2.3 · Power Set Using Bit Manipulation**

**WHAT IT'S ASKING**

Generate the power set (all subsets) of an array using bitmasking instead of recursion.

**TOPIC / PATTERN**

Bit Manipulation — Binary representation as subset selector

**CORE IDEA**

Each number from 0 to  $2^N-1$ , in binary, represents a unique subset: bit  $i$  set means include element  $i$ .

**ALGORITHM**

- For mask = 0 to  $2^N-1$ :
- For  $i = 0$  to  $N-1$ : if mask has bit  $i$  set  $\rightarrow$  include  $arr[i]$  in this subset.

**VISUALIZATION**

```
arr = [A, B, C], N=3, 2^3=8 subsets
mask=000  $\rightarrow$  {} mask=100  $\rightarrow$  {C}
mask=001  $\rightarrow$  {A} mask=101  $\rightarrow$  {A,C}
mask=010  $\rightarrow$  {B} mask=110  $\rightarrow$  {B,C}
mask=011  $\rightarrow$  {A,B} mask=111  $\rightarrow$  {A,B,C}
```

**COMPLEXITY**

| Approach            | Time              | Space  | Notes                               |
|---------------------|-------------------|--------|-------------------------------------|
| Bitmask Enumeration | $O(2^N \times N)$ | $O(N)$ | $2^N$ subsets, $O(N)$ to build each |

**■ Interview Trigger**

Binary digits of mask directly encode which elements are 'in' the subset.

**8.2.4 · XOR of Numbers from L to R****WHAT IT'S ASKING**

Compute the XOR of all integers from L to R.

**TOPIC / PATTERN**

Bit Manipulation — XOR prefix trick

**CORE IDEA**

$XOR(L, R) = XOR(1..R) \oplus XOR(1..L-1)$ .  $XOR(1..n)$  follows a repeating pattern of period 4.

**XOR(1..N) PATTERN**

$n \% 4 == 0 \rightarrow n$   $n \% 4 == 1 \rightarrow 1$   $n \% 4 == 2 \rightarrow n+1$   $n \% 4 == 3 \rightarrow 0$

**ALGORITHM**

- Result =  $computeXOR(R) \oplus computeXOR(L-1)$ .

**COMPLEXITY**

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|                 |          |        |                           |
|-----------------|----------|--------|---------------------------|
| Brute           | $O(R-L)$ | $O(1)$ | XOR every number in range |
| Pattern Formula | $O(1)$   | $O(1)$ | Use mod-4 pattern         |

#### ■ Interview Trigger

XOR(1..n) cycles every 4 numbers —  $O(1)$  formula via mod 4.

### 8.3 Advanced Maths

## 8.3.1 / 8.3.2 · Prime Factors and All Divisors

#### WHAT IT'S ASKING

Find all prime factors of a number, and list all of its divisors, efficiently.

#### TOPIC / PATTERN

Maths — Trial division up to  $\sqrt{N}$

#### PRIME FACTORIZATION

- For  $i = 2$  to  $\sqrt{N}$ : while  $N \% i == 0 \rightarrow$  print  $i$ ,  $N /= i$ .
- If  $N > 1$  after the loop,  $N$  itself is a prime factor.

#### ALL DIVISORS

- For  $i = 1$  to  $\sqrt{N}$ : if  $N \% i == 0 \rightarrow i$  is a divisor, and  $N/i$  is a divisor too.
- Special case: if  $i == N/i$ , add only once (perfect square case).

#### COMPLEXITY

| Approach         | Time          | Space  | Notes                                  |
|------------------|---------------|--------|----------------------------------------|
| Naive (1 to N)   | $O(N)$        | $O(1)$ | Check every number                     |
| $\sqrt{N}$ trick | $O(\sqrt{N})$ | $O(1)$ | Divisors come in pairs ( $i$ , $N/i$ ) |

#### ■ Interview Trigger

Only check up to  $\sqrt{N}$  — divisors/factors come in pairs ( $i$ ,  $N/i$ ).

## 8.3.3 / 8.3.4 · Sieve of Eratosthenes and Prime Factorization using Sieve

#### WHAT IT'S ASKING

Use the Sieve of Eratosthenes to find all primes up to  $N$ , and to prime-factorize numbers quickly.

#### TOPIC / PATTERN

Maths — Precompute primality for a range

#### CORE IDEA

Mark all multiples of each prime as composite, starting from  $p^2$ . Whatever remains unmarked is prime.

#### ALGORITHM

- 1. `isPrime[2..N] = True` initially.
- 2. For  $p = 2$  to  $\sqrt{N}$ : if `isPrime[p]` → mark all multiples of  $p$  (starting from  $p^2$ ) as `False`.
- 3. Remaining `True` entries are primes.

**WHY START AT  $P^2$ ?**

- Smaller multiples of  $p$  (like  $2p, 3p\dots$ ) were already marked by smaller primes.

**PRIME FACTORIZATION USING SIEVE (SMALLEST PRIME FACTOR)**

- Precompute `SPF[i]` = smallest prime factor of  $i$  using a modified sieve.
- Then factorize any number in  $O(\log N)$  by repeatedly dividing by `SPF[n]`.

**COMPLEXITY**

| Approach              | Time               | Space  | Notes                                  |
|-----------------------|--------------------|--------|----------------------------------------|
| Sieve of Eratosthenes | $O(N \log \log N)$ | $O(N)$ | Precompute primality up to $N$         |
| SPF + Factorize       | $O(N \log \log N)$ | $O(N)$ | $O(\log N)$ per query after precompute |

SPF + Factorize  $O(N \log \log N)$  precompute  $O(N)$   $O(\log N)$  per query after precompute

*Related: [LeetCode 204 Count Primes](#)*

**■ Interview Trigger**

Sieve precomputes once in  $O(N \log \log N)$  — then every primality/factorization query is fast.

**STEP 9 - Stack and Queues**

9.1 / 9.2 Learning + Conversions

**9.1 · Implementing Stack & Queue (Array, LL, Each Other)****WHAT IT'S ASKING**

Implement a stack and a queue from scratch using arrays, linked lists, and each other.

**TOPIC / PATTERN**

Data Structure Implementation

**STACK — LIFO (LAST IN FIRST OUT)**

- Array-based: push/pop at the end —  $O(1)$ . Fixed capacity issue.
- LL-based: push/pop at head —  $O(1)$ . No capacity limit.

**QUEUE — FIFO (FIRST IN FIRST OUT)**

- Array-based: circular array to reuse space, or two-pointer (front/rear).
- LL-based: push at tail, pop at head — both  $O(1)$  with tail pointer.

**STACK USING TWO QUEUES / QUEUE USING TWO STACKS**

- Queue using 2 stacks: push → push to `stack1`. Pop → if `stack2` empty, move all from `stack1` to `stack2`

(reverses order), then pop from stack2.

- Stack using 2 queues: push → add to empty queue, then dequeue all from the other queue and enqueue

them after, so newest is always at front.

#### COMPLEXITY

| Approach                       | Time             | Space  | Notes                                  |
|--------------------------------|------------------|--------|----------------------------------------|
| Queue via 2 Stacks (amortized) | $O(1)$ amortized | $O(N)$ | Each element moved at most twice total |
| Stack via 2 Queues             | $O(N)$ per push  | $O(N)$ | Reorder on every push                  |

Queue via 2 Stacks (amortized)

| Approach | Time   | Space  | Notes                                    |
|----------|--------|--------|------------------------------------------|
| Stack    | $O(N)$ | $O(N)$ | One pass, stack stores unmatched openers |

#### ■ Interview Trigger

Queue from 2 stacks = reverse order twice = original order. Amortized  $O(1)$ .

## 9.1.7 · Check for Balanced Parentheses

### WHAT IT'S ASKING

Given a string of brackets, check whether the parentheses are balanced.

### TOPIC / PATTERN

Stack — Match opening with most recent closing

### ALGORITHM

- 1. For each char: if opening bracket → push to stack.
- 2. If closing bracket → check if stack top is the matching opener. If yes, pop. If no, return False.
- 3. At the end, stack must be empty for balanced string.

### COMPLEXITY

Related: [LeetCode 20 Valid Parentheses](#)

#### ■ Interview Trigger

Stack remembers the most recent unmatched opener — perfect LIFO matching.

## 9.1.8 · Implement Min Stack

### WHAT IT'S ASKING

Design a stack that supports push, pop, top, and retrieving the minimum element, all in  $O(1)$ .

### TOPIC / PATTERN

Stack — Auxiliary stack or encoded values

#### APPROACH 1 — AUXILIARY MIN STACK

– Maintain a second stack that tracks the minimum at each level. On push: `minStack.push(min(value, minStack.top()))`.

#### APPROACH 2 — O(1) SPACE (ENCODING TRICK)

– Store a single 'min' variable. When pushing a new minimum, push an encoded value:  $2 \times \text{value} - \text{oldMin}$ .

On pop, decode to restore previous min.

#### COMPLEXITY

| Approach        | Time | Space      | Notes                                           |
|-----------------|------|------------|-------------------------------------------------|
| Auxiliary Stack | O(1) | O(N)       | Simple, extra stack                             |
| Encoding Trick  | O(1) | O(1) extra | No extra stack — store min as a single variable |

*Related: LeetCode 155 Min Stack*

#### ■ Interview Trigger

Track min alongside each push — auxiliary stack gives O(1) `getMin` in exchange for O(N) space.

## 9.2 · Infix / Prefix / Postfix Conversions

### WHAT IT'S ASKING

Convert an infix expression to prefix and postfix form and vice versa.

### TOPIC / PATTERN

Stack — Operator precedence and order conversion

#### INFIX TO POSTFIX (SHUNTING YARD)

- 1. Scan left to right. Operand → add to output.
- 2. '(' → push to stack.
- 3. ')' → pop to output until '(' found, discard '('.
- 4. Operator → pop higher/equal precedence operators to output, then push current.
- 5. At end, pop all remaining operators to output.

#### INFIX TO PREFIX

- Reverse the infix string (swap brackets), convert to postfix, reverse the result.

#### POSTFIX TO INFIX

- Use a stack of strings. On operand, push. On operator, pop 2, combine as '(op2 OP op1)', push back.

#### PREFIX TO POSTFIX / POSTFIX TO PREFIX

- Similar stack approach scanning in appropriate direction (right-to-left for prefix).

#### COMPLEXITY

| Approach    | Time   | Space  | Notes                      |
|-------------|--------|--------|----------------------------|
| Stack-based | $O(N)$ | $O(N)$ | All conversions are linear |

#### ■ Interview Trigger

Operator precedence + stack = systematic conversion between all three notations.

### 9.3 Monotonic Stack / Queue Problems

The Pattern: A monotonic stack maintains elements in increasing or decreasing order. When a new element breaks the order, pop and resolve those elements — this is how you find 'next greater/smaller element' in  $O(N)$  instead of  $O(N^2)$ .

## 9.3.1 / 9.3.2 / 9.3.3 · Next Greater/Smaller Element (incl. Circular)

### WHAT IT'S ASKING

For every element, find the next/previous greater or smaller element in the array (including a circular version).

### TOPIC / PATTERN

Monotonic Stack — Decreasing stack for 'next greater'

### CORE IDEA

Traverse right to left (or left to right with reverse processing). Maintain a stack of 'candidates' in decreasing order. Pop elements smaller than current — they found their 'next greater'.

### ALGORITHM — NEXT GREATER ELEMENT (RIGHT TO LEFT)

- 1. Stack = empty, result[N-1..0].
- 2. For  $i = N-1$  down to 0:
- 3. While stack not empty and  $\text{stack.top()} \leq \text{arr}[i]$ : pop.
- 4.  $\text{result}[i] = \text{stack.top()}$  if stack not empty else -1.
- 5. Push  $\text{arr}[i]$ .

### VISUALIZATION

```
arr = [4, 5, 2, 10]
i=3 (10): stack=[] → NGE=-1, push 10. stack=[10]
i=2 (2): 10>2 → NGE=10, push 2. stack=[10,2]
i=1 (5): pop 2(≤5), 10>5 → NGE=10, push 5. stack=[10,5]
i=0 (4): 5>4 → NGE=5, push 4. stack=[10,5,4]
Result: [5, 10, 10, -1] ✓
```

### CIRCULAR ARRAY VARIANT (NEXT GREATER ELEMENT II)

- Iterate  $2N$  times ( $i \% N$ ), without actually duplicating the array, to simulate wraparound.

### NEXT SMALLER ELEMENT

- Same logic, but maintain an INCREASING stack — pop elements GREATER than current.

### COMPLEXITY

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|                 |          |        |                                 |
|-----------------|----------|--------|---------------------------------|
| Brute Force     | $O(N^2)$ | $O(1)$ | Check every pair                |
| Monotonic Stack | $O(N)$   | $O(N)$ | Each element pushed/popped once |

Related: [LeetCode 496](#), [503 Next Greater Element](#), [LeetCode 739 Daily Temperatures](#)

#### ■ Interview Trigger

Maintain a decreasing stack — pop smaller elements, they just found their next greater.

## 9.3.5 · Trapping Rainwater

### WHAT IT'S ASKING

Given elevation heights, compute how much rainwater is trapped between the bars.

### TOPIC / PATTERN

Monotonic Stack OR Two Pointers

### CORE IDEA

Water trapped at index  $i = \min(\text{maxLeft}[i], \text{maxRight}[i]) - \text{height}[i]$ . The water level is bounded by the SHORTER of the two tallest walls on either side.

### APPROACH 1 — PREFIX/SUFFIX MAX ARRAYS

- Precompute  $\text{maxLeft}[i]$  and  $\text{maxRight}[i]$  for every index. Sum up the trapped water.

### APPROACH 2 — TWO POINTERS (OPTIMAL, $O(1)$ SPACE)

- 1.  $\text{left}=0, \text{right}=N-1, \text{leftMax}=0, \text{rightMax}=0, \text{water}=0$ .
- 2. While  $\text{left} < \text{right}$ :
- 3. If  $\text{height}[\text{left}] < \text{height}[\text{right}]$ :
- 4. If  $\text{height}[\text{left}] \geq \text{leftMax}$ :  $\text{leftMax} = \text{height}[\text{left}]$ . Else  $\text{water} += \text{leftMax} - \text{height}[\text{left}]$ .
- 5.  $\text{left}++$ .
- 6. Else: symmetric logic with  $\text{right}, \text{rightMax}$ .  $\text{right}--$ .

### WHY TWO POINTERS WORK

We only move the side with the SMALLER current height, because we know the larger side guarantees a wall at least that tall — water level is bound by the smaller side.

### APPROACH 3 — MONOTONIC STACK

- Maintain a decreasing stack of indices. When a taller bar appears, pop and compute trapped water in the

'valley' formed between bars.

### COMPLEXITY

| Approach          | Time     | Space  | Notes                                     |
|-------------------|----------|--------|-------------------------------------------|
| Brute Force       | $O(N^2)$ | $O(1)$ | For each bar, scan left and right for max |
| Prefix/Suffix Max | $O(N)$   | $O(N)$ | Precompute arrays                         |

|                 |        |        |                                                  |
|-----------------|--------|--------|--------------------------------------------------|
| Two Pointers    | $O(N)$ | $O(1)$ | Optimal — no extra space                         |
| Monotonic Stack | $O(N)$ | $O(N)$ | Alternative, useful for histogram-style problems |

Related: [LeetCode 42 Trapping Rain Water](#)

#### ■ Interview Trigger

Water at  $i = \min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$ . Two pointers track this in  $O(1)$  space.

## 9.3.6 · Sum of Subarray Minimums

### WHAT IT'S ASKING

For each element in an array, find the sum of the minimum of every subarray that contains it.

### TOPIC / PATTERN

Monotonic Stack — Contribution Technique

### CORE IDEA

For each element, find how many subarrays have it as the MINIMUM. Contribution =  $\text{arr}[i] \times (\text{count of subarrays where it's the min})$ . Use 'previous smaller element' and 'next smaller element' distances.

### ALGORITHM

1. For each  $i$ , find  $\text{left}[i]$  = distance to previous strictly smaller element (or start).
2. Find  $\text{right}[i]$  = distance to next smaller-or-equal element (or end) — careful with ties.
3. Contribution of  $\text{arr}[i]$  =  $\text{arr}[i] \times \text{left}[i] \times \text{right}[i]$ .
4. Sum all contributions, mod  $10^9+7$  (often required due to large sums).

### COMMON MISTAKES

- Using strict inequality on BOTH sides causes double-counting for duplicate values — use strict on one

side, non-strict on the other.

### COMPLEXITY

| Approach | Time     | Space  | Notes                        |
|----------|----------|--------|------------------------------|
| Brute    | $O(N^2)$ | $O(1)$ | All subarrays, find min each |

| Approach        | Time   | Space  | Notes                  |
|-----------------|--------|--------|------------------------|
| Monotonic Stack | $O(N)$ | $O(N)$ | Contribution technique |

Related: [LeetCode 907 Sum of Subarray Minimums](#), [LeetCode 2104 Sum of Subarray Ranges](#)

#### ■ Interview Trigger

Each element's contribution = value  $\times$  (ways it's the min) — use prev/next smaller distances.

### 9.3.7 · Asteroid Collision

#### WHAT IT'S ASKING

Simulate asteroids moving in a line and colliding according to their direction and size.

#### TOPIC / PATTERN

Stack — Simulate collisions

#### CORE IDEA

Positive = moving right, negative = moving left. Collision only happens when a right-moving asteroid is followed by a left-moving one. Use a stack to simulate.

#### ALGORITHM

- For each asteroid:
- While stack not empty,  $\text{stack.top()} > 0$ , and  $\text{current} < 0$  (collision possible):
- If  $|\text{stack.top()}| < |\text{current}|$ : pop (top destroyed), continue checking.
- If  $|\text{stack.top()}| == |\text{current}|$ : pop (both destroyed), mark current as destroyed, break.
- If  $|\text{stack.top()}| > |\text{current}|$ : current destroyed, break.
- If current survives: push it.

#### COMPLEXITY

| Approach         | Time   | Space  | Notes                                    |
|------------------|--------|--------|------------------------------------------|
| Stack Simulation | $O(N)$ | $O(N)$ | Each asteroid pushed/popped at most once |

Related: [LeetCode 735 Asteroid Collision](#)

#### ■ Interview Trigger

Stack simulates collisions — only right-then-left adjacent pairs ever collide.

### 9.3.9 · Remove K Digits (Smallest Resulting Number)

#### WHAT IT'S ASKING

Given a number as a string, remove K digits so the resulting number is the smallest possible.

#### TOPIC / PATTERN

Monotonic Stack — Greedy removal of larger preceding digits

#### CORE IDEA

To get the smallest number, remove digits that are larger than the digit right after them — greedily, using an increasing stack.

#### ALGORITHM

- 1. For each digit: while stack not empty,  $k > 0$ , and  $\text{stack.top()} > \text{digit}$ : pop,  $k--$ .
- 2. Push digit.
- 3. If  $k$  still  $> 0$  after processing all digits, remove from the end ( $k$  digits).

- 4. Strip leading zeros. If result is empty, return '0'.

### VISUALIZATION

```
num='1432219', k=3
1,4: 4>3? no push. stack=[1,4]
3: 4>3 pop(k=2). stack=[1,3]
2: 3>2 pop(k=1). stack=[1,2]
```

2: 2>2? no, push. stack=[1,2,2] 1: 2>1 pop(k=0). stack=[1,2,1] 9: push. stack=[1,2,1,9] Result: '1219' ✓

### COMPLEXITY

| Approach        | Time | Space | Notes                         |
|-----------------|------|-------|-------------------------------|
| Monotonic Stack | O(N) | O(N)  | Each digit pushed/popped once |

Related: [LeetCode 402 Remove K Digits](#)

#### ■ Interview Trigger

Increasing stack — pop a digit if it's bigger than what comes right after it.

## 9.3.10 / 9.3.11 · Largest Rectangle in Histogram and Maximal Rectangle

### WHAT IT'S ASKING

Find the area of the largest rectangle that fits under a histogram, and the largest all-1s rectangle in a binary matrix.

### TOPIC / PATTERN

Monotonic Stack — Previous/Next Smaller Element

### CORE IDEA

For each bar, the largest rectangle using that bar's height extends from the previous smaller bar to the next smaller bar. width = (nextSmaller - prevSmaller - 1).

### ALGORITHM (SINGLE PASS WITH STACK)

- 1. Maintain an increasing stack of indices.
- 2. For each bar i: while stack not empty and height[stack.top()] >= height[i]:
- 3. top = stack.pop(). width = i - stack.top() - 1 (or i if stack empty).
- 4. area = height[top] \* width. Track max area.
- 5. Push i. After loop, process remaining stack with width extending to N.

### VISUALIZATION

```
heights = [2, 1, 5, 6, 2, 3]
Bar at idx 3 (height=6): prevSmaller=idx2(5≥6? no wait, find correctly)
Bar height 5,6 form rect of area 6×2=10 (using width between bars of ht ≥5,6)
Max area = 10 ✓
```

### MAXIMAL RECTANGLE (2D — ALL 1S IN BINARY MATRIX)

Convert each row into a 'histogram' where height[j] = consecutive 1s ending at this row in column j. Apply Largest Rectangle in Histogram to each row's histogram. Track global max.

**COMPLEXITY**

| Approach          | Time            | Space  | Notes                                   |
|-------------------|-----------------|--------|-----------------------------------------|
| Brute (Histogram) | $O(N^2)$        | $O(1)$ | For each bar expand left/right          |
| Monotonic Stack   | $O(N)$          | $O(N)$ | Optimal — each bar pushed/popped once   |
| Maximal Rectangle | $O(M \times N)$ | $O(N)$ | Apply histogram trick to each of M rows |

*Related: [LeetCode 84 Largest Rectangle in Histogram](#), [LeetCode 85 Maximal Rectangle](#)*

**■ Interview Trigger**

Width of rectangle at bar  $i$  = gap between previous smaller and next smaller bar.

**9.4 Implementation Problems****9.4.1 · Sliding Window Maximum****WHAT IT'S ASKING**

Find the maximum of every window of size  $K$  as it slides across the array.

**TOPIC / PATTERN**

Monotonic Deque — Maintain decreasing order of candidates

**CORE IDEA**

Use a deque storing INDICES in decreasing order of value. Front of deque is always the max of the current window. Remove indices that fall out of window range, and remove smaller elements from the back.

**ALGORITHM**

- 1. For each index  $i$ :
- 2. Remove indices from front if they're outside the window ( $i - k$ ).
- 3. Remove indices from back while  $\text{arr}[\text{back}] \leq \text{arr}[i]$  (they'll never be the max).
- 4. Push  $i$  to back.
- 5. If  $i \geq k-1$ : front of deque is the window's max — record it.

**VISUALIZATION**

```
arr=[1,3,-1,-3,5,3,6,7], k=3
Window [1,3,-1]: deque=[idx1(3)] → max=3
Window [3,-1,-3]: deque=[idx1(3)] → max=3
Window [-1,-3,5]: deque=[idx4(5)] → max=5
Result: [3,3,5,5,6,7]
```

**COMPLEXITY**

| Approach | Time            | Space  | Notes            |
|----------|-----------------|--------|------------------|
| Brute    | $O(N \times K)$ | $O(1)$ | Scan each window |

|                 |        |        |                                         |
|-----------------|--------|--------|-----------------------------------------|
| Monotonic Deque | $O(N)$ | $O(K)$ | Each index pushed/popped once — optimal |
|-----------------|--------|--------|-----------------------------------------|

Related: [LeetCode 239 Sliding Window Maximum](#)

#### ■ Interview Trigger

Decreasing deque of indices — front is always max, pop smaller elements from back.

## 9.4.2 · Stock Span Problem

### WHAT IT'S ASKING

For each day's stock price, find how many consecutive previous days had a price  $\leq$  today's (stock span).

### TOPIC / PATTERN

Monotonic Stack — Previous Greater Element distance

### CORE IDEA

Span of day  $i$  = number of consecutive days (including today) where price  $\leq$  today's price. This equals the distance to the previous GREATER element.

### ALGORITHM

- Maintain a stack of (price, span) pairs.
- For each new price: while stack not empty and  $\text{stack.top().price} \leq \text{price}$ : accumulate span, pop.
- Push (price, accumulated\_span + 1).

### COMPLEXITY

| Approach        | Time             | Space  | Notes                           |
|-----------------|------------------|--------|---------------------------------|
| Monotonic Stack | $O(N)$ amortized | $O(N)$ | Each element pushed/popped once |

Related: [LeetCode 901 Online Stock Span](#)

#### ■ Interview Trigger

Span = distance to the previous greater price — classic monotonic stack application.

## 9.4.3 · The Celebrity Problem

### WHAT IT'S ASKING

Design a system that identifies the celebrity in a group — someone everyone knows but who knows no one.

### TOPIC / PATTERN

Stack / Two Pointers — Eliminate non-celebrities

### CORE IDEA

A celebrity is known by everyone but knows no one. Use two pointers to eliminate candidates: if A knows B, A can't be celebrity. If A doesn't know B, B can't be celebrity.

**ALGORITHM**

- 1. top=0, bottom=N-1.
- 2. While top < bottom: if knows(top, bottom) → top++ (top can't be celeb). Else bottom-- (bottom can't be).
- 3. Candidate = top (or bottom, they converge).
- 4. Verify: candidate knows no one, AND everyone knows candidate.

**COMPLEXITY**

| Approach                | Time     | Space  | Notes                                  |
|-------------------------|----------|--------|----------------------------------------|
| Brute (Matrix)          | $O(N^2)$ | $O(1)$ | Check every pair                       |
| Two Pointer Elimination | $O(N)$   | $O(1)$ | Eliminate one candidate per comparison |

*Related: LeetCode 277 Find the Celebrity*

**■ Interview Trigger**

Each comparison eliminates exactly one non-celebrity — converges to 1 candidate in  $O(N)$ .

**9.4.5 · LRU Cache (Important — Frequently Asked)****WHAT IT'S ASKING**

Design a Least Recently Used (LRU) cache with  $O(1)$  get and put operations.

**TOPIC / PATTERN**

Design — HashMap + Doubly Linked List

**CORE IDEA**

Need  $O(1)$  get and put. HashMap gives  $O(1)$  lookup. Doubly Linked List gives  $O(1)$  removal/insertion at any position (needed to move recently-used items to front and evict from the back).

**DESIGN**

- HashMap for  $O(1)$  access to any node.
- DLL maintains usage order: head = most recently used, tail = least recently used.
- Dummy head and tail nodes simplify edge cases (no null checks needed).

**GET(KEY)**

- 1. If key not in map → return -1.
- 2. Move the corresponding node to the front (most recently used).
- 3. Return its value.

**PUT(KEY, VALUE)**

- 1. If key exists → update value, move node to front.
- 2. Else: create new node, add to front, add to map.
- 3. If capacity exceeded → remove the tail's neighbor (LRU item), remove from map too.

**COMPLEXITY**

| Approach      | Time   | Space                | Notes                       |
|---------------|--------|----------------------|-----------------------------|
| HashMap + DLL | $O(1)$ | $O(\text{capacity})$ | Both get and put are $O(1)$ |

**COMMON MISTAKES**

- Forgetting to remove the evicted node from the HashMap as well — memory leak / stale entries.
- Not using dummy head/tail — leads to messy null-pointer edge case handling.

*Related: LeetCode 146 LRU Cache, LeetCode 460 LFU Cache (harder variant)*

**■ Interview Trigger**

HashMap for  $O(1)$  lookup + DLL for  $O(1)$  reordering — the canonical LRU design.

Addendum — 3 Missing Problems (Restored) Step 8 (Bit Manipulation / Advanced Maths) was missing Power( $n, m$ ). Step 9 (Monotonic Stack) was missing two problems: Number of NGEs to the right, and Sum of Subarray Ranges. Added back below in their correct subsections.

## 8.3 Advanced Maths (Addendum)

**8.3.5 · Power( $n, m$ )****WHAT IT'S ASKING**

Compute  $n$  raised to the power  $m$  efficiently (fast exponentiation variant).

**TOPIC / PATTERN**

Fast exponentiation (binary exponentiation)

**ALGORITHM**

- Compute  $n^m$  in  $O(\log m)$  instead of  $O(m)$  repeated multiplication.
- Iterative form:  $\text{result} = 1$ ; while  $m > 0$ : if  $m$  is odd,  $\text{result} *= n$ ; then  $n *= n$  and  $m //= 2$ .
- For a negative exponent  $m$ , compute  $\text{power} = \text{Power}(n, -m)$  then return  $1 / \text{power}$ .
- If working under a modulus, take the mod after every multiplication to avoid overflow.

**COMPLEXITY**

| Approach              | Time        | Space                                    | Notes                                                   |
|-----------------------|-------------|------------------------------------------|---------------------------------------------------------|
| Binary exponentiation | $O(\log m)$ | $O(1)$ iterative / $O(\log m)$ recursive | Halve the exponent each step, square the base each step |

$O(\log m)$  base each step recursive

*Related: Count Good Numbers (7.1.3)*

**■ Interview Trigger**

Any “raise to a large power” problem → binary exponentiation: square the base, halve the exponent, multiply into the result only on odd exponents.

## 9.3 Monotonic Stack / Queue Problems (Addendum)

### 9.3.4 · Number of NGEs to the Right

#### WHAT IT'S ASKING

For each element, count how many elements to its right are strictly greater (Next Greater Element count).

#### TOPIC / PATTERN

For every index, count elements to its right that are strictly greater

#### ALGORITHM (FENWICK / BIT ON VALUE RANKS)

- Process the array from right to left, maintaining a Fenwick Tree (Binary Indexed Tree) indexed by the

VALUE RANK of elements seen so far (i.e. elements to the current index's right).

- For the current element, query the BIT for how many already-inserted values are strictly greater than it —

that count is the answer for this index.

- Insert the current element's rank into the BIT, then move one step further left and repeat.

#### WHY NOT A PLAIN MONOTONIC STACK

- A monotonic stack alone finds only the nearest/first greater element — it does not directly give a COUNT of

all greater elements to the right. Counting needs an order-statistics structure (BIT / order-statistics tree) over the suffix seen so far.

#### COMPLEXITY

| Approach                  | Time   | Space  | Notes                                                            |
|---------------------------|--------|--------|------------------------------------------------------------------|
| 4x monotonic stack passes | $O(N)$ | $O(N)$ | Two passes for the max-sum half, two passes for the min-sum half |

the BIT size

*Related: Next Greater Element, Count Inversions (Step 3), Reverse Pairs (Step 3)*

#### ■ Interview Trigger

“Count of greater/smaller elements to the right for every index” → think Fenwick Tree over ranks processed right-to-left, not a monotonic stack (which only finds the nearest one).

### 9.3.8 · Sum of Subarray Ranges

#### WHAT IT'S ASKING

For every subarray, sum up (max – min), summed across all subarrays.

#### TOPIC / PATTERN

Range = max – min of a subarray; split into two independent monotonic-stack problems

#### CORE IDEA

- $\text{sum}(\text{range}) \text{ over all subarrays} = \text{sum}(\text{max over all subarrays}) - \text{sum}(\text{min over all subarrays})$ .
- Each half is solved independently with the classic “contribution technique.”

**CONTRIBUTION TECHNIQUE FOR SUM OF MAXIMUMS**

- For each index  $i$ , find the Previous Greater Element index (PGE) and the Next Greater-or-Equal Element

index (NGE) using a monotonic decreasing stack (one pass each direction).

- $arr[i]$  is the maximum of exactly  $(i - \text{PGE}) \times (\text{NGE} - i)$  subarrays — add  $arr[i] \times$  that count to the running

sum.

**MIRROR FOR SUM OF MINIMUMS**

- Repeat with Previous Smaller Element and Next Smaller-or-Equal Element (monotonic increasing stack) to

get each element's contribution as a minimum.

**COMPLEXITY**

passes for the min-sum half

*Related: Sum of Subarray Minimums, Largest Rectangle in Histogram*

**■ Interview Trigger**

“Sum of max/min over all subarrays” is never brute force — split into independent contribution-technique passes with a monotonic stack, using  $(i - \text{prevGreater}) \times (\text{nextGreater} - i)$  as each element's subarray count.

**BIT MANIPULATION + STACK/QUEUE — MASTER****CHEATSHEET****BIT MANIPULATION QUICK REFERENCE**

| Operation              | Formula                            | Use Case                                   |
|------------------------|------------------------------------|--------------------------------------------|
| Check $i$ -th bit      | $(n \gg i) \& 1$                   | Bitmask DP, subset enumeration             |
| Set $i$ -th bit        | $n \mid (1 \ll i)$                 | Mark element as included                   |
| Clear $i$ -th bit      | $n \& \sim(1 \ll i)$               | Mark element as excluded                   |
| Check power of 2       | $n \& (n-1) == 0$                  | $O(1)$ power check                         |
| Count set bits         | Brian Kernighan: $n \& (n-1)$ loop | Hamming weight                             |
| Isolate lowest set bit | $n \& (-n)$                        | Splitting XOR results (two single numbers) |
| XOR self-cancel        | $a^a=0, a^0=a$                     | Find single/odd-occurring number           |

**STACK / QUEUE PATTERN TRIGGERS**

| Pattern              | Trigger Keywords                         | Key Tool                       |
|----------------------|------------------------------------------|--------------------------------|
| Matching/Balance     | 'valid parentheses', 'balanced brackets' | Stack of openers               |
| Next Greater/Smaller | 'next greater', 'daily temperatures'     | Monotonic stack, traverse RfIL |

|                         |                                          |                                |
|-------------------------|------------------------------------------|--------------------------------|
| Histogram Width         | 'largest rectangle', 'maximal rectangle' | Prev/Next smaller via stack    |
| Sliding Window Extremum | 'max/min in every window of size K'      | Monotonic deque                |
| Span / Distance         | 'stock span', 'days until warmer'        | Monotonic stack with distances |
| O(1) Recent Access      | 'LRU cache', 'most recent K'             | HashMap + Doubly Linked List   |
| Simulate Collisions     | 'asteroid collision', 'remove adjacent'  | Stack simulates interactions   |

### THE GOLDEN RULES

- 'Next/previous greater or smaller element' → ALWAYS monotonic stack, never brute force  $O(N^2)$ .
- Decreasing stack finds 'next GREATER'. Increasing stack finds 'next SMALLER'.
- Sliding window max/min → monotonic DEQUE (not stack — need to remove from both ends).
- XOR is the bit-manipulation Swiss army knife: cancellation, swap, single-number detection.
- Any 'O(1) cache with eviction' design question → HashMap + Doubly Linked List.

DSA Revision Notes Striver's A2Z DSA Sheet

# DSA Revision Notes

## Striver's A2Z DSA Sheet

### Batch 7 · Steps 10, 11 & 12: Sliding Window + Heaps + Greedy

Problems 31 problems — Window techniques, Heaps, Greedy strategies

Patterns Variable Window · Two Heaps · Activity Selection · Interval Scheduling

Format Pattern · Intuition · Algorithm · Complexity · Traps · Trigger

Three different 'optimal local choice' philosophies: shrink/expand, top-of-heap, and greedy sort-first.

#### TABLE OF CONTENTS

##### 10.1 SLIDING WINDOW — MEDIUM

- 10.1.1 Longest Substring Without Repeating Characters
- 10.1.2 Max Consecutive Ones III
- 10.1.3 Fruit Into Baskets
- 10.1.4 Longest Repeating Character Replacement
- 10.1.5 Binary Subarray with Sum
- 10.1.6 Count Number of Nice Subarrays
- 10.1.7 Number of Substrings Containing All Three Characters
- 10.1.8 Maximum Points You Can Obtain from Cards

##### 10.2 SLIDING WINDOW — HARD

- 10.2.1 Longest Substring with At Most K Distinct Characters
- 10.2.2 Subarray with K Different Integers
- 10.2.3 Minimum Window Substring
- 10.2.4 Minimum Window Subsequence

##### 11.1 / 11.2 / 11.3 HEAPS

- 11.1 Binary Heap Theory — Min/Max Heap, Heapify
- 11.2.1 Kth Largest Element
- 11.2.2 Kth Smallest Element
- 11.2.4 Merge M Sorted Lists
- 11.2.6 Task Scheduler
- 11.2.7 Hands of Straights
- 11.3.1 Design Twitter
- 11.3.2 Connect N Ropes (Min Cost)
- 11.3.5 Find Median from Data Stream
- 11.3.6 K Most Frequent Elements

##### 12.1 / 12.2 GREEDY ALGORITHMS

- 12.1.1 Assign Cookies
- 12.1.2 Fractional Knapsack
- 12.1.3 Minimum Coins (Greedy)
- 12.1.4 Lemonade Change
- 12.2.1 N Meetings in One Room
- 12.2.2 Jump Game / Jump Game 2
- 12.2.4 Minimum Platforms

12.2.5 Job Sequencing Problem

12.2.6 Candy

12.2.8 Insert Interval / Merge Intervals

12.2.10 Non-overlapping Intervals

**STEP 10 — Sliding Window & Two Pointer**

Universal Variable Sliding Window Template

left = 0

for right in range(N):

# expand window: include arr[right]

while window\_invalid():

# shrink window: remove arr[left]

left += 1

# window [left..right] is now valid

update\_answer(right - left + 1)

– Every index visited by 'right' at most once, every index visited by 'left' at most once →  $O(N)$  total, NOT  $O(N^2)$ .

– 'At most K' problems: window can grow/shrink freely. 'Exactly K' problems:  $\text{atMost}(K) - \text{atMost}(K-1)$ .

10.1 Medium Problems

**10.1.1 · Longest Substring Without Repeating Characters****WHAT IT'S ASKING**

Find the length of the longest substring without repeating characters.

**TOPIC / PATTERN**

Sliding Window — Variable size with HashMap

**ALGORITHM**

- 1. Maintain a map of last-seen index for each character.
- 2. For each right: if arr[right] seen before AND its index  $\geq$  left, move left = lastIndex+1.
- 3. Update lastSeen[arr[right]] = right.
- 4. maxLen = max(maxLen, right - left + 1).

**COMPLEXITY**

| Approach       | Time                 | Space  | Notes                         |
|----------------|----------------------|--------|-------------------------------|
| Brute          | $O(N^2)$ or $O(N^3)$ | $O(K)$ | Check every substring         |
| Sliding Window | $O(N)$               | $O(K)$ | K = charset size, single pass |

Related: [LeetCode 3 Longest Substring Without Repeating Characters](#)**■ Interview Trigger**

Track last-seen index — jump left pointer directly instead of shrinking one by one.

**10.1.2 · Max Consecutive Ones III (At Most K Zero Flips)**

**WHAT IT'S ASKING**

Find the longest subarray with at most K zeros flipped to 1s (max consecutive ones III).

**TOPIC / PATTERN**

Sliding Window — Variable size with violation counter

**ALGORITHM**

- 1. zeroCount = 0, left = 0.
- 2. For right: if arr[right]==0, zeroCount++.
- 3. While zeroCount > K: if arr[left]==0, zeroCount--. left++.
- 4. maxLen = max(maxLen, right - left + 1).

**COMPLEXITY**

| Approach       | Time | Space | Notes                                   |
|----------------|------|-------|-----------------------------------------|
| Sliding Window | O(N) | O(1)  | Window never shrinks more than it grows |

*Related: LeetCode 1004 Max Consecutive Ones III*

**■ Interview Trigger**

Allow up to K zeros in window — shrink only when zero count exceeds K.

**10.1.3 · Fruit Into Baskets****WHAT IT'S ASKING**

Given baskets that can hold only two types of fruit, find the maximum fruits you can collect from a row of trees.

**TOPIC / PATTERN**

Sliding Window — At most 2 distinct types (generalizes to K)

**CORE IDEA**

This is literally 'longest subarray with at most 2 distinct elements' in disguise.

**ALGORITHM**

- Maintain a frequency map. Shrink window while distinct count > 2.
- maxLen = max(maxLen, right - left + 1) at every step.

**COMPLEXITY**

| Approach               | Time   | Space | Notes                 |
|------------------------|--------|-------|-----------------------|
| Sliding Window + HashM | aOp(N) | O(1)  | At most 2 keys in map |

Sliding Window + HashMap

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|                |        |         |                                  |
|----------------|--------|---------|----------------------------------|
| Sliding Window | $O(N)$ | $O(26)$ | Fixed alphabet size for freq map |
|----------------|--------|---------|----------------------------------|

*Related: LeetCode 904 Fruit Into Baskets*

#### ■ Interview Trigger

Re-read the problem: 'fruit baskets' = longest window with  $\leq 2$  distinct values.

## 10.1.4 · Longest Repeating Character Replacement

### WHAT IT'S ASKING

Find the length of the longest substring after replacing at most  $K$  characters to make all characters the same.

### TOPIC / PATTERN

Sliding Window — Track max frequency character in window

### CORE IDEA

A window of length  $L$  is valid if  $(L - \text{maxFreqChar}) \leq K$  (we can replace the non-majority characters with the majority one).

### ALGORITHM

- 1. Maintain frequency map.  $\text{maxFreq}$  = max count of any single char in window.
- 2. For right: update freq map,  $\text{maxFreq} = \max(\text{maxFreq}, \text{freq}[\text{arr}[\text{right}]])$ .
- 3. While  $(\text{window length} - \text{maxFreq}) > K$ : shrink left, update freq.
- 4.  $\text{maxLen} = \max(\text{maxLen}, \text{window length})$ .

### COMMON MISTAKES

- $\text{maxFreq}$  doesn't need to be recalculated on shrink — it's safe to leave it stale since we only care about

the maximum achievable window length, not exact validity at every point.

### COMPLEXITY

*Related: LeetCode 424 Longest Repeating Character Replacement*

#### ■ Interview Trigger

Window valid if  $\text{length} - \text{maxFreqInWindow} \leq K$  — replace the rest.

## 10.1.5 / 10.1.6 / 10.1.7 · Binary Subarray Sum, Nice Subarrays, 3-Character

### WHAT IT'S ASKING

Count subarrays whose sum equals a goal in a binary array, count subarrays with exactly  $K$  odd numbers, and count substrings containing all three characters at least once.

Substrings

### TOPIC / PATTERN

Sliding Window — 'Exactly K' via  $\text{atMost}(K) - \text{atMost}(K-1)$

#### CORE TRICK — EXACTLY K PATTERN

'Exactly K' conditions are hard to maintain directly with shrink/grow logic. Instead:  $\text{count}(\text{exactly } K) = \text{count}(\text{at most } K) - \text{count}(\text{at most } K-1)$ .

#### BINARY SUBARRAY WITH SUM = GOAL

- $\text{atMostSum}(S)$ : count subarrays with  $\text{sum} \leq S$  using standard sliding window ( $S \geq 0$ ).
- $\text{Answer} = \text{atMostSum}(\text{goal}) - \text{atMostSum}(\text{goal}-1)$ .

#### COUNT NICE SUBARRAYS (EXACTLY K ODD NUMBERS)

- $\text{atMostOdd}(K)$ : count subarrays with at most K odd numbers.
- $\text{Answer} = \text{atMostOdd}(K) - \text{atMostOdd}(K-1)$ .

#### NUMBER OF SUBSTRINGS CONTAINING ALL 3 CHARACTERS (A,B,C)

- Different pattern: for each right, find the smallest left where window is still valid (contains all 3).
- Count += left (every smaller window from 0 to left-1 as start is also valid).

#### COMPLEXITY

| Approach            | Time   | Space  | Notes                                   |
|---------------------|--------|--------|-----------------------------------------|
| Sliding Window (2x) | $O(N)$ | $O(1)$ | Two passes via $\text{atMost}(K)$ trick |

*Related: LeetCode 930, 992, 1248, 1358 — all 'exactly K' sliding window variants*

#### ■ Interview Trigger

'Exactly K' =  $\text{atMost}(K) - \text{atMost}(K-1)$ . Turns a hard constraint into two easy ones.

## 10.1.8 · Maximum Points You Can Obtain from Cards

#### WHAT IT'S ASKING

Given cards in a row and K picks from either end, maximize the sum of picked cards.

#### TOPIC / PATTERN

Sliding Window — Reframe as 'minimum sum subarray to EXCLUDE'

#### CORE IDEA

Picking K cards from either end is equivalent to EXCLUDING a contiguous subarray of size  $(N-K)$  from the middle. Minimize that excluded sum to maximize the picked sum.

#### ALGORITHM

1.  $\text{windowSize} = N - K$ . Find minimum sum subarray of this fixed size (standard fixed window).
2.  $\text{Answer} = \text{totalSum} - \text{minExcludedSum}$ .

#### COMPLEXITY

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|              |        |        |                                                        |
|--------------|--------|--------|--------------------------------------------------------|
| Fixed Window | $O(N)$ | $O(1)$ | Single pass — much simpler than picking from both ends |
|--------------|--------|--------|--------------------------------------------------------|

Related: [LeetCode 1423 Maximum Points You Can Obtain from Cards](#)

#### ■ Interview Trigger

Picking from both ends = excluding a fixed-size middle window. Minimize that instead.

### 10.2 Hard Problems

## 10.2.1 / 10.2.2 · Longest Substring with At Most K Distinct, Subarray with K

### WHAT IT'S ASKING

Find the longest substring containing at most K distinct characters, and count subarrays with exactly K distinct integers.

Different Integers

### TOPIC / PATTERN

Sliding Window — Distinct count tracking

### AT MOST K DISTINCT

- Standard variable window: shrink while distinct count (map size)  $> K$ .

### EXACTLY K DIFFERENT INTEGERS

- Answer =  $\text{atMostK}(K) - \text{atMostK}(K-1)$  (the exactly-K trick again).

### COMPLEXITY

| Approach       | Time   | Space  | Notes                 |
|----------------|--------|--------|-----------------------|
| Sliding Window | $O(N)$ | $O(K)$ | Map size bounded by K |

Related: [LeetCode 340 Longest Substring with At Most K Distinct](#), [LeetCode 992 Subarrays with K Different Integers](#)

#### ■ Interview Trigger

Distinct count = map size. Exactly K reduces to  $\text{atMost}(K) - \text{atMost}(K-1)$  again.

## 10.2.3 · Minimum Window Substring

### WHAT IT'S ASKING

Find the smallest window in a string that contains every character of another string.

### TOPIC / PATTERN

Sliding Window — Variable window, minimize valid window size

### CORE IDEA

Expand right until window contains all characters of pattern t (with correct frequencies). Then shrink left as much as possible while staying valid, recording the minimum.

**ALGORITHM**

- 1. need = frequency map of t. required = number of unique chars in t.
- 2. formed = 0 (how many unique chars currently satisfy their required frequency).
- 3. Expand right: update window freq. If freq matches need for that char, formed++.
- 4. While formed == required: update min window. Shrink left, adjust formed if it breaks.

**COMPLEXITY**

| Approach       | Time     | Space  | Notes                                                 |
|----------------|----------|--------|-------------------------------------------------------|
| Brute          | $O(N^2)$ | $O(K)$ | Check every substring                                 |
| Sliding Window | $O(N+M)$ | $O(K)$ | $M=\text{len}(t)$ , $K=\text{charset size}$ — optimal |

**COMMON MISTAKES**

- Comparing the ENTIRE frequency maps on every step (slow) — use the 'formed' counter trick instead.

*Related: [LeetCode 76 Minimum Window Substring](#)*

**■ Interview Trigger**

Expand until valid (formed==required), then greedily shrink while still valid.

## 10.2.4 · Minimum Window Subsequence

**WHAT IT'S ASKING**

Find the smallest window in string S that contains string T as a subsequence (not just characters, in order).

**TOPIC / PATTERN**

Sliding Window — Two-pointer with backward verification (subsequence  $\neq$  substring of chars)

**CORE IDEA**

Unlike Minimum Window Substring, here T must appear as a SUBSEQUENCE (order matters, no frequency map trick). Use two pointers: forward match, then backward shrink.

**ALGORITHM**

- 1. Forward: find the first window in S where T appears as a subsequence (two pointers).
- 2. Backward: from the end of that window, walk backward matching T in reverse to find the tightest start.
- 3. Record window length. Continue searching from start+1 for potentially smaller windows.

**COMPLEXITY**

| Approach             | Time            | Space  | Notes                                                       |
|----------------------|-----------------|--------|-------------------------------------------------------------|
| Two-Pass Two Pointer | $O(N \times M)$ | $O(1)$ | $N=\text{len}(S)$ , $M=\text{len}(T)$ — worst case retraces |

*Related: [LeetCode 727 Minimum Window Subsequence](#)*

### ■ Interview Trigger

Subsequence (order matters) needs forward-match then backward-tighten — not a frequency map.

## STEP 11 - Heaps

### 11.1 Learning — Heap Theory

## 11.1 · Binary Heap Theory — Min/Max Heap, Heapify

### WHAT IT'S ASKING

Understand how a binary min/max heap is stored in an array and how heapify restores the heap property.

### TOPIC / PATTERN

Heap — Complete binary tree stored as an array, satisfying heap property

### CORE PROPERTIES

- Max Heap: parent  $\geq$  both children. Root = maximum element.
- Min Heap: parent  $\leq$  both children. Root = minimum element.
- Array indexing: for node at index  $i \rightarrow$  left child =  $2i+1$ , right child =  $2i+2$ , parent =  $(i-1)/2$ .

### KEY OPERATIONS

- Insert: Add to end, 'bubble up' (swap with parent while violating heap property).  $O(\log N)$ .
- Extract Min/Max: Swap root with last element, remove last, 'bubble down' (heapify) the new root.  $O(\log N)$ .
- Build Heap from array: Heapify from the last non-leaf node backward to root.  $O(N)$ , NOT  $O(N \log N)$ !

### WHY BUILD-HEAP IS $O(N)$ , NOT $O(N \log N)$

Most nodes are near the bottom (leaves), where heapify does almost no work. The math works out to a geometric series that sums to  $O(N)$ .

### COMPLEXITY SUMMARY

| Approach        | Time        | Space  | Notes                                 |
|-----------------|-------------|--------|---------------------------------------|
| Insert          | $O(\log N)$ | $O(1)$ | Bubble up                             |
| Extract Min/Max | $O(\log N)$ | $O(1)$ | Bubble down (heapify)                 |
| Peek Min/Max    | $O(1)$      | $O(1)$ | Just look at root                     |
| Build Heap      | $O(N)$      | $O(1)$ | Bottom-up heapify — NOT $O(N \log N)$ |

### ■ Interview Trigger

Heap = complete binary tree in an array. Build is  $O(N)$ , insert/extract are  $O(\log N)$ .

## 11.2.1 / 11.2.2 · Kth Largest and Kth Smallest Element

**WHAT IT'S ASKING**

Find the Kth largest and Kth smallest element in an unsorted array using a heap.

**TOPIC / PATTERN**

Heap — Fixed-size heap technique

**KTH LARGEST — USE MIN HEAP OF SIZE K**

Counterintuitive but key insight: maintain a min-heap of the K LARGEST elements seen so far. The root (minimum of these K) is the Kth largest overall.

- 1. Push elements into a min-heap. If heap size > K, pop the smallest.
- 2. After processing all elements, heap root = Kth largest.

**KTH SMALLEST — USE MAX HEAP OF SIZE K (SYMMETRIC LOGIC)****WHY THIS APPROACH?**

- Keeps heap size bounded at K (not N) →  $O(N \log K)$  instead of  $O(N \log N)$ .

**COMPLEXITY**

| Approach       | Time          | Space  | Notes                                     |
|----------------|---------------|--------|-------------------------------------------|
| Sort           | $O(N \log N)$ | $O(1)$ | Simple but not optimal for streaming data |
| Heap of size K | $O(N \log K)$ | $O(K)$ | Better when $K \ll N$                     |
| Quickselect    | $O(N)$ avg    | $O(1)$ | Optimal average case, $O(N^2)$ worst      |

*Related: LeetCode 215 Kth Largest Element in an Array, LeetCode 703 Kth Largest in a Stream*

**■ Interview Trigger**

Kth largest = min-heap of size K. Kth smallest = max-heap of size K.

**11.2.4 · Merge M Sorted Lists****WHAT IT'S ASKING**

Merge K sorted linked lists into one sorted linked list.

**TOPIC / PATTERN**

Heap — K-way merge using a min-heap

**CORE IDEA**

Push the first element of each list into a min-heap (with list index tracked). Pop the minimum, push the next element from that same list. Repeat.

**ALGORITHM**

- 1. Push (value, listIndex, elementIndex) for the first element of each list.
- 2. While heap not empty: pop min, add to result.
- 3. If that list has a next element, push it onto the heap.

**COMPLEXITY**

| Approach              | Time          | Space  | Notes                                      |
|-----------------------|---------------|--------|--------------------------------------------|
| Brute — Concat & Sort | $O(N \log N)$ | $O(N)$ | $N$ = total elements across all lists      |
| Min Heap              | $O(N \log M)$ | $O(M)$ | $M$ = number of lists, heap bounded by $M$ |

Related: [LeetCode 23 Merge K Sorted Lists](#)

#### ■ Interview Trigger

Min-heap holds one 'frontier' element per list — always pop the global minimum next.

## 11.2.6 · Task Scheduler

### WHAT IT'S ASKING

Schedule CPU tasks with a cooldown period between identical tasks, minimizing total time.

### TOPIC / PATTERN

Heap / Greedy — Max heap with cooldown tracking

### CORE IDEA

Always schedule the most frequent remaining task first (greedy). Use a max-heap of frequencies + a cooldown queue to track when tasks become available again.

### ALGORITHM (HEAP + QUEUE)

- 1. Max-heap of task frequencies. Cooldown queue holds (count, available\_time).
- 2. At each time unit: pop most frequent task from heap, decrement count, push to cooldown queue with

availableTime = time + n + 1.

- 3. If cooldown queue's front task is now available (time matches), push back into heap.

### ALTERNATIVE — MATH FORMULA (NO SIMULATION)

- maxFreq = highest task frequency. idleSlots = (maxFreq - 1) × n.
- Fill idle slots with other tasks; if more tasks than idle slots, no idle time needed.
- Answer = max(len(tasks), (maxFreq - 1) × (n + 1) + countOfTasksWithMaxFreq).

### COMPLEXITY

| Approach        | Time           | Space   | Notes                          |
|-----------------|----------------|---------|--------------------------------|
| Heap Simulation | $O(N \log 26)$ | $O(26)$ | Simulate time unit by unit     |
| Math Formula    | $O(N)$         | $O(26)$ | Direct formula — no simulation |

Related: [LeetCode 621 Task Scheduler](#)

#### ■ Interview Trigger

Greedy: always run the most frequent task. Or use the idle-slot math formula directly.

## 11.2.7 · Hands of Straights

### WHAT IT'S ASKING

Group numbers into groups of consecutive integers of a required size (Hands of Straights).

### TOPIC / PATTERN

Greedy + HashMap/TreeMap — Form consecutive groups

### ALGORITHM

- 1. Count frequency of each card. Use a sorted structure (TreeMap/sorted keys).
- 2. For each smallest remaining card with freq > 0: try to form a consecutive group of size W starting from
- it. Decrement frequency of each card in that group.
- 3. If any required card is missing → return False.

### COMPLEXITY

| Approach        | Time          | Space  | Notes                           |
|-----------------|---------------|--------|---------------------------------|
| Sorted + Greedy | $O(N \log N)$ | $O(N)$ | Sort once, then greedy grouping |

*Related: LeetCode 846 Hand of Straights, LeetCode 1296 Divide Array in Sets of K Consecutive*

#### ■ Interview Trigger

Always start a group from the smallest available card — greedy consecutive grouping.

## 11.3.2 · Connect N Ropes with Minimum Cost

### WHAT IT'S ASKING

Given rope lengths, connect them all into one rope while minimizing the total connection cost.

### TOPIC / PATTERN

Heap — Greedy, always combine the two smallest

### CORE IDEA

To minimize total cost, always merge the two SMALLEST ropes first — this is the same pattern as Huffman coding.

### ALGORITHM

- 1. Push all rope lengths into a min-heap.
- 2. While heap.size() > 1: pop two smallest, cost += their sum, push the sum back.
- 3. Return total accumulated cost.

### COMPLEXITY

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|          |               |        |                                        |
|----------|---------------|--------|----------------------------------------|
| Min Heap | $O(N \log N)$ | $O(N)$ | N-1 merge operations, each $O(\log N)$ |
|----------|---------------|--------|----------------------------------------|

*Related: GFG Minimum Cost of ropes, Huffman Encoding (same core idea)*

#### ■ Interview Trigger

Greedily combine the two smallest values — min-heap is the natural structure.

## 11.3.5 · Find Median from Data Stream

### WHAT IT'S ASKING

Design a data structure that returns the median of all numbers seen so far in a stream.

### TOPIC / PATTERN

Heap — Two Heaps (max-heap for lower half, min-heap for upper half)

### CORE IDEA

Maintain two heaps that split the data in half: a MAX-heap holding the smaller half, and a MIN-heap holding the larger half. Keep them balanced (size difference  $\leq 1$ ). Median is either the top of the larger heap, or the average of both tops.

### ALGORITHM — INSERT

- 1. Push new number to maxHeap (lower half).
- 2. Move maxHeap's top to minHeap (ensures lower half  $\leq$  upper half).
- 3. Rebalance: if  $\text{minHeap.size()} > \text{maxHeap.size()}$ , move minHeap's top back to maxHeap.

### GET MEDIAN

- If sizes equal: average of both heap tops. If maxHeap is larger: maxHeap's top.

### VISUALIZATION

```
Stream: 5, 15, 1, 3
maxHeap (lower half) ← [5,3,1] minHeap (upper half) → [15]
After balancing: maxHeap=[3,1], minHeap=[5,15]
Median = (max(maxHeap) + min(minHeap)) / 2 = (3+5)/2 = 4
```

### COMPLEXITY

| Approach            | Time                   | Space  | Notes                               |
|---------------------|------------------------|--------|-------------------------------------|
| Sorted Array Insert | $O(N)$ per insert      | $O(N)$ | Insert maintains sort — slow insert |
| Two Heaps           | $O(\log N)$ per insert | $O(N)$ | getMedian is $O(1)$ — optimal       |

*Related: LeetCode 295 Find Median from Data Stream*

#### ■ Interview Trigger

Two heaps split data at the median — max-heap below, min-heap above, keep balanced.

## 11.3.6 · K Most Frequent Elements

### WHAT IT'S ASKING

Find the K most frequently occurring elements in an array.

### TOPIC / PATTERN

Heap — Min-heap of size K, or Bucket Sort

### APPROACH 1 — MIN HEAP OF SIZE K

- Count frequencies. Push (freq, element) into min-heap. If size > K, pop smallest. Result = heap contents.

### APPROACH 2 — BUCKET SORT (OPTIMAL $O(N^2)$ )

- Create buckets indexed by frequency (1 to N). Place each element in bucket[freq].
- Traverse buckets from highest frequency down, collecting elements until K are found.

### COMPLEXITY

| Approach           | Time          | Space  | Notes                               |
|--------------------|---------------|--------|-------------------------------------|
| Sort by Frequency  | $O(N \log N)$ | $O(N)$ | Simple                              |
| Min Heap of Size K | $O(N \log K)$ | $O(N)$ | Better when $K \ll N$               |
| Bucket Sort        | $O(N)$        | $O(N)$ | Optimal — frequency is bounded by N |

*Related: LeetCode 347 Top K Frequent Elements*

#### ■ Interview Trigger

Bucket sort by frequency gives  $O(N)$  — frequency values are bounded by array length.

### ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING

## 11.3.1 · Design Twitter

### WHAT IT'S ASKING

Design a simplified Twitter: post tweets and fetch a user's 10 most recent tweets from themselves and everyone they follow.

### TOPIC / PATTERN

Heaps — Design + Merge K Sorted Lists idea

### CORE IDEA

Store each user's tweets with a global timestamp, and each user's follow list in a set. For a news feed, merge the most recent tweets from the user and everyone they follow using a max-heap, keeping only the top 10.

### ALGORITHM

1. postTweet(user, id): append (timestamp++, id) to user's tweet list.
2. follow(a, b) / unfollow(a, b): maintain a  $\text{HashMap}<\text{user}, \text{Set}<\text{followee}>>$ .

- 3. `getNewsFeed(user)`: push the most recent tweet from user and each followee into a max-heap keyed by timestamp; pop the top 10, pushing each popped tweet's predecessor back in.

#### COMPLEXITY

- Time:  $O(F \log F)$  per `getNewsFeed`,  $F$  = number of followees
- Space:  $O(T + F)$  for tweets and follow graph

#### ■ Interview Trigger

One pointer per followee's tweet list + a max-heap = merge  $K$  sorted lists, capped at 10.

## STEP 12 - Greedy Algorithms

### 12.1 Easy Problems

**Greedy Mindset:** Make the locally optimal choice at each step, trusting that it leads to a globally optimal solution. Works only when the problem has 'greedy choice property' — usually provable via exchange argument or sorting.

#### 12.1.1 · Assign Cookies

##### WHAT IT'S ASKING

Assign cookies to children to satisfy as many of their greed factors as possible.

##### TOPIC / PATTERN

Greedy — Sort both arrays, match smallest feasible

##### ALGORITHM

- 1. Sort children's greed factors and cookie sizes.
- 2. Two pointers: for each child (smallest greed first), find the smallest cookie that satisfies them.
- 3. If satisfied, move both pointers. Else, move only the cookie pointer.

##### COMPLEXITY

| Approach           | Time          | Space  | Notes             |
|--------------------|---------------|--------|-------------------|
| Sort + Two Pointer | $O(N \log N)$ | $O(1)$ | Sorting dominates |

*Related: [LeetCode 455 Assign Cookies](#)*

#### ■ Interview Trigger

Sort both, greedily satisfy the least greedy child with the smallest sufficient cookie.

#### 12.1.2 · Fractional Knapsack

##### WHAT IT'S ASKING

Maximize the total value you can carry when items can be taken in fractions (fractional knapsack).

##### TOPIC / PATTERN

Greedy — Sort by value/weight ratio

**CORE IDEA**

Unlike 0/1 Knapsack, items CAN be broken into fractions. Greedily take items with the highest value-per-weight ratio first.

**ALGORITHM**

- 1. Sort items by (value/weight) descending.
- 2. For each item: if it fits entirely, take it all. Else take the fraction that fits, then stop.

**WHY GREEDY WORKS HERE (BUT NOT FOR 0/1 KNAPSACK)**

- Fractional allows taking PART of an item — no risk of 'wasted' capacity, so greedy ratio-based selection

is always optimal. 0/1 Knapsack lacks this flexibility, requiring DP.

**COMPLEXITY**

| Approach      | Time          | Space  | Notes          |
|---------------|---------------|--------|----------------|
| Sort + Greedy | $O(N \log N)$ | $O(1)$ | Sort dominates |

**■ Interview Trigger**

Fractional allowed → greedy by value/weight ratio works. 0/1 → needs DP instead.

**12.1.3 · Minimum Number of Coins (Greedy)****WHAT IT'S ASKING**

Find the minimum number of coins needed to make a given amount using a greedy approach.

**TOPIC / PATTERN**

Greedy — Largest denomination first

**ALGORITHM**

- Sort denominations descending. Repeatedly take the largest coin that fits remaining amount.

**CRITICAL CAVEAT**

Greedy coin change ONLY works for 'canonical' coin systems (like standard currency: 1,2,5,10...). For arbitrary denominations (e.g. [1,3,4] for amount 6), greedy gives suboptimal answers (greedy: 4+1+1=3 coins; optimal: 3+3=2 coins). General case needs DP (Coin Change problem).

**COMPLEXITY**

| Approach                | Time          | Space  | Notes              |
|-------------------------|---------------|--------|--------------------|
| Greedy (canonical only) | $O(N \log N)$ | $O(1)$ | Sort + linear scan |

**■ Interview Trigger**

Greedy coin change is a TRAP for non-canonical systems — verify or default to DP.

**12.1.4 · Lemonade Change**

**WHAT IT'S ASKING**

Given a \$5/\$10/\$20 bill queue, determine if you can give correct change to every customer.

**TOPIC / PATTERN**

Greedy — Prefer giving larger bills as change first

**ALGORITHM**

- Track count of \$5 and \$10 bills.
- For \$5 bill: just collect it (five++).
- For \$10 bill: need one \$5 as change. five--, ten++.
- For \$20 bill: prefer giving one \$10 + one \$5 (greedy — save more \$5s for flexibility). If no \$10, give three

\$5s.

**WHY PREFER \$10+\$5 OVER THREE \$5S?**

\$5 bills are more versatile (needed for both \$10 and \$20 change) — conserve them when possible.

**COMPLEXITY**

| Approach       | Time | Space | Notes                     |
|----------------|------|-------|---------------------------|
| Greedy Counter | O(N) | O(1)  | Single pass, two counters |

*Related: [LeetCode 860 Lemonade Change](#)*

**■ Interview Trigger**

\$5 bills are precious — always prefer using a \$10 for change over two \$5s.

12.2 Medium / Hard Problems

**12.2.1 · N Meetings in One Room****WHAT IT'S ASKING**

Given meeting start/end times, find the maximum number of meetings that can be held in one room.

**TOPIC / PATTERN**

Greedy — Activity Selection (Sort by END time)

**CORE IDEA**

To maximize the number of meetings in a single room, always pick the meeting that ENDS earliest among remaining options. This leaves the most room for future meetings.

**ALGORITHM**

- 1. Sort meetings by end time.
- 2. Take the first meeting. lastEnd = its end time.
- 3. For each subsequent meeting: if start > lastEnd → take it, update lastEnd.

**WHY SORT BY END (NOT START OR DURATION)?**

- Sorting by start time can pick a long meeting first, blocking many short ones. Sorting by end time always

leaves maximum remaining time — provably optimal via exchange argument.

### COMPLEXITY

| Approach      | Time          | Space  | Notes          |
|---------------|---------------|--------|----------------|
| Sort + Greedy | $O(N \log N)$ | $O(1)$ | Sort dominates |

*Related: LeetCode 435 Non-overlapping Intervals (inverse framing), Activity Selection Problem*

#### ■ Interview Trigger

THE classic greedy: always sort intervals by END time, not start time.

## 12.2.2 · Jump Game and Jump Game II

### WHAT IT'S ASKING

Determine if you can reach the last index of an array via jumps, and find the minimum number of jumps needed.

### TOPIC / PATTERN

Greedy — Track farthest reachable index

#### JUMP GAME I — CAN YOU REACH THE END?

- Track maxReach. For each index  $i$  (if  $i \leq \text{maxReach}$ ):  $\text{maxReach} = \max(\text{maxReach}, i + \text{arr}[i])$ .
- If  $\text{maxReach} \geq N-1$  at any point  $\rightarrow$  return True. If  $i > \text{maxReach} \rightarrow$  return False (stuck).

#### JUMP GAME II — MINIMUM JUMPS TO REACH THE END

BFS-like greedy: track the current jump's boundary and the farthest reachable in the next jump. When you exhaust the current boundary, increment jump count.

- 1. jumps=0, currentEnd=0, farthest=0.
- 2. For  $i = 0$  to  $N-2$ :  $\text{farthest} = \max(\text{farthest}, i + \text{arr}[i])$ .
- 3. If  $i == \text{currentEnd}$ : jumps++, currentEnd = farthest.

### VISUALIZATION (JUMP GAME II)

```
arr = [2,3,1,1,4]
i=0: farthest=2, i==currentEnd(0) → jumps=1, currentEnd=2
i=1: farthest=4
i=2: farthest=4, i==currentEnd(2) → jumps=2, currentEnd=4
Done – minimum jumps = 2 ✓
```

### COMPLEXITY

| Approach     | Time   | Space  | Notes                              |
|--------------|--------|--------|------------------------------------|
| Jump Game I  | $O(N)$ | $O(1)$ | Track max reachable, single pass   |
| Jump Game II | $O(N)$ | $O(1)$ | BFS-level-like greedy, single pass |

*Related: LeetCode 55 Jump Game, LeetCode 45 Jump Game II*

**■ Interview Trigger**

Track farthest reachable index — when current jump's range ends, force a new jump.

## 12.2.4 · Minimum Number of Platforms

### WHAT IT'S ASKING

Given arrival and departure times of trains, find the minimum number of platforms needed.

### TOPIC / PATTERN

Greedy — Sort arrivals and departures separately, two-pointer sweep

### ALGORITHM

- 1. Sort arrival times and departure times independently.
- 2. Two pointers  $i$  (arrivals),  $j$  (departures).  $platforms=0$ ,  $maxPlatforms=0$ .
- 3. If  $arrival[i] \leq departure[j]$ :  $platforms++$ ,  $i++$  (train arrives, needs new platform).
- 4. Else:  $platforms--$ ,  $j++$  (train departs, frees a platform).
- 5.  $maxPlatforms = \max(maxPlatforms, platforms)$  at every step.

### COMPLEXITY

| Approach           | Time          | Space  | Notes          |
|--------------------|---------------|--------|----------------|
| Sort + Two Pointer | $O(N \log N)$ | $O(1)$ | Sort dominates |

### COMMON MISTAKES

- Using  $\leq$  consistently matters — if arrival equals departure, the platform IS still needed simultaneously.

*Related: GFG Minimum Platforms*

**■ Interview Trigger**

Sweep sorted arrivals and departures together — count overlapping intervals at peak.

## 12.2.5 · Job Sequencing Problem

### WHAT IT'S ASKING

Schedule jobs with deadlines and profits to maximize total profit, doing at most one job per slot.

### TOPIC / PATTERN

Greedy — Sort by profit, schedule as late as possible

### CORE IDEA

Each job has a deadline and profit; each job takes 1 unit of time. Greedily process jobs by HIGHEST profit first, scheduling each as LATE as possible (closest to its deadline) to leave earlier slots open for other jobs.

### ALGORITHM

- 1. Sort jobs by profit descending.

- 2. For each job: find a free slot from its deadline backward to slot 1. If found, schedule it there.
- 3. Use a boolean array (or Disjoint Set for optimization) to track occupied slots.

**COMPLEXITY**

| Approach            | Time                           | Space                 | Notes                                  |
|---------------------|--------------------------------|-----------------------|----------------------------------------|
| Sort + Linear Scan  | $O(N \times \text{maxDeadli})$ | $O(\text{maxDeadli})$ | Simple slot search                     |
| Sort + Disjoint Set | $O(N \log N)$                  | $O(\text{maxDeadli})$ | Optimal — $O(1)$ amortized slot lookup |

Sort + Linear Scan  $O(N \times \text{maxDeadline})$   $O(\text{maxDeadline})$  Simple slot search

Optimal —  $O(1)$  amortized slot lookup

*Related: GFG Job Sequencing Problem*

■ Interview Trigger

Highest profit first, schedule as late as possible — preserves flexibility for others.

## 12.2.6 · Candy

**WHAT IT'S ASKING**

Distribute candies to children standing in a line so higher-rated children get more candy than their neighbors.

**TOPIC / PATTERN**

Greedy — Two-pass (left-to-right, then right-to-left)

**CORE IDEA**

Each child needs at least 1 candy, and a child with a higher rating than a neighbor must get more candy than that neighbor. Solve left-constraint and right-constraint separately, then take the max.

**ALGORITHM**

- 1.  $\text{candies}[i] = 1$  for all, initially.
- 2. Left to right: if  $\text{rating}[i] > \text{rating}[i-1] \rightarrow \text{candies}[i] = \text{candies}[i-1] + 1$ .
- 3. Right to left: if  $\text{rating}[i] > \text{rating}[i+1] \rightarrow \text{candies}[i] = \max(\text{candies}[i], \text{candies}[i+1] + 1)$ .
- 4. Sum all  $\text{candies}[i]$ .

**WHY TWO PASSES?**

A single pass can't satisfy both 'greater than left neighbor' AND 'greater than right neighbor' constraints simultaneously — need both directions, then merge with max.

**COMPLEXITY**

| Approach        | Time   | Space  | Notes                       |
|-----------------|--------|--------|-----------------------------|
| Two-Pass Greedy | $O(N)$ | $O(N)$ | Linear, two passes plus sum |

*Related: LeetCode 135 Candy*

**■ Interview Trigger**

Two passes (L→R then R→L), each enforcing one direction's constraint — merge with max.

## 12.2.8 / 12.2.9 / 12.2.10 · Insert Interval, Merge Intervals, Non-overlapping

### WHAT IT'S ASKING

Insert a new interval into a sorted list of intervals, merge overlapping intervals, and remove the minimum intervals to make the rest non-overlapping.

Intervals

### TOPIC / PATTERN

Greedy — Sort by start (or end), sweep and merge/count

### INSERT INTERVAL

- 1. Add all intervals ending before new interval starts (no overlap, before).
- 2. Merge all overlapping intervals with the new one into a single merged interval.
- 3. Add all remaining intervals (no overlap, after).

### MERGE INTERVALS

- Sort by start. Sweep — merge into the last interval in result if overlapping, else append new.

### NON-OVERLAPPING INTERVALS (MINIMUM REMOVALS TO MAKE NON-OVERLAPPING)

Sort by END time (activity selection style). Greedily keep an interval if its start is  $\geq$  the last kept interval's end. Count of removed = total - count of kept.

### COMPLEXITY

| Approach        | Time          | Space  | Notes                                     |
|-----------------|---------------|--------|-------------------------------------------|
| Insert Interval | $O(N)$        | $O(N)$ | Single pass since input is already sorted |
| Merge Intervals | $O(N \log N)$ | $O(N)$ | Sort dominates                            |

| Approach        | Time          | Space  | Notes                      |
|-----------------|---------------|--------|----------------------------|
| Non-overlapping | $O(N \log N)$ | $O(1)$ | Sort by end + greedy count |

*Related: [LeetCode 56 Merge Intervals](#), [LeetCode 57 Insert Interval](#), [LeetCode 435 Non-overlapping Intervals](#)*

**■ Interview Trigger**

Interval problems: sort by start to merge, sort by END to maximize non-overlapping count.

### WINDOW + HEAP + GREEDY — MASTER CHEATSHEET

### SLIDING WINDOW PATTERN TRIGGERS

| Pattern | Trigger Keywords | Key Tool |
|---------|------------------|----------|
|---------|------------------|----------|

|                     |                                    |                                        |
|---------------------|------------------------------------|----------------------------------------|
| Fixed Window        | 'subarray of size K'               | Add right, remove left, fixed size     |
| Variable — Maximize | 'longest substring with condition' | Expand right, shrink when invalid      |
| Variable — Minimize | 'minimum window containing X'      | Expand until valid, shrink while valid |
| Exactly K Trick     | 'exactly K distinct/odd/sum'       | atMost(K) - atMost(K-1)                |
| Distinct Count      | 'at most K distinct characters'    | Window size = map size                 |

### HEAP PATTERN TRIGGERS

| Pattern              | Trigger Keywords                  | Key Tool                            |
|----------------------|-----------------------------------|-------------------------------------|
| Kth Largest/Smallest | 'Kth largest', 'top K'            | Min/Max heap of size K              |
| K-Way Merge          | 'merge K sorted lists/arrays'     | Min heap with one element per list  |
| Running Median       | 'median of data stream'           | Two heaps (max below, min above)    |
| Greedy Combine       | 'minimum cost to combine/connect' | Min heap, always combine 2 smallest |
| Frequency Ranking    | 'K most frequent elements'        | Min heap of size K, or bucket sort  |

### GREEDY PATTERN TRIGGERS

| Pattern                  | Trigger Keywords                          | Key Tool                                    |
|--------------------------|-------------------------------------------|---------------------------------------------|
| Activity Selection       | 'max meetings', 'max non-overlapping'     | Sort by END time                            |
| Interval Merging         | 'merge intervals', 'insert interval'      | Sort by START time                          |
| Reachability             | 'jump game', 'can you reach the end'      | Track farthest reachable index              |
| Ratio Optimization       | 'fractional knapsack', 'max value/weight' | Sort by ratio descending                    |
| Two-Direction Constraint | 'candy', 'satisfies both neighbors'       | Two passes (LfiR then RfiL), merge with max |
| Resource Counting        | 'min platforms', 'min meeting rooms'      | Sort starts/ends separately, sweep          |

Two-Direction Constraint 'candy', 'satisfies both neighbors' Two passes ( $L \rightarrow R$  then  $R \rightarrow L$ ), merge with max

### THE GOLDEN RULES

- Sliding window is  $O(N)$  total because left and right pointers each move at most  $N$  times — never re-scan.
- 'Kth largest/smallest' from a stream  $\rightarrow$  heap of size  $K$ , never sort the whole thing.
- Greedy is risky — it must be PROVABLE (exchange argument). If unsure, default to DP.
- 'Minimize coins/largest first' greedy only works for canonical systems — verify before trusting it.
- Interval problems: the sort key (start vs end) determines whether you're merging or selecting.

Batch 7 Addendum · Steps 11 & 12 — Heaps + Greedy Addendum — 5 Missing Problems (Restored)

Step 11 (Heaps) was missing 4 problems from 11.2/11.3. Step 12 (Greedy) was missing 1 problem from 12.2. Added back below in their correct subsections.

## 11.2 / 11.3 Medium &amp; Hard Problems (Addendum)

**11.2.3 · Sort a K-Sorted (Nearly Sorted) Array****WHAT IT'S ASKING**

Sort an array where every element is at most  $K$  positions away from its sorted position, efficiently using a heap.

**TOPIC / PATTERN**

Min-Heap — bounded window of size  $K+1$

**ALGORITHM**

- Every element is at most  $K$  positions away from its sorted position.
- Push the first  $K+1$  elements into a min-heap.
- For each remaining element: pop the heap minimum (it is now finalized, append to output), then push the new element.
- After the input is exhausted, pop all remaining heap elements in order.

**COMPLEXITY**

| Approach               | Time          | Space  | Notes                                     |
|------------------------|---------------|--------|-------------------------------------------|
| Min-Heap (size $K+1$ ) | $O(N \log K)$ | $O(K)$ | Heap never holds more than $K+1$ elements |

**COMMON MISTAKES**

- Using a full sort ( $O(N \log N)$ ) — ignores the  $K$ -bound structure, wastes the given constraint.
- Forgetting to drain the heap after the main loop — leaves the last  $K$  elements unsorted.

*Related: LeetCode 378 (variant), Kth Largest Element in an Array (Step 11)*

**■ Interview Trigger**

Bounded displacement + need-the-minimum-next → fixed-size min-heap window.

**11.2.5 · Replace Each Array Element by Its Rank****WHAT IT'S ASKING**

Replace each element of an array with its rank (1-indexed) among all elements.

**TOPIC / PATTERN**

Sorting + Hashing — coordinate compression

**ALGORITHM**

- Create a copy of the array and sort it to determine relative order.
- Assign rank 1 to the smallest (unique) value, incrementing rank only when the value changes, and store value→rank in a HashMap.
- Traverse the original array and replace each element with its rank from the map.

**COMPLEXITY**

| Approach       | Time          | Space  | Notes                                     |
|----------------|---------------|--------|-------------------------------------------|
| Sort + HashMap | $O(N \log N)$ | $O(N)$ | Equal elements must receive the same rank |

**COMMON MISTAKES**

- Incrementing rank for every index instead of only on distinct-value boundaries — breaks tie handling.
- Re-sorting on every query instead of precomputing the rank map once.

*Related: Coordinate Compression pattern, Replace with Rank (GfG)*

■ Interview Trigger

Sort once to learn order, hash to answer  $O(1)$  per element afterward.

**11.3.3 · Kth Largest Element in a Stream of Running Integers****WHAT IT'S ASKING**

Find the Kth largest element in a continuously growing stream of numbers.

**TOPIC / PATTERN**

Min-Heap — maintain top K seen so far

**ALGORITHM**

- Maintain a min-heap of size K containing the K largest elements seen so far.
- On each new element: push it, then if heap size exceeds K, pop the minimum.
- The heap's minimum (top) is always the current Kth largest.

**COMPLEXITY**

| Approach           | Time                  | Space  | Notes                                         |
|--------------------|-----------------------|--------|-----------------------------------------------|
| Min-Heap of size K | $O(\log K)$ per inser | $O(K)$ | Design-style problem — class with an add(val) |

**COMMON MISTAKES**

- Rebuilding/sorting the entire stream on every query instead of maintaining a running heap.
- Using a max-heap of all elements — wastes  $O(N)$  space when only the top K matter.

*Related: LeetCode 703 Kth Largest Element in a Stream, Kth Largest in an Array (Step 11)*

■ Interview Trigger

Streaming + 'Kth largest so far' → fixed-size K min-heap, answer = heap top.

**11.3.4 · Maximum Sum Combination****WHAT IT'S ASKING**

From two arrays, pick one element from each to form N pairs, and find the K pairs with the maximum sum.

**TOPIC / PATTERN**

Max-Heap — pairwise sum exploration with visited-pair pruning

**ALGORITHM**

- Sort both arrays A and B in descending order (indices now represent 'strength' — smaller index = larger contribution).
- Push  $(A[0]+B[0], i=0, j=0)$  into a max-heap; track visited  $(i,j)$  pairs in a set to avoid

duplicate pushes. • Pop the max sum K times; each pop pushes its two neighbors (i+1,j) and (i,j+1) if not already visited.

#### COMPLEXITY

| Approach               | Time          | Space  | Notes                                           |
|------------------------|---------------|--------|-------------------------------------------------|
| Max-Heap + visited set | $O(K \log K)$ | $O(K)$ | Only explores promising neighbor pairs, not all |

#### COMMON MISTAKES

- Generating all  $N^2$  pair sums and sorting — defeats the purpose when only the top K are needed.
- Forgetting the visited-pair set — causes duplicate heap entries and wrong results.

*Related: LeetCode 373 Find K Pairs with Smallest Sums (mirror problem), Merge K Sorted Lists (Step 11)*

#### ■ Interview Trigger

Top-K from two sorted sequences → max-heap seeded at the best pair, expand lazily.

### 12.2 Medium/Hard Problems (Addendum)

## 12.2.7 · Shortest Job First (Minimum Average Waiting Time)

#### WHAT IT'S ASKING

Given jobs with processing times, order them to minimize the average waiting time (Shortest Job First).

#### TOPIC / PATTERN

Greedy — sort by burst/processing time

#### ALGORITHM

- Sort all jobs (processes) by their burst time (execution duration) in ascending order.
- Process jobs strictly in this sorted order on a single machine.
- For each job, waiting time = sum of burst times of all jobs processed before it.
- Total/average waiting time = sum of individual waiting times divided by N.

#### COMPLEXITY

| Approach           | Time          | Space        | Notes                           |
|--------------------|---------------|--------------|---------------------------------|
| Sort by burst time | $O(N \log N)$ | $O(1)$ extra | Same idea as SJF CPU scheduling |

#### COMMON MISTAKES

- Sorting by arrival time instead of burst time — SJF greediness depends on shortest job first, not FCFS.
- Forgetting that waiting time excludes the job's own burst time — only prior jobs count.

*Related: OS CPU Scheduling — SJF, N Meetings in One Room (Step 12)*

#### ■ Interview Trigger

Minimize average waiting time → always run the shortest remaining job next.

DSA Revision Notes Striver's A2Z DSA Sheet

**ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING**

## 12.1.5 · Valid Parenthesis Checker

### WHAT IT'S ASKING

Check whether a string of brackets is balanced and properly nested.

### TOPIC / PATTERN

Greedy / Stack — Balanced bracket validation

### CORE IDEA

Push opening brackets onto a stack; on a closing bracket, it must match the top of the stack, otherwise the string is invalid.

### ALGORITHM

- 1. For each character: if it's an opening bracket, push it.
- 2. If it's a closing bracket, check the stack isn't empty and its top matches the corresponding opener — if so pop, else return false.
- 3. At the end, the string is valid only if the stack is empty.

### COMPLEXITY

- Time:  $O(N)$
- Space:  $O(N)$

#### ■ Interview Trigger

Stack of openers; every closer must cancel the most recent unmatched opener.

# DSA Revision Notes

## Striver's A2Z DSA Sheet

### Batch 8 · Steps 13 & 14: Binary Trees + BST

Problems 38 problems — Traversals, Tree properties, BST operations

Patterns DFS/BFS Traversals · Recursive Tree DP · BST Property Exploitation

Format Pattern · Intuition · Algorithm · Complexity · Traps · Trigger

Trees = recursion on a branching structure. BST = trees + the sorted-order superpower.

#### TABLE OF CONTENTS

##### 13.1 TRAVERSALS

- 13.1.1 Preorder, Inorder, Postorder (Recursive)
- 13.1.2 Level Order Traversal (BFS)
- 13.1.3 Iterative Preorder Traversal
- 13.1.4 Iterative Inorder Traversal
- 13.1.5 Postorder Using 2 Stacks / 1 Stack
- 13.1.6 All Traversals in One Pass

##### 13.2 MEDIUM PROBLEMS

- 13.2.1 Height of Binary Tree
- 13.2.2 Check if Height-Balanced
- 13.2.3 Diameter of Binary Tree
- 13.2.4 Maximum Path Sum
- 13.2.5 Check if Two Trees are Identical
- 13.2.6 Zigzag Traversal
- 13.2.7 Boundary Traversal
- 13.2.8 Vertical Order Traversal
- 13.2.9 Top View / Bottom View
- 13.2.11 Right/Left View

##### 13.3 HARD PROBLEMS

- 13.3.2 Construct Tree from Inorder + Preorder
- 13.3.4 Symmetric Binary Tree
- 13.3.5 Flatten Binary Tree to Linked List
- 13.3.7 LCA in Binary Tree
- 13.3.8 Maximum Width of Binary Tree
- 13.3.9 Nodes at Distance K
- 13.3.10 Time to Burn the Binary Tree
- 13.3.11 Count Nodes in Complete Binary Tree

##### 14.1 / 14.2 BINARY SEARCH TREE

- 14.1.1 Search in BST
- 14.1.3 Find Min/Max in BST
- 14.2.1 Ceil in BST
- 14.2.2 Floor in BST
- 14.2.3 Insert a Node in BST

- 14.2.4 Delete a Node in BST
- 14.2.5 Kth Smallest/Largest in BST
- 14.2.6 Check if Tree is BST
- 14.2.7 LCA in BST
- 14.2.8 Construct BST from Preorder
- 14.2.9 Inorder Successor/Predecessor
- 14.2.10 BST Iterator
- 14.2.11 Two Sum in BST
- 14.2.12 Recover BST
- 14.2.13 Largest BST in Binary Tree

## STEP 13 — Binary Trees

### 13.1 Traversals

#### 13.1.1 · Preorder, Inorder, Postorder — Recursive

##### WHAT IT'S ASKING

Traverse a binary tree in preorder, inorder, and postorder using recursion.

##### TOPIC / PATTERN

Tree — DFS Recursive Traversal

##### DEFINITIONS

- Preorder: Root → Left → Right
- Inorder: Left → Root → Right (gives SORTED order for a BST)
- Postorder: Left → Right → Root

##### ALGORITHM (INORDER EXAMPLE)

- 1. If node is null → return.
- 2. Recurse on node.left.
- 3. Process node.val.
- 4. Recurse on node.right.

##### VISUALIZATION

```

1
/ \
2 3
/ \
4 5
Preorder: 1 2 4 5 3
Inorder: 4 2 5 1 3
Postorder: 4 5 2 3 1

```

##### COMPLEXITY

| Approach      | Time   | Space  | Notes                             |
|---------------|--------|--------|-----------------------------------|
| Recursive DFS | $O(N)$ | $O(H)$ | H = height, recursion stack depth |

**■ Interview Trigger**

Position of 'Root' in the name tells you the order: PRE-order, IN-order, POST-order.

## 13.1.2 · Level Order Traversal (BFS)

### WHAT IT'S ASKING

Traverse a binary tree level by level (breadth-first).

### TOPIC / PATTERN

Tree — BFS using a Queue

### ALGORITHM

- 1. Queue = [root].
- 2. While queue not empty: levelSize = queue.size().
- 3. For i = 0 to levelSize-1: pop node, process it, push its children.
- 4. (This loop structure naturally groups nodes by level.)

### COMPLEXITY

| Approach       | Time | Space | Notes                                       |
|----------------|------|-------|---------------------------------------------|
| BFS with Queue | O(N) | O(N)  | Queue holds up to N/2 nodes at widest level |

**■ Interview Trigger**

Capture queue.size() before the inner loop — that's how you separate levels.

## 13.1.3 / 13.1.4 / 13.1.5 · Iterative Pre/In/Postorder using Stacks

### WHAT IT'S ASKING

Perform preorder, inorder, and postorder traversal iteratively using an explicit stack.

### TOPIC / PATTERN

Tree — Iterative DFS using explicit Stack

### ITERATIVE PREORDER (1 STACK)

- Push root. While stack not empty: pop, process, push right child then left child (so left is processed first).

### ITERATIVE INORDER (1 STACK)

- Push all left children onto stack. Pop, process, move to right child, repeat pushing left children.

### ITERATIVE POSTORDER — 2 STACKS

- Use stack1 to do a modified preorder (Root→Right→Left). Push each popped node to stack2.
- Final order of stack2 (popped) = Left→Right→Root = postorder.

### ITERATIVE POSTORDER — 1 STACK

- Trickier: track the last visited node to determine if we've already processed the right subtree.

**COMPLEXITY**

| Approach             | Time   | Space  | Notes                                        |
|----------------------|--------|--------|----------------------------------------------|
| Iterative with Stack | $O(N)$ | $O(H)$ | Explicit stack replaces recursion call stack |

**■ Interview Trigger**

Preorder iterative: push right then left. Postorder 2-stack: reverse of modified preorder.

**13.1.6 · Preorder, Inorder, Postorder in a Single Traversal****WHAT IT'S ASKING**

Perform preorder, inorder, and postorder traversal of a tree in a single pass using one or two stacks.

**TOPIC / PATTERN**

Tree — Single stack with state tracking (1/2/3 visits per node)

**CORE IDEA**

Push (node, state=1) onto stack. Each time you pop a node, increment its state and push it back if not done. State 1 = preorder add, push left. State 2 = inorder add, push right. State 3 = postorder add, done.

**ALGORITHM**

- 1. Stack holds (node, state) pairs, starting with (root, 1).
- 2. Pop (node, state). If state==1: add to preorder, push (node,2), push (node.left,1) if exists.
- 3. If state==2: add to inorder, push (node,3), push (node.right,1) if exists.
- 4. If state==3: add to postorder. Don't push back.

**COMPLEXITY**

| Approach              | Time   | Space  | Notes                             |
|-----------------------|--------|--------|-----------------------------------|
| Single Pass, 3 States | $O(N)$ | $O(H)$ | Each node visited exactly 3 times |

**■ Interview Trigger**

Track visit count (1,2,3) per node on the stack — generates all 3 traversals in one pass.

13.2 Medium Problems

**13.2.1 / 13.2.2 · Height of Tree and Check Height-Balanced****WHAT IT'S ASKING**

Find the height of a binary tree, and check whether it is height-balanced.

**TOPIC / PATTERN**

Tree — Bottom-up recursion

**HEIGHT ALGORITHM**

- $\text{height}(\text{node}) = 1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$ . Base case:  $\text{null} \rightarrow 0$ .

#### BALANCED CHECK — EFFICIENT $O(N)$ (NOT $O(N^2)$ )

Naive approach recalculates height repeatedly ( $O(N^2)$ ). Optimal: compute height and check balance SIMULTANEOUSLY in one bottom-up pass, returning -1 as a 'not balanced' signal.

- 1.  $\text{solve}(\text{node})$ : if  $\text{null} \rightarrow$  return 0.
- 2.  $\text{leftHeight} = \text{solve}(\text{node.left})$ . If  $\text{leftHeight} == -1 \rightarrow$  return -1 (propagate failure).
- 3.  $\text{rightHeight} = \text{solve}(\text{node.right})$ . If  $\text{rightHeight} == -1 \rightarrow$  return -1.
- 4. If  $|\text{leftHeight} - \text{rightHeight}| > 1 \rightarrow$  return -1.
- 5. Return  $1 + \max(\text{leftHeight}, \text{rightHeight})$ .

#### COMPLEXITY

| Approach                 | Time     | Space  | Notes                                |
|--------------------------|----------|--------|--------------------------------------|
| Naive (recompute height) | $O(N^2)$ | $O(H)$ | Recalculates height at every node    |
| Optimal (single pass)    | $O(N)$   | $O(H)$ | -1 signals imbalance, short-circuits |

Related: [LeetCode 104 Maximum Depth](#), [LeetCode 110 Balanced Binary Tree](#)

#### ■ Interview Trigger

Combine height computation and balance check in ONE pass — use -1 as a failure signal.

## 13.2.3 · Diameter of Binary Tree

### WHAT IT'S ASKING

Find the diameter of a binary tree — the longest path between any two nodes.

### TOPIC / PATTERN

Tree — Bottom-up recursion with global tracking

### CORE IDEA

Diameter = longest path between any two nodes (may or may not pass through root). At each node, the diameter THROUGH that node =  $\text{leftHeight} + \text{rightHeight}$ . Track the global maximum across all nodes.

### ALGORITHM

- 1.  $\text{maxDiameter} = 0$  (global/instance variable).
- 2.  $\text{solve}(\text{node})$ : if  $\text{null} \rightarrow$  return 0.
- 3.  $\text{leftHeight} = \text{solve}(\text{node.left})$ .  $\text{rightHeight} = \text{solve}(\text{node.right})$ .
- 4.  $\text{maxDiameter} = \max(\text{maxDiameter}, \text{leftHeight} + \text{rightHeight})$ .
- 5. Return  $1 + \max(\text{leftHeight}, \text{rightHeight})$  (height, not diameter, is returned upward).

### COMPLEXITY

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|             |      |      |                                              |
|-------------|------|------|----------------------------------------------|
| Single Pass | O(N) | O(H) | Height computation doubles as diameter check |
|-------------|------|------|----------------------------------------------|

Related: [LeetCode 543 Diameter of Binary Tree](#)

#### ■ Interview Trigger

Diameter through a node = leftHeight + rightHeight — track the max while computing height.

## 13.2.4 · Maximum Path Sum

### WHAT IT'S ASKING

Find the maximum path sum between any two nodes in a binary tree.

### TOPIC / PATTERN

Tree — Bottom-up recursion with global tracking (similar to diameter)

### CORE IDEA

Path can start and end anywhere (not necessarily root-to-leaf). At each node, the best 'through this node' path =  $\text{node.val} + \max(0, \text{leftGain}) + \max(0, \text{rightGain})$ . But what we RETURN upward is only  $\text{node.val} + \max(0, \max(\text{leftGain}, \text{rightGain}))$  — a path can't branch in both directions once it goes up to the parent.

- 1. maxSum = -infinity (global).
- 2. solve(node): if null → return 0.
- 3. leftGain =  $\max(0, \text{solve}(\text{node.left}))$ . rightGain =  $\max(0, \text{solve}(\text{node.right}))$ .
- 4. maxSum =  $\max(\text{maxSum}, \text{node.val} + \text{leftGain} + \text{rightGain})$ .
- 5. Return  $\text{node.val} + \max(\text{leftGain}, \text{rightGain})$  (only one branch can continue upward).

### COMMON MISTAKES

- Returning leftGain+rightGain upward — WRONG, a path passed to parent can only use ONE child branch.
- Forgetting  $\max(0, \dots)$  — negative subtree contributions should be excluded, not added.

### COMPLEXITY

| Approach    | Time | Space | Notes                       |
|-------------|------|-------|-----------------------------|
| Single Pass | O(N) | O(H)  | Bottom-up, track global max |

Related: [LeetCode 124 Binary Tree Maximum Path Sum](#)

#### ■ Interview Trigger

Track global max with BOTH children, but return upward with only ONE child — paths can't branch twice.

## 13.2.8 / 13.2.9 / 13.2.11 · Vertical, Top, Bottom, Left/Right View

### WHAT IT'S ASKING

Get the vertical order, top view, bottom view, and left/right view of a binary tree.

### TOPIC / PATTERN

Tree — BFS with coordinate tracking (vertical/horizontal distance)

### VERTICAL ORDER TRAVERSAL

Assign each node a (row, col) coordinate: root=(0,0), left child=(row+1,col-1), right child=(row+1,col+1). BFS level by level, group by col, sort by row then value within same (row,col).

### TOP VIEW / BOTTOM VIEW

Same coordinate system, but only track ONE node per column: Top view = first node encountered at each col (BFS, level order ensures topmost first). Bottom view = LAST node encountered at each col (keep overwriting).

### LEFT VIEW / RIGHT VIEW

Level order BFS: Left view = first node of each level. Right view = last node of each level. Or recursive: DFS preferring left(or right) first, only record first visit to each depth.

### ZIGZAG (SPIRAL) TRAVERSAL

– Standard level order BFS, but reverse the order of alternating levels before adding to result.

### COMPLEXITY

| Approach       | Time          | Space  | Notes                                  |
|----------------|---------------|--------|----------------------------------------|
| Vertical Order | $O(N \log N)$ | $O(N)$ | Sorting needed for same (row,col) ties |

| Approach        | Time          | Space  | Notes                                  |
|-----------------|---------------|--------|----------------------------------------|
| Top/Bottom View | $O(N \log N)$ | $O(N)$ | Map keyed by column, sorted at the end |
| Left/Right View | $O(N)$        | $O(H)$ | No sorting needed — simple BFS or DFS  |

*Related: LeetCode 199 Binary Tree Right Side View, LeetCode 987 Vertical Order Traversal*

#### ■ Interview Trigger

Assign (row,col) coords via BFS — top/bottom/vertical views are just different ways to bucket by col.

### ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING

## 13.2.5 · Check if Two Trees are Identical

### WHAT IT'S ASKING

Determine whether two binary trees are structurally identical with the same node values.

### TOPIC / PATTERN

Binary Trees — Simultaneous Recursion

### CORE IDEA

Two trees are identical only if their root values match AND both their left subtrees and right subtrees are identical — check both trees node-by-node in lockstep.

#### ALGORITHM

- 1. If both nodes are null → return true. If exactly one is null, or values differ → return false.
- 2. Otherwise return isIdentical(p.left, q.left) AND isIdentical(p.right, q.right).

#### COMPLEXITY

- Time:  $O(\min(N, M))$
- Space:  $O(H)$

#### ■ Interview Trigger

Walk both trees together — any mismatch in structure or value fails immediately.

### 13.2.6 · Zigzag (Spiral) Traversal of Binary Tree

#### WHAT IT'S ASKING

Traverse a binary tree level by level, alternating left-to-right and right-to-left on successive levels.

#### TOPIC / PATTERN

Binary Trees — Level Order with alternating direction

#### CORE IDEA

Same as level-order BFS, but reverse the order of nodes collected on every other level to create a zigzag reading pattern.

#### ALGORITHM

- 1. BFS level by level using a queue, exactly like standard level order traversal.
- 2. Collect each level's values into a list.
- 3. If the current level index is odd, reverse that level's list before adding it to the result.

#### COMPLEXITY

- Time:  $O(N)$
- Space:  $O(N)$

#### ■ Interview Trigger

Plain level-order BFS, just reverse alternate levels before storing them.

### 13.2.7 · Boundary Traversal

#### WHAT IT'S ASKING

Print the boundary nodes of a binary tree in anti-clockwise order.

#### TOPIC / PATTERN

Tree — Combine 3 sub-traversals (left boundary, leaves, right boundary)

#### ALGORITHM

- 1. Add root (if not a leaf).

- 2. Add LEFT boundary top-to-bottom (excluding leaves) — go left, then right if no left child.
- 3. Add ALL leaf nodes left-to-right (simple DFS, check `node.left==null` and `node.right==null`).
- 4. Add RIGHT boundary bottom-to-top (excluding leaves) — collect then reverse.

**COMPLEXITY**

| Approach              | Time   | Space  | Notes                                      |
|-----------------------|--------|--------|--------------------------------------------|
| 3 Separate DFS Passes | $O(N)$ | $O(H)$ | Each pass is linear, combined still $O(N)$ |

**■ Interview Trigger**

Boundary = left edge (top-down) + leaves (L-to-R) + right edge (bottom-up, reversed).

## 13.3 Hard Problems

**13.3.2 · Construct Tree from Inorder + Preorder****WHAT IT'S ASKING**

Reconstruct a unique binary tree given its inorder and preorder traversals.

**TOPIC / PATTERN**

Tree — Recursive construction using array boundaries

**CORE IDEA**

Preorder's first element is always the ROOT. Find that root's position in inorder — everything left of it in inorder is the left subtree, everything right is the right subtree.

**ALGORITHM**

- 1. Use a HashMap to store value→index in inorder for  $O(1)$  lookup.
- 2. `rootVal = preorder[preStart]`. Create node.
- 3. `rootIdxInorder = map[rootVal]`. `leftSize = rootIdxInorder - inStart`.
- 4. Recurse: left subtree uses `preorder[preStart+1 .. preStart+leftSize]`, `inorder[inStart .. rootIdxInorder-1]`.
- 5. Recurse: right subtree uses the remaining ranges.

**INORDER + POSTORDER VARIANT**

- Postorder's LAST element is the root (instead of first). Same logic, mirrored.

**COMPLEXITY**

| Approach        | Time     | Space  | Notes                                  |
|-----------------|----------|--------|----------------------------------------|
| Without HashMap | $O(N^2)$ | $O(N)$ | Linear search for root index each time |
| With HashMap    | $O(N)$   | $O(N)$ | $O(1)$ root lookup via precomputed map |

*Related: [LeetCode 105 Construct Binary Tree from Preorder and Inorder](#), [LeetCode 106 \(Inorder+Postorder\)](#)*

### ■ Interview Trigger

Preorder[0] = root. Find it in inorder to split left/right subtree ranges.

## 13.3.7 · Lowest Common Ancestor in Binary Tree

### WHAT IT'S ASKING

Find the lowest common ancestor of two given nodes in a binary tree.

### TOPIC / PATTERN

Tree — Bottom-up recursion

### CORE IDEA

Recursively search both subtrees. If a node finds BOTH targets in different subtrees, that node IS the LCA. If only one subtree has a match, propagate that match upward.

### ALGORITHM

- 1. solve(node): if node==null or node==p or node==q → return node.
- 2. left = solve(node.left). right = solve(node.right).
- 3. If left and right are both non-null → node is the LCA, return node.
- 4. If only left is non-null → return left. Else return right.

### VISUALIZATION

```

3
/ \
5 1 find LCA(5, 1)
solve(5)=5, solve(1)=1 → both non-null at node 3 → LCA = 3 ✓

```

### COMPLEXITY

| Approach        | Time | Space | Notes                  |
|-----------------|------|-------|------------------------|
| Single DFS Pass | O(N) | O(H)  | Each node visited once |

*Related: LeetCode 236 Lowest Common Ancestor of a Binary Tree*

### ■ Interview Trigger

If both subtrees return non-null, current node is the LCA — classic bottom-up signal propagation.

## 13.3.8 · Maximum Width of Binary Tree

### WHAT IT'S ASKING

Find the maximum width (widest level) of a binary tree.

### TOPIC / PATTERN

Tree — BFS with index numbering (like a complete binary tree's array index)

### CORE IDEA

Assign each node an index as if the tree were a complete binary tree:  $\text{root}=0$ ,  $\text{left child}=2 \times \text{idx}$ ,  $\text{right child}=2 \times \text{idx}+1$ . Width at a level =  $\text{lastIdx} - \text{firstIdx} + 1$ .

#### ALGORITHM

- 1. BFS with (node, index) pairs in the queue.
- 2. At each level:  $\text{width} = \text{lastIdx} - \text{firstIdx} + 1$ . Track max width.
- 3. IMPORTANT: normalize indices at each level (subtract firstIdx) to prevent integer overflow.

#### COMPLEXITY

| Approach                | Time   | Space  | Notes                                 |
|-------------------------|--------|--------|---------------------------------------|
| BFS with Index Tracking | $O(N)$ | $O(N)$ | Standard BFS, extra index bookkeeping |

#### COMMON MISTAKES

- Not normalizing indices per level — indices can overflow for skewed deep trees.

*Related: LeetCode 662 Maximum Width of Binary Tree*

#### ■ Interview Trigger

Index nodes like a complete binary tree ( $2i, 2i+1$ ) — width = index range at each level.

## 13.3.10 · Time to Burn the Binary Tree from a Node

### WHAT IT'S ASKING

Find how long it takes for a fire that starts at a given node to burn the entire binary tree.

### TOPIC / PATTERN

Tree → Graph conversion + BFS

### CORE IDEA

Fire spreads to ALL adjacent nodes (parent AND children), not just children. Convert the tree to an undirected graph (build a parent map via DFS/BFS), then run BFS from the target node — answer = number of BFS levels.

### ALGORITHM

- 1. Build a parent map: for every node, store a pointer to its parent (via DFS or BFS).
- 2. Find the starting node (the target to burn from).
- 3. BFS from start, treating left, right, AND parent as neighbors. Mark visited.
- 4. Count BFS levels until queue is empty — that's the total time.

### COMPLEXITY

| Approach  | Time   | Space  | Notes                                     |
|-----------|--------|--------|-------------------------------------------|
| Map + BFS | $O(N)$ | $O(N)$ | Build parent map $O(N)$ , then BFS $O(N)$ |

*Related: LeetCode 863 All Nodes Distance K in Binary Tree (same parent-map + BFS technique)*

### ■ Interview Trigger

Add parent pointers via a map — fire spreads in 3 directions, not just down to children.

## 13.3.11 · Count Total Nodes in a Complete Binary Tree

### WHAT IT'S ASKING

Count the total number of nodes in a complete binary tree faster than a plain  $O(N)$  traversal.

### TOPIC / PATTERN

Tree — Exploit Complete Binary Tree structure for  $O(\log^2 N)$

### CORE IDEA

In a complete binary tree, if  $\text{leftHeight} == \text{rightHeight}$ , the LEFT subtree is a PERFECT binary tree → its node count is known instantly via formula ( $2^h - 1$ ). Only recurse into the side that ISN'T perfect.

### ALGORITHM

- 1.  $\text{leftHeight}$  = height by always going left.  $\text{rightHeight}$  = height by always going right.
- 2. If  $\text{leftHeight} == \text{rightHeight}$  → this subtree is PERFECT: return  $2^{\text{leftHeight}} - 1$ .
- 3. Else → return  $1 + \text{countNodes}(\text{left}) + \text{countNodes}(\text{right})$  (recurse normally).

### COMPLEXITY

| Approach                | Time          | Space       | Notes                                         |
|-------------------------|---------------|-------------|-----------------------------------------------|
| Naive Traversal         | $O(N)$        | $O(H)$      | Visits every node — doesn't exploit structure |
| Height Comparison Trick | $O(\log^2 N)$ | $O(\log N)$ | Optimal — exploits completeness               |

*Related: [LeetCode 222 Count Complete Tree Nodes](#)*

### ■ Interview Trigger

Compare left-height vs right-height — equal means PERFECT subtree, use formula instead of recursing.

### ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING

## 13.3.4 · Symmetric Binary Tree

### WHAT IT'S ASKING

Check whether a binary tree is a mirror image of itself around its center.

### TOPIC / PATTERN

Binary Trees — Mirrored Recursion

### CORE IDEA

A tree is symmetric if its left subtree is a mirror image of its right subtree — check two nodes in tandem instead of one.

**ALGORITHM**

- 1. isMirror(left, right): both null → true; one null → false; values differ → false.
- 2. Otherwise return isMirror(left.left, right.right) AND isMirror(left.right, right.left).
- 3. Call isMirror(root.left, root.right) to start.

**COMPLEXITY**

- Time:  $O(N)$
- Space:  $O(H)$  recursion stack

**■ Interview Trigger**

Compare left.left with right.right, and left.right with right.left — that's the mirror check.

## 13.3.5 · Flatten Binary Tree to LinkedList

**WHAT IT'S ASKING**

Flatten a binary tree in place into a right-skewed linked list that follows preorder traversal order.

**TOPIC / PATTERN**

Binary Trees — Morris-style in-place reordering

**CORE IDEA**

Reshape the tree in place into a right-skewed "linked list" that follows preorder traversal order, using only the right pointers.

**ALGORITHM**

- 1. Reverse-preorder (right, left, root) with a running "prev" pointer: set curr.right = prev, curr.left = null, prev = curr. This naturally produces preorder order when done right-to-left.
- 2.  $O(1)$ -space alternative: Morris-style — for each node, if it has a left child, find the rightmost node of the left subtree, attach the current right subtree there, then move left subtree to the right and null out left.

**COMPLEXITY**

- Time:  $O(N)$
- Space:  $O(H)$  recursive, or  $O(1)$  with the Morris-style approach

**■ Interview Trigger**

Process right subtree, then left, then root — linking each to the previously processed node's right pointer.

## 13.3.9 · Print Nodes at Distance K in a Binary Tree

**WHAT IT'S ASKING**

Given a target node, print all nodes that are exactly K edges away from it (in any direction).

**TOPIC / PATTERN**

Binary Trees — Parent Map + BFS

**CORE IDEA**

Trees only have child pointers, so first build a map from each node to its parent, then BFS outward from the target node treating it like an undirected graph.

#### ALGORITHM

- 1. DFS/BFS once to build a `HashMap<node, parent>`.
- 2. BFS from the target node, level by level, moving to left child, right child, AND parent (skip the node you came from).
- 3. Stop and collect all nodes when the BFS level reaches K.

#### COMPLEXITY

- Time:  $O(N)$
- Space:  $O(N)$

#### ■ Interview Trigger

Add parent pointers, then it's just BFS from the target treating the tree as a graph.

## STEP 14 - Binary Search Trees

### 14.1 Concepts

The BST Property: For every node, all values in the left subtree are smaller, all values in the right subtree are larger. This gives  $O(\log N)$  search/insert/delete on a balanced tree ( $O(N)$  worst case if skewed).

### 14.1.1 / 14.1.3 · Search in BST and Find Min/Max

#### WHAT IT'S ASKING

Search for a value in a BST, and find its minimum and maximum values.

#### TOPIC / PATTERN

BST — Exploit sorted property to eliminate half the tree each step

#### SEARCH ALGORITHM

- If `node.val == target` → found. If `target < node.val` → go left. Else go right.

#### FIND MIN/MAX

- Min = keep going left until null. Max = keep going right until null.

#### COMPLEXITY

| Approach      | Time   | Space            | Notes                                                     |
|---------------|--------|------------------|-----------------------------------------------------------|
| BST Traversal | $O(H)$ | $O(1)$ iterative | Height = H; recursive $O(\log N)$ balanced, $O(N)$ skewed |

H=height; recursive  $O(\log N)$  balanced,  $O(N)$  skewed

#### ■ Interview Trigger

BST search = binary search on a tree — go left or right, never both.

## 14.2.1 / 14.2.2 · Ceil and Floor in a BST

### WHAT IT'S ASKING

Find the ceil and floor of a given value in a BST.

### TOPIC / PATTERN

BST — Track best candidate while traversing

### CEIL(X) — SMALLEST VALUE $\geq X$

– Traverse: if  $\text{node.val} == x \rightarrow \text{return } x$ . If  $\text{node.val} > x \rightarrow \text{candidate} = \text{node.val}$ , go left (look for smaller valid). Else go right.

### FLOOR(X) — LARGEST VALUE $\leq X$

– Symmetric: if  $\text{node.val} < x \rightarrow \text{candidate} = \text{node.val}$ , go right. Else go left.

### COMPLEXITY

| Approach      | Time   | Space  | Notes                               |
|---------------|--------|--------|-------------------------------------|
| BST Traversal | $O(H)$ | $O(1)$ | Single pass, no backtracking needed |

#### ■ Interview Trigger

Ceil: go left but remember when you go too high. Floor: go right but remember when too low.

## 14.2.3 / 14.2.4 · Insert and Delete a Node in BST

### WHAT IT'S ASKING

Insert a new value into a BST, and delete a given node from a BST.

### TOPIC / PATTERN

BST — Maintain BST property during modification

### INSERT ALGORITHM

– Traverse like search. When you hit null, that's where the new node goes.

### DELETE ALGORITHM — 3 CASES

- Case 1 — Leaf node: Simply remove it (set parent's pointer to null).
- Case 2 — One child: Replace node with its single child.
- Case 3 — Two children: Find the INORDER SUCCESSOR (smallest in right subtree, i.e. leftmost node

of right subtree). Copy its value to current node, then delete the successor (which has at most one child, reducing to Case 1 or 2).

### VISUALIZATION — DELETE WITH TWO CHILDREN

```

5 5
/ \ delete 5 / \
3 8 → 3 8
/ \ / \
6 9 6 9
Successor = 6 (leftmost of right subtree)
Replace 5 with 6, then delete original 6 node

```

**COMPLEXITY**

| Approach | Time | Space          | Notes                             |
|----------|------|----------------|-----------------------------------|
| Insert   | O(H) | O(1) iterative | H=height of tree                  |
| Delete   | O(H) | O(1) iterative | Finding successor also takes O(H) |

*Related: LeetCode 701 Insert into a BST, LeetCode 450 Delete Node in a BST*

**■ Interview Trigger**

Delete with 2 children: replace with inorder successor, then delete the successor (now simpler case).

**14.2.5 · Kth Smallest/Largest Element in BST****WHAT IT'S ASKING**

Find the Kth smallest and Kth largest element in a BST.

**TOPIC / PATTERN**

BST — Inorder traversal gives sorted order

**CORE IDEA**

Inorder traversal of a BST visits nodes in ASCENDING order. The Kth element visited = Kth smallest. For Kth largest, do reverse-inorder (right→root→left).

**ALGORITHM (ITERATIVE WITH EARLY STOP)**

– Use the iterative inorder stack approach. Stop as soon as you've popped K elements — avoids traversing the whole tree.

**COMPLEXITY**

| Approach          | Time   | Space | Notes                             |
|-------------------|--------|-------|-----------------------------------|
| Iterative Inorder | O(H+K) | O(H)  | Stop early once Kth element found |

*Related: LeetCode 230 Kth Smallest Element in a BST*

**■ Interview Trigger**

Inorder traversal of a BST = sorted order — Kth visited node is the answer.

## 14.2.6 · Check if a Tree is a Valid BST

### WHAT IT'S ASKING

Check whether a given binary tree satisfies the BST property.

### TOPIC / PATTERN

BST — Range validation (NOT just comparing with immediate children)

### COMMON MISTAKE TO AVOID

Just checking `node.left.val < node.val < node.right.val` is WRONG — it doesn't enforce that ALL nodes in the left subtree are smaller, not just the immediate child.

### CORRECT ALGORITHM — RANGE PROPAGATION

- 1. `solve(node, minBound, maxBound)`: if null → return True.
- 2. If `node.val <= minBound` or `node.val >= maxBound` → return False.
- 3. Return `solve(node.left, minBound, node.val) AND solve(node.right, node.val, maxBound)`.

### ALTERNATIVE — INORDER SHOULD BE SORTED

- Do an inorder traversal — if the result isn't strictly increasing, it's not a valid BST.

### COMPLEXITY

| Approach          | Time   | Space  | Notes                                            |
|-------------------|--------|--------|--------------------------------------------------|
| Range Propagation | $O(N)$ | $O(H)$ | Each node checked once against tightening bounds |

*Related: LeetCode 98 Validate Binary Search Tree*

#### ■ Interview Trigger

Pass down (min,max) bounds — checking only immediate parent-child is the classic WRONG approach.

## 14.2.10 · BST Iterator

### WHAT IT'S ASKING

Design an iterator over a BST that returns the next smallest number in sorted order in  $O(1)$  amortized time.

### TOPIC / PATTERN

BST — Controlled inorder traversal using an explicit stack

### CORE IDEA

Simulate iterative inorder traversal, but pause/resume using a stack stored as state, instead of doing the whole traversal upfront.

### DESIGN

- Constructor: push all left-children of root onto stack (like the start of iterative inorder).
- `next()`: pop top, that's the current smallest. If it has a right child, push all its left-children.
- `hasNext()`: stack is non-empty.

**COMPLEXITY**

| Approach  | Time           | Space | Notes                               |
|-----------|----------------|-------|-------------------------------------|
| next()    | O(1) amortized | O(H)  | Total work across all calls is O(N) |
| hasNext() | O(1)           | O(1)  | Just check stack emptiness          |

*Related: LeetCode 173 Binary Search Tree Iterator*

■ **Interview Trigger**

Stack-based lazy inorder — push left-children only as needed, amortized O(1) per next().

## 14.2.11 · Two Sum in BST

### WHAT IT'S ASKING

Check if there exist two elements in a BST whose sum equals a target.

### TOPIC / PATTERN

BST — Two Pointer using BST Iterators (forward + backward)

### CORE IDEA

Use TWO BST iterators: one giving values in ascending order (next), one giving values in descending order (before/prev). This is exactly the two-pointer technique on a sorted array, but using BST structure instead of converting to array.

### ALGORITHM

- 1. left pointer = BSTIterator (ascending), right pointer = BSTIterator (descending).
- 2. lo = left.next(), hi = right.next().
- 3. While lo < hi: if lo+hi==target → found. If sum<target → lo=left.next(). Else hi=right.next().

**COMPLEXITY**

| Approach                 | Time | Space | Notes                                       |
|--------------------------|------|-------|---------------------------------------------|
| Inorder to Array + Two P | O(N) | O(N)  | Simple but uses extra space                 |
| Two BST Iterators        | O(N) | O(H)  | Better space — avoids full array conversion |

Inorder to Array + Two Pointer

| Approach          | Time | Space | Notes                                |
|-------------------|------|-------|--------------------------------------|
| Inorder Traversal | O(N) | O(H)  | Single pass, constant extra tracking |

*Related: LeetCode 653 Two Sum IV - Input is a BST*

**■ Interview Trigger**

Two BST iterators (ascending + descending) = two pointers without materializing the array.

## 14.2.12 · Recover BST (Two Nodes Swapped)

### WHAT IT'S ASKING

Two nodes of a BST were swapped by mistake — recover the BST without changing its structure.

### TOPIC / PATTERN

BST — Inorder traversal, find out-of-order pairs

### CORE IDEA

A correct BST's inorder traversal is strictly increasing. If exactly 2 nodes are swapped, the inorder sequence will have 1 or 2 'violations' (places where  $\text{arr}[i] > \text{arr}[i+1]$ ).

### ALGORITHM

- 1. Do inorder traversal, tracking prev node.
- 2. Whenever  $\text{prev.val} > \text{curr.val}$  (violation): if first violation, mark  $\text{first}=\text{prev}$ ,  $\text{second}=\text{curr}$ .
- 3. If a SECOND violation occurs, update  $\text{second}=\text{curr}$  (the two swapped nodes might not be adjacent).
- 4. After traversal, swap  $\text{first.val}$  and  $\text{second.val}$ .

### WHY TRACK 'FIRST' ONLY ONCE BUT UPDATE 'SECOND'?

If swapped nodes are ADJACENT in inorder → only 1 violation, first and second are direct neighbors. If NOT adjacent → 2 violations, first is the first node of violation 1, second is the second node of violation 2.

### COMPLEXITY

*Related: LeetCode 99 Recover Binary Search Tree*

**■ Interview Trigger**

Inorder violations reveal swapped nodes — 1 violation = adjacent swap, 2 = distant swap.

## 14.2.13 · Size of the Largest BST in a Binary Tree

### WHAT IT'S ASKING

Find the size of the largest subtree within a binary tree that is itself a valid BST.

### TOPIC / PATTERN

Tree — Bottom-up recursion returning a tuple of info

### CORE IDEA

For each subtree, we need to know: is it a valid BST? What's its min and max value? How many nodes does it have? Return all this info bottom-up, and a parent can decide if combining with its children forms a larger valid

### BST.

### ALGORITHM

- 1. solve(node) returns (isBST, minVal, maxVal, size).
- 2. Base case (null): return (True, +infinity, -infinity, 0).
- 3. Get left and right results recursively.
- 4. If left.isBST and right.isBST and left.maxVal < node.val < right.minVal:
- 5. This subtree IS a BST. size = left.size + right.size + 1. Update global maxBSTSize.
- 6. Return (True, min(left.minVal, node.val), max(right.maxVal, node.val), size).
- 7. Else: return (False, -inf, +inf, max(left.size, right.size)) (propagate failure but track size for safety).

### COMPLEXITY

| Approach              | Time | Space | Notes                                        |
|-----------------------|------|-------|----------------------------------------------|
| Single Bottom-Up Pass | O(N) | O(H)  | Each node processed once, O(1) work per node |

*Related: LeetCode 333 Largest BST Subtree*

#### ■ Interview Trigger

Return a tuple (isBST, min, max, size) bottom-up — parent decides if it extends a valid BST.

### TREES + BST — MASTER CHEATSHEET

#### TREE PATTERN TRIGGERS

| Pattern                   | Trigger Keywords                                 | Key Tool                                         |
|---------------------------|--------------------------------------------------|--------------------------------------------------|
| Bottom-Up Recursion       | 'height', 'diameter', 'balanced', 'max path sum' | Return value bubbles up, track global max        |
| Top-Down Recursion        | 'path from root', 'depth at this node'           | Pass accumulated state DOWN as paramete          |
| BFS Level Order           | 'level by level', 'zigzag', 'right side view'    | Queue, capture level size                        |
| Coordinate Tracking       | 'vertical order', 'top/bottom view'              | (row,col) via BFS or DFS                         |
| LCA Pattern               | 'lowest common ancestor'                         | Both children return non-null = LCA found        |
| Tuple Return              | 'largest BST subtree', 'validate+aggregate'      | Return (valid, min, max, size) bottom-up         |
| Construct from Traversals | 'build tree from inorder+preorder'               | First/last of pre/post = root, split via inorder |

Construct from Traversals 'build tree from inorder+preorder' First/last of pre/post = root, split via inorder

#### BST PATTERN TRIGGERS

| Pattern                   | Trigger Keywords                    | Key Tool                             |
|---------------------------|-------------------------------------|--------------------------------------|
| Sorted Order Exploitation | 'Kth smallest', 'inorder successor' | Inorder traversal = sorted sequence  |
| Range Validation          | 'valid BST', 'is this a BST'        | Pass (min,max) bounds down recursion |

|                           |                                     |                                             |
|---------------------------|-------------------------------------|---------------------------------------------|
| Search Direction          | 'search', 'insert', 'ceil', 'floor' | Compare with node, go left or right only    |
| Two Pointer on BST        | 'two sum in BST'                    | Ascending + descending BST iterators        |
| Delete 2-Children Case    | 'delete node in BST'                | Replace with inorder successor              |
| Lazy/Controlled Traversal | 'BST iterator'                      | Stack holding only the 'pending left spine' |

Sorted Order Exploitation 'Kth smallest', 'inorder successor' Inorder traversal = sorted sequence

Lazy/Controlled Traversal 'BST iterator' Stack holding only the 'pending left spine'

### THE GOLDEN RULES

- If the answer can be computed from children's answers → bottom-up recursion (return value upward).
- If the answer needs info accumulated from the root → top-down recursion (pass value downward as parameter).
- Inorder traversal of a BST is ALWAYS sorted — this is the single most powerful BST fact.
- 'Path sum' style problems: track global max separately from what gets RETURNED upward (they differ).
- Validating a BST: NEVER just check immediate children — always propagate (min,max) range bounds.

Batch 8 Addendum · Step 13 — Binary Trees Addendum — 5 Missing Problems (Restored)

Step 13 (Binary Trees) was missing 5 problems across 13.2 and 13.3. Added back below in their correct subsections.

13.2 / 13.3 Medium & Hard Problems (Addendum)

## 13.2.12 · Find the Centre of a Binary Tree / Path

### WHAT IT'S ASKING

Find the centre of a binary tree, or the middle node of the path between two given nodes.

### TOPIC / PATTERN

Tree — diameter endpoints via two BFS/DFS passes

### ALGORITHM

- Run a BFS/DFS from any node to find the farthest node U (one end of the diameter/longest path).
- Run a second BFS/DFS from U to find the farthest node V — U to V is the diameter path.
- The centre is the middle node (or middle edge, if the path length is odd) of the U→V path.

### COMPLEXITY

| Approach         | Time | Space | Notes                                               |
|------------------|------|-------|-----------------------------------------------------|
| Two-pass BFS/DFS | O(N) | O(N)  | Classic 'tree diameter via 2 BFS' trick (originally |

### COMMON MISTAKES

- Confusing this with the root — the geometric centre of a tree is rarely the root.
- Off-by-one when the diameter path has an even number of nodes (two centres, not one).

*Related: Diameter of Binary Tree (Step 13), Minimum Height Trees (LeetCode 310)*

#### ■ Interview Trigger

Farthest node from any node is a diameter endpoint — BFS twice, walk to the midpoint.

### 13.3.3 · Construct Binary Tree from Inorder and Postorder Traversal

#### WHAT IT'S ASKING

Reconstruct a unique binary tree given its inorder and postorder traversals.

#### TOPIC / PATTERN

Tree Construction — postorder end gives root, inorder splits subtrees

#### ALGORITHM

- The last element of postorder is always the root of the current subtree.
- Locate the root's index in inorder (use a HashMap for  $O(1)$  lookup) — everything left of it is the left subtree, everything right is the right subtree.
- Recurse: build the right subtree first (since postorder is Left, Right, Root — popping from the back gives Root, Right, Left), then the left subtree.
- Use a shrinking postorder index pointer (from the end) across recursive calls.

#### COMPLEXITY

| Approach                  | Time   | Space  | Notes                                        |
|---------------------------|--------|--------|----------------------------------------------|
| Recursion + index HashMap | $O(N)$ | $O(N)$ | HashMap avoids $O(N)$ linear search per call |

#### COMMON MISTAKES

- Building left subtree before right — postorder popped from the back yields the right subtree first.
- Recomputing inorder root-index via linear search each call — degrades to  $O(N^2)$ .

*Related: Construct Binary Tree from Inorder + Preorder (Step 13), LeetCode 106*

#### ■ Interview Trigger

Postorder's last element = root; split inorder around it; recurse right-before-left.

### 13.3.6 · Check for Children Sum Property

#### WHAT IT'S ASKING

Check whether every node's value equals the sum of the values of its immediate children.

#### TOPIC / PATTERN

Tree — value of node equals sum of its children's values

#### ALGORITHM

- For every node with two children, verify  $\text{node.val} == \text{left.val} + \text{right.val}$  (treat a missing child as 0).
- To convert a tree to satisfy this property: recursively fix child values first (top-down), then on the way back

up, update the parent to the sum of (possibly updated) children. • Leaf nodes trivially satisfy the property (no children to sum).

#### COMPLEXITY

| Approach               | Time   | Space  | Notes                           |
|------------------------|--------|--------|---------------------------------|
| Recursive top-down fix | $O(N)$ | $O(H)$ | H = height, for recursion stack |

#### COMMON MISTAKES

- Checking the property bottom-up without also propagating value increases downward first — a node's value can only be increased, never decreased, which can break an already-valid child.
- Not handling single-child nodes (treat the missing side as contributing 0).

*Related: Sum Tree / Balanced Tree property problems (GfG)*

#### ■ Interview Trigger

Push value down first (child gets at least parent's value), then sum children back up.

## 13.3.12 · Requirements Needed to Construct a Unique Binary Tree

### WHAT IT'S ASKING

Determine how many additional traversals are needed, on top of what's given, to uniquely construct a binary tree.

### TOPIC / PATTERN

Tree Construction — theory: which traversal pairs uniquely determine a tree

### ALGORITHM

- Inorder + Preorder → always uniquely determines the tree.
- Inorder + Postorder → always uniquely determines the tree.
- Preorder + Postorder → uniquely determines the tree ONLY IF it is known to be a full binary tree (every node has 0 or 2 children).
- Preorder or Postorder or Inorder ALONE → never sufficient; multiple trees can share one traversal.

### COMPLEXITY

| Approach            | Time | Space | Notes                                           |
|---------------------|------|-------|-------------------------------------------------|
| Conceptual / theory | N/A  | N/A   | Frequently asked as a follow-up to construction |

### COMMON MISTAKES

- Assuming Preorder+Postorder always works — it fails whenever some node has exactly one child.
- Assuming Inorder alone plus tree size is enough — it is not, shape is still ambiguous.

*Related: Construct from Inorder+Preorder, Construct from Inorder+Postorder (Step 13)*

#### ■ Interview Trigger

Inorder is the key ingredient — pair it with either pre- or postorder for a guaranteed-unique tree.

### 13.3.13 · Construct Binary Tree from String with Bracket Representation

#### WHAT IT'S ASKING

Construct a binary tree from a string that encodes it using parentheses to represent child subtrees.

#### TOPIC / PATTERN

Recursion / Stack — parse 'val(left)(right)' notation

#### ALGORITHM

- Format: "val(leftSubtreeStr)(rightSubtreeStr)", where either bracket group may be absent.
- Parse the leading number (handle multi-digit and negative signs) as the current node's value.
- If the next character is '(', find its matching close-bracket (track bracket depth) to isolate the left subtree substring, then recurse on it.
- Repeat for a second bracket group to isolate and recurse on the right subtree substring.

#### COMPLEXITY

| Approach                  | Time   | Space  | Notes                                               |
|---------------------------|--------|--------|-----------------------------------------------------|
| Recursive descent parsing | $O(N)$ | $O(N)$ | $N$ = length of the input string; recursion depth = |

#### COMMON MISTAKES

- Using the first ')' found instead of the matching one — nested subtrees contain their own parentheses, so depth must be tracked.
- Not handling a value with no parentheses at all (a leaf node with an empty string).

*Related: LeetCode 536 Construct Binary Tree from String, Mini Parser style problems*

#### ■ Interview Trigger

Track bracket depth to find matching parens — this is 'balanced parentheses' recursion applied to tree parsing.

DSA Revision Notes Striver's A2Z DSA Sheet

#### ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING

### 14.2.7 · LCA in Binary Search Tree

#### WHAT IT'S ASKING

Find the lowest common ancestor of two given nodes in a Binary Search Tree.

#### TOPIC / PATTERN

Binary Search Trees — Use BST ordering to skip subtrees

#### CORE IDEA

Unlike a plain binary tree, a BST lets you decide which side to go without searching both — if both targets are smaller, go left; if both are bigger, go right; otherwise you're at the LCA.

#### ALGORITHM

- 1. Start at root. If both  $p$  and  $q$  are less than  $node.val \rightarrow$  recurse/move left.
- 2. If both are greater  $\rightarrow$  recurse/move right.

- 3. Otherwise (values split, or one equals node) → this node is the LCA.

#### COMPLEXITY

- Time:  $O(H)$
- Space:  $O(1)$  iterative,  $O(H)$  recursive

#### ■ Interview Trigger

BST ordering tells you which side to go — no need to search both subtrees like a plain binary tree LCA.

## 14.2.8 · Construct a BST from a Preorder Traversal

### WHAT IT'S ASKING

Reconstruct a unique Binary Search Tree given only its preorder traversal.

### TOPIC / PATTERN

Binary Search Trees — Bounds-based Recursion

### CORE IDEA

The first element is the root; use a valid (min, max) range to decide, in one linear pass, which subsequent elements belong in the left vs. right subtree — no need for inorder like a plain binary tree.

### ALGORITHM

- 1. Keep a global index into preorder. `build(bound)`: if index out of range or `preorder[index]` violates bound → return null.
- 2. Otherwise take `val = preorder[index++]`, create node.
- 3. `node.left = build(bound updated to (... , val))`; `node.right = build(bound updated to (val, ...))`.

### COMPLEXITY

- Time:  $O(N)$
- Space:  $O(H)$

#### ■ Interview Trigger

BST property means preorder alone is enough — track a valid value range instead of needing inorder too.

## 14.2.9 · Inorder Successor/Predecessor in BST

### WHAT IT'S ASKING

Find the next-larger (successor) or next-smaller (predecessor) node of a given node in a BST.

### TOPIC / PATTERN

Binary Search Trees — Directed descent using BST ordering

### CORE IDEA

The successor is the smallest node greater than the target; the predecessor is the largest node smaller than the target. Both can be found in  $O(H)$  without a full inorder traversal.

**ALGORITHM**

- 1. Successor: walk down from root; whenever  $\text{curr.val} > \text{target}$ , record curr as a candidate and go left; else go right.
- 2. Predecessor: symmetric — whenever  $\text{curr.val} < \text{target}$ , record curr as a candidate and go right; else go left.
- 3. Return the last recorded candidate.

**COMPLEXITY**

- Time:  $O(H)$
- Space:  $O(1)$

**■ Interview Trigger**

Track the last node where you turned "the right way" — that's your successor/predecessor, no inorder needed.

# DSA Revision Notes

## Striver's A2Z DSA Sheet

### Batch 9 · Step 15: Graphs

Problems 43 problems — BFS/DFS, Topo Sort, Shortest Paths, MST, Advanced

Patterns BFS/DFS · Topological Sort · Dijkstra/Bellman-Ford · DSU · Bridges/SCC

Format Pattern · Intuition · Algorithm · Complexity · Traps · Trigger

The biggest topic in the sheet. Master BFS/DFS first — everything else builds on them.

### TABLE OF CONTENTS

#### 15.1 LEARNING — FUNDAMENTALS

15.1.1 Graph Types and Representation

15.1.3 Connected Components

15.1.4 BFS (Breadth First Search)

15.1.5 DFS (Depth First Search)

#### 15.2 PROBLEMS ON BFS/DFS

15.2.1 Number of Provinces

15.2.3 Rotten Oranges (Multi-source BFS)

15.2.4 Flood Fill

15.2.5 Cycle Detection — Undirected (BFS & DFS)

15.2.7 0/1 Matrix (BFS)

15.2.8 Surrounded Regions

15.2.9 Number of Enclaves

15.2.10 Word Ladder I & II

15.2.12 Number of Distinct Islands

15.2.13 Bipartite Check (BFS & DFS)

#### 15.3 TOPOLOGICAL SORT & PROBLEMS

15.3.1 Topo Sort — BFS (Kahn's Algorithm)

15.3.2 Topo Sort — DFS

15.3.4 Cycle Detection — Directed Graph

15.3.6 Course Schedule I & II

15.3.8 Eventual Safe States

15.3.10 Alien Dictionary

15.3.11 Shortest Path in DAG

#### 15.4 SHORTEST PATH ALGORITHMS

15.4.1 Dijkstra's Algorithm

15.4.2 Bellman-Ford Algorithm

15.4.3 Floyd-Warshall Algorithm

15.4.4 Smallest Number of Neighbors at Threshold

15.4.5 Cheapest Flights Within K Stops

15.4.6 Network Delay Time

15.4.8 Minimum Multiplications to Reach End

## 15.5 MST / DISJOINT SET

- 15.5.1 Minimum Spanning Tree — Concept
- 15.5.2 Prim's Algorithm
- 15.5.3 Disjoint Set — Union by Rank/Size
- 15.5.5 Kruskal's Algorithm
- 15.5.8 Accounts Merge
- 15.5.10 Making a Large Island

## 15.6 OTHER ALGORITHMS

- 15.6.1 Bridges in Graph (Tarjan's)
- 15.6.2 Articulation Point
- 15.6.3 Kosaraju's Algorithm (SCC)

## STEP 15 — Graphs

- 15.1 Learning — Fundamentals

### 15.1.1 · Graph Types and Representation

#### WHAT IT'S ASKING

Understand how a graph is represented (adjacency list/matrix) and the types of graphs (directed, weighted, cyclic).

#### TOPIC / PATTERN

Graph — Representation choices

#### ADJACENCY LIST (MOST COMMON)

- Map/array of lists:  $\text{adj}[u] = [\text{list of neighbors}]$ . Space  $O(V+E)$ .

#### ADJACENCY MATRIX

- 2D array:  $\text{matrix}[u][v] = 1$  if edge exists. Space  $O(V^2)$ . Good for dense graphs,  $O(1)$  edge lookup.

#### DIRECTED VS UNDIRECTED

- Undirected: add edge both ways ( $u \rightarrow v$  and  $v \rightarrow u$ ). Directed: add edge one way only.

#### WEIGHTED VS UNWEIGHTED

- Weighted: store (neighbor, weight) pairs instead of just neighbor.

#### COMPLEXITY

| Approach         | Time           | Space              | Notes                                    |
|------------------|----------------|--------------------|------------------------------------------|
| Adjacency List   | $O(V+E)$ space | $O(\text{degree})$ | Best for sparse graphs — most common     |
| Adjacency Matrix | $O(V^2)$ space | $O(1)$             | Best for dense graphs, edge-lookup-heavy |

#### ■ Interview Trigger

Adjacency list = default choice. Use matrix only when graph is dense or you need  $O(1)$  edge checks.

## 15.1.4 / 15.1.5 · BFS and DFS — The Foundation

### WHAT IT'S ASKING

Traverse a graph breadth-first and depth-first from a starting node, visiting every reachable node.

### TOPIC / PATTERN

Graph — The two fundamental traversal algorithms

#### BFS — BREADTH FIRST SEARCH

- Queue-based. Explores level by level. Use when you need SHORTEST PATH in an unweighted graph.
- 1. Queue=[start], visited={start}.
- 2. While queue not empty: pop node, process it.
- 3. For each unvisited neighbor: mark visited, push to queue.

#### DFS — DEPTH FIRST SEARCH

- Stack-based (or recursive call stack). Explores as deep as possible before backtracking. Use for connectivity, cycle detection, topological sort, exploring all paths.
- 1. Mark current node visited, process it.
- 2. For each unvisited neighbor: recursively DFS.

#### BFS VS DFS — WHEN TO USE WHICH

| Approach | Time     | Space  | Notes                                                         |
|----------|----------|--------|---------------------------------------------------------------|
| BFS      | $O(V+E)$ | $O(V)$ | Shortest path (unweighted), level-by-level processing         |
| DFS      | $O(V+E)$ | $O(V)$ | Connectivity, cycles, topo sort, path existence, backtracking |

#### COMMON MISTAKES

- Forgetting to mark visited BEFORE pushing to queue in BFS — can cause duplicate processing.

#### ■ Interview Trigger

'Shortest path, unweighted' → BFS. 'Explore all paths / connectivity / cycles' → DFS.

15.2 Problems on BFS/DFS

## 15.2.1 · Number of Provinces (Connected Components)

### WHAT IT'S ASKING

Count the number of connected components ('provinces') in an undirected graph.

### TOPIC / PATTERN

Graph — Count connected components via BFS/DFS or DSU

### ALGORITHM

- 1. For each unvisited node: increment component count, run BFS/DFS to mark entire component visited.
- 2. Repeat until all nodes visited.

**COMPLEXITY**

| Approach           | Time                      | Space  | Notes                               |
|--------------------|---------------------------|--------|-------------------------------------|
| BFS/DFS            | $O(V+E)$                  | $O(V)$ | Each node/edge visited once total   |
| Disjoint Set Union | $O(V+E \times \alpha(V))$ | $O(V)$ | Alternative — near $O(1)$ per union |

*Related: LeetCode 547 Number of Provinces*

**■ Interview Trigger**

Connected components = count of times you START a fresh BFS/DFS.

### 15.2.3 · Rotten Oranges — Multi-Source BFS

**WHAT IT'S ASKING**

Given a grid of oranges, find the minimum time for all fresh oranges to rot using multi-source BFS.

**TOPIC / PATTERN**

Graph — Multi-source BFS (start from ALL rotten oranges simultaneously)

**CORE IDEA**

Instead of running BFS from one source  $N$  times, push ALL initial rotten oranges into the queue at once. This naturally computes the minimum time for the infection to spread.

**ALGORITHM**

- 1. Push all initially rotten oranges into queue with time=0. Count fresh oranges.
- 2. BFS level by level: each level = 1 unit of time.
- 3. For each rotten orange popped, rot its fresh neighbors, push them, decrement fresh count.
- 4. Answer = max time reached. If fresh count  $> 0$  at the end  $\rightarrow$  impossible, return -1.

**COMPLEXITY**

| Approach         | Time            | Space           | Notes                               |
|------------------|-----------------|-----------------|-------------------------------------|
| Multi-Source BFS | $O(M \times N)$ | $O(M \times N)$ | Grid size, each cell processed once |

*Related: LeetCode 994 Rotting Oranges*

**■ Interview Trigger**

Multiple starting points spreading simultaneously  $\rightarrow$  push them ALL into the queue first.

### 15.2.5 · Cycle Detection in Undirected Graph (BFS & DFS)

**WHAT IT'S ASKING**

Detect whether an undirected graph contains a cycle, using both BFS and DFS.

**TOPIC / PATTERN**

Graph — Track parent to distinguish back-edge from trivial back-traversal

**DFS APPROACH**

- 1. DFS(node, parent): mark visited.
- 2. For each neighbor: if unvisited → DFS(neighbor, node). If visited AND neighbor != parent → CYCLE

found.

**BFS APPROACH**

- 1. Queue holds (node, parent) pairs.
- 2. For each neighbor: if unvisited → mark visited, push (neighbor, node).
- 3. If visited AND neighbor != parent → CYCLE found.

**WHY TRACK PARENT?**

In an undirected graph, the edge  $u-v$  is stored both ways. Without tracking parent, we'd immediately detect a 'false cycle' going straight back to where we came from.

**COMPLEXITY**

| Approach                     | Time     | Space  | Notes                                   |
|------------------------------|----------|--------|-----------------------------------------|
| BFS/DFS with Parent Tracking | $O(V+E)$ | $O(V)$ | Standard traversal with one extra check |

BFS/DFS with Parent Tracking

| Approach         | Time            | Space           | Notes                    |
|------------------|-----------------|-----------------|--------------------------|
| Multi-Source BFS | $O(M \times N)$ | $O(M \times N)$ | Each cell processed once |

**■ Interview Trigger**

A visited neighbor that ISN'T your immediate parent = genuine cycle.

**15.2.7 · 0/1 Matrix — Distance to Nearest 0****WHAT IT'S ASKING**

Given a binary matrix, find the distance of every cell to the nearest 0.

**TOPIC / PATTERN**

Graph — Multi-source BFS (same pattern as Rotten Oranges)

**ALGORITHM**

- 1. Push ALL cells with value 0 into the queue with distance=0.
- 2. BFS outward: for each cell popped, update unvisited neighbors with distance+1, push them.

**COMPLEXITY**

Related: [LeetCode 542 01 Matrix](#)

#### ■ Interview Trigger

Distance-to-nearest-X problems = multi-source BFS starting from all X's at once.

## 15.2.8 / 15.2.9 · Surrounded Regions and Number of Enclaves

### WHAT IT'S ASKING

Capture regions of 'O's fully surrounded by 'X's in a matrix, and count regions of land not touching the border.

### TOPIC / PATTERN

Graph — Boundary-first DFS/BFS (mark what CAN'T be captured)

### CORE IDEA

Instead of checking if each region is surrounded (hard), start DFS/BFS from ALL boundary 'O' cells — these and everything connected to them CANNOT be captured. Everything else gets flipped/counted.

### ALGORITHM (SURROUNDED REGIONS)

- 1. DFS/BFS from every boundary cell with value 'O', marking them as safe (e.g. '#').
- 2. Traverse the grid: flip remaining 'O' to 'X' (captured), flip '#' back to 'O' (safe).

### NUMBER OF ENCLAVES (SAME IDEA, DIFFERENT OUTPUT)

- Mark boundary-connected land as visited. Count remaining unvisited land cells (the enclaves).

### COMPLEXITY

| Approach         | Time            | Space           | Notes                  |
|------------------|-----------------|-----------------|------------------------|
| Boundary DFS/BFS | $O(M \times N)$ | $O(M \times N)$ | Each cell visited once |

Related: [LeetCode 130 Surrounded Regions](#), [LeetCode 1020 Number of Enclaves](#)

#### ■ Interview Trigger

Flip the problem: find what's connected to the BOUNDARY first — that's what's safe/unsurrounded.

## 15.2.10 · Word Ladder I & II

### WHAT IT'S ASKING

Transform a word into another word one letter at a time via valid dictionary words, finding the shortest such sequence(s).

### TOPIC / PATTERN

Graph — BFS on implicit graph (words are nodes, 1-char-diff = edge)

### WORD LADDER I — SHORTEST TRANSFORMATION LENGTH

- 1. BFS from startWord. At each step, try changing each character to each of 26 letters.
- 2. If resulting word is in the word set and unvisited → push to queue, mark visited.
- 3. Return level when endWord is reached.

**WORD LADDER II — ALL SHORTEST TRANSFORMATION SEQUENCES**

Harder: need to reconstruct all paths of shortest length. BFS level by level, but track parent pointers (or build paths directly), and remove ALL words used at the current level only after finishing that level (to avoid cutting off other valid same-length paths).

**COMPLEXITY**

| Approach       | Time                      | Space           | Notes                             |
|----------------|---------------------------|-----------------|-----------------------------------|
| Word Ladder I  | $O(N \times L \times 26)$ | $O(N)$          | N=word count, L=word length       |
| Word Ladder II | $O(N \times L \times 26)$ | $O(N \times L)$ | Path reconstruction adds overhead |

*Related: LeetCode 127 Word Ladder, LeetCode 126 Word Ladder II*

**■ Interview Trigger**

Implicit graph: try all 26 single-character substitutions as 'edges' to other valid words.

**15.2.12 · Number of Distinct Islands****WHAT IT'S ASKING**

Count the number of islands in a grid that have a shape not seen elsewhere (distinct islands).

**TOPIC / PATTERN**

Graph — DFS with shape encoding for deduplication

**CORE IDEA**

Two islands are 'the same shape' if their relative cell positions match (translation-invariant). Encode each island's shape as a string of relative coordinates or movement directions, then use a Set to count unique shapes.

**ALGORITHM**

- 1. For each unvisited land cell: DFS, recording (row - startRow, col - startCol) for every visited cell.
- 2. Convert this list of relative coordinates to a canonical string/tuple.
- 3. Add to a Set. Answer = size of the set.

**COMPLEXITY**

| Approach             | Time            | Space           | Notes                                      |
|----------------------|-----------------|-----------------|--------------------------------------------|
| DFS + Shape Encoding | $O(M \times N)$ | $O(M \times N)$ | Each cell visited once, plus shape strings |

*Related: LeetCode 694 Number of Distinct Islands*

**■ Interview Trigger**

Encode shape as RELATIVE coordinates (not absolute) — translation-invariant comparison via a Set.

### 15.2.13 · Bipartite Graph Check (BFS & DFS)

#### WHAT IT'S ASKING

Determine whether a graph can be 2-colored such that no two adjacent nodes share a color (bipartite check).

#### TOPIC / PATTERN

Graph — 2-coloring problem

#### CORE IDEA

A graph is bipartite if you can color it with 2 colors such that no two adjacent nodes share a color. Equivalent to: graph has NO ODD-LENGTH CYCLE.

#### BFS ALGORITHM

- 1.  $\text{color}[\text{start}] = 0$ .  $\text{Queue} = [\text{start}]$ .
- 2. For each neighbor: if uncolored  $\rightarrow \text{color} = 1 - \text{color}[\text{current}]$ , push.
- 3. If neighbor has SAME color as current  $\rightarrow$  NOT bipartite, return False.

#### DFS ALGORITHM

- Same idea recursively:  $\text{DFS}(\text{node}, \text{color})$ . Assign opposite color to neighbors, check for conflicts, recurse

on uncolored neighbors.

#### COMPLEXITY

| Approach           | Time     | Space  | Notes                               |
|--------------------|----------|--------|-------------------------------------|
| BFS/DFS 2-Coloring | $O(V+E)$ | $O(V)$ | Standard traversal with color array |

*Related: LeetCode 785 Is Graph Bipartite?*

#### ■ Interview Trigger

Bipartite = 2-colorable = no odd cycle. Color neighbors opposite — conflict means not bipartite.

### 15.3 Topological Sort & Problems

Topological Sort: A linear ordering of nodes in a DIRECTED ACYCLIC GRAPH (DAG) such that for every edge  $u \rightarrow v$ ,  $u$  comes before  $v$ . Only exists if the graph has no cycle.

### 15.3.1 / 15.3.2 · Topological Sort — BFS (Kahn's) and DFS

#### WHAT IT'S ASKING

Produce a valid topological ordering of a directed acyclic graph, using both Kahn's BFS algorithm and DFS.

#### TOPIC / PATTERN

Graph — Ordering based on dependencies (DAG only)

#### KAHN'S ALGORITHM — BFS APPROACH

- 1. Compute in-degree of every node.
- 2. Push all nodes with in-degree 0 into a queue.

- 3. Pop node, add to result. For each neighbor: decrement in-degree. If it becomes 0, push to queue.
- 4. If result size  $< V$  at the end  $\rightarrow$  cycle exists, no valid topo order.

**DFS APPROACH**

- 1. DFS each unvisited node. After exploring ALL neighbors (post-order), push current node to a stack.
- 2. Topological order = stack contents reversed (popped from top).

**WHY DFS POST-ORDER WORKS**

A node is only fully 'finished' (pushed to stack) after all its dependencies are processed — so the LAST finished nodes have no dependents, and should come last in topo order, hence the reversal.

**COMPLEXITY**

| Approach     | Time     | Space  | Notes                                         |
|--------------|----------|--------|-----------------------------------------------|
| Kahn's (BFS) | $O(V+E)$ | $O(V)$ | Detects cycle naturally via incomplete result |
| DFS-based    | $O(V+E)$ | $O(V)$ | Needs separate cycle check or visited states  |

*Related: LeetCode 207/210 Course Schedule (Topo sort applications)*

**■ Interview Trigger**

Kahn's: process in-degree-0 nodes first. DFS: post-order finish time, then reverse.

**15.3.4 · Cycle Detection in Directed Graph (BFS & DFS)****WHAT IT'S ASKING**

Detect whether a directed graph contains a cycle, using both BFS (Kahn's) and DFS.

**TOPIC / PATTERN**

Graph — Detect cycles in directed graphs (different from undirected!)

**DFS APPROACH — PATH/RECURSION STACK TRACKING**

- 1. Maintain TWO states: visited (ever visited) and inRecursionStack (currently on the DFS path).
- 2. DFS(node): mark visited AND inRecursionStack.
- 3. For each neighbor: if inRecursionStack[neighbor]  $\rightarrow$  CYCLE found.
- 4. Else if unvisited  $\rightarrow$  recurse.
- 5. Before returning, unmark inRecursionStack[node] (backtrack).

**BFS APPROACH — KAHN'S ALGORITHM**

– Run Kahn's topo sort. If the resulting order has fewer than  $V$  nodes  $\rightarrow$  a cycle exists (some nodes never reached in-degree 0).

**WHY DIRECTED CYCLE DETECTION DIFFERS FROM UNDIRECTED**

In directed graphs, simply checking 'visited and not parent' is WRONG — you need to distinguish between a 'cross edge' to an already-finished branch vs a genuine 'back edge' to an ancestor still on the

current path.

#### COMPLEXITY

| Approach                 | Time     | Space  | Notes                        |
|--------------------------|----------|--------|------------------------------|
| DFS with Recursion Stack | $O(V+E)$ | $O(V)$ | Two boolean arrays needed    |
| Kahn's (BFS)             | $O(V+E)$ | $O(V)$ | Cycle n topo sort incomplete |

#### ■ Interview Trigger

Directed cycle = node revisited while still on the CURRENT recursion path (not just 'visited').

## 15.3.6 · Course Schedule I & II

### WHAT IT'S ASKING

Given prerequisite pairs for courses, determine if all courses can be finished, and in what order.

### TOPIC / PATTERN

Graph — Direct application of topological sort

#### COURSE SCHEDULE I — CAN ALL COURSES BE FINISHED?

– Build graph: prerequisite  $\rightarrow$  course (edge direction matters). Run Kahn's. If topo order includes all courses  $\rightarrow$  True. Else  $\rightarrow$  cycle exists  $\rightarrow$  False.

#### COURSE SCHEDULE II — RETURN THE ACTUAL ORDER

– Same as above, but return the topo order itself (or empty array if impossible).

#### COMPLEXITY

| Approach         | Time     | Space    | Notes                                 |
|------------------|----------|----------|---------------------------------------|
| Kahn's Algorithm | $O(V+E)$ | $O(V+E)$ | $V$ =courses, $E$ =prerequisite pairs |

*Related: LeetCode 207 Course Schedule, LeetCode 210 Course Schedule II*

#### ■ Interview Trigger

'Can all tasks be completed given dependencies' = pure topological sort / cycle detection.

## 15.3.8 · Eventual Safe States

### WHAT IT'S ASKING

Find all 'safe' nodes in a directed graph — nodes from which every path eventually leads to a terminal node.

### TOPIC / PATTERN

Graph — Reverse graph + Topological sort, or DFS with cycle tracking

**CORE IDEA**

A node is 'safe' if every path from it eventually leads to a terminal node (no cycles). Trick: REVERSE all edges, then nodes with in-degree 0 in the reversed graph are terminal/safe. Run Kahn's on the reversed graph.

**ALGORITHM — REVERSE GRAPH + KAHN'S**

- 1. Build the REVERSE graph (flip every edge direction).
- 2. Compute in-degrees on the reversed graph. Push in-degree-0 nodes to queue.
- 3. Standard Kahn's BFS — all nodes that get processed are SAFE.

**ALTERNATIVE — DFS WITH 3-COLORING**

- States: unvisited, visiting (on current path), visited-safe. If DFS revisits a 'visiting' node → cycle → not

safe. Otherwise mark safe after full exploration.

**COMPLEXITY**

| Approach               | Time     | Space    | Notes                         |
|------------------------|----------|----------|-------------------------------|
| Reverse Graph + Kahn's | $O(V+E)$ | $O(V+E)$ | Standard topo sort complexity |

*Related: LeetCode 802 Find Eventual Safe States*

**■ Interview Trigger**

Reverse the graph — terminal/safe nodes become the in-degree-0 starting points for Kahn's.

**15.3.10 · Alien Dictionary****WHAT IT'S ASKING**

Given words sorted according to an unknown alien alphabet, determine the order of the letters.

**TOPIC / PATTERN**

Graph — Build edges from word ordering, then topo sort

**CORE IDEA**

Compare each pair of ADJACENT words. The first differing character gives an ordering edge (char1 comes before char2 in the alien alphabet). Build a graph of characters, then topologically sort it.

**ALGORITHM**

- 1. For each pair of adjacent words: find the first index where they differ.
- 2. Add edge: word1[diffIdx] → word2[diffIdx] (this char comes before that char).
- 3. Special case: if word1 is a PREFIX of word2 but LONGER → invalid ordering, return "".
- 4. Run topological sort on the character graph (26 max nodes).

**COMPLEXITY**

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|                         |                      |         |                                                |
|-------------------------|----------------------|---------|------------------------------------------------|
| Build Graph + Topo Sort | $O(N \times L + 26)$ | $O(26)$ | $N = \text{words}, L = \text{avg word length}$ |
|-------------------------|----------------------|---------|------------------------------------------------|

Related: [LeetCode 269 Alien Dictionary](#)

#### ■ Interview Trigger

Adjacent word pairs reveal character ordering edges — then it's just topological sort.

### 15.3.11 · Shortest Path in a DAG

#### WHAT IT'S ASKING

Find the shortest path from a source to every node in a Directed Acyclic Graph using topological sort.

#### TOPIC / PATTERN

Graph — Topological Sort + Relaxation (faster than Dijkstra for DAGs)

#### CORE IDEA

Since a DAG has no cycles, processing nodes in TOPOLOGICAL ORDER guarantees that when we process a node, all paths leading to it have already been finalized. No need for a priority queue like Dijkstra.

#### ALGORITHM

- 1. Topologically sort the DAG.
- 2.  $\text{dist}[\text{source}] = 0$ , all others = infinity.
- 3. Process nodes in topo order: for each node  $u$ , for each edge  $(u, v, \text{weight})$ : if  $\text{dist}[u] + \text{weight} < \text{dist}[v] \rightarrow$

relax (update  $\text{dist}[v]$ ).

#### COMPLEXITY

| Approach           | Time              | Space  | Notes                           |
|--------------------|-------------------|--------|---------------------------------|
| Dijkstra (general) | $O((V+E) \log V)$ | $O(V)$ | Works for any graph, but slower |
| Topo Sort + Relax  | $O(V+E)$          | $O(V)$ | Faster — only works for DAGs    |

#### ■ Interview Trigger

DAG + shortest path  $\rightarrow$  topological order + relaxation beats Dijkstra (no heap needed).

### 15.4 Shortest Path Algorithms

#### 15.4.1 · Dijkstra's Algorithm

##### WHAT IT'S ASKING

Find the shortest distance from a source to all nodes in a weighted graph with non-negative edges.

##### TOPIC / PATTERN

Graph — Single-source shortest path (NON-negative weights only)

**CORE IDEA**

Greedy process the node with the SMALLEST known distance first (using a min-heap). Once a node is popped with its final distance, that distance is guaranteed optimal (works because weights are non-negative — no shorter path can appear later).

**ALGORITHM**

- 1.  $\text{dist}[\text{source}] = 0$ , all others = infinity. Min-heap =  $[(0, \text{source})]$ .
- 2. While heap not empty: pop  $(d, \text{node})$  with smallest  $d$ .
- 3. If  $d > \text{dist}[\text{node}] \rightarrow$  stale entry, skip (we already found a better path).
- 4. For each neighbor  $(v, \text{weight})$ : if  $\text{dist}[\text{node}] + \text{weight} < \text{dist}[v] \rightarrow$  relax: update  $\text{dist}[v]$ , push  $(\text{dist}[v], v)$  to

heap.

**WHY IT FAILS WITH NEGATIVE WEIGHTS**

Dijkstra assumes once a node is finalized (popped), its distance can't improve. Negative weights can create a shorter path discovered LATER, breaking this assumption.

**COMPLEXITY**

| Approach                  | Time              | Space  | Notes                             |
|---------------------------|-------------------|--------|-----------------------------------|
| Brute (no heap)           | $O(V^2)$          | $O(V)$ | Good for dense graphs             |
| Min-Heap (Priority Queue) | $O((V+E) \log V)$ | $O(V)$ | Standard — good for sparse graphs |

*Related: LeetCode 743 Network Delay Time, LeetCode 1631 Path With Minimum Effort*

**■ Interview Trigger**

Non-negative weights, single source  $\rightarrow$  Dijkstra with min-heap. NEVER use with negative edges.

**15.4.2 · Bellman-Ford Algorithm****WHAT IT'S ASKING**

Find shortest paths from a source in a graph that may contain negative edge weights, and detect negative cycles.

**TOPIC / PATTERN**

Graph — Single-source shortest path, handles NEGATIVE weights + detects negative cycles

**CORE IDEA**

Relax ALL edges  $V-1$  times (the longest possible shortest-path has at most  $V-1$  edges in a graph with no negative cycle). A relaxation possible on the  $V$ th iteration indicates a negative weight cycle.

**ALGORITHM**

- 1.  $\text{dist}[\text{source}] = 0$ , all others = infinity.
- 2. Repeat  $V-1$  times: for every edge  $(u, v, \text{weight})$ : if  $\text{dist}[u] + \text{weight} < \text{dist}[v] \rightarrow$  relax.
- 3. Run ONE more relaxation pass. If anything still relaxes  $\rightarrow$  NEGATIVE CYCLE exists.

**WHY V-1 ITERATIONS?**

A shortest simple path can have at most  $V-1$  edges (visiting each of  $V$  nodes once). After  $V-1$  rounds of relaxing ALL edges, every shortest path is guaranteed found.

#### COMPLEXITY

| Approach     | Time            | Space  | Notes                                             |
|--------------|-----------------|--------|---------------------------------------------------|
| Bellman-Ford | $O(V \times E)$ | $O(V)$ | Slower than Dijkstra but handles negative weights |

*Related: LeetCode 787 Cheapest Flights Within K Stops (Bellman-Ford-like relaxation)*

#### ■ Interview Trigger

Negative weights present → Bellman-Ford. Extra relaxation pass detects negative cycles.

### 15.4.3 · Floyd-Warshall Algorithm

#### WHAT IT'S ASKING

Find the shortest distances between every pair of nodes in a graph, including with negative edges.

#### TOPIC / PATTERN

Graph — ALL-PAIRS shortest path (every node to every other node)

#### CORE IDEA

Dynamic programming: try every node  $K$  as a possible INTERMEDIATE point in the path from  $i$  to  $j$ . If going through  $K$  is shorter than the direct path, update.

#### ALGORITHM

- 1.  $dist$  = adjacency matrix (infinity where no direct edge, 0 on diagonal).
- 2. For  $k = 0$  to  $V-1$ : for  $i = 0$  to  $V-1$ : for  $j = 0$  to  $V-1$ :
- 3.  $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$ .

#### CRITICAL DETAIL — $K$ MUST BE THE OUTERMOST LOOP

This ordering ensures that when we consider path  $i \rightarrow j$  via  $k$ , all shorter paths using intermediate nodes  $0..k-1$  have already been finalized.

#### DETECTING NEGATIVE CYCLES

- After running, if  $dist[i][i] < 0$  for any  $i \rightarrow$  negative cycle exists.

#### COMPLEXITY

| Approach       | Time     | Space    | Notes                                                 |
|----------------|----------|----------|-------------------------------------------------------|
| Floyd-Warshall | $O(V^3)$ | $O(V^2)$ | All-pairs — use only when $V$ is small ( $\leq 500$ ) |

*Related: LeetCode 1334 Smallest City With Threshold Distance*

#### ■ Interview Trigger

Need ALL-PAIRS shortest paths → Floyd-Warshall.  $K$  (intermediate node) MUST be the outer loop.

## 15.4.5 · Cheapest Flights Within K Stops

### WHAT IT'S ASKING

Find the cheapest flight route from a source to a destination with at most K stopovers allowed.

### TOPIC / PATTERN

Graph — Modified Bellman-Ford (limited number of relaxation rounds)

### CORE IDEA

K stops means at most K+1 edges. Run Bellman-Ford-style relaxation, but limit to K+1 iterations, and use a COPY of the distance array each round to avoid using updates from the SAME round (which would imply more than K stops).

### ALGORITHM

- 1.  $\text{dist}[\text{source}] = 0$ , rest = infinity.
- 2. For  $i = 0$  to  $K$  ( $K+1$  rounds):  $\text{tempDist} = \text{copy}(\text{dist})$ .
- 3. For each edge  $(u, v, \text{price})$ : if  $\text{dist}[u] + \text{price} < \text{tempDist}[v] \rightarrow \text{tempDist}[v] = \text{dist}[u] + \text{price}$ .
- 4.  $\text{dist} = \text{tempDist}$ .

### WHY COPY THE ARRAY EACH ROUND?

Without copying, a node could be updated and then immediately used to update another node in the SAME round, effectively using more than the allowed number of edges.

### COMPLEXITY

| Approach             | Time            | Space  | Notes                     |
|----------------------|-----------------|--------|---------------------------|
| Limited Bellman-Ford | $O(K \times E)$ | $O(V)$ | K+1 rounds instead of V-1 |

*Related: LeetCode 787 Cheapest Flights Within K Stops*

#### ■ Interview Trigger

Limited number of edges allowed  $\rightarrow$  Bellman-Ford with a COPY of distances per round, not in-place.

## 15.4.8 · Minimum Multiplications to Reach End

### WHAT IT'S ASKING

Find the minimum number of multiplications (mod M) needed to transform a start number into an end number.

### TOPIC / PATTERN

Graph — Implicit BFS (each possible value 0-99999 is a node)

### CORE IDEA

Each number from 0 to 99999 is a 'node'. Applying any multiplier from the array creates an edge to  $(\text{current} \times \text{multiplier}) \bmod 100000$ . Find shortest path (min operations) from start to end using BFS (unweighted — each operation costs 1).

### ALGORITHM

- 1. BFS from start. Queue holds (currentValue, steps).
- 2. For each multiplier m: next = (current × m) % 100000.
- 3. If next unvisited: mark visited, push (next, steps+1). If next == end → return steps+1.

**COMPLEXITY**

| Approach              | Time                 | Space       | Notes                     |
|-----------------------|----------------------|-------------|---------------------------|
| BFS on Implicit Graph | $O(100000 \times M)$ | $O(100000)$ | M = number of multipliers |

**■ Interview Trigger**

State space (values 0-99999) IS the graph — BFS finds minimum operations like shortest path.

## 15.5 Minimum Spanning Tree / Disjoint Set

**15.5.1 · Minimum Spanning Tree — Concept****WHAT IT'S ASKING**

Understand what a Minimum Spanning Tree is and why it matters for weighted, connected graphs.

**TOPIC / PATTERN**

Graph — Connect all nodes with minimum total edge weight, no cycles

**DEFINITION**

A spanning tree connects all V nodes using exactly V-1 edges with no cycles. The MINIMUM spanning tree has the smallest possible sum of edge weights among all spanning trees.

**TWO CLASSIC ALGORITHMS**

- Prim's: Grow the tree one edge at a time, always picking the cheapest edge connecting the current tree

to a new node. Similar to Dijkstra's greedy structure.

- Kruskal's: Sort ALL edges by weight. Greedily add edges if they don't form a cycle (checked via Disjoint

Set Union).

**■ Interview Trigger**

MST = connect all nodes, minimum total weight, no cycles — exactly V-1 edges.

**15.5.2 · Prim's Algorithm****WHAT IT'S ASKING**

Build the Minimum Spanning Tree of a graph using Prim's algorithm.

**TOPIC / PATTERN**

Graph — MST via greedy node-by-node growth (min-heap based)

**ALGORITHM**

- 1. Start from any node. Min-heap = [(0, startNode)]. visited = {}.

- 2. While heap not empty: pop (weight, node). If already visited, skip.
- 3. Mark visited. Add weight to MST total.
- 4. For each neighbor (v, edgeWeight): if v unvisited → push (edgeWeight, v) to heap.

**COMPLEXITY**

| Approach       | Time          | Space    | Notes                         |
|----------------|---------------|----------|-------------------------------|
| Min-Heap Based | $O(E \log E)$ | $O(V+E)$ | Similar structure to Dijkstra |

**■ Interview Trigger**

Prim's grows the tree from a single point — always add the cheapest edge to a NEW node.

**15.5.3 · Disjoint Set — Union by Rank / Size + Path Compression****WHAT IT'S ASKING**

Implement the Disjoint Set (Union-Find) data structure with union by rank/size and path compression.

**TOPIC / PATTERN**

Data Structure — Track connected components with near- $O(1)$  union/find

**CORE OPERATIONS**

- find(x): Returns the representative (root) of x's set.
- union(x, y): Merges the sets containing x and y.

**PATH COMPRESSION (IN FIND)**

While finding the root, make every node on the path point DIRECTLY to the root. Flattens the tree for future queries.

**UNION BY RANK/SIZE**

Always attach the SMALLER tree under the root of the LARGER tree. Prevents the tree from becoming a long chain.

**WHY BOTH OPTIMIZATIONS MATTER**

Path compression alone gives  $O(\log N)$  amortized. Union by rank/size alone gives  $O(\log N)$ . TOGETHER, they give nearly  $O(1)$  amortized — technically  $O(\alpha(N))$ , the inverse Ackermann function.

**COMPLEXITY**

| Approach                         | Time             | Space  | Notes                                         |
|----------------------------------|------------------|--------|-----------------------------------------------|
| Naive (no optimization)          | $O(N)$ per op    | $O(N)$ | Can degrade to a linked list                  |
| Path Compression + Union by Rank | $O(1)$ amortized | $O(N)$ | Effectively $O(1)$ for all practical purposes |

Path Compression + Union  $O(\alpha(N))$

| Approach | Time          | Space    | Notes                                        |
|----------|---------------|----------|----------------------------------------------|
| Prim's   | $O(E \log V)$ | $O(V+E)$ | Better for DENSE graphs, grows from one node |

|           |               |          |                                     |
|-----------|---------------|----------|-------------------------------------|
| Kruskal's | $O(E \log E)$ | $O(V+E)$ | Better for SPARSE graphs, needs DSU |
|-----------|---------------|----------|-------------------------------------|

#### ■ Interview Trigger

DSU = path compression (flatten on find) + union by rank/size (attach small to big).

## 15.5.5 · Kruskal's Algorithm

### WHAT IT'S ASKING

Build the Minimum Spanning Tree of a graph using Kruskal's algorithm, sorting edges by weight.

### TOPIC / PATTERN

Graph — MST via greedy edge-by-edge selection using DSU

### ALGORITHM

- 1. Sort all edges by weight ascending.
- 2. For each edge (u, v, weight) in sorted order:
- 3. If  $\text{find}(u) \neq \text{find}(v)$  (different components, no cycle):  $\text{union}(u, v)$ , add weight to MST total.
- 4. Stop when  $V-1$  edges have been added.

### PRIM'S VS KRUSKAL'S

*Related: LeetCode 1584 Min Cost to Connect All Points*

#### ■ Interview Trigger

Sort edges, greedily add if no cycle (DSU find check) — that's Kruskal's MST.

## 15.5.8 · Accounts Merge

### WHAT IT'S ASKING

Given bank accounts that share emails, merge accounts belonging to the same person.

### TOPIC / PATTERN

Graph — Disjoint Set Union to merge connected accounts

### CORE IDEA

Accounts sharing ANY email belong together. Use DSU: union accounts that share an email. Then group all emails by their final DSU root.

### ALGORITHM

- 1. Map each email to the FIRST account index that owns it.
- 2. For each account, for each email: if email already mapped to another account →  $\text{union}(\text{currentAccount}, \text{mappedAccount})$ .
- 3. Else map email → currentAccount index.
- 4. Group all emails by their DSU root account. Sort emails within each group, prepend name.

### COMPLEXITY

| Approach       | Time                                                 | Space           | Notes                                |
|----------------|------------------------------------------------------|-----------------|--------------------------------------|
| DSU + Grouping | $O(N \times K \times \alpha(N) + N \times K \log K)$ | $O(N \times K)$ | N=accounts, K=avg emails per account |

DSU + Grouping  $O(N \times K \times \alpha(N) + N \times K O(N \times K) \log K)$  N=accounts, K=avg emails per account

*Related: LeetCode 721 Accounts Merge*

#### ■ Interview Trigger

Shared email = union the accounts. Classic 'merge groups with common element' = DSU.

## 15.5.10 · Making a Large Island

### WHAT IT'S ASKING

Given a grid of 0s and 1s, flip one 0 to 1 to make the largest possible island.

### TOPIC / PATTERN

Graph — DSU or DFS, simulate flipping ONE 0 to 1

### CORE IDEA

Pre-compute the size of every existing island (label them with unique IDs). For each 0 cell, check the UNIQUE island IDs of its 4 neighbors — flipping this 0 would merge those islands. Sum their sizes + 1.

### ALGORITHM

- 1. DFS/BFS each island, label cells with a unique ID, store size per ID in a map.
- 2. For each 0 cell: collect unique neighbor island IDs (use a Set to avoid double-counting same island).
- 3. potentialSize = 1 + sum(islandSizes[id] for each unique neighboring id).
- 4. Track the maximum potential size across all 0 cells. Handle edge case: if no 0 exists, answer = total

grid size.

### COMPLEXITY

| Approach     | Time            | Space           | Notes                    |
|--------------|-----------------|-----------------|--------------------------|
| Label + Scan | $O(M \times N)$ | $O(M \times N)$ | Two passes over the grid |

*Related: LeetCode 827 Making a Large Island*

#### ■ Interview Trigger

Pre-label islands with unique IDs and sizes — for each 0, sum sizes of DISTINCT neighboring islands.

15.6 Other Algorithms — Bridges, Articulation Points, SCC

These all build on DFS with discovery/low-link times — Tarjan's family of algorithms.

## 15.6.1 · Bridges in Graph (Tarjan's Algorithm)

**WHAT IT'S ASKING**

Find all 'bridges' in a graph — edges whose removal disconnects the graph — using Tarjan's algorithm.

**TOPIC / PATTERN**

Graph — DFS with discovery time and low-link value

**DEFINITION**

A bridge is an edge whose removal increases the number of connected components (i.e. it's the ONLY connection between two parts of the graph).

**CORE IDEA — DISCOVERY TIME AND LOW-LINK VALUE**

- $disc[u]$ : time when  $u$  was first visited in DFS.
- $low[u]$ : the EARLIEST discovery time reachable from  $u$ 's subtree (including back-edges).
- Edge  $(u,v)$  is a bridge if  $low[v] > disc[u]$  — meaning  $v$ 's subtree has NO back-edge to  $u$  or earlier.

**ALGORITHM**

1. DFS(node, parent):  $disc[node] = low[node] = timer++$ .
2. For each neighbor: if neighbor == parent → skip (don't count trivial back-edge).
3. If unvisited: DFS(neighbor, node).  $low[node] = \min(low[node], low[neighbor])$ .
4. If  $low[neighbor] > disc[node]$  → edge (node,neighbor) IS A BRIDGE.
5. Else (already visited, not parent):  $low[node] = \min(low[node], disc[neighbor])$  (back-edge).

**COMPLEXITY**

| Approach     | Time     | Space  | Notes                                     |
|--------------|----------|--------|-------------------------------------------|
| Tarjan's DFS | $O(V+E)$ | $O(V)$ | Single DFS pass with discovery/low arrays |

Related: [LeetCode 1192 Critical Connections in a Network](#)

**■ Interview Trigger**

$low[v] > disc[u]$  means  $v$ 's subtree can't reach back to  $u$  or earlier — that edge is a bridge.

**15.6.2 · Articulation Point****WHAT IT'S ASKING**

Find all articulation points in a graph — nodes whose removal disconnects the graph.

**TOPIC / PATTERN**

Graph — Same DFS technique as bridges, slightly different condition

**DEFINITION**

An articulation point (cut vertex) is a node whose REMOVAL increases the number of connected components.

**CORE CONDITION**

- Non-root node  $u$  is an articulation point if ANY child  $v$  has  $low[v] \geq disc[u]$  ( $v$ 's subtree cannot reach above  $u$  without going through  $u$ ).

- Root node is an articulation point if it has 2 OR MORE children in the DFS tree.

#### ALGORITHM

- Same DFS structure as bridges, but track childCount for root, and use  $\geq$  instead of  $>$  in the comparison.

#### BRIDGE VS ARTICULATION POINT — KEY DIFFERENCE

| Approach | Time                             | Space             | Notes                   |
|----------|----------------------------------|-------------------|-------------------------|
| Bridge   | $\text{low}[v] > \text{disc}[u]$ | Strict inequality | edge itself is critical |

| Approach           | Time                                | Space        | Notes                   |
|--------------------|-------------------------------------|--------------|-------------------------|
| Articulation Point | $\text{low}[v] \geq \text{disc}[u]$ | Non-strict — | node itself is critical |

*Related: GFG Articulation Points, LeetCode (no direct equivalent, but related to network reliability)*

#### ■ Interview Trigger

Same low-link DFS as bridges, but check  $\geq$  instead of  $>$  and handle the root specially.

### 15.6.3 · Kosaraju's Algorithm — Strongly Connected Components

#### WHAT IT'S ASKING

Find all strongly connected components of a directed graph using Kosaraju's algorithm.

#### TOPIC / PATTERN

Graph — Find SCCs in a directed graph using 2 DFS passes

#### DEFINITION

A Strongly Connected Component (SCC) is a maximal set of nodes where every node can reach every other node within the set.

#### ALGORITHM — 3 STEPS

- 1. Run DFS on the original graph, pushing each node to a STACK upon finishing (post-order, like topo sort).
- 2. REVERSE all edges in the graph (transpose graph).
- 3. Pop nodes from the stack one by one. For each unvisited node, run DFS on the REVERSED graph —

everything reached in this DFS forms ONE SCC.

#### WHY THIS WORKS

Step 1 orders nodes such that 'finishing late' nodes are likely SCC representatives. Reversing edges in step 2 means a DFS from a true SCC root can ONLY reach its own SCC members (since reversed edges block 'escaping' to nodes that were already finished earlier).

#### COMPLEXITY

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|                           |          |          |                                                |
|---------------------------|----------|----------|------------------------------------------------|
| Kosaraju's (2 DFS passes) | $O(V+E)$ | $O(V+E)$ | Linear — same as a single DFS, just done twice |
|---------------------------|----------|----------|------------------------------------------------|

*Related: LeetCode does not have a direct SCC problem, but it's foundational for compiler design, circuit analysis*

#### ■ Interview Trigger

Finish-time order (DFS) + reverse graph + DFS again in that order = SCCs revealed.

## GRAPHS — MASTER CHEATSHEET

### ALGORITHM SELECTION GUIDE

| Need                            | Use This Algorithm               | Complexity            | Constraint                       |
|---------------------------------|----------------------------------|-----------------------|----------------------------------|
| Shortest path, unweighted       | BFS                              | $O(V+E)$              | None                             |
| Shortest path, non-negative wei | Dijkstra                         | $O((V+E) \log V)$     | No negative edges                |
| Shortest path, negative weights | Bellman-Ford                     | $O(V \times E)$       | Detects negative cycles too      |
| Shortest path, ALL pairs        | Floyd-Warshall                   | $O(V^3)$              | Small V only (≤500)              |
| Shortest path in DAG            | Topo Sort + Relax                | $O(V+E)$              | Must be acyclic                  |
| Minimum Spanning Tree           | Prim's or Kruskal's              | $O(E \log V)$         | Connected, undirected            |
| Cycle detection, undirected     | DFS/BFS + parent tracking        | $O(V+E)$              | None                             |
| Cycle detection, directed       | DFS + recursion stack, or Kahn's | $O(V+E)$              | None                             |
| Ordering with dependencies      | Topological Sort                 | $O(V+E)$              | Must be a DAG                    |
| Connected components, dynam     | Disjoint Set Union               | $O(\alpha(V))$ per op | Near-constant with optimizations |
| Bridges / cut vertices          | Tarjan's (low-link DFS)          | $O(V+E)$              | Undirected graph                 |
| Strongly connected component    | Kosaraju's or Tarjan's           | $O(V+E)$              | Directed graph                   |

Shortest path, non-negative weights

| Trigger Keyword(s)                                    | Maps To                                   |
|-------------------------------------------------------|-------------------------------------------|
| 'minimum operations', 'shortest transformation'       | BFS on implicit graph                     |
| 'multi-source', 'simultaneously spreading'            | Multi-source BFS (push all sources first) |
| 'can all tasks be completed', 'order of dependencies' | Topological Sort                          |
| 'connect all', 'minimum cost network'                 | MST — Prim's or Kruskal's                 |
| 'merge groups', 'same component'                      | Disjoint Set Union                        |
| 'critical edge/node', 'single point of failure'       | Bridges / Articulation Points             |
| 'two-color', 'no adjacent conflict'                   | Bipartite Check                           |

'cheapest with at most K stops'

Limited Bellman-Ford

Shortest path, negative weights OK

**PATTERN RECOGNITION TRIGGERS**

Trigger Keyword(s) Maps To

'minimum operations', 'shortest transformation' BFS on implicit graph

'multi-source', 'simultaneously spreading' Multi-source BFS (push all sources first)

'can all tasks be completed', 'order of dependencies' Topological Sort

'connect all', 'minimum cost network' MST — Prim's or Kruskal's

'merge groups', 'same component' Disjoint Set Union

'critical edge/node', 'single point of failure' Bridges / Articulation Points

'two-color', 'no adjacent conflict' Bipartite Check

'cheapest with at most K stops' Limited Bellman-Ford

**THE GOLDEN RULES**

- BFS = shortest path in UNWEIGHTED graphs. Never use BFS for weighted shortest path.
- Undirected cycle detection: track parent. Directed cycle detection: track recursion stack (NOT just visited).
- Dijkstra breaks with negative weights — switch to Bellman-Ford immediately if you see negative edges.
- DSU with path compression + union by rank is the go-to for 'are these connected / merge these groups' questions.

- Bridges/articulation points always reduce to DFS with discovery time + low-link value tracking.

Batch 9 Addendum · Step 15 — Graphs Addendum — 7 Missing Problems (Restored)

Step 15 (Graphs) was missing 7 problems across 15.2, 15.3, 15.4 and 15.5. Added back below in their correct subsections.

15.2 / 15.3 / 15.4 / 15.5 (Addendum)

**15.2.2 · Connected Components Problem in a Matrix****WHAT IT'S ASKING**

Count the number of connected components of 1s in a 2D binary matrix, treating it as a graph.

**TOPIC / PATTERN**

Grid Graph — BFS/DFS flood fill to label components

**ALGORITHM**

- Treat each cell as a graph node; two cells are connected if adjacent (4- or 8-directional, per problem) and share the required cell value.
- Iterate over every unvisited cell; on finding an unvisited 'valid' cell, run BFS/DFS to mark the entire connected component with a unique component ID.
- Increment the component counter each time a new BFS/DFS is launched from an unvisited cell.

**COMPLEXITY**

| Approach           | Time            | Space           | Notes                               |
|--------------------|-----------------|-----------------|-------------------------------------|
| BFS/DFS flood fill | $O(N \times M)$ | $O(N \times M)$ | Same core idea as Number of Islands |

**COMMON MISTAKES**

- Only checking 4-directional adjacency when the problem intends 8-directional (or vice versa).
- Not marking visited immediately when enqueueing (in BFS) — causes duplicate enqueues of the same cell.

*Related: Number of Provinces, Number of Islands (Step 15), Flood Fill*

**■ Interview Trigger**

Grid + 'connected group' language → flood-fill BFS/DFS, count launches from unvisited cells.

**15.3.12 · Shortest Path in an Undirected Graph (Unit Weights)****WHAT IT'S ASKING**

Find the shortest path between two nodes in an undirected graph where every edge has weight 1.

**TOPIC / PATTERN**

BFS — unweighted shortest path

**ALGORITHM**

- Build an adjacency list; since edges are unweighted, BFS naturally finds shortest paths in terms of edge count.
- Initialize `dist[]` to infinity except `dist[source] = 0`; use a queue starting with the source.
- For each dequeued node, relax all neighbors: if `dist[node] + 1 < dist[neighbor]`, update and enqueue the neighbor.
- Optionally track a `parent[]` array to reconstruct the actual path via backtracking from destination.

**COMPLEXITY**

| Approach  | Time     | Space    | Notes                                         |
|-----------|----------|----------|-----------------------------------------------|
| Plain BFS | $O(V+E)$ | $O(V+E)$ | Works because every edge has identical weight |

**COMMON MISTAKES**

- Using Dijkstra for a plain unweighted graph — unnecessary  $O((V+E) \log V)$  overhead when BFS suffices.
- Marking visited only at pop-time instead of at push-time — can enqueue the same node multiple times.

*Related: Shortest Path in DAG (Step 15), 0/1 Matrix — Distance to Nearest 0*

**■ Interview Trigger**

All edge weights equal → BFS IS shortest path; save Dijkstra for actually-weighted graphs.

**15.4.7 · Number of Ways to Arrive at Destination****WHAT IT'S ASKING**

Count the number of shortest (minimum-weight) paths from source to destination in a weighted graph, modulo  $10^9+7$ , using a Dijkstra-style relaxation.

### TOPIC / PATTERN

Modified Dijkstra — count shortest paths while relaxing

### ALGORITHM

- Run Dijkstra from the source, maintaining `dist[]` as usual, plus a `ways[]` array initialized to `ways[source] = 1`.
- When relaxing an edge ( $u \rightarrow v$ ) with weight  $w$ : if  $\text{dist}[u] + w < \text{dist}[v]$ , a strictly shorter path is found  $\rightarrow$  update `dist[v]` and set `ways[v] = ways[u]` (overwrite, don't add).
- If  $\text{dist}[u] + w == \text{dist}[v]$  (an equally short alternative path is found), accumulate: `ways[v] = (ways[v] + ways[u]) mod 1e9+7`.
- Answer is `ways[destination]` after Dijkstra completes.

### COMPLEXITY

| Approach                  | Time          | Space    | Notes                                          |
|---------------------------|---------------|----------|------------------------------------------------|
| Dijkstra + counting array | $O(E \log V)$ | $O(V+E)$ | Take result modulo $10^9+7$ as per problem con |

### COMMON MISTAKES

- Adding to `ways[v]` on a strict improvement instead of overwriting — carries over stale path counts from a longer route.
- Forgetting the modulo — path counts can overflow for dense graphs.

*Related: LeetCode 1976 Number of Ways to Arrive at Destination, Dijkstra's Algorithm (Step 15)*

#### ■ Interview Trigger

'Count shortest paths'  $\rightarrow$  Dijkstra, but branch on strict-improve (overwrite) vs tie (accumulate).

## 15.4.9 · Swim in Rising Water

### WHAT IT'S ASKING

Given a grid where cells become passable over time, find the minimum time needed to swim from top-left to bottom-right.

### TOPIC / PATTERN

Binary Search on Answer + BFS/DFS, or Min-Heap (Dijkstra-style)

### ALGORITHM

- Heap approach: push the start cell with its elevation into a min-heap; pop the minimum-elevation reachable cell each step.
- The answer (time to reach the destination) is the maximum elevation encountered along the min-max path so far — track a running 'water level' as the max popped so far.
- Push all unvisited neighbors of the popped cell; stop when the destination cell is popped — the tracked running maximum is the answer.
- Alternative: binary search on the time  $T$ , then BFS/DFS using only cells with elevation  $\leq T$  to check if destination is reachable.

### COMPLEXITY

| Approach                  | Time            | Space    | Notes                                                   |
|---------------------------|-----------------|----------|---------------------------------------------------------|
| Min-Heap (Dijkstra-style) | $O(N^2 \log N)$ | $O(N^2)$ | $N \times N$ grid; 'widest path / minimax path' pattern |

|                     |                                     |          |                                                     |
|---------------------|-------------------------------------|----------|-----------------------------------------------------|
| Binary Search + BFS | $O(N^2 \log(\max\text{Elevation}))$ | $O(N^2)$ | Alternative when heap approach feels less intuitive |
|---------------------|-------------------------------------|----------|-----------------------------------------------------|

Binary Search + BFS  $O(N^2 \log(\max\text{Elevation}))$   $O(N^2)$  Alternative when heap approach feels less intuitive

### COMMON MISTAKES

- Summing elevations instead of tracking the running maximum — this is a minimax path, not a shortest-sum path.
- Forgetting to mark cells visited when pushed to the heap — causes redundant re-processing.

*Related: LeetCode 778 Swim in Rising Water, Path with Minimum Effort (similar minimax pattern)*

#### ■ Interview Trigger

'Minimize the maximum edge/cell on a path' → minimax path via Dijkstra-style heap or binary-search+BFS.

## 15.5.6 · Number of Operations to Make the Network Connected

### WHAT IT'S ASKING

Given a network of connected computers, find the minimum number of cable operations needed to connect all computers.

### TOPIC / PATTERN

Disjoint Set Union — count redundant edges and components

### ALGORITHM

- If the number of edges is less than  $(N-1)$ , it is impossible to connect all  $N$  nodes — return -1 immediately.
- Use DSU: for each edge  $(u, v)$ , union the two nodes; if they are already in the same set, this edge is 'redundant' (extra cable).
- After processing all edges, count the number of distinct components remaining (distinct DSU roots).
- Answer = (number of components - 1) — each redundant edge can be repurposed to connect one more component.

### COMPLEXITY

| Approach         | Time                   | Space  | Notes                                            |
|------------------|------------------------|--------|--------------------------------------------------|
| DSU (Union-Find) | $O(E \cdot \alpha(N))$ | $O(N)$ | $\alpha$ = inverse Ackermann, effectively $O(1)$ |

DSU (Union-Find)  $O(E \cdot \alpha(N))$   $O(N)$   $\alpha$  = inverse Ackermann, effectively  $O(1)$

### COMMON MISTAKES

- Forgetting the edge-count feasibility check ( $\text{edges} < N-1 \Rightarrow$  impossible, return -1) before running DSU.
- Counting redundant edges instead of counting remaining components minus one — both work but must be applied consistently.

*Related: Disjoint Set (Step 15), Number of Provinces*

#### ■ Interview Trigger

Redundant edges = spare cables; answer = components needing connection = (#components - 1).

### 15.5.7 · Most Stones Removed with Same Row or Column

#### WHAT IT'S ASKING

Remove as many stones as possible where each removed stone shares a row or column with another remaining stone.

#### TOPIC / PATTERN

Disjoint Set Union — stones sharing a row/column are 'connected'

#### ALGORITHM

- Union every pair of stones that share the same row OR the same column (union stone  $i$  with stone  $j$  if  $\text{row}[i] == \text{row}[j]$  or  $\text{col}[i] == \text{col}[j]$ ).
- A common trick: union each stone's row-index and column-index directly (using offset column indices, e.g.  $\text{col} + 10001$ , to keep row/col ID spaces disjoint) instead of comparing all stone pairs.
- Count the number of distinct connected components (distinct DSU roots among the stones) after all unions.
- In each connected component, exactly one stone can remain (all others can be chained-removed) —  $\text{answer} = \text{totalStones} - \text{numberOfComponents}$ .

#### COMPLEXITY

| Approach           | Time                   | Space  | Notes                                          |
|--------------------|------------------------|--------|------------------------------------------------|
| DSU on row/col IDs | $O(N \cdot \alpha(N))$ | $O(N)$ | Avoids the naive $O(N^2)$ pairwise comparisons |

DSU on row/col IDs  $O(N \cdot \alpha(N))$   $O(N)$  Avoids the naive  $O(N^2)$  pairwise comparison

#### COMMON MISTAKES

- Comparing every pair of stones directly ( $O(N^2)$ ) instead of unioning through shared row/column IDs.
- Forgetting to offset column indices from row indices — causes false unions between unrelated row 5 and column 5.

*Related: Number of Provinces, Accounts Merge (Step 15) — both use the 'union through a shared attribute' trick*

#### ■ Interview Trigger

Union stones via their row/col IDs (not pairwise) — components = stones you must keep, rest removable.

### 15.5.9 · Number of Islands II (Online Query)

#### WHAT IT'S ASKING

Given a grid where land cells are added one at a time, track the size of the largest island after each addition.

#### TOPIC / PATTERN

Disjoint Set Union — incremental land addition with live component count

#### ALGORITHM

- Start with an all-water grid and a running island count = 0. Process land-addition queries one at a time.
- For each new land cell: increment the running count by 1 (assume it's a new island), then check its up-to-4 neighbors.
- For every neighboring cell that is already land, union it with the current cell in the DSU; each successful union (merging two previously-different components) decrements the running

count by 1. • After processing each query, append the current running count to the answer list — this is the 'online' (incremental) answer after that query.

### COMPLEXITY

| Approach              | Time                   | Space           | Notes                                                                   |
|-----------------------|------------------------|-----------------|-------------------------------------------------------------------------|
| DSU with live counter | $O(Q \cdot \alpha(N))$ | $O(N \times M)$ | Q = number of queries; avoids re-flood-filling the whole grid per query |

DSU with live counter  $O(Q \cdot \alpha(N \times M))$   $O(N \times M)$  Q = number of queries; avoids re-flood-filling the whole grid per que

### COMMON MISTAKES

- Re-running full BFS/DFS flood fill on the entire grid after every single query — far too slow for many queries.
- Double-unioning the same pair of components (must check if  $\text{find}(a) \neq \text{find}(b)$  before decrementing the count).

*Related: Number of Provinces, Making a Large Island (Step 15)*

#### ■ Interview Trigger

'Online/incremental' + 'count connected components as edges/nodes stream in' → DSU with a running counter.

DSA Revision Notes Striver's A2Z DSA Sheet

### ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING

## 15.2.4 · Flood Fill

### WHAT IT'S ASKING

Starting from a pixel, recolor every connected pixel of the same original color to a new color.

### TOPIC / PATTERN

Graphs — DFS/BFS on a grid

### CORE IDEA

Starting from a pixel, change every connected pixel of the same original color to the new color — identical machinery to Number of Islands.

### ALGORITHM

1. Record the starting color. If it equals the new color, return immediately (avoids infinite recursion).
2. DFS/BFS from the start cell: if a neighbor is in bounds and matches the starting color, recolor it and recurse/enqueue.

### COMPLEXITY

- Time:  $O(N \times M)$
- Space:  $O(N \times M)$  recursion/queue in the worst case

#### ■ Interview Trigger

Same as Number of Islands, but you're painting instead of counting.

## 15.4.4 · City With the Smallest Number of Neighbors at a Threshold Distance

### WHAT IT'S ASKING

Find the city that has the fewest other cities reachable within a given distance threshold.

### TOPIC / PATTERN

Graphs — All-Pairs Shortest Path (Floyd-Warshall)

### CORE IDEA

Compute shortest distances between every pair of cities, then for each city count how many others are reachable within the threshold — pick the city with the fewest (tie → highest city number).

### ALGORITHM

- 1. Run Floyd-Warshall to get  $\text{dist}[i][j]$  for all pairs.
- 2. For each city  $i$ , count  $j \neq i$  where  $\text{dist}[i][j] \leq \text{threshold}$ .
- 3. Track the minimum count seen; on ties prefer the larger city index.

### COMPLEXITY

- Time:  $O(V^3)$  for Floyd-Warshall
- Space:  $O(V^2)$

#### ■ Interview Trigger

Small  $V$  and "all pairs" phrasing → Floyd-Warshall is usually the intended tool.

## 15.4.6 · Network Delay Time

### WHAT IT'S ASKING

Find the time for a signal starting at one node to reach every other node in a weighted directed graph.

### TOPIC / PATTERN

Graphs — Dijkstra's Algorithm

### CORE IDEA

A signal starts at node  $K$  and travels along weighted directed edges; find the time for it to reach every node, and answer is the MAXIMUM of those times (or  $-1$  if some node is unreachable).

### ALGORITHM

- 1. Run Dijkstra from source  $K$  to get shortest time to every node.
- 2. If any node's distance is still infinity → return  $-1$  (not all nodes reachable).
- 3. Otherwise return the maximum distance across all nodes.

### COMPLEXITY

- Time:  $O(E \log V)$
- Space:  $O(V + E)$

#### ■ Interview Trigger

Textbook Dijkstra — the twist is just taking the max of the resulting distance array.

# DSA Revision Notes

## Striver's A2Z DSA Sheet

### Batch 10 · Steps 16 & 17: Dynamic Programming + Tries (FINAL BATCH)

Problems 58 problems — 1D/2D DP, Subsequences, Strings, Stocks, LIS, MCM, Tries

Patterns Top-Down/Bottom-Up · Take/Not-Take · State Machine · Partition DP · Trie XOR

Format Pattern · Intuition · Algorithm · Complexity · Traps · Trigger

DP = identify overlapping subproblems, define state, write recurrence, memoize or tabulate.

#### TABLE OF CONTENTS

##### DP FOUNDATION

DP-0 The Universal DP Framework — 5 Steps

DP-0 Top-Down vs Bottom-Up — When to Use Which

##### 16.2 1D DP

DP-1 Climbing Stairs

DP-3 Frog Jump

DP-4 Frog Jump with K Distances

DP-5 Maximum Sum of Non-Adjacent Elements

DP-6 House Robber

##### 16.3 2D / 3D DP AND GRIDS

DP-7 Ninja's Training

DP-8 Grid Unique Paths

DP-10 Minimum Path Sum in Grid

DP-11 Triangle — Min Path Sum

DP-12 Min/Max Falling Path Sum

DP-13 Ninja and His Friends (3D DP)

##### 16.4 DP ON SUBSEQUENCES

DP-14 Subset Sum Equal to Target

DP-15 Partition Equal Subset Sum

DP-17 Count Subsets with Sum K

DP-19 0/1 Knapsack

DP-20 Coin Change (Min Coins)

DP-21 Target Sum

DP-22 Coin Change 2 (Count Ways)

DP-23 Unbounded Knapsack

DP-24 Rod Cutting

##### 16.5 DP ON STRINGS

DP-25 Longest Common Subsequence

DP-27 Longest Common Substring

DP-28 Longest Palindromic Subsequence

DP-29 Min Insertions to Make Palindrome

DP-30 Min Insertions/Deletions to Convert A to B

DP-31 Shortest Common Supersequence  
 DP-32 Number of Distinct Subsequences  
 DP-33 Edit Distance  
 DP-34 Wildcard Matching

## 16.6 DP ON STOCKS

DP-35 Buy/Sell Stock I (Single Transaction)  
 DP-36 Buy/Sell Stock II (Unlimited)  
 DP-37 Buy/Sell Stock III (At Most 2)  
 DP-38 Buy/Sell Stock IV (At Most K)  
 DP-39 Stocks with Cooldown  
 DP-40 Stocks with Transaction Fee

## 16.7 DP ON LIS

DP-41 Longest Increasing Subsequence  
 DP-43 LIS using Binary Search  
 DP-44 Largest Divisible Subset  
 DP-46 Longest Bitonic Subsequence

## 16.8 MCM / PARTITION DP

DP-48 Matrix Chain Multiplication  
 DP-50 Minimum Cost to Cut a Stick  
 DP-51 Burst Balloons  
 DP-52 Evaluate Boolean Expression to True  
 DP-53 Palindrome Partitioning II  
 DP-54 Partition Array for Maximum Sum

## 16.9 DP ON SQUARES

DP-55 Maximal Rectangle with All 1s  
 DP-56 Count Square Submatrices with All 1s

## 17 TRIES

17.1.1 Implement Trie I & II  
 17.1.3 Longest String with All Prefixes  
 17.2.1 Maximum XOR of Two Numbers in Array

## STEP 16 — Dynamic Programming

DP Foundation — Framework and Mindset

---

## DP-0 · The Universal DP Framework — 5 Steps

### WHAT IT'S ASKING

Learn the 5-step framework (states, transitions, base cases, top-down vs bottom-up) used to approach any DP problem.

### THE 5-STEP DP PROCESS

– Step 1 — Identify the overlapping subproblem: Can the problem be expressed as smaller versions of

itself? Does the same sub-problem repeat?

- Step 2 — Define the DP state: What parameters uniquely describe a subproblem? (e.g. index, capacity, remaining count)
- Step 3 — Write the recurrence relation: Express  $dp[state]$  in terms of smaller states.
- Step 4 — Identify base cases: What are the smallest inputs with known answers?
- Step 5 — Choose top-down or bottom-up: Memoization (recursive + cache) or tabulation (iterative table).

### TOP-DOWN VS BOTTOM-UP

Top-Down (Memoization) Bottom-Up (Tabulation)

|          | Top-Down (Memoization)              | Bottom-Up (Tabulation)                  |
|----------|-------------------------------------|-----------------------------------------|
| How      | Recursive + cache                   | Iterative DP table                      |
| Order    | Natural recursion order             | Must figure out fill order              |
| Space    | $O(N)$ call stack extra             | No extra stack — saves space            |
| Ease     | Easier to write (same as recursion) | Harder initially, but more efficient    |
| Use when | Subproblems not all needed          | All subproblems needed, memory critical |

#### ■ Interview Trigger

DP = Overlapping Subproblems + Optimal Substructure. If recursion is exponential → check for DP.

## 16.2 1D DP

### DP-1 · Climbing Stairs / Fibonacci Pattern

#### WHAT IT'S ASKING

Find the number of ways to climb  $N$  stairs taking 1 or 2 steps at a time (Fibonacci-style DP).

#### TOPIC / PATTERN

1D DP — Linear recurrence, two previous states

#### STATE & RECURRENCE

- $dp[i]$  = number of ways to reach step  $i$ .
- Recurrence:  $dp[i] = dp[i-1] + dp[i-2]$  (come from step  $i-1$  or  $i-2$ ).
- Base cases:  $dp[0]=1$  (start),  $dp[1]=1$ .

#### SPACE OPTIMIZATION

- Only need the last 2 values — use two variables (prev1, prev2) instead of an array.

#### COMPLEXITY

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|                     |          |        |                                     |
|---------------------|----------|--------|-------------------------------------|
| Recursion (no memo) | $O(2^N)$ | $O(N)$ | Exponential — classic DP motivation |
| DP (tabulation)     | $O(N)$   | $O(N)$ | Or $O(1)$ space with 2 variables    |

Related: [LeetCode 70 Climbing Stairs](#), [LeetCode 509 Fibonacci Number](#)

#### ■ Interview Trigger

$dp[i] = dp[i-1] + dp[i-2]$ . Only 2 previous states needed — space-optimize to  $O(1)$ .

## DP-3 / DP-4 · Frog Jump and Frog Jump with K Distances

### WHAT IT'S ASKING

A frog jumps between stones minimizing total energy cost, either one step at a time or up to K steps.

### TOPIC / PATTERN

1D DP — Min-cost path with jump choices

#### FROG JUMP (JUMP 1 OR 2 STEPS)

- $dp[i] = \text{min cost to reach stone } i$ .
- $dp[i] = \min(dp[i-1] + |h[i]-h[i-1]|, dp[i-2] + |h[i]-h[i-2]|)$ .

#### FROG JUMP WITH K DISTANCES

- $dp[i] = \text{min over } j=1 \text{ to } K \text{ of } (dp[i-j] + |h[i]-h[i-j]|)$ .

### COMPLEXITY

| Approach   | Time            | Space            | Notes              |
|------------|-----------------|------------------|--------------------|
| Frog Jump  | $O(N)$          | $O(N)$ or $O(1)$ | 2-state recurrence |
| K-distance | $O(N \times K)$ | $O(N)$           | K choices per step |

#### ■ Interview Trigger

Cost at each stone = min over all valid previous stones. Classic min-cost DP.

## DP-5 / DP-6 · Max Sum Non-Adjacent Elements / House Robber

### WHAT IT'S ASKING

Find the maximum sum of non-adjacent elements in an array, and the related House Robber problem.

### TOPIC / PATTERN

1D DP — Pick or skip, no two adjacent allowed

#### STATE & RECURRENCE

- $dp[i] = \text{max sum using elements from index } 0 \text{ to } i$ .
- Recurrence:  $dp[i] = \max(dp[i-1], dp[i-2] + arr[i])$ . Pick  $arr[i]$  (skip  $i-1$ ) or skip  $arr[i]$  (take  $dp[i-1]$ ).

#### HOUSE ROBBER (CIRCULAR ARRAY VARIANT)

- Can't rob both first and last house. Run max-non-adjacent on  $\text{arr}[0..N-2]$  and  $\text{arr}[1..N-1]$  separately, take the max.

#### COMPLEXITY

| Approach | Time   | Space  | Notes                        |
|----------|--------|--------|------------------------------|
| 1D DP    | $O(N)$ | $O(1)$ | Two variables: pick and skip |

Related: [LeetCode 198 House Robber](#), [LeetCode 213 House Robber II](#)

#### ■ Interview Trigger

Take or skip:  $\text{dp}[i] = \max(\text{dp}[i-1], \text{dp}[i-2] + \text{arr}[i])$ . Circular = run twice, take max.

### 16.3 2D/3D DP and Grids

## DP-8 / DP-9 · Grid Unique Paths I & II

#### WHAT IT'S ASKING

Find the number of unique paths, and the minimum path sum, in a grid where you can only move right or down.

#### TOPIC / PATTERN

2D DP — Count paths in a grid (move right or down only)

#### STATE & RECURRENCE

- $\text{dp}[i][j]$  = number of ways to reach cell  $(i, j)$ .
- $\text{dp}[i][j] = \text{dp}[i-1][j] + \text{dp}[i][j-1]$  (come from above or left).
- Base cases:  $\text{dp}[0][j] = 1$  and  $\text{dp}[i][0] = 1$  (only one way along edges).

#### WITH OBSTACLES (UNIQUE PATHS II)

- If  $\text{grid}[i][j] == 1$  (obstacle):  $\text{dp}[i][j] = 0$ . Else apply the same recurrence.

#### COMPLEXITY

| Approach | Time            | Space                     | Notes                             |
|----------|-----------------|---------------------------|-----------------------------------|
| 2D DP    | $O(M \times N)$ | $O(M \times N)$ or $O(N)$ | Space-optimize using a single row |

Space-optimize using a single row

Related: [LeetCode 62 Unique Paths](#), [LeetCode 63 Unique Paths II](#)

#### ■ Interview Trigger

Grid paths = sum of ways from top and from left — obstacle cells get 0.

## DP-10 / DP-11 / DP-12 · Min Path Sum, Triangle, Falling Path

**WHAT IT'S ASKING**

Find the minimum path sum from top to bottom of a triangle, and the min/max sum path falling through a matrix.

**TOPIC / PATTERN**

2D DP — Minimum cost path with directional constraints

**MINIMUM PATH SUM IN GRID**

–  $dp[i][j] = \text{min cost to reach } (i,j)$ .  $dp[i][j] = \text{grid}[i][j] + \min(dp[i-1][j], dp[i][j-1])$ .

**TRIANGLE — MIN PATH FROM TOP TO BOTTOM**

- Move bottom-up:  $dp[i][j] = \text{triangle}[i][j] + \min(dp[i+1][j], dp[i+1][j+1])$ .
- Or top-down:  $dp[i][j] = \text{triangle}[i][j] + \min(dp[i-1][j-1], dp[i-1][j])$ .
- Bottom-up is cleaner — modify in-place, answer is min of last row.

**MIN/MAX FALLING PATH SUM**

– From each cell  $(i,j)$ , can move to  $(i+1,j-1)$ ,  $(i+1,j)$ , or  $(i+1,j+1)$ . DP from top to bottom.

**COMPLEXITY**

| Approach             | Time            | Space                     | Notes                            |
|----------------------|-----------------|---------------------------|----------------------------------|
| 2D DP (all variants) | $O(M \times N)$ | $O(M \times N)$ or $O(N)$ | Space-optimize using one row/col |

Space-optimize using one row/col

**■ Interview Trigger**

Grid min-path: come from feasible directions, take the minimum + current cell cost.

**DP-13 · 3D DP — Ninja and His Friends****WHAT IT'S ASKING**

A ninja and two friends collect the maximum cherries possible moving simultaneously through a grid.

**TOPIC / PATTERN**

3D DP — Two entities moving simultaneously (2 column variables per row)

**CORE IDEA**

Two people start from column 0 and N-1 of row 0, both moving down. State: (row, col1, col2). They collect chocolates — if on the same cell, count once.

**STATE**

- $dp[\text{row}][\text{col1}][\text{col2}] = \text{max chocolates collected from row 0 to 'row', given positions (col1, col2)}$ .
- At each row, each person can move to 3 positions → 9 combinations to try.

**COMPLEXITY**

| Approach | Time | Space | Notes |
|----------|------|-------|-------|
|----------|------|-------|-------|

|       |                                   |                          |                                                   |
|-------|-----------------------------------|--------------------------|---------------------------------------------------|
| 3D DP | $O(R \times C \times C \times 9)$ | $O(R \times C \times C)$ | R=rows, C=cols — reduce to $O(C^2)$ space per row |
|-------|-----------------------------------|--------------------------|---------------------------------------------------|

#### ■ Interview Trigger

Two movers = track two column variables. 3D state (row, c1, c2). Try all 9 next-step combos.

### 16.4 DP on Subsequences

## DP-19 · 0/1 Knapsack

### WHAT IT'S ASKING

Solve the classic 0/1 Knapsack problem — maximize value in a knapsack where each item can be taken at most once.

### TOPIC / PATTERN

DP — Take or Not-Take each item, track remaining capacity

### STATE & RECURRENCE

- $dp[i][W]$  = max value using first  $i$  items with capacity  $W$ .
- Not take:  $dp[i][W] = dp[i-1][W]$ .
- Take (if  $weight[i] \leq W$ ):  $dp[i][W] = value[i] + dp[i-1][W - weight[i]]$ .
- $dp[i][W] = \max(\text{not-take}, \text{take})$ .

### SPACE OPTIMIZATION

– Only need the previous row → use a 1D dp array. Traverse  $W$  from RIGHT TO LEFT to avoid using updated values.

### TABULATION TEMPLATE

for  $i$  in  $1..N$ : for  $W$  in  $maxW..0$  (reverse): # KEY: right to left if  $weight[i] \leq W$ :  $dp[W] = \max(dp[W], val[i] + dp[W - weight[i]])$

### COMPLEXITY

| Approach          | Time            | Space           | Notes                               |
|-------------------|-----------------|-----------------|-------------------------------------|
| 2D DP             | $O(N \times W)$ | $O(N \times W)$ | Standard                            |
| 1D DP (optimized) | $O(N \times W)$ | $O(W)$          | Traverse right to left — essential! |

Related: Classic 0/1 Knapsack, LeetCode 416 Partition Equal Subset Sum

#### ■ Interview Trigger

0/1 Knapsack 1D: MUST traverse capacity right to left (prevents reusing same item).

## DP-20 / DP-22 / DP-23 · Coin Change, Coin Change 2, Unbounded Knapsack

### WHAT IT'S ASKING

Find the minimum coins (or number of ways) to make a target amount, with unlimited or limited supply of each coin.

### TOPIC / PATTERN

DP — Unbounded items (each item usable unlimited times)

### COIN CHANGE (MINIMUM COINS)

- $dp[amount]$  = min coins to make 'amount'.  $dp[0]=0$ , rest=infinity.
- For each coin  $c$ , for each amount  $W$  from  $c$  to target:  $dp[W] = \min(dp[W], 1+dp[W-c])$ .

### COIN CHANGE 2 (COUNT WAYS)

- $dp[amount]$  = number of ways.  $dp[0]=1$ .
- For each coin  $c$ , for each  $W$  from  $c$  to target:  $dp[W] += dp[W-c]$ .

### KEY DIFFERENCE FROM 0/1 KNAPSACK

- Unbounded = traverse  $W$  from LEFT TO RIGHT (can reuse same coin). 0/1 = RIGHT TO LEFT (each item once).

### COMPLEXITY

| Approach    | Time                        | Space              | Notes                   |
|-------------|-----------------------------|--------------------|-------------------------|
| Coin Change | $O(N \times \text{Amount})$ | $O(\text{Amount})$ | Left-to-right traversal |

| Approach      | Time                        | Space              | Notes                               |
|---------------|-----------------------------|--------------------|-------------------------------------|
| Coin Change 2 | $O(N \times \text{Amount})$ | $O(\text{Amount})$ | Same, but count ways instead of min |

Related: [LeetCode 322 Coin Change](#), [LeetCode 518 Coin Change II](#)

#### ■ Interview Trigger

Unbounded = left-to-right traversal (reuse allowed). 0/1 = right-to-left (no reuse).

## DP-21 · Target Sum

### WHAT IT'S ASKING

Determine if elements of an array can be assigned + or - signs to reach a given target sum.

### TOPIC / PATTERN

DP — Count subsets with specific sum difference

### CORE IDEA

Assign + or - to each number. Let  $S1$  = sum of positives,  $S2$  = sum of negatives.  $S1 - S2 = \text{target}$  AND  $S1 + S2 = \text{totalSum}$ , so  $S1 = (\text{totalSum} + \text{target}) / 2$ . Count subsets with sum =  $S1$ .

### ALGORITHM

- If  $(\text{totalSum} + \text{target})$  is odd or  $\text{target} > \text{totalSum} \rightarrow$  return 0.
- Otherwise, count subsets with sum =  $(\text{totalSum} + \text{target}) / 2$  using standard subset sum DP.

**COMPLEXITY**

| Approach        | Time                     | Space           | Notes                                   |
|-----------------|--------------------------|-----------------|-----------------------------------------|
| Subset Count DP | $O(N \times \text{Sum})$ | $O(\text{Sum})$ | Reduce to count subsets with target sum |

Related: [LeetCode 494 Target Sum](#)

### ■ Interview Trigger

Target Sum reduces to: count subsets with  $\text{sum} = (\text{totalSum} + \text{target}) / 2$ .

16.5 DP on Strings

## DP-25 · Longest Common Subsequence

### WHAT IT'S ASKING

Find the length of the Longest Common Subsequence between two strings.

### TOPIC / PATTERN

String DP — 2D table comparing two strings character by character

### STATE & RECURRENCE

- $\text{dp}[i][j]$  = LCS of first  $i$  chars of  $s1$  and first  $j$  chars of  $s2$ .
- If  $s1[i-1] == s2[j-1]$ :  $\text{dp}[i][j] = 1 + \text{dp}[i-1][j-1]$  (both chars match, extend LCS).
- Else:  $\text{dp}[i][j] = \max(\text{dp}[i-1][j], \text{dp}[i][j-1])$  (skip one char from either string).

### VISUALIZATION

$s1 = \text{'ABCDE'}$ ,  $s2 = \text{'ACE'}$

" A C E

" 0 0 0 0 A 0 1 1 1 B 0 1 1 1 C 0 1 2 2 D 0 1 2 2 E 0 1 2 3 → LCS length = 3 ✓

### WHY LCS IS THE FOUNDATION

- LCS directly solves: LCS itself, Shortest Common Supersequence, Min Insertions/Deletions, LCS-based

Edit Distance.

### COMPLEXITY

| Approach | Time            | Space                     | Notes                       |
|----------|-----------------|---------------------------|-----------------------------|
| 2D DP    | $O(N \times M)$ | $O(N \times M)$ or $O(M)$ | Space-optimize using 2 rows |

Space-optimize using 2 rows

Related: [LeetCode 1143 LCS](#), [LeetCode 1092 SCS](#), [LeetCode 583 Delete to Make Equal](#)

### ■ Interview Trigger

LCS: match → diagonal+1. No match → max of left or above. Master this table.

DP-27 to DP-31 · LCS Variants — Substring, Palindromic, Supersequence

### TOPIC / PATTERN

String DP — All derived from the core LCS framework

### LONGEST COMMON SUBSTRING (CONTIGUOUS)

- Same table as LCS, but reset  $dp[i][j]=0$  when characters don't match. Track max value overall.

### LONGEST PALINDROMIC SUBSEQUENCE

- $LPS(s) = LCS(s, reverse(s))$ .

### MIN INSERTIONS TO MAKE STRING PALINDROME

- Answer =  $N - LPS(s)$  (insert the missing characters of LPS to both ends).

### MIN INSERTIONS/DELETIONS TO CONVERT A TO B

- Delete from A:  $len(A) - LCS(A,B)$ . Insert into A:  $len(B) - LCS(A,B)$ . Total =  $(len(A)+len(B)) - 2*LCS$ .

### SHORTEST COMMON SUPERSEQUENCE

- SCS length =  $len(A) + len(B) - LCS(A,B)$ . Reconstruct by tracing the LCS table diagonally.

### COMPLEXITY

| Approach         | Time            | Space           | Notes                                          |
|------------------|-----------------|-----------------|------------------------------------------------|
| All LCS Variants | $O(N \times M)$ | $O(N \times M)$ | Same table structure, different interpretation |

#### ■ Interview Trigger

$LPS = LCS(s, rev(s))$ .  $SCS = len(A)+len(B)-LCS$ . Min del/ins = lengths minus LCS.

## DP-32 · Number of Distinct Subsequences

### WHAT IT'S ASKING

Count the number of distinct subsequences of one string that equal another string.

### TOPIC / PATTERN

String DP — Count ways to form target as subsequence of source

### STATE & RECURRENCE

- $dp[i][j]$  = number of times  $t[0..j-1]$  appears as a subsequence in  $s[0..i-1]$ .
- If  $s[i-1]=t[j-1]$ :  $dp[i][j] = dp[i-1][j-1] + dp[i-1][j]$ . Match: use it OR skip it.
- Else:  $dp[i][j] = dp[i-1][j]$ . No match: skip this char of s.

### COMPLEXITY

| Approach | Time            | Space                     | Notes                                   |
|----------|-----------------|---------------------------|-----------------------------------------|
| 2D DP    | $O(N \times M)$ | $O(N \times M)$ or $O(M)$ | Use modular arithmetic for large counts |

Use modular arithmetic for large counts

Related: [LeetCode 115 Distinct Subsequences](#)

### ■ Interview Trigger

Match: BOTH use it (diagonal) and skip it (above). No match: only skip (above).

## DP-33 · Edit Distance

### WHAT IT'S ASKING

Find the minimum number of insertions, deletions, and substitutions to convert one string into another.

### TOPIC / PATTERN

String DP — Minimum insert, delete, replace operations to convert s1 to s2

### STATE & RECURRENCE

- $dp[i][j]$  = min operations to convert  $s1[0..i-1]$  to  $s2[0..j-1]$ .
- If  $s1[i-1] == s2[j-1]$ :  $dp[i][j] = dp[i-1][j-1]$  (no operation needed, carry forward).
- Else:  $dp[i][j] = 1 + \min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])$ .

### THE THREE OPERATIONS

| Approach       | Time               | Space                            | Notes |
|----------------|--------------------|----------------------------------|-------|
| Replace        | $1 + dp[i-1][j-1]$ | Replace $s1[i-1]$ with $s2[j-1]$ |       |
| Delete from s1 | $1 + dp[i-1][j]$   | Remove current char of s1        |       |
| Insert into s1 | $1 + dp[i][j-1]$   | Add char from s2 into s1         |       |

### COMPLEXITY

| Approach | Time            | Space                     | Notes                 |
|----------|-----------------|---------------------------|-----------------------|
| 2D DP    | $O(N \times M)$ | $O(N \times M)$ or $O(M)$ | Same structure as LCS |

Same structure as LCS

Related: [LeetCode 72 Edit Distance](#)

### ■ Interview Trigger

Match → diagonal. No match →  $1 + \min(\text{replace}=\text{diagonal}, \text{delete}=\text{above}, \text{insert}=\text{left})$ .

## DP-34 · Wildcard Matching

### WHAT IT'S ASKING

Determine whether a string with '?' and '\*' wildcards matches a given pattern.

### TOPIC / PATTERN

String DP — Pattern matching with ? (any single char) and \* (any sequence)

### STATE & RECURRENCE

- $dp[i][j]$  = True if pattern  $p[0..i-1]$  matches string  $s[0..j-1]$ .

- If  $p[i-1] == '?'$  or  $p[i-1] == s[j-1]$ :  $dp[i][j] = dp[i-1][j-1]$ .
- If  $p[i-1] == '*'$ :  $dp[i][j] = dp[i-1][j]$  (star matches empty) OR  $dp[i][j-1]$  (star matches one more char).

### COMPLEXITY

| Approach | Time            | Space           | Notes                    |
|----------|-----------------|-----------------|--------------------------|
| 2D DP    | $O(N \times M)$ | $O(N \times M)$ | Similar to edit distance |

Related: [LeetCode 44 Wildcard Matching](#), [LeetCode 10 Regular Expression Matching](#)

### ■ Interview Trigger

'\*' is the tricky case: either it matches empty (look up) or one more char (look left).

## 16.6 DP on Stocks

Stock problems follow a state machine pattern: At each day, you're in one of several states (holding/not-holding, transactions remaining). The key is defining state as (day, holding, transactions\_remaining) and writing transitions.

DP-35 to DP-40 · All Stock Buy/Sell Problems — Unified Framework

### STATE MACHINE TEMPLATE

- State: (day, canBuy, transactionsLeft). 'canBuy' = 1 if not holding stock, 0 if holding.
- If canBuy: Either buy (pay prices[day]) and move to canBuy=0, or skip.
- If holding (canBuy=0): Either sell (earn prices[day]) and move to canBuy=1, or skip.

### PROBLEM VARIANTS

| Approach               | Time            | Space  | Notes                                         |
|------------------------|-----------------|--------|-----------------------------------------------|
| I — Single transaction | $O(N)$          | $O(1)$ | Track min price seen — no DP needed (greedy)  |
| II — Unlimited         | $O(N)$          | $O(1)$ | Add profit whenever $prices[i] > prices[i-1]$ |
| III — At most 2        | $O(N)$          | $O(1)$ | 4 states: buy1, sell1, buy2, sell2            |
| IV — At most K         | $O(N \times K)$ | $O(K)$ | DP with K as 3rd dimension                    |
| With Cooldown          | $O(N)$          | $O(1)$ | After sell, skip one day before buying        |
| With Fee               | $O(N)$          | $O(1)$ | Subtract fee on each sell                     |

### STOCK I — GREEDY (MOST COMMON INTERVIEW QUESTION)

–  $maxProfit = 0$ ,  $minPrice = prices[0]$ . For each price:  $minPrice = \min(minPrice, price)$ ,  $maxProfit = \max(maxProfit, price - minPrice)$ .

### STOCK II — UNLIMITED TRANSACTIONS

- Sum all positive differences: if  $prices[i] > prices[i-1]$ , add  $prices[i] - prices[i-1]$ .

### STOCK III / IV — STATE MACHINE DP

- $dp[canBuy][k]$  where  $k$  = remaining transactions. Memoize or tabulate.

### COOLDOWN

- After sell on day  $i$ , next buy can only be on day  $i+2$ . Handle by setting `canBuy` from day  $i+2$ .

Related: [LeetCode 121](#), [122](#), [123](#), [188](#), [309](#), [714](#)

### ■ Interview Trigger

All stock problems = state machine: (holding?, transactions\_left). Write transitions, memoize.

## 16.7 DP on LIS (Longest Increasing Subsequence)

## DP-41 / DP-43 · LIS — DP Approach and Binary Search Approach

### WHAT IT'S ASKING

Find the length of the Longest Increasing Subsequence, both with an  $O(N^2)$  DP approach and an  $O(N \log N)$  binary search approach.

### TOPIC / PATTERN

LIS — Find the length of the longest strictly increasing subsequence

### DP APPROACH — $O(N^2)$

- $dp[i]$  = LIS ending at index  $i$ .
- $dp[i] = 1 + \max(dp[j])$  for  $j < i$  where  $arr[j] < arr[i]$ .
- Answer =  $\max(dp[i])$  for all  $i$ .

### BINARY SEARCH APPROACH — $O(N \log N)$

Maintain a 'tails' array:  $tails[i]$  = smallest tail element of all increasing subsequences of length  $i+1$ . For each element, binary search for its position in tails and replace or extend.

1.  $tails = []$ .
2. For each `num`: if `num > tails[-1]` → append (extends LIS). Else → binary search for first element in tails  $\geq num$ , replace it (maintains optimal candidates).
3. Answer =  $len(tails)$ .

### WHY BINARY SEARCH WORKS

'tails' is always sorted. Replacing a larger tail with a smaller one for the same length keeps options open for future elements — it doesn't change the current LIS LENGTH, just who can extend it.

### COMPLEXITY

| Approach      | Time          | Space  | Notes                                                     |
|---------------|---------------|--------|-----------------------------------------------------------|
| DP            | $O(N^2)$      | $O(N)$ | Good for small $N$ or when subsequence needs reconstructi |
| Binary Search | $O(N \log N)$ | $O(N)$ | Optimal length — tails array is NOT the actual LIS        |

Related: [LeetCode 300 LIS](#), [LeetCode 354 Russian Doll Envelopes \(2D LIS\)](#)

**■ Interview Trigger**

LIS in  $O(N \log N)$ : maintain sorted 'tails' array, binary search to replace. `Length = tails.size()`.

**DP-44 / DP-46 · LIS Variants — Divisible Subset, Bitonic Subsequence****WHAT IT'S ASKING**

Find LIS variants — the largest subset where each element divides the next, and the longest bitonic subsequence.

**LARGEST DIVISIBLE SUBSET**

Sort array. Same DP as LIS but condition changes to `arr[j] % arr[i] == 0` (or vice versa). Also track the actual subset by storing the previous index for reconstruction.

**LONGEST BITONIC SUBSEQUENCE**

A bitonic sequence first increases then decreases. Compute LIS from LEFT (`dp1[i]` = LIS ending at *i*) and LIS from RIGHT (`dp2[i]` = LIS starting at *i*, i.e. LDS ending at *i*). Answer = `max(dp1[i] + dp2[i] - 1)` for all *i*.

**COMPLEXITY**

| Approach         | Time     | Space  | Notes                           |
|------------------|----------|--------|---------------------------------|
| Divisible Subset | $O(N^2)$ | $O(N)$ | LIS with divisibility condition |
| Bitonic          | $O(N^2)$ | $O(N)$ | Two LIS passes (LfiR and RfiL)  |

*Related: LeetCode 368 Largest Divisible Subset, LeetCode 673 Number of LIS*

**■ Interview Trigger**

Bitonic = LIS from left + LIS from right - 1 at each index. Find the maximum.

**16.8 MCM / Partition DP**

The Partition DP Template: For problems of the form 'split array/string into parts optimally', try every split point *k* in `[i..j-1]`. The answer for range `[i..j]` = min/max over all *k* of `(solve(i,k) + solve(k+1,j) + cost_of_combining)`.

**DP-48 / DP-49 · Matrix Chain Multiplication****WHAT IT'S ASKING**

Find the minimum number of scalar multiplications needed to multiply a chain of matrices in the optimal order.

**TOPIC / PATTERN**

Partition DP — Find optimal order to multiply a chain of matrices

**CORE IDEA**

Matrix multiplication is associative — order of grouping doesn't change the result but DRASTICALLY changes the number of scalar multiplications. Find the minimum.

**STATE & RECURRENCE**

- $dp[i][j]$  = min cost to multiply matrices from index  $i$  to  $j$ .
- Try every split point  $k$  in  $[i, j-1]$ :
- $dp[i][j] = \min(dp[i][k] + dp[k+1][j] + \text{dims}[i-1] * \text{dims}[k] * \text{dims}[j])$  for  $k = i$  to  $j-1$ .
- Base case:  $dp[i][i] = 0$  (single matrix, no multiplication needed).

#### FILLING ORDER

- Fill diagonally by LENGTH of interval (len=2, then len=3, ..., then len=N). Or use memoization.

#### COMPLEXITY

| Approach                  | Time     | Space    | Notes                                  |
|---------------------------|----------|----------|----------------------------------------|
| Brute (try all orderings) | $O(2^N)$ | $O(N)$   | Exponential                            |
| Partition DP              | $O(N^3)$ | $O(N^2)$ | Try all split points for all intervals |

Related: Classic MCM, LeetCode 1039 Minimum Score Triangulation, LeetCode 1000 Minimum Cost to Merge Stones

#### ■ Interview Trigger

MCM template:  $dp[i][j] = \min \text{ over } k \text{ of } dp[i][k] + dp[k+1][j] + \text{merge\_cost}(i,k,j)$ .

## DP-51 · Burst Balloons

#### WHAT IT'S ASKING

Given balloons with numbers on them, find the maximum coins from bursting them all in the best order.

#### TOPIC / PATTERN

Partition DP — Think of LAST balloon burst, not first

#### CORE IDEA

Counterintuitive: instead of thinking which balloon to burst FIRST, think which balloon is burst LAST in range  $[i, j]$ . If balloon  $k$  is last in range  $[i, j]$ , its neighbors are  $\text{nums}[i-1]$  and  $\text{nums}[j+1]$  at the time of bursting.

#### STATE & RECURRENCE

- $dp[i][j]$  = max coins from bursting all balloons in range  $(i, j)$  exclusive.
- $dp[i][j] = \max \text{ over } k \text{ in } (i, j) \text{ of } dp[i][k] + \text{nums}[i] * \text{nums}[k] * \text{nums}[j] + dp[k][j]$ .

#### COMPLEXITY

| Approach     | Time     | Space    | Notes                  |
|--------------|----------|----------|------------------------|
| Partition DP | $O(N^3)$ | $O(N^2)$ | Same MCM-style filling |

Related: LeetCode 312 Burst Balloons

#### ■ Interview Trigger

Burst Balloons: think of LAST balloon burst in the range, not the first — neighbors are then known.

## DP-53 · Palindrome Partitioning II (Min Cuts)

### WHAT IT'S ASKING

Find the minimum number of cuts needed to partition a string into palindromic substrings.

### TOPIC / PATTERN

Partition DP — Min cuts to make all partitions palindromes

### STATE & RECURRENCE

- $dp[i]$  = min cuts for substring  $s[0..i]$ .
- For each  $i$ : try every split  $j$  ( $0$  to  $i$ ). If  $s[j..i]$  is palindrome:  $dp[i] = \min(dp[i], dp[j-1]+1)$ .
- Precompute  $isPalin[j][i]$  in  $O(N^2)$  to avoid  $O(N)$  checks inside.

### COMPLEXITY

| Approach               | Time     | Space    | Notes                               |
|------------------------|----------|----------|-------------------------------------|
| Brute                  | $O(N^3)$ | $O(N^2)$ | Check palindrome for each substring |
| DP + Precomputed Palin | $O(N^2)$ | $O(N^2)$ | Precompute palindromes, then 1D DP  |

DP + Precomputed Palindrome

*Related: LeetCode 132 Palindrome Partitioning II, LeetCode 131 (all partitions — backtracking)*

#### ■ Interview Trigger

Precompute  $isPalin[i][j]$  in  $O(N^2)$ , then 1D DP for min cuts in  $O(N^2)$  total.

16.9 DP on Squares

## DP-55 / DP-56 · Maximal Rectangle and Count Square Submatrices

### WHAT IT'S ASKING

Find the largest rectangle of 1s in a binary matrix, and count the number of square submatrices filled with 1s.

### TOPIC / PATTERN

2D DP — Build on top of histogram techniques

#### MAXIMAL RECTANGLE WITH ALL 1S

- Convert each row to a histogram ( $height[j]$  = consecutive 1s ending at this row in column  $j$ ).
- Apply 'Largest Rectangle in Histogram' (monotonic stack) to each row's histogram.
- Track the global maximum area.

#### COUNT SQUARE SUBMATRICES WITH ALL 1S

- $dp[i][j]$  = size of the largest square with bottom-right corner at  $(i,j)$ .
- $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$  if  $grid[i][j] == 1$ .
- Every  $dp[i][j] = k$  means there are  $k$  squares (of sizes  $1 \times 1, 2 \times 2, \dots, k \times k$ ) at this corner.
- Answer = sum of all  $dp[i][j]$ .

**COMPLEXITY**

| Approach          | Time            | Space                     | Notes                      |
|-------------------|-----------------|---------------------------|----------------------------|
| Maximal Rectangle | $O(M \times N)$ | $O(N)$                    | Histogram approach per row |
| Count Squares     | $O(M \times N)$ | $O(M \times N)$ or $O(N)$ | DP directly on grid        |

DP directly on grid

*Related: LeetCode 85 Maximal Rectangle, LeetCode 221 Maximal Square, LeetCode 1277 Count Square Submatrices*

#### ■ Interview Trigger

Maximal Rectangle = histogram trick per row. Count Squares = min of 3 neighbors + 1.

### ADDITIONAL COVERAGE — TO COMPLETE OFFICIAL 452-PROBLEM NUMBERING

## DP-7 · Ninja's Training (Single Ninja)

### WHAT IT'S ASKING

Choose one activity per day to maximize total points, never repeating the same activity on two consecutive days.

### TOPIC / PATTERN

1D/2D DP — Choice with an adjacency constraint

### CORE IDEA

Each day the ninja picks one of 3 activities for points, but can't repeat the same activity on two consecutive days — track the best total given which activity was last chosen.

### ALGORITHM

- 1.  $dp[day][last]$  = max points achievable up to 'day' given activity 'last' was NOT done today.
- 2. For each day, for each activity  $\neq last$ ,  $dp = points[day][activity] + dp[day-1][activity]$ .
- 3. Take the max over the 2 other allowed activities.

### COMPLEXITY

- Time:  $O(days \times 4)$
- Space:  $O(days \times 4)$ , reducible to  $O(4)$

#### ■ Interview Trigger

State = (day, activity done today) — forbid repeating the previous day's activity.

## DP-14 · Subset Sum Equal to Target

### WHAT IT'S ASKING

Determine whether any subset of an array sums to exactly a given target.

### TOPIC / PATTERN

DP on Subsequences — 0/1 take/not-take

### CORE IDEA

Classic take/not-take: does some subset of the array sum to exactly the target?

### ALGORITHM

- 1.  $dp[i][target] = \text{true}$  if some subset of  $arr[0..i]$  sums to target.
- 2.  $dp[i][target] = dp[i-1][target]$  (not take) OR  $(target \geq arr[i] \text{ AND } dp[i-1][target - arr[i]])$  (take).
- 3. Base case:  $dp[i][0] = \text{true}$  always (empty subset).

### COMPLEXITY

- Time:  $O(N \times target)$
- Space:  $O(N \times target)$ , reducible to  $O(target)$

#### ■ Interview Trigger

Same take/not-take template as 0/1 Knapsack, just a boolean instead of a max.

## DP-15 · Partition Equal Subset Sum

### WHAT IT'S ASKING

Determine whether an array can be split into two subsets with equal sums.

### TOPIC / PATTERN

DP on Subsequences — Subset Sum applied

### CORE IDEA

The array can be split into two equal-sum halves only if the total sum is even AND some subset sums to exactly  $totalSum/2$  — a direct call to Subset Sum Equal to Target.

### ALGORITHM

- 1. If  $totalSum$  is odd  $\rightarrow$  return false immediately.
- 2. Otherwise run Subset Sum Equal to Target with  $target = totalSum / 2$ .

### COMPLEXITY

- Time:  $O(N \times totalSum)$
- Space:  $O(totalSum)$

#### ■ Interview Trigger

Odd total  $\rightarrow$  impossible immediately; otherwise it's Subset Sum with  $target = sum/2$ .

## DP-17 · Count Subsets with Sum K

### WHAT IT'S ASKING

Count how many subsets of an array sum to exactly K.

### TOPIC / PATTERN

DP on Subsequences — Counting variant of Subset Sum

**CORE IDEA**

Same recurrence as Subset Sum Equal to Target, but ADD the take and not-take counts instead of OR-ing booleans.

**ALGORITHM**

- 1.  $dp[i][target]$  = number of subsets of  $arr[0..i]$  summing to target.
- 2.  $dp[i][target] = dp[i-1][target] + (target \geq arr[i] ? dp[i-1][target - arr[i]] : 0)$ .
- 3. Watch for zeros in the array — they double-count subsets, requiring special base-case handling ( $2^{(\text{count of zeros})}$ ).

**COMPLEXITY**

- Time:  $O(N \times target)$
- Space:  $O(target)$

**■ Interview Trigger**

Same shape as Subset Sum Equal to Target — swap the boolean OR for an integer addition.

**DP-24 · Rod Cutting Problem****WHAT IT'S ASKING**

Cut a rod into pieces of integer lengths to maximize total sale value, given a price for each length.

**TOPIC / PATTERN**

DP on Subsequences — Unbounded Knapsack in disguise

**CORE IDEA**

Cutting a rod of length  $N$  into pieces to maximize price is exactly Unbounded Knapsack, where "piece length  $i$ " is an item you can pick unlimited times with  $weight=i$ ,  $value=price[i]$ .

**ALGORITHM**

- 1.  $dp[len]$  = max value obtainable from a rod of length 'len'.
- 2. For each length from 1 to  $N$ , try every cut length  $i \leq length$ :  $dp[length] = \max(dp[length], price[i] + dp[length-i])$ .

**COMPLEXITY**

- Time:  $O(N^2)$
- Space:  $O(N)$

**■ Interview Trigger**

"Cut into pieces, maximize value, reuse lengths freely" = Unbounded Knapsack with  $weight = length$ .

**DP-27 · Longest Common Substring****WHAT IT'S ASKING**

Find the length of the longest contiguous substring common to two strings.

**TOPIC / PATTERN**

DP on Strings — LCS variant requiring contiguity

### CORE IDEA

Unlike LCS (subsequence), a substring must be contiguous — so the moment characters mismatch, the running match length resets to 0 instead of carrying over.

### ALGORITHM

- 1.  $dp[i][j]$  = length of the common substring ending exactly at  $s1[i-1]$  and  $s2[j-1]$ .
- 2. If  $s1[i-1] == s2[j-1]$ :  $dp[i][j] = 1 + dp[i-1][j-1]$ . Else  $dp[i][j] = 0$  (hard reset, unlike LCS).
- 3. Track the maximum dp value seen anywhere as the answer.

### COMPLEXITY

- Time:  $O(N \times M)$
- Space:  $O(N \times M)$ , reducible to  $O(M)$

#### ■ Interview Trigger

Mismatch resets the counter to 0 — that's the one change from Longest Common Subsequence.

## DP-28 · Longest Palindromic Subsequence

### WHAT IT'S ASKING

Find the length of the longest subsequence of a string that reads the same forwards and backwards.

### TOPIC / PATTERN

DP on Strings — LCS of a string with its reverse

### CORE IDEA

The longest palindromic subsequence of  $s$  equals the Longest Common Subsequence of  $s$  and  $\text{reverse}(s)$ .

### ALGORITHM

- 1. Reverse the string to get  $s2$ .
- 2. Run standard  $\text{LCS}(s, s2)$ .
- 3. The LCS length is the answer.

### COMPLEXITY

- Time:  $O(N^2)$
- Space:  $O(N^2)$ , reducible to  $O(N)$

#### ■ Interview Trigger

Palindromic subsequence of  $s = \text{LCS}(s, \text{reverse}(s))$  — no new DP needed.

## DP-29 · Minimum Insertions to Make String Palindrome

### WHAT IT'S ASKING

Find the minimum number of characters to insert into a string to make it a palindrome.

### TOPIC / PATTERN

DP on Strings — Built on Longest Palindromic Subsequence

### CORE IDEA

Every character NOT part of the longest palindromic subsequence needs a matching insertion — so the answer is  $N$  minus the LPS length.

### ALGORITHM

- 1. Compute Longest Palindromic Subsequence (LPS) length of the string.
- 2. Answer =  $N$  - LPS length.

### COMPLEXITY

- Time:  $O(N^2)$
- Space:  $O(N)$

#### ■ Interview Trigger

Insertions needed = length – longest palindromic subsequence.

## DP-30 · Minimum Insertions/Deletions to Convert String A to B

### WHAT IT'S ASKING

Find the minimum number of insertions and deletions needed to transform string A into string B.

### TOPIC / PATTERN

DP on Strings — Built on LCS

### CORE IDEA

Keep the Longest Common Subsequence intact; delete everything else from A, insert everything else from B.

### ALGORITHM

- 1. Compute LCS(A, B) length.
- 2. Deletions needed =  $\text{len}(A)$  - LCS length.
- 3. Insertions needed =  $\text{len}(B)$  - LCS length.

### COMPLEXITY

- Time:  $O(N \times M)$
- Space:  $O(N \times M)$ , reducible to  $O(M)$

#### ■ Interview Trigger

Everything outside the LCS in A gets deleted; everything outside the LCS in B gets inserted.

## DP-31 · Shortest Common Supersequence

### WHAT IT'S ASKING

Find the shortest string that contains both given strings as subsequences.

### TOPIC / PATTERN

DP on Strings — Built on LCS, reconstruction required

**CORE IDEA**

Merge both strings into the shortest string containing both as subsequences by walking the LCS DP table backward: matched characters are written once, unmatched characters from either string are written as-is.

**ALGORITHM**

- 1. Build the standard LCS DP table for A and B.
- 2. Walk back from  $dp[N][M]$ : if characters match, write it once and move diagonally.
- 3. Otherwise move toward (and write the character from) whichever cell (up or left) has the larger dp value.
- 4. Append any remaining prefix of A or B once the table walk reaches an edge.

**COMPLEXITY**

- Time:  $O(N \times M)$
- Space:  $O(N \times M)$

**■ Interview Trigger**

Build the LCS table, then walk it backward — shared characters written once, the rest written from both sides.

## DP-35 / DP-36 / DP-37 / DP-38 / DP-39 / DP-40 · DP on Stocks (Buy & Sell Stock I–VI)

**WHAT IT'S ASKING**

Maximize profit from buying and selling a stock under varying rules: one transaction, unlimited transactions, at most 2 or K transactions, or with a cooldown/fee.

**TOPIC / PATTERN**

DP on Stocks — State machine over (day, holding?, transactions used)

**CORE IDEA**

All six stock problems share one state machine: on each day you're either holding a stock or not, and the variants add constraints — one transaction only, unlimited transactions, at most K transactions, or a cooldown/fee after selling.

**ALGORITHM**

- 1. Best Time to Buy/Sell (I): track `minPriceSoFar`,  $\text{answer} = \max(\text{profit}, \text{price} - \text{minPriceSoFar})$  in one pass.
- 2. Buy/Sell II (unlimited transactions): greedily add every positive price difference between consecutive days.
- 3. Buy/Sell III / IV (at most 2 / at most K transactions):  $dp[\text{day}][\text{transactionsLeft}][\text{holding}] = \max \text{profit}$ ; transition is buy, sell, or do nothing.
- 4. With Cooldown: after selling, you must skip a day before buying again — add a 'cooldown' state.
- 5. With Transaction Fee: subtract a fixed fee at the moment you sell.

**COMPLEXITY**

- Time:  $O(N)$  for I & II,  $O(N \times K)$  for III/IV,  $O(N)$  for cooldown/fee variants
- Space:  $O(1)$  to  $O(K)$  depending on variant

**■ Interview Trigger**

Always the same 2-3 states (holding or not, transactions left) — each variant just tweaks the transition rule.

**DP-50 · Minimum Cost to Cut the Stick****WHAT IT'S ASKING**

Given required cut positions on a stick, find the minimum total cost to make all cuts, where each cut costs the current segment's length.

**TOPIC / PATTERN**

Partition DP — MCM-style interval DP

**CORE IDEA**

Add 0 and stick length as sentinel cut points; the cost of a cut equals the current segment's length, and cutting order matters, so try every cut as the LAST one made within a range.

**ALGORITHM**

- 1. Sort the cut positions, add 0 and length as boundary sentinels.
- 2.  $dp[i][j]$  = min cost to make all cuts strictly between sentinel index  $i$  and  $j$ .
- 3. For each cut index  $k$  between  $i$  and  $j$ :  $dp[i][j] = \min(dp[i][j], (cuts[j] - cuts[i]) + dp[i][k] + dp[k][j])$ .

**COMPLEXITY**

- Time:  $O(C^3)$ ,  $C$  = number of cuts
- Space:  $O(C^2)$

**■ Interview Trigger**

Add boundary sentinels, then it's Matrix Chain Multiplication with "cost of a cut = segment length."

**DP-52 · Evaluate Boolean Expression to True****WHAT IT'S ASKING**

Count the number of ways to parenthesize a boolean expression (with AND/OR/XOR) so it evaluates to true.

**TOPIC / PATTERN**

Partition DP — Count ways over operator positions

**CORE IDEA**

For every operator in the expression, split into left/right halves and count ways each half evaluates to true/false, then combine counts according to the operator (AND/OR/XOR) truth table.

**ALGORITHM**

- 1.  $dp[i][j][true/false]$  = number of ways the substring expression  $[i..j]$  evaluates to true or false.
- 2. For every operator position  $k$  between  $i$  and  $j$ , combine leftTrue/leftFalse with rightTrue/rightFalse per the operator's truth table.
- 3. Sum contributions across all valid split points  $k$ .

**COMPLEXITY**

- Time:  $O(N^3)$
- Space:  $O(N^2)$ , 2 states each

**■ Interview Trigger**

Interval DP like MCM, but each split combines TRUE/FALSE counts through the operator's truth table.

**DP-54 · Partition Array for Maximum Sum****WHAT IT'S ASKING**

Partition an array into contiguous subarrays of length at most K, replacing each subarray's elements with its max, to maximize the total sum.

**TOPIC / PATTERN**

1D DP — Bounded look-back window

**CORE IDEA**

Partition the array into contiguous subarrays of length at most K, where every element in a partition is replaced by that partition's max — maximize the total sum.

**ALGORITHM**

- 1.  $dp[i]$  = max sum achievable for the first i elements.
- 2. For each i, try every partition length L from 1 to K ending at i: track the max element in that trailing window,  $dp[i] = \max(dp[i], dp[i-L] + \max \times L)$ .

**COMPLEXITY**

- Time:  $O(N \times K)$
- Space:  $O(N)$

**■ Interview Trigger**

Try every partition length up to K ending at the current index, tracking the running max in that window.

**STEP 17 - Tries**

17.1 / 17.2 Trie Implementation and Problems

**17.1.1 / 17.1.2 · Implement Trie I and II****WHAT IT'S ASKING**

Implement Insert and Search (with wildcard '.') operations for a Trie (prefix tree).

**TOPIC / PATTERN**

Trie — Prefix tree for  $O(L)$  string insert/search/prefix-check

**STRUCTURE**

- Each node has 26 children (for lowercase a-z) and a boolean 'isEnd'.

- TrieNode: children[26] = null, isEnd = false.

#### INSERT(WORD)

- Start at root. For each char in word: if child doesn't exist → create it. Move to child. Mark isEnd at last node.

#### SEARCH(WORD)

- Traverse char by char. If any child missing → return false. At the end: return node.isEnd.

#### STARTSWITH(PREFIX)

- Same as search but don't check isEnd — just need to traverse successfully.

#### TRIE II ADDITIONS

- Track countOfWordsEndingHere and countOfWordsWithPrefix per node for frequency counting.

#### COMPLEXITY

| Approach      | Time   | Space            | Notes                                        |
|---------------|--------|------------------|----------------------------------------------|
| Insert        | $O(L)$ | $O(L \times 26)$ | $L$ = word length; each node has 26 pointers |
| Search/Prefix | $O(L)$ | $O(1)$           | Just traverse, no extra space                |

*Related: LeetCode 208 Implement Trie, LeetCode 211 Design Add and Search Words*

#### ■ Interview Trigger

Trie =  $O(L)$  insert and search ( $L$ =word length). Much faster than HashMap for prefix queries.

## 17.1.3 · Longest String with All Prefixes

### WHAT IT'S ASKING

Find the longest string in an array such that every prefix of it is also present in the array.

### TOPIC / PATTERN

Trie — Check if all prefixes of a word exist in the set

### CORE IDEA

A word is valid if ALL its prefixes are also words in the set. Insert all words into a Trie. Then DFS through the Trie, only traversing nodes marked isEnd=True — these nodes form valid prefix chains.

### ALGORITHM

- 1. Insert all words into the Trie.
- 2. For each word: traverse the Trie. At every node, check isEnd. If any node's isEnd is false, this word is invalid.
- 3. Among all valid words, return the longest (break ties alphabetically).

### COMPLEXITY

| Approach   | Time                            | Space                           | Notes                     |
|------------|---------------------------------|---------------------------------|---------------------------|
| Trie + DFS | $O(\text{sum of word lengths})$ | $O(\text{sum of word lengths})$ | Build Trie, then traverse |

Trie + DFS  $O(\text{sum of word lengths})$   $O(\text{sum of lengths Build} \times 26)$  Trie then traverse

*Related: LeetCode 720 Longest Word in Dictionary*

#### ■ Interview Trigger

Valid prefix chain = every prefix also in the set. Trie DFS only through isEnd nodes.

## 17.2.1 · Maximum XOR of Two Numbers in an Array

### WHAT IT'S ASKING

Find the maximum XOR value obtainable by pairing any two numbers in an array, using a Trie.

### TOPIC / PATTERN

Trie — Binary Trie for bit-by-bit XOR maximization

### CORE IDEA

Build a BINARY TRIE where each number is inserted bit by bit (MSB to LSB). For each number  $x$ , to maximize XOR, greedily try to take the OPPOSITE bit at each level (if  $x$  has bit 0 here, try to go to bit-1 branch in the Trie).

### ALGORITHM

- 1. Insert all numbers into binary trie (bit 31 to bit 0).
- 2. For each number  $x$ : traverse trie, at each bit position try the opposite branch.
- 3. If opposite branch exists → take it (contributes 1 to XOR at this bit). Else take same branch.
- 4. Track the XOR value built so far. After 32 levels, that's the max XOR with  $x$ .
- 5. Answer = max over all  $x$  of the XOR found.

### COMPLEXITY

| Approach          | Time             | Space            | Notes                                    |
|-------------------|------------------|------------------|------------------------------------------|
| Brute (all pairs) | $O(N^2)$         | $O(1)$           | Check all pairs                          |
| Binary Trie       | $O(N \times 32)$ | $O(N \times 32)$ | Build trie + query each number — optimal |

*Related: LeetCode 421 Maximum XOR of Two Numbers in an Array, LeetCode 1707 Maximum XOR with an Element from*

Array

#### ■ Interview Trigger

Binary Trie: for each number, greedily take the OPPOSITE bit at each level to maximize XOR.

Addendum — 9 Missing Problems (Restored) These 9 problems exist in Striver's official A2Z sheet (55 DP problems + 7 Trie problems) but were dropped from the original batch when companion/variant problems got merged into combined write-ups. Added back below in their correct subsections.

## 16.4 DP on Subsequences (Addendum)

**DP-16 · Partition Set Into 2 Subsets With Min Absolute Sum Diff****WHAT IT'S ASKING**

Partition an array into two subsets minimizing the absolute difference of their sums.

**TOPIC / PATTERN**

DP on Subsequences — extension of Subset Sum

**STATE & RECURRENCE**

- Run the standard Subset-Sum DP:  $dp[i][s] = \text{true}$  if some subset of the first  $i$  elements sums to  $s$ .
- Compute  $dp$  for every achievable sum  $s$  from 0 to  $\text{totalSum}$  using the array (Subset Sum, DP-14 pattern).
- For every  $s$  where  $dp[n][s]$  is true, one partition is  $(s, \text{totalSum} - s)$ ; minimise  $|\text{totalSum} - 2s|$ .

**SPACE OPTIMIZATION**

- Only need a 1D boolean array of size  $\text{totalSum}+1$ , updated right-to-left per element (same as 0/1 Knapsack).

**COMPLEXITY**

| Approach | Time                     | Space           | Notes                                                          |
|----------|--------------------------|-----------------|----------------------------------------------------------------|
| 1D DP    | $O(N \times \text{sum})$ | $O(\text{sum})$ | Reuses Subset-Sum table; scan achievable sums for the min diff |

sums for the min diff

*Related: Subset Sum Equal to Target (DP-14), Partition Equal Subset Sum (DP-15)*

**■ Interview Trigger**

Reduces to Subset Sum: generate every reachable sum up to  $\text{totalSum}/2$ , then minimise  $|\text{totalSum} - 2 \times \text{sum}|$ .

**DP-18 · Count Partitions with Given Difference****WHAT IT'S ASKING**

Count the number of ways to partition an array into two subsets with a given difference between their sums.

**TOPIC / PATTERN**

DP on Subsequences — reduces to Count Subsets with Sum  $K$

**STATE & RECURRENCE**

- Want subsets  $S_1, S_2$  with  $S_1 - S_2 = \text{diff}$  and  $S_1 + S_2 = \text{totalSum}$ , so  $S_1 = (\text{totalSum} + \text{diff}) / 2$ .
- If  $(\text{totalSum} + \text{diff})$  is odd, or  $\text{diff} > \text{totalSum}$ , the answer is 0 — no valid partition exists.
- Otherwise the answer is exactly “Count Subsets with Sum  $K$ ” (DP-17) with  $K = (\text{totalSum} + \text{diff}) / 2$ .

**EDGE CASE — ZEROS**

- Array zeros double-count subsets (a zero can be in S1 or S2 without changing the sum) — the DP-17

recurrence already handles this by adding both branches when  $\text{arr}[i] == 0$ .

#### COMPLEXITY

| Approach | Time            | Space  | Notes                                                 |
|----------|-----------------|--------|-------------------------------------------------------|
| 1D DP    | $O(N \times K)$ | $O(K)$ | Identical implementation to DP-17, different target K |

target K

*Related: Count Subsets with Sum K (DP-17), Target Sum (DP-21)*

#### ■ Interview Trigger

See “difference” between two subset sums → convert to a single target sum  $K = (\text{total} + \text{diff})/2$ , then run Count Subsets with Sum K.

16.5 DP on Strings (Addendum)

## DP-26 · Print Longest Common Subsequence

### WHAT IT'S ASKING

Not just the length, but actually print/reconstruct one Longest Common Subsequence between two strings.

### TOPIC / PATTERN

DP on Strings — backtracking through an LCS table

### ALGORITHM

- First build the full  $(n+1) \times (m+1)$  LCS DP table exactly as in DP-25 (do NOT space-optimize — the full table

is needed to backtrack).

- Start at  $i=n, j=m$ . If  $s1[i-1] == s2[j-1]$ : append that character to the result and move to  $(i-1, j-1)$ .
- Else move toward the larger neighbour: to  $(i-1, j)$  if  $\text{dp}[i-1][j] \geq \text{dp}[i][j-1]$ , otherwise to  $(i, j-1)$ .
- Stop when  $i==0$  or  $j==0$ , then reverse the collected characters to get the LCS string.

### COMPLEXITY

| Approach          | Time            | Space           | Notes                                                     |
|-------------------|-----------------|-----------------|-----------------------------------------------------------|
| 2D DP + backtrack | $O(N \times M)$ | $O(N \times M)$ | Full table required — cannot space-optimize when printing |

when printing

*Related: Longest Common Subsequence (DP-25)*

#### ■ Interview Trigger

Printing (not just counting) a DP string result almost always means: keep the full 2D table, then walk it backward from  $\text{dp}[n][m]$ .

## 16.7 DP on LIS (Addendum)

**DP-42 · Printing Longest Increasing Subsequence****WHAT IT'S ASKING**

Not just the length, but actually print/reconstruct one Longest Increasing Subsequence of an array.

**TOPIC / PATTERN**

DP on LIS — backtracking with a parent/hash array

**ALGORITHM**

- Run the  $O(N^2)$  LIS DP:  $dp[i]$  = length of the LIS ending at index  $i$ , alongside a  $hash[i]$  parent pointer.
- When updating  $dp[i] = dp[j] + 1$  for some  $j < i$  with  $arr[j] < arr[i]$ , also set  $hash[i] = j$ .
- Initialise  $hash[i] = i$  for all  $i$  (self-loop marks the start of a chain).
- Find the index with the maximum  $dp$  value, then walk  $hash[]$  backward until  $hash[i] == i$ , collecting elements.
- Reverse the collected elements to get the actual LIS (not just its length).

**COMPLEXITY**

| Approach                   | Time     | Space  | Notes                                                                         |
|----------------------------|----------|--------|-------------------------------------------------------------------------------|
| $O(N^2)$ DP + parent array | $O(N^2)$ | $O(N)$ | Binary-search $O(N \log N)$<br>LIS does not directly reconstruct the sequence |

directly reconstruct the sequence

*Related: Longest Increasing Subsequence (DP-41), LIS using Binary Search (DP-43)*

**■ Interview Trigger**

2 Need the actual subsequence, not just its length → use the  $O(N^2)$  DP with a parent/hash array; the  $O(N \log N)$  tails trick only gives length.

**DP-45 · Longest String Chain****WHAT IT'S ASKING**

Find the longest chain of words where each word is formed by adding exactly one letter to the previous word.

**TOPIC / PATTERN**

DP on LIS-style chains over strings

**STATE & RECURRENCE**

- Sort all words by length ascending, so any predecessor is processed before its successor.
- $dp[word] = 1 + \max(dp[predecessor])$  over every predecessor formed by deleting exactly one character from word.
- A word with no valid predecessor in the set has  $dp[word] = 1$ .

– Answer =  $\max(dp[word])$  over all words.

### IMPLEMENTATION

– Store dp in a hashmap keyed by word. For each word, try removing each character (one at a time) and

look up the resulting string.

### COMPLEXITY

| Approach   | Time                      | Space  | Notes                                                             |
|------------|---------------------------|--------|-------------------------------------------------------------------|
| HashMap DP | $O((cid:229)len(word)^2)$ | $O(N)$ | Same “take/skip predecessor” shape as LIS, generalised to strings |

generalised to strings

*Related: Largest Divisible Subset (DP-44)*

#### ■ Interview Trigger

“Chain where each step differs by one character” is LIS on strings — sort by length, DP over single-character-deletion predecessors.

## DP-47 · Number of Longest Increasing Subsequences

### WHAT IT'S ASKING

Count how many distinct Longest Increasing Subsequences exist in an array.

### TOPIC / PATTERN

DP on LIS — counting alongside length

### STATE & RECURRENCE

- Maintain  $dp[i]$  = length of LIS ending at  $i$ , and  $cnt[i]$  = number of LIS of that length ending at  $i$ .
- For  $j < i$  with  $arr[j] < arr[i]$ : if  $dp[j] + 1 > dp[i]$ , a strictly longer chain is found  $\rightarrow dp[i] = dp[j] + 1$ ,  $cnt[i] = cnt[j]$ .
- If  $dp[j] + 1 == dp[i]$  (a tie at the current best length), accumulate:  $cnt[i] += cnt[j]$ .
- Answer = sum of  $cnt[i]$  over every index  $i$  where  $dp[i]$  equals the overall maximum LIS length.

### COMPLEXITY

| Approach             | Time     | Space  | Notes                                                |
|----------------------|----------|--------|------------------------------------------------------|
| $O(N^2)$ DP + counts | $O(N^2)$ | $O(N)$ | Two parallel arrays: length and count-of-that-length |

count-of-that-length

*Related: Longest Increasing Subsequence (DP-41)*

#### ■ Interview Trigger

Whenever a DP asks “how many ways” alongside an optimum, carry a parallel count array and accumulate on ties, overwrite on strict improvement.

## 17 Tries (Addendum)

## 17.1.4 · Number of Distinct Substrings in a String

## WHAT IT'S ASKING

Count the number of distinct substrings (including the empty prefix) of a string efficiently using a Trie.

## TOPIC / PATTERN

Trie insertion — count new nodes created

## ALGORITHM

- Insert every suffix of the string into a Trie, one suffix at a time (n suffixes total).
- Each time an insertion creates a brand-new node, that corresponds to exactly one new distinct substring.
- Include the empty substring in the final count if the problem asks for it (add 1 at the end).

## COMPLEXITY

| Approach             | Time     | Space    | Notes                                                                   |
|----------------------|----------|----------|-------------------------------------------------------------------------|
| Trie of all suffixes | $O(N^2)$ | $O(N^2)$ | N suffixes, each up to length N; a suffix automaton gets this to $O(N)$ |

2 2

automaton gets this to  $O(N)$

*Related: Implement Trie I & II (17.1.1)*

## ■ Interview Trigger

“Count distinct substrings” → insert all suffixes into a Trie and count new-node creations, not the substrings themselves.

## 17.2.0 · Bit PreRequisites for Trie Problems

## WHAT IT'S ASKING

Understand bit-manipulation prerequisites (XOR properties, binary trie structure) needed for Trie-based bit problems.

## TOPIC / PATTERN

Concept — Binary (XOR) Trie

## CORE IDEA

- Store each number as a fixed-width binary string (commonly 32 bits, MSB first) as a path down the Trie.
- Each Trie node has at most 2 children: bit 0 and bit 1 — this is what makes XOR queries  $O(\text{bit-width})$  instead of  $O(N)$ .

- To maximise XOR with a query number, greedily descend toward the child representing the OPPOSITE bit

at each level, when available.

### WHY IT WORKS

- XOR is maximised bit-by-bit from the most significant bit down — a mismatched bit contributes more value

than any combination of lower bits, so greedy MSB-first choice is always optimal.

*Related: Maximum XOR of Two Numbers in Array (17.2.1)*

#### ■ Interview Trigger

Any “maximum XOR” problem is a Binary Trie problem: insert fixed-width bit paths, then greedily walk toward opposite bits.

## 17.2.2 · Maximum XOR With an Element From Array

### WHAT IT'S ASKING

Given an array and queries, find the maximum XOR of a query value with any array element using a Trie.

### TOPIC / PATTERN

Binary Trie — offline queries with a constraint

### ALGORITHM

- Each query is a pair  $(x_i, m_i)$ : find the max XOR of  $x_i$  with any array element  $\leq m_i$  (or  $-1$  if none qualify).
- Sort the array ascending, and sort queries by  $m_i$  ascending — this makes the constraint monotonic.
- Sweep through queries in that order, inserting array elements into the Binary Trie while they stay  $\leq$  the

current  $m_i$ .

- For each query, if no elements have been inserted yet, answer  $-1$ ; otherwise greedily query the Trie for the

max XOR with  $x_i$ .

### COMPLEXITY

| Approach           | Time                         | Space            | Notes                                                             |
|--------------------|------------------------------|------------------|-------------------------------------------------------------------|
| Sort + Binary Trie | $O((N+Q)\log + Q \times 32)$ | $O(N \times 32)$ | Offline processing turns a per-query filter into one linear sweep |

$Q \times 32$ ) into one linear sweep

*Related: Maximum XOR of Two Numbers in Array (17.2.1), Bit PreRequisites for Trie Problems*

#### ■ Interview Trigger

“Max XOR subject to a constraint”  $\rightarrow$  sort queries by the constraint, insert into the Trie incrementally (offline sweep), not one Trie rebuild per query.

## DP + TRIES — MASTER CHEATSHEET

## DP PROBLEM TYPE RECOGNITION

| Problem Type        | State Definition                 | Recurrence Pattern                                     | Key Trick                            |
|---------------------|----------------------------------|--------------------------------------------------------|--------------------------------------|
| 1D Linear           | $dp[i]$                          | $dp[i] = f(dp[i-1], dp[i-2])$                          | Often $O(1)$ space with 2 vars       |
| 2D Grid             | $dp[i][j]$                       | $dp[i][j] = f(dp[i-1][j], dp[i][j-1])$                 | Space-optimize to 1 row              |
| Knapsack 0/1        | $dp[i][W]$                       | Take or skip, trace W right-to-left                    | 1D array, RIGHT-to-LEFT              |
| Knapsack Unbounded  | $dp[W]$                          | Take repeatedly, trace W left-to-right                 | 1D array, LEFT-to-RIGHT              |
| String LCS          | $dp[i][j]$                       | Match/diagonal+1, else $\max(\text{up}, \text{left})$  | All string DP derives from this      |
| Edit Distance       | $dp[i][j]$                       | Match/diagonal, else $1 + \min(3 \text{ dirs})$        | Replace, delete, insert              |
| Stock State Machine | $dp[\text{day}][\text{hold}][k]$ | Transitions: buy/sell/skip                             | Most variants optimize to $O(1)$     |
| LIS                 | $dp[i]$                          | $dp[i] = 1 + \max(dp[j])$ for $j < i, arr[j] < arr[i]$ | $O(N \log N)$ with binary search     |
| Partition/MCM       | $dp[i][j]$                       | min/max over split point k                             | Fill by interval length (outer: len) |
| Count Ways          | $dp[\text{target}]$              | Accumulate counts, not max/min                         | Watch mod for large answers          |

## TRIE TRIGGER KEYWORDS

| Trigger                         | Use Case                          | Complexity Benefit                                 |
|---------------------------------|-----------------------------------|----------------------------------------------------|
| 'prefix search', 'autocomplete' | Trie insert + search              | $O(L)$ vs $O(N \times L)$ for hashmap prefix check |
| 'all prefixes exist'            | Trie DFS through isEnd nodes only | Avoids re-checking each prefix                     |
| 'maximum XOR'                   | Binary Trie (bits as branches)    | $O(N \times 32)$ vs $O(N^2)$ brute force           |
| 'distinct substrings count'     | Trie or suffix array              | Avoids duplicate substring tracking                |

## THE GOLDEN RULES OF DP

- If your recursion is exponential and the same (index, ...) keeps repeating — it IS a DP problem.
- Define state PRECISELY before writing the recurrence. Wrong state = wrong DP.
- 0/1 Knapsack 1D: RIGHT-to-LEFT. Unbounded Knapsack 1D: LEFT-to-RIGHT. Never mix these up.
- All string DP (LCS, Edit Distance, Wildcard) use a 2D table over (index1, index2) — the recurrence just differs.
- Partition DP (MCM, Burst Balloons): outer loop = interval LENGTH, inner loops = i and j. Try every split k.

– Stock problems: define state as (day, holding, transactions\_remaining). Write transitions. Most optimise

to  $O(1)$  space.

– LIS  $O(N \log N)$ : 'tails' array is sorted but is NOT the actual subsequence — it gives only the LENGTH.

– Trie beats HashMap for prefix operations. Binary Trie beats brute-force for XOR maximization.

You've Completed the Full A2Z Sheet!

| Batch | Steps Covered | Topics                                      |
|-------|---------------|---------------------------------------------|
| 1     | 1–2           | Basics + Sorting                            |
| 2     | 3             | Arrays (Easy to Hard)                       |
| 3     | 4             | Binary Search (1D, Answer, 2D)              |
| 4     | 5 & 18        | Strings (Basic → Advanced, KMP, Z-Function) |
| 5     | 6 & 7         | Linked List + Recursion & Backtracking      |
| 6     | 8 & 9         | Bit Manipulation + Stacks & Queues          |
| 7     | 10, 11 & 12   | Sliding Window + Heaps + Greedy             |
| 8     | 13 & 14       | Binary Trees + BST                          |
| 9     | 15            | Graphs (BFS/DFS, Topo, Dijkstra, DSU, SCC)  |
| 10    | 16 & 17       | Dynamic Programming + Tries                 |

452 Problems · 18 Steps · 10 Batches

Good luck in your interviews!